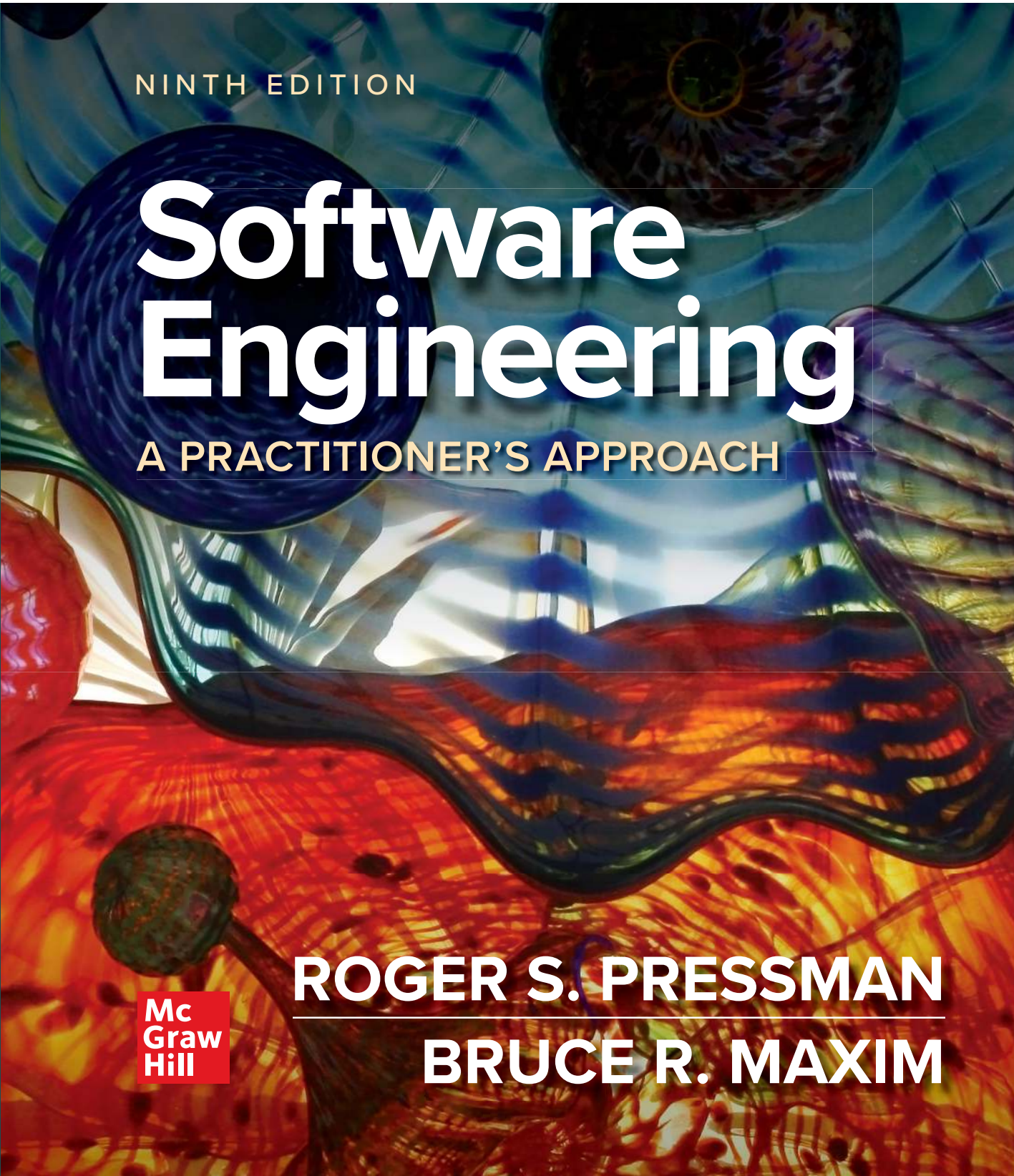
This International Student Edition is for use outside of the U.S.



NINTH EDITION

Software Engineering

A PRACTITIONER'S APPROACH

ROGER S. PRESSMAN

BRUCE R. MAXIM

Mc Graw Hill

Software Engineering

A PRACTITIONER'S APPROACH

Software Engineering

A PRACTITIONER'S APPROACH

NINTH EDITION

Roger S. Pressman, Ph.D.
Bruce R. Maxim, Ph.D.

Mc Graw Hill



SOFTWARE ENGINEERING: A PRACTITIONER'S APPROACH

Published by McGraw-Hill Education, 2 Penn Plaza, New York, NY 10121. Copyright © 2020 by McGraw-Hill Education. All rights reserved. Printed in the United States of America. No part of this publication may be reproduced or distributed in any form or by any means, or stored in a database or retrieval system, without the prior written consent of McGraw-Hill Education, including, but not limited to, in any network or other electronic storage or transmission, or broadcast for distance learning.

Some ancillaries, including electronic and print components, may not be available to customers outside the United States.

This book is printed on acid-free paper.

1 2 3 4 5 6 7 8 9 LCR 24 23 22 21 20 19

ISBN 978-1-260-54800-6
MHID 1-260-54800-7

Cover Image: ©*R.L. Hausdorf/Shutterstock*

All credits appearing on page or at the end of the book are considered to be an extension of the copyright page.

The Internet addresses listed in the text were accurate at the time of publication. The inclusion of a website does not indicate an endorsement by the authors or McGraw-Hill Education, and McGraw-Hill Education does not guarantee the accuracy of the information presented at these sites.

mheducation.com/highered

To Barbara, Matt, Mike,
Shiri, Adam, Lily,
and Maya.

—Roger S. Pressman

To my family who
support me in all
that I do.

—Bruce R. Maxim

About the Authors



Courtesy of Roger Pressman

Roger S. Pressman is an internationally recognized consultant and author in software engineering. For almost five decades, he has worked as a software engineer, a manager, a professor, an author, a consultant, and an entrepreneur.

Dr. Pressman was president of R. S. Pressman & Associates, Inc., a consulting firm that specialized in helping companies establish effective software engineering strategies. Over the years he developed a set of techniques and tools that improved software engineering practice. He is also the founder and chief technology officer of EVANNEX®, an automotive aftermarket company that specializes in the design and manufacture of accessories for the Tesla line of electric vehicles.

Dr. Pressman is the author of ten books, including two novels, and many technical and management papers. He has been on the editorial boards of *IEEE Software* and *The Cutter IT Journal* and was editor of the "Manager" column in *IEEE Software.*

Dr. Pressman is a well-known speaker, keynoting a number of major industry conferences. He has presented tutorials at the International Conference on Software Engineering and at many other industry meetings. He has been a member of the ACM, IEEE, Tau Beta Pi, Phi Kappa Phi, Eta Kappa Nu, and Pi Tau Sigma.



Michigan Creative/
UM-Dearborn

Bruce R. Maxim has worked as a software engineer, project manager, professor, author, and consultant for more than thirty years. His research interests include software engineering, user experience design, serious game development, artificial intelligence, and engineering education.

Dr. Maxim is professor of computer and information science and collegiate professor of engineering at the University of Michigan–Dearborn. He established the GAME Lab in the College of Engineering and Computer Science. He has published papers on computer algorithm animation, game development, and engineering education. He is coauthor of a best-selling introductory computer science text and two edited collections of software engineering research papers. Dr. Maxim has supervised several hundred industry-based software development projects as part of his work at the University of Michigan–Dearborn.

Dr. Maxim's professional experience includes managing research information systems at a medical school, directing instructional computing for a medical campus, and working as a statistical programmer. Dr. Maxim served as the chief technology officer for a game development company.

Dr. Maxim was the recipient of several distinguished teaching awards, a distinguished community service award, and a distinguished faculty governance award. He is a member of Sigma Xi, Upsilon Pi Epsilon, Pi Mu Epsilon, Association of Computing Machinery, IEEE Computer Society, American Society for Engineering Education, Society of Women Engineers, and International Game Developers Association.

Contents at a Glance

CHAPTER 1 Software and Software Engineering 1

PART ONE THE SOFTWARE PROCESS 19

CHAPTER 2 Process Models 20
CHAPTER 3 Agility and Process 37
CHAPTER 4 Recommended Process Model 54
CHAPTER 5 Human Aspects of Software Engineering 74

PART TWO MODELING 83

CHAPTER 6 Principles That Guide Practice 84
CHAPTER 7 Understanding Requirements 102
CHAPTER 8 Requirements Modeling—A Recommended Approach 126
CHAPTER 9 Design Concepts 156
CHAPTER 10 Architectural Design—A Recommended Approach 181
CHAPTER 11 Component-Level Design 206
CHAPTER 12 User Experience Design 233
CHAPTER 13 Design for Mobility 264
CHAPTER 14 Pattern-Based Design 289

PART THREE QUALITY AND SECURITY 309

CHAPTER 15 Quality Concepts 310
CHAPTER 16 Reviews—A Recommended Approach 325
CHAPTER 17 Software Quality Assurance 339
CHAPTER 18 Software Security Engineering 356
CHAPTER 19 Software Testing—Component Level 372
CHAPTER 20 Software Testing—Integration Level 395
CHAPTER 21 Software Testing—Specialized Testing for Mobility 412
CHAPTER 22 Software Configuration Management 437
CHAPTER 23 Software Metrics and Analytics 460

Mc Graw Hill 501-1111

PART FOUR MANAGING SOFTWARE PROJECTS 489

CHAPTER 24 Project Management Concepts 490
CHAPTER 25 Creating a Viable Software Plan 504
CHAPTER 26 Risk Management 532
CHAPTER 27 A Strategy for Software Support 549

PART FIVE ADVANCED TOPICS 567

CHAPTER 28 Software Process Improvement 568
CHAPTER 29 Emerging Trends in Software Engineering 583
CHAPTER 30 Concluding Comments 602

APPENDIX 1 An Introduction to UML 611
APPENDIX 2 Data Science for Software Engineers 629
REFERENCES 639
INDEX 659

Table of Contents

Preface *xxvii*

CHAPTER 1 SOFTWARE AND SOFTWARE ENGINEERING 1

1.1 The Nature of Software 4
- 1.1.1 Defining Software 5
- 1.1.2 Software Application Domains 7
- 1.1.3 Legacy Software 8

1.2 Defining the Discipline 8

1.3 The Software Process 9
- 1.3.1 The Process Framework 10
- 1.3.2 Umbrella Activities 11
- 1.3.3 Process Adaptation 11

1.4 Software Engineering Practice 12
- 1.4.1 The Essence of Practice 12
- 1.4.2 General Principles 14

1.5 How It All Starts 15

1.6 Summary 17

PART ONE THE SOFTWARE PROCESS 19

CHAPTER 2 PROCESS MODELS 20

2.1 A Generic Process Model 21

2.2 Defining a Framework Activity 23

2.3 Identifying a Task Set 23

2.4 Process Assessment and Improvement 24

2.5 Prescriptive Process Models 25
- 2.5.1 The Waterfall Model 25
- 2.5.2 Prototyping Process Model 26
- 2.5.3 Evolutionary Process Model 29
- 2.5.4 Unified Process Model 31

2.6 Product and Process 33

2.7 Summary 35

CHAPTER 3 AGILITY AND PROCESS 37

3.1 What Is Agility? 38
3.2 Agility and the Cost of Change 39
3.3 What Is an Agile Process? 40
3.3.1 Agility Principles 40
3.3.2 The Politics of Agile Development 41
3.4 Scrum 42
3.4.1 Scrum Teams and Artifacts 43
3.4.2 Sprint Planning Meeting 44
3.4.3 Daily Scrum Meeting 44
3.4.4 Sprint Review Meeting 45
3.4.5 Sprint Retrospective 45
3.5 Other Agile Frameworks 46
3.5.1 The XP Framework 46
3.5.2 Kanban 48
3.5.3 DevOps 50
3.6 Summary 51

CHAPTER 4 RECOMMENDED PROCESS MODEL 54

4.1 Requirements Definition 57
4.2 Preliminary Architectural Design 59
4.3 Resource Estimation 60
4.4 First Prototype Construction 61
4.5 Prototype Evaluation 64
4.6 Go, No-Go Decision 65
4.7 Prototype Evolution 67
4.7.1 New Prototype Scope 67
4.7.2 Constructing New Prototypes 68
4.7.3 Testing New Prototypes 68
4.8 Prototype Release 68
4.9 Maintain Release Software 69
4.10 Summary 72

CHAPTER 5 HUMAN ASPECTS OF SOFTWARE ENGINEERING 74

5.1 Characteristics of a Software Engineer 75
5.2 The Psychology of Software Engineering 75

5.3 The Software Team 76
5.4 Team Structures 78
5.5 The Impact of Social Media 79
5.6 Global Teams 80
5.7 Summary 81

PART TWO MODELING 83

CHAPTER 6 PRINCIPLES THAT GUIDE PRACTICE 84

6.1 Core Principles 85
6.1.1 Principles That Guide Process 85
6.1.2 Principles That Guide Practice 86
6.2 Principles That Guide Each Framework Activity 88
6.2.1 Communication Principles 88
6.2.2 Planning Principles 91
6.2.3 Modeling Principles 92
6.2.4 Construction Principles 95
6.2.5 Deployment Principles 98
6.3 Summary 100

CHAPTER 7 UNDERSTANDING REQUIREMENTS 102

7.1 Requirements Engineering 103
7.1.1 Inception 104
7.1.2 Elicitation 104
7.1.3 Elaboration 104
7.1.4 Negotiation 105
7.1.5 Specification 105
7.1.6 Validation 105
7.1.7 Requirements Management 106
7.2 Establishing the Groundwork 107
7.2.1 Identifying Stakeholders 107
7.2.2 Recognizing Multiple Viewpoints 107
7.2.3 Working Toward Collaboration 108
7.2.4 Asking the First Questions 108
7.2.5 Nonfunctional Requirements 109
7.2.6 Traceability 109

7.3 Requirements Gathering 110
7.3.1 Collaborative Requirements Gathering 110
7.3.2 Usage Scenarios 113
7.3.3 Elicitation Work Products 114
7.4 Developing Use Cases 114
7.5 Building the Analysis Model 118
7.5.1 Elements of the Analysis Model 119
7.5.2 Analysis Patterns 122
7.6 Negotiating Requirements 122
7.7 Requirements Monitoring 123
7.8 Validating Requirements 123
7.9 Summary 124

CHAPTER 8 REQUIREMENTS MODELING—A RECOMMENDED APPROACH 126

8.1 Requirements Analysis 127
8.1.1 Overall Objectives and Philosophy 128
8.1.2 Analysis Rules of Thumb 128
8.1.3 Requirements Modeling Principles 129
8.2 Scenario-Based Modeling 130
8.2.1 Actors and User Profiles 131
8.2.2 Creating Use Cases 131
8.2.3 Documenting Use Cases 135
8.3 Class-Based Modeling 137
8.3.1 Identifying Analysis Classes 137
8.3.2 Defining Attributes and Operations 140
8.3.3 UML Class Models 141
8.3.4 Class-Responsibility-Collaborator Modeling 144
8.4 Functional Modeling 146
8.4.1 A Procedural View 146
8.4.2 UML Sequence Diagrams 148
8.5 Behavioral Modeling 149
8.5.1 Identifying Events with the Use Case 149
8.5.2 UML State Diagrams 150
8.5.3 UML Activity Diagrams 151
8.6 Summary 154

CHAPTER 9 DESIGN CONCEPTS 156

9.1 Design Within the Context of Software Engineering 157
9.2 The Design Process 159
9.2.1 Software Quality Guidelines and Attributes 160
9.2.2 The Evolution of Software Design 161
9.3 Design Concepts 163
9.3.1 Abstraction 163
9.3.2 Architecture 163
9.3.3 Patterns 164
9.3.4 Separation of Concerns 165
9.3.5 Modularity 165
9.3.6 Information Hiding 166
9.3.7 Functional Independence 167
9.3.8 Stepwise Refinement 167
9.3.9 Refactoring 168
9.3.10 Design Classes 169
9.4 The Design Model 171
9.4.1 Design Modeling Principles 173
9.4.2 Data Design Elements 174
9.4.3 Architectural Design Elements 175
9.4.4 Interface Design Elements 175
9.4.5 Component-Level Design Elements 176
9.4.6 Deployment-Level Design Elements 177
9.5 Summary 178

CHAPTER 10 ARCHITECTURAL DESIGN— A RECOMMENDED APPROACH 181

10.1 Software Architecture 182
10.1.1 What Is Architecture? 182
10.1.2 Why Is Architecture Important? 183
10.1.3 Architectural Descriptions 183
10.1.4 Architectural Decisions 184
10.2 Agility and Architecture 185
10.3 Architectural Styles 186
10.3.1 A Brief Taxonomy of Architectural Styles 187
10.3.2 Architectural Patterns 192
10.3.3 Organization and Refinement 193

10.4 Architectural Considerations 193
10.5 Architectural Decisions 195
10.6 Architectural Design 196
10.6.1 Representing the System in Context 196
10.6.2 Defining Archetypes 197
10.6.3 Refining the Architecture into Components 198
10.6.4 Describing Instantiations of the System 200
10.7 Assessing Alternative Architectural Designs 201
10.7.1 Architectural Reviews 202
10.7.2 Pattern-Based Architecture Review 203
10.7.3 Architecture Conformance Checking 204
10.8 Summary 204

CHAPTER 11 COMPONENT-LEVEL DESIGN 206

11.1 What Is a Component? 207
11.1.1 An Object-Oriented View 207
11.1.2 The Traditional View 209
11.1.3 A Process-Related View 211
11.2 Designing Class-Based Components 212
11.2.1 Basic Design Principles 212
11.2.2 Component-Level Design Guidelines 215
11.2.3 Cohesion 216
11.2.4 Coupling 218
11.3 Conducting Component-Level Design 219
11.4 Specialized Component-Level Design 225
11.4.1 Component-Level Design for WebApps 226
11.4.2 Component-Level Design for Mobile Apps 226
11.4.3 Designing Traditional Components 227
11.4.4 Component-Based Development 228
11.5 Component Refactoring 230
11.6 Summary 231

CHAPTER 12 USER EXPERIENCE DESIGN 233

12.1 User Experience Design Elements 234
12.1.1 Information Architecture 235
12.1.2 User Interaction Design 236
12.1.3 Usability Engineering 236
12.1.4 Visual Design 237

12.2 The Golden Rules 238
12.2.1 Place the User in Control 238
12.2.2 Reduce the User's Memory Load 239
12.2.3 Make the Interface Consistent 240
12.3 User Interface Analysis and Design 241
12.3.1 Interface Analysis and Design Models 241
12.3.2 The Process 242
12.4 User Experience Analysis 243
12.4.1 User Research 244
12.4.2 User Modeling 245
12.4.3 Task Analysis 247
12.4.4 Work Environment Analysis 248
12.5 User Experience Design 249
12.6 User Interface Design 250
12.6.1 Applying Interface Design Steps 251
12.6.2 User Interface Design Patterns 252
12.7 Design Evaluation 253
12.7.1 Prototype Review 253
12.7.2 User Testing 255
12.8 Usability and Accessibility 255
12.8.1 Usability Guidelines 257
12.8.2 Accessibility Guidelines 259
12.9 Conventional Software UX and Mobility 261
12.10 Summary 261

CHAPTER 13 DESIGN FOR MOBILITY 264

13.1 The Challenges 265
13.1.1 Development Considerations 265
13.1.2 Technical Considerations 266
13.2 Mobile Development Life Cycle 268
13.2.1 User Interface Design 270
13.2.2 Lessons Learned 271
13.3 Mobile Architectures 273
13.4 Context-Aware Apps 274
13.5 Web Design Pyramid 275
13.5.1 WebApp Interface Design 275
13.5.2 Aesthetic Design 277
13.5.3 Content Design 277
13.5.4 Architecture Design 278
13.5.5 Navigation Design 280

13.6 Component-Level Design 282
13.7 Mobility and Design Quality 282
13.8 Mobility Design Best Practices 285
13.9 Summary 287

CHAPTER 14 PATTERN-BASED DESIGN 289

14.1 Design Patterns 290
14.1.1 Kinds of Patterns 291
14.1.2 Frameworks 293
14.1.3 Describing a Pattern 293
14.1.4 Machine Learning and Pattern Discovery 294
14.2 Pattern-Based Software Design 295
14.2.1 Pattern-Based Design in Context 295
14.2.2 Thinking in Patterns 296
14,2.3 Design Tasks 297
14.2.4 Building a Pattern-Organizing Table 298
14.2.5 Common Design Mistakes 298
14.3 Architectural Patterns 299
14.4 Component-Level Design Patterns 300
14.5 Anti-Patterns 302
14.6 User Interface Design Patterns 304
14.7 Mobility Design Patterns 305
14.8 Summary 306

PART THREE QUALITY AND SECURITY 309

CHAPTER 15 QUALITY CONCEPTS 310

15.1 What Is Quality? 311
15.2 Software Quality 312
15.2.1 Quality Factors 312
15.2.2 Qualitative Quality Assessment 314
15.2.3 Quantitative Quality Assessment 315
15.3 The Software Quality Dilemma 315
15.3.1 “Good Enough” Software 316
15.3.2 The Cost of Quality 317
15.3.3 Risks 319
15.3.4 Negligence and Liability 320

15.3.5 Quality and Security 320
15.3.6 The Impact of Management Actions 321
15.4 Achieving Software Quality 321
15.4.1 Software Engineering Methods 322
15.4.2 Project Management Techniques 322
15.4.3 Machine Learning and Defect Prediction 322
15.4.4 Quality Control 322
15.4.5 Quality Assurance 323
15.5 Summary 323

CHAPTER 16 REVIEWS—A RECOMMENDED APPROACH 325

16.1 Cost Impact of Software Defects 326
16.2 Defect Amplification and Removal 327
16.3 Review Metrics and Their Use 327
16.4 Criteria for Types of Reviews 330
16.5 Informal Reviews 331
16.6 Formal Technical Reviews 332
16.6.1 The Review Meeting 332
16.6.2 Review Reporting and Record Keeping 333
16.6.3 Review Guidelines 334
16.7 Postmortem Evaluations 336
16.8 Agile Reviews 336
16.9 Summary 337

CHAPTER 17 SOFTWARE QUALITY ASSURANCE 339

17.1 Background Issues 341
17.2 Elements of Software Quality Assurance 341
17.3 SQA Processes and Product Characteristics 343
17.4 SQA Tasks, Goals, and Metrics 343
17.4.1 SQA Tasks 343
17.4.2 Goals, Attributes, and Metrics 345
17.5 Formal Approaches to SQA 347
17.6 Statistical Software Quality Assurance 347

17.6.1 A Generic Example 347
17.6.2 Six Sigma for Software Engineering 349
17.7 Software Reliability 350
17.7.1 Measures of Reliability and Availability 350
17.7.2 Use of AI to Model Reliability 351
17.7.3 Software Safety 352
17.8 The ISO 9000 Quality Standards 353
17.9 The SQA Plan 354
17.10 Summary 355

CHAPTER 18 SOFTWARE SECURITY ENGINEERING 356

18.1 Why Software Security Information Is Important 357
18.2 Security Life-Cycle Models 357
18.3 Secure Development Life-Cycle Activities 359
18.4 Security Requirements Engineering 360
18.4.1 SQUARE 360
18.4.2 The SQUARE Process 360
18.5 Misuse and Abuse Cases and Attack Patterns 363
18.6 Security Risk Analysis 364
18.7 Threat Modeling, Prioritization, and Mitigation 365
18.8 Attack Surface 366
18.9 Secure Coding 367
18.10 Measurement 368
18.11 Security Process Improvement and Maturity Models 370
18.12 Summary 370

CHAPTER 19 SOFTWARE TESTING—COMPONENT LEVEL 372

19.1 A Strategic Approach to Software Testing 373
19.1.1 Verification and Validation 373
19.1.2 Organizing for Software Testing 374
19.1.3 The Big Picture 375
19.1.4 Criteria for “Done” 377

19.2 Planning and Recordkeeping 378
19.2.1 Role of Scaffolding 379
19.2.2 Cost-Effective Testing 379
19.3 Test-Case Design 381
19.3.1 Requirements and Use Cases 382
19.3.2 Traceability 383
19.4 White-Box Testing 383
19.4.1 Basis Path Testing 384
19.4.2 Control Structure Testing 386
19.5 Black-Box Testing 388
19.5.1 Interface Testing 388
19.5.2 Equivalence Partitioning 389
19.5.3 Boundary Value Analysis 389
19.6 Object-Oriented Testing 390
19.6.1 Class Testing 390
19.6.2 Behavioral Testing 392
19.7 Summary 393

CHAPTER 20 SOFTWARE TESTING—INTEGRATION LEVEL 395

20.1 Software Testing Fundamentals 396
20.1.1 Black-Box Testing 397
20.1.2 White-Box Testing 397
20.2 Integration Testing 398
20.2.1 Top-Down Integration 398
20.2.2 Bottom-Up Integration 399
20.2.3 Continuous Integration 400
20.2.4 Integration Test Work Products 402
20.3 Artificial Intelligence and Regression Testing 402
20.4 Integration Testing in the OO Context 404
20.4.1 Fault-Based Test-Case Design 405
20.4.2 Scenario-Based Test-Case Design 406
20.5 Validation Testing 407
20.6 Testing Patterns 409
20.7 Summary 409

CHAPTER 21 SOFTWARE TESTING—SPECIALIZED TESTING FOR MOBILITY 412

21.1 Mobile Testing Guidelines 413
21.2 The Testing Strategies 414
21.3 User Experience Testing Issues 415
21.3.1 Gesture Testing 415
21.3.2 Virtual Keyboard Input 416
21.3.3 Voice Input and Recognition 416
21.3.4 Alerts and Extraordinary Conditions 417
21.4 Web Application Testing 418
21.5 Web Testing Strategies 418
21.5.1 Content Testing 420
21.5.2 Interface Testing 421
21.5.3 Navigation Testing 421
21.6 Internationalization 423
21.7 Security Testing 423
21.8 Performance Testing 424
21.9 Real-Time Testing 426
21.10 Testing AI Systems 428
21.10.1 Static and Dynamic Testing 429
21.10.2 Model-Based Testing 429
21.11 Testing Virtual Environments 430
21.11.1 Usability Testing 430
21.11.2 Accessibility Testing 433
21.11.3 Playability Testing 433
21.12 Testing Documentation and Help Facilities 434
21.13 Summary 435

CHAPTER 22 SOFTWARE CONFIGURATION MANAGEMENT 437

22.1 Software Configuration Management 438
22.1.1 An SCM Scenario 439
22.1.2 Elements of a Configuration Management System 440
22.1.3 Baselines 441

22.1.4 Software Configuration Items 441
22.1.5 Management of Dependencies and Changes 442
22.2 The SCM Repository 443
22.2.1 General Features and Content 444
22.2.2 SCM Features 444
22.3 Version Control Systems 445
22.4 Continuous Integration 446
22.5 The Change Management Process 447
22.5.1 Change Control 448
22.5.2 Impact Management 451
22.5.3 Configuration Audit 452
22.5.4 Status Reporting 452
22.6 Mobility and Agile Change Management 453
22.6.1 e-Change Control 453
22.6.2 Content Management 455
22.6.3 Integration and Publishing 455
22.6.4 Version Control 457
22.6.5 Auditing and Reporting 458
22.7 Summary 458

CHAPTER 23 SOFTWARE METRICS AND ANALYTICS 460

23.1 Software Measurement 461
23.1.1 Measures, Metrics, and Indicators 461
23.1.2 Attributes of Effective Software Metrics 462
23.2 Software Analytics 462
23.3 Product Metrics 463
23.3.1 Metrics for the Requirements Model 464
23.3.2 Design Metrics for Conventional Software 466
23.3.3 Design Metrics for Object-Oriented Software 468
23.3.4 User Interface Design Metrics 471
23.3.5 Metrics for Source Code 473
23.4 Metrics for Testing 474
23.5 Metrics for Maintenance 476
23.6 Process and Project Metrics 476

23.7 Software Measurement 479
23.8 Metrics for Software Quality 482
23.9 Establishing Software Metrics Programs 485
23.10 Summary 487

PART FOUR MANAGING SOFTWARE PROJECTS 489

CHAPTER 24 PROJECT MANAGEMENT CONCEPTS 490

24.1 The Management Spectrum 491
24.1.1 The People 491
24.1.2 The Product 491
24.1.3 The Process 492
24.1.4 The Project 492
24.2 People 493
24.2.1 The Stakeholders 493
24.2.2 Team Leaders 493
24.2.3 The Software Team 494
24.2.4 Coordination and Communications Issues 496
24.3 Product 497
24.3.1 Software Scope 497
24.3.2 Problem Decomposition 497
24.4 Process 498
24.4.1 Melding the Product and the Process 498
24.4.2 Process Decomposition 498
24.5 Project 500
24.6 The W^5HH Principle 501
24.7 Critical Practices 502
24.8 Summary 502

CHAPTER 25 CREATING A VIABLE SOFTWARE PLAN 504

25.1 Comments on Estimation 505
25.2 The Project Planning Process 506

25.3 Software Scope and Feasibility 507
25.4 Resources 507
25.4.1 Human Resources 508
25.4.2 Reusable Software Resources 509
25.4.3 Environmental Resources 509
25.5 Data Analytics and Software Project Estimation 509
25.6 Decomposition and Estimation Techniques 511
25.6.1 Software Sizing 511
25.6.2 Problem-Based Estimation 512
25.6.3 An Example of LOC-Based Estimation 512
25.6.4 An Example of FP-Based Estimation 514
25.6.5 An Example of Process-Based Estimation 515
25.6.6 An Example of Estimation Using Use Case Points 517
25.6.7 Reconciling Estimates 518
25.6.8 Estimation for Agile Development 519
25.7 Project Scheduling 520
25.7.1 Basic Principles 521
25.7.2 The Relationship Between People and Effort 522
25.8 Defining a Project Task Set 523
25.8.1 A Task Set Example 524
25.8.2 Refinement of Major Tasks 524
25.9 Defining a Task Network 525
25.10 Scheduling 226
25.10.1 Time-Line Charts 526
25.10.2 Tracking the Schedule 528
25.11 Summary 530

CHAPTER 26 RISK MANAGEMENT 532

26.1 Reactive Versus Proactive Risk Strategies 533
26.2 Software Risks 534
26.3 Risk Identification 535
26.3.1 Assessing Overall Project Risk 536
26.3.2 Risk Components and Drivers 537
26.4 Risk Projection 538

26.4.1 Developing a Risk Table 538
26.4.2 Assessing Risk Impact 540
26.5 Risk Refinement 542
26.6 Risk Mitigation, Monitoring, and Management 543
26.7 The RMMM Plan 546
26.8 Summary 547

CHAPTER 27 A STRATEGY FOR SOFTWARE SUPPORT 549

27.1 Software Support 550
27.2 Software Maintenance 552
27.2.1 Maintenance Types 553
27.2.2 Maintenance Tasks 554
27.2.3 Reverse Engineering 555
27.3 Proactive Software Support 557
27.3.1 Use of Software Analytics 558
27.3.2 Role of Social Media 559
27.3.3 Cost of Support 559
27.4 Refactoring 560
27.4.1 Data Refactoring 561
24.4.2 Code Refactoring 561
27.4.3 Architecture Refactoring 561
27.5 Software Evolution 562
27.5.1 Inventory Analysis 563
27.5.2 Document Restructuring 564
27.5.3 Reverse Engineering 564
27.5.4 Code Refactoring 564
27.7.5 Data Refactoring 564
27.5.6 Forward Engineering 565
27.6 Summary 565

PART FIVE ADVANCED TOPICS 567

CHAPTER 28 SOFTWARE PROCESS IMPROVEMENT 568

28.1 What Is SPI? 569
28.1.1 Approaches to SPI 569
28.1.2 Maturity Models 570
28.1.3 Is SPI for Everyone? 571

28.2 The SPI Process 571
28.2.1 Assessment and GAP Analysis 572
28.2.2 Education and Training 573
28.2.3 Selection and Justification 573
28.2.4 Installation/Migration 574
28.2.5 Evaluation 575
28.2.6 Risk Management for SPI 575
28.3 The CMMI 576
28.4 Other SPI Frameworks 579
28.4.1 SPICE 579
28.4.2 TickIT Plus 579
28.5 SPI Return on Investment 580
28.6 SPI Trends 580
28.7 Summary 581

CHAPTER 29 EMERGING TRENDS IN SOFTWARE ENGINEERING 583

29.1 Technology Evolution 584
29.2 Software Engineering as a Discipline 585
29.3 Observing Software Engineering Trends 586
29.4 Identifying “Soft Trends” 587
29.4.1 Managing Complexity 588
29.4.2 Open-World Software 589
29.4.3 Emergent Requirements 590
29.4.4 The Talent Mix 591
29.4.5 Software Building Blocks 591
29.4.6 Changing Perceptions of “Value” 592
29.4.7 Open Source 592
29.5 Technology Directions 593
29.5.1 Process Trends 593
29.5.2 The Grand Challenge 594
29.5.3 Collaborative Development 595
29.5.4 Requirements Engineering 596
29.5.5 Model-Driven Software Development 596
29.5.6 Search-Based Software Engineering 597
29.5.7 Test-Driven Development 598
29.6 Tools-Related Trends 599
29.7 Summary 600

CHAPTER 30 CONCLUDING COMMENTS 602

30.1 The Importance of Software—Revisited 603
30.2 People and the Way They Build Systems 603
30.3 Knowledge Discovery 605
30.4 The Long View 606
30.5 The Software Engineer's Responsibility 607
30.6 A Final Comment from RSP 609

APPENDIX 1 An Introduction to UML 611
APPENDIX 2 Data Science for Software Engineers 629
REFERENCES 639
INDEX 659

PREFACE

When computer software succeeds—when it meets the needs of the people who use it, when it performs flawlessly over a long period of time, when it is easy to modify and even easier to use—it can and does change things for the better. But when software fails—when its users are dissatisfied, when it is error prone, when it is difficult to change and even harder to use—bad things can and do happen. We all want to build software that makes things better, avoiding the bad things that lurk in the shadow of failed efforts. To succeed, we need discipline when software is designed and built. We need an engineering approach.

It has been nearly four decades since the first edition of this book was written. During that time, software engineering has evolved from an obscure idea practiced by a relatively small number of zealots to a legitimate engineering discipline. Today, it is recognized as a subject worthy of serious research, conscientious study, and tumultuous debate. Throughout the industry, software engineer has replaced programmer or coder as the job title of preference. Software process models, software engineering methods, and software tools have been adopted successfully across a broad spectrum of industry segments.

Although managers and practitioners alike recognize the need for a more disciplined approach to software, they continue to debate the manner in which discipline is to be applied. Many individuals and companies still develop software haphazardly, even as they build systems to service today's most advanced technologies. Many professionals and students are unaware of modern methods. And as a result, the quality of the software that we produce suffers, and bad things happen. In addition, debate, and controversy about the true nature of the software engineering approach continue. The status of software engineering is a study in contrasts. Attitudes have changed, progress has been made, but much remains to be done before the discipline reaches full maturity.

NEW TO THE NINTH EDITION

The ninth edition of *Software Engineering: A Practitioner's Approach* is intended to serve as a guide to a maturing engineering discipline. The ninth edition, like the eight editions that preceded it, is intended for both students and practitioners, retaining its appeal as a guide for the industry professional and a comprehensive introduction to the student at the upper-level undergraduate or first-year graduate level.

The ninth edition is considerably more than a simple update. The book has been revised and restructured to improve pedagogical flow and emphasize new and important software engineering processes and practices. In addition, we have further enhanced the popular "support system" for the book, providing a comprehensive set of student, instructor, and professional resources to complement the content of the book.

Readers of the past few editions of *Software Engineering: A Practitioner's Approach* will note that the ninth edition has actually been reduced in page length. Our goal was concision, making the book stronger from a pedagogical viewpoint and less daunting for the reader who desires to journey through the entire book. An anecdote, attributed to Blaise Pascal, the famous mathematician and physicist, goes like this: In writing a overly long letter to a friend, Pascal ended with this sentence. "I wanted to write you a shorter letter, but I didn't have the time." As we worked on concision for the ninth edition, we came to appreciate Pascal's words.

The 30 chapters of the ninth edition are organized into five parts. This organization better compartmentalizes topics and assists instructors who may not have the time to complete the entire book in one term.

Part 1, *The Software Process,* presents a variety of different views of software process, considering several important process models and frameworks that allow us to address the debate between prescriptive and agile process philosophies. Part 2, *Modeling,* presents analysis and design methods with an emphasis on object-oriented techniques and UML modeling. Pattern-based design and design for mobility computing applications are also considered. The coverage of user experience design has been expanded in this section. Part 3, *Quality and Security,* presents the concepts, procedures, techniques, and methods that enable a software team to assess software quality, review software engineering work products, conduct SQA procedures, and apply an effective testing strategy and tactics. In addition, we present software security practices that can be inserted into incremental software development models. Part 4, *Managing Software Projects,* presents topics that are relevant to those who plan, manage, and control a software development project. Part 5, *Advanced Topics,* considers software process improvement and software engineering trends. Boxed features are included throughout the book to present the trials and tribulations of a (fictional) software team and to provide supplementary materials about methods and tools that are relevant to chapter topics.

The five-part organization of the ninth edition enables an instructor to "cluster" topics based on available time and student need. An entire one-term course can be built around one or more of the five parts. A software engineering survey course would select chapters from all five parts. A software engineering course that emphasizes analysis and design would select topics from Parts 1 and 2. A testing-oriented software engineering course would select topics from Parts 1 and 3, with a brief foray into Part 2. A "management course" would stress Parts 1 and 4. By organizing the ninth edition in this way,

we have attempted to provide an instructor with a number of teaching options. In every case the content of the ninth edition is complemented by the following elements of the *SEPA, 9/e Support System.*

Additional Resources

A wide variety of resources can be accessed through the instructor website, including an extensive online learning center encompassing problem solutions, a variety of Web-based resources with software engineering checklists, an evolving collection of "tiny tools," and comprehensive case studies. *Professional Resources* provide several hundred categorized web references which allow students to explore software engineering in greater detail, along with a reference library with links to several hundred downloadable references providing an in-depth source of advanced software engineering information. Additionally, a complete online *Instructor's Guide* and supplementary teaching materials along with several hundred PowerPoint slides that may be used for lectures are included.

The *Instructor's Guide for Software Engineering: A Practitioner's Approach* presents suggestions for conducting various types of software engineering courses, recommendations for a variety of software projects to be conducted in conjunction with a course, solutions to selected problems, and a number of useful teaching aids.

When coupled with its online support system, the ninth edition of *Software Engineering: A Practitioner's Approach* provides flexibility and depth of content that cannot be achieved by a textbook alone.

Bruce Maxim has taken the lead in developing new content for the ninth edition of *Software Engineering: A Practitioner's Approach,* while Roger Pressman has served as editor-in-chief as well as providing contributions in select circumstances.

Acknowledgments Special thanks go to Nancy Mead from Software Engineering Institute at Carnegie Mellon University who wrote the chapter on software security engineering; Tim Lethbridge of the University of Ottawa who assisted us in the development of UML and OCL examples and developed the case study that accompanies this book; Dale Skrien of Colby College who developed the UML tutorial in Appendix 1; William Grosky of the University of Michigan–Dearborn who developed the overview of data science in Appendix 2 with his student Terry Ruas; and our Australian colleague Margaret Kellow for updating the pedagogical Web materials that accompany this book. In addition, we would like to thank Austin Krauss for providing insight into software development in the video game industry from his perspective as a senior software engineer.

Special Thanks BRM: I am grateful to have had the opportunity to work with Roger on the ninth edition of this book. During the time I have been working on this book, my son, Benjamin, has become a software engineering manager and my daughter, Katherine, used her art background to create the figures that appear in the book chapters. I am quite pleased to see the adults they have become and enjoy my time with their children

(Isla, Emma, and Thelma). I am very grateful to my wife, Norma, for her enthusiastic support as I filled my free time working on this book.

RSP: As the editions of this book have evolved, my sons, Mathew and Michael, have grown from boys to men. Their maturity, character, and success in the real world have been an inspiration to me. After many years of following our own professional paths, the three of us now work together in a business that we founded in 2012. Nothing has filled me with more pride. Both of my sons now have children of their own, Maya and Lily, who start still another generation. Finally, to my wife, Barbara, my love and thanks for tolerating the many, many hours in the office and encouraging still another edition of "the book."

Bruce R. Maxim
Roger S. Pressman

Design element: Quick Look icon magnifying glass: © Roger Pressman

Affordability & Outcomes = Academic Freedom!

You deserve choice, flexibility and control. You know what's best for your students and selecting the course materials that will help them succeed should be in your hands.

That's why providing you with a wide range of options that lower costs and drive better outcomes is our highest priority.

Mc Graw Hill connect®

Students—study more efficiently, retain more and achieve better outcomes. Instructors—focus on what you love—teaching.



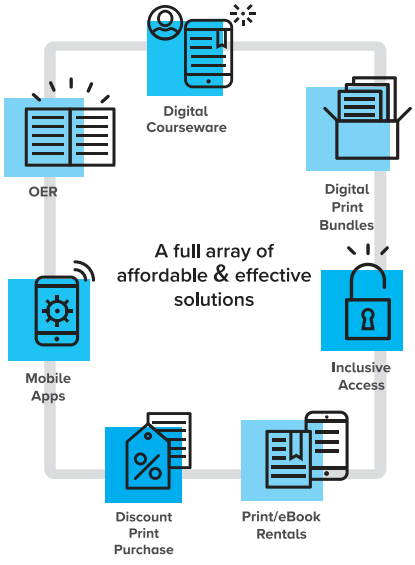
Laptop: McGraw-Hill Education

They'll thank you for it.

Study resources in Connect help your students be better prepared in less time. You can transform your class time from dull definitions to dynamic discussion. Hear from your peers about the benefits of Connect at **www.mheducation.com/highered/connect/smartbook**

Make it simple, make it affordable.

Connect makes it easy with seamless integration using any of the major Learning Management Systems—Blackboard®, Canvas, and D2L, among others—to let you organize your course in one convenient location. Give your students access to digital materials at a discount with our inclusive access program. Ask your McGraw-Hill representative for more information.



Learning for everyone.

McGraw-Hill works directly with Accessibility Services Departments and faculty to meet the learning needs of all students. Please contact your Accessibility Services office and ask them to email accessibility@mheducation.com, or visit **www.mheducation.com/about/accessibility.html** for more information.

Learn more at: www.mheducation.com/realvalue

Rent It

Affordable print and digital rental options through our partnerships with leading textbook distributors including Amazon, Barnes & Noble, Chegg, Follett, and more.

Go Digital

A full and flexible range of affordable digital solutions ranging from Connect, ALEKS, inclusive access, mobile apps, OER and more.

Get Print

Students who purchase digital materials can get a loose-leaf print version at a significantly reduced rate to meet their individual preferences and budget.

CHAPTER

1 SOFTWARE AND SOFTWARE ENGINEERING

As he finished showing me the latest build of one of the world's most popular first-person shooter video games, the young developer laughed.

"You're not a gamer, are you?" he asked.

I smiled. "How'd you guess?"

The young man was dressed in shorts and a tee shirt. His leg bounced up and down like a piston, burning the nervous energy that seemed to be commonplace among his co-workers.

KEY CONCEPTS

application domains 7
failure curves 5
framework activities 10
general principles 14
legacy software 8
principles 14
problem solving 12
SafeHome 16
software,
  definition 5
  nature of 4
  process 9
  questions about 4
software engineering,
  definition 3
  layers 9
  practice 12
umbrella activities 11
wear 5

QUICK LOOK

What is it? Computer software is a work product that software professionals build and then support over many years. These work products include programs that execute within computers of any size and architecture. Software engineering encompasses a process, a collection of methods (practice), and an array of tools that allow professionals to build high-quality computer software.

Who does it? Software engineers build and support software, and virtually everyone in the industrialized world uses it. Software engineers apply the software engineering process.

Why is it important? Software engineering is important because it enables us to build complex systems in a timely manner and with high quality. It imposes discipline to work that can become quite chaotic, but it also allows the people who build computer software to adapt their approach in a manner that best suits their needs.

What are the steps? You build computer software like you build any successful product, by applying an agile, adaptable process that leads to a high-quality result that meets the needs of the people who will use the product.

What is the work product? From the software engineer's point of view, the work product is the set of programs, content (data), and other work products that support computer software. But from the user's point of view, the work product is a tool or product that somehow makes the user's world better.

How do I ensure that I've done it right? Read the remainder of this book, select those ideas that are applicable to the software that you build, and apply them to your work.

"Because if you were," he said, "you'd be a lot more excited. You've gotten a peek at our next generation product and that's something that our customers would kill for . . . no pun intended."

We sat in a development area at one of the most successful game developers on the planet. Over the years, earlier generations of the game he demoed sold over 50 million copies and generated billions of dollars in revenue.

"So, when will this version be on the market?" I asked.

He shrugged. "In about five months, and we've still got a lot of work to do."

He had responsibility for game play and artificial intelligence functionality in an application that encompassed more than three million lines of code.

"Do you guys use any software engineering techniques?" I asked, half-expecting that he'd laugh and shake his head.

He paused and thought for a moment. Then he slowly nodded. "We adapt them to our needs, but sure, we use them."

"Where?" I asked, probing.

"Our problem is often translating the requirements the creatives give us."

"The creatives?" I interrupted.

"You know, the guys who design the story, the characters, all the stuff that make the game a hit. We have to take what they give us and produce a set of technical requirements that allow us to build the game."

"And after the requirements are established?"

He shrugged. "We have to extend and adapt the architecture of the previous version of the game and create a new product. We have to create code from the requirements, test the code with daily builds, and do lots of things that your book recommends."

"You know my book?" I was honestly surprised.

"Sure, used it in school. There's a lot there."

"I've talked to some of your buddies here, and they're more skeptical about the stuff in my book."

He frowned. "Look, we're not an IT department or an aerospace company, so we have to customize what you advocate. But the bottom line is the same—we need to produce a high-quality product, and the only way we can accomplish that in a repeatable fashion is to adapt our own subset of software engineering techniques."

"And how will your subset change as the years pass?"

He paused as if to ponder the future. "Games will become bigger and more complex, that's for sure. And our development timelines will shrink as more competition emerges. Slowly, the games themselves will force us to apply a bit more development discipline. If we don't, we're dead."

*****

Computer software continues to be the single most important technology on the world stage. And it's also a prime example of the law of unintended consequences. Sixty years ago no one could have predicted that software would become an indispensable technology for business, science, and engineering; that software would enable the creation of new technologies (e.g., genetic engineering and nanotechnology), the extension of existing technologies (e.g., telecommunications), and the radical change in older technologies (e.g., the media); that software would be the driving force behind the personal computer revolution; that software applications would be purchased by

consumers using their mobile devices; that software would slowly evolve from a product to a service as "on-demand" software companies deliver just-in-time functionality via a Web browser; that a software company would become larger and more influential than all industrial-era companies; or that a vast software-driven network would evolve and change everything from library research to consumer shopping to political discourse to the dating habits of young (and not so young) adults.

As software's importance has grown, the software community has continually attempted to develop technologies that will make it easier, faster, and less expensive to build and support high-quality computer programs. Some of these technologies are targeted at a specific application domain (e.g., website design and implementation); others focus on a technology domain (e.g., object-oriented systems or aspect-oriented programming); and still others are broad based (e.g., operating systems such as Linux). However, we have yet to develop a software technology that does it all, and the likelihood of one arising in the future is small. And yet, people bet their jobs, their comforts, their safety, their entertainment, their decisions, and their very lives on computer software. It better be right.

This book presents a framework that can be used by those who build computer software—people who must get it right. The framework encompasses a process, a set of methods, and an array of tools that we call *software engineering.*

To build software that is ready to meet the challenges of the twenty-first century, you must recognize a few simple realities:

- Software has become deeply embedded in virtually every aspect of our lives. The number of people who have an interest in the features and functions provided by a specific application[1] has grown dramatically. *A concerted effort should be made to understand the problem before a software solution is developed.*
- The information technology requirements demanded by individuals, businesses, and governments grow increasingly complex with each passing year. Large teams of people now create computer programs. Sophisticated software that was once implemented in a predictable, self-contained computing environment is now embedded inside everything from consumer electronics to medical devices to autonomous vehicles. *Design has become a pivotal activity.*
- Individuals, businesses, and governments increasingly rely on software for strategic and tactical decision making as well as day-to-day operations and control. If the software fails, people and major enterprises can experience anything from minor inconvenience to catastrophic consequences. *Software should exhibit high quality.*
- As the perceived value of a specific application grows, the likelihood is that its user base and longevity will also grow. As its user base and time in use increase, demands for adaptation and enhancement will also grow. *Software should be maintainable.*

These simple realities lead to one conclusion: *Software in all its forms and across all its application domains should be engineered.* And that leads us to the topic of this book—*software engineering.*

1 We will call these people "stakeholders" later in this book.

1.1 The Nature of Software

Today, software takes on a dual role. It is a product, and the vehicle for delivering a product. As a product, it delivers the computing potential embodied by computer hardware or, more broadly, by a network of computers that are accessible by local hardware. Whether it resides within a mobile device, on the desktop, in the cloud, or within a mainframe computer or autonomous machine, software is an information transformer—producing, managing, acquiring, modifying, displaying, or transmitting information that can be as simple as a single bit or as complex as an augmented-reality representation derived from data acquired from dozens of independent sources and then overlaid on the real world. As the vehicle used to deliver a product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environments).

Software delivers the most important product of our time—*information.* It transforms personal data (e.g., an individual's financial transactions) so that the data can be more useful in a local context; it manages business information to enhance competitiveness; it provides a gateway to worldwide information networks (e.g., the Internet); and provides the means for acquiring information in all its forms. It also provides a vehicle that can threaten personal privacy and a gateway that enables those with malicious intent to commit criminal acts.

The role of computer software has undergone significant change over the last 60 years. Dramatic improvements in hardware performance, profound changes in computing architectures, vast increases in memory and storage capacity, and a wide variety of exotic input and output options have all precipitated more sophisticated and complex computer-based systems. Sophistication and complexity can produce dazzling results when a system succeeds, but they can also pose huge problems for those who must build and protect complex systems.

Today, a huge software industry has become a dominant factor in the economies of the industrialized world. Teams of software specialists, each focusing on one part of the technology required to deliver a complex application, have replaced the lone programmer of an earlier era. And yet, the questions that were asked of the lone programmer are the same questions that are asked when modern computer-based systems are built:[2]

- Why does it take so long to get software finished?
- Why are development costs so high?
- Why can't we find all errors before we give the software to our customers?
- Why do we spend so much time and effort maintaining existing programs?
- Why do we continue to have difficulty in measuring progress as software is being developed and maintained?

2 In an excellent book of essays on the software business, Tom DeMarco [DeM95] argues the counterpoint. He states: "Instead of asking why software costs so much, we need to begin asking 'What have we done to make it possible for today's software to cost so little?' The answer to that question will help us continue the extraordinary level of achievement that has always distinguished the software industry."

These, and many other questions, are a manifestation of the concern about software and how it is developed—a concern that has led to the adoption of software engineering practice.

1.1.1 Defining Software

Today, most professionals and many members of the public at large feel that they understand software. But do they?

A textbook description of software might take the following form:

> Software is: (1) instructions (computer programs) that when executed provide desired features, function, and performance; (2) data structures that enable the programs to adequately manipulate information; and (3) descriptive information in both hard copy and virtual forms that describes the operation and use of the programs.

There is no question that other more complete definitions could be offered. But a more formal definition probably won't measurably improve your understanding. To accomplish that, it's important to examine the characteristics of software that make it different from other things that human beings build. Software is a logical rather than a physical system element. Therefore, software has one fundamental characteristic that makes it considerably different from hardware: *Software doesn't "wear out."*

Figure 1.1 depicts failure rate as a function of time for hardware. The relationship, often called the "bathtub curve," indicates that hardware exhibits relatively high failure rates early in its life (these failures are often attributable to design or manufacturing defects); defects are corrected, and the failure rate drops to a steady-state level (hopefully, quite low) for some period of time. As time passes, however, the failure rate rises again as hardware components suffer from the cumulative effects of dust,

FIGURE 1.1

Failure curve for hardware

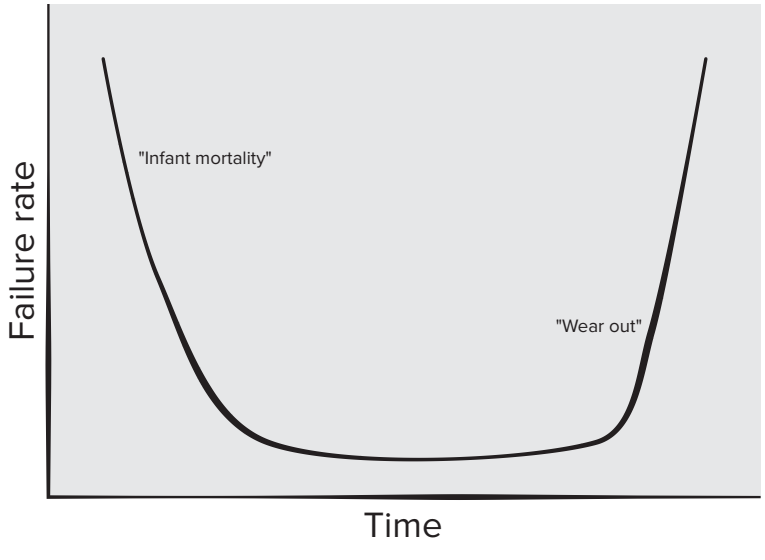
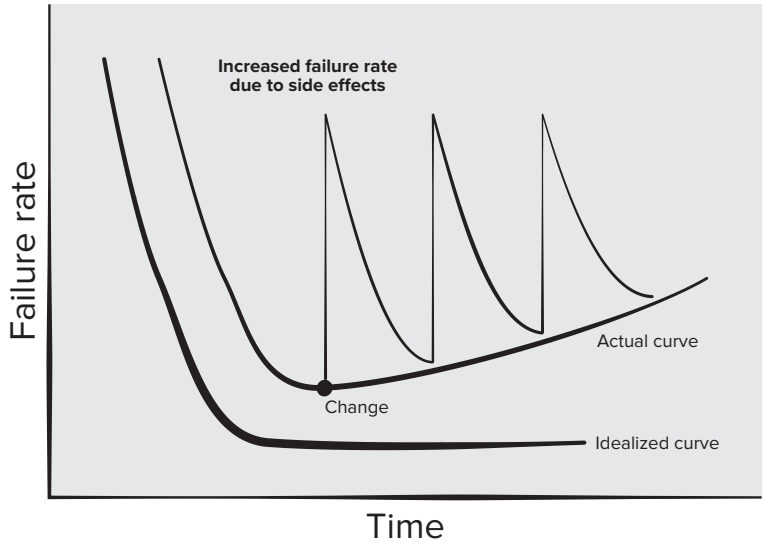


FIGURE 1.2

Failure curves for software



vibration, abuse, temperature extremes, and many other environmental maladies. Stated simply, the hardware begins to *wear out.*

Software is not susceptible to the environmental maladies that cause hardware to wear out. In theory, therefore, the failure rate curve for software should take the form of the "idealized curve" shown in Figure 1.2. Undiscovered defects will cause high failure rates early in the life of a program. However, these are corrected and the curve flattens as shown. The idealized curve is a gross oversimplification of actual failure models for software. However, the implication is clear—software doesn't wear out. But it does *deteriorate!*

This seeming contradiction can best be explained by considering the actual curve in Figure 1.2. During its life,[3] software will undergo change. As changes are made, it is likely that errors will be introduced, causing the failure rate curve to spike as shown in the "actual curve" (Figure 1.2). Before the curve can return to the original steady-state failure rate, another change is requested, causing the curve to spike again. Slowly, the minimum failure rate level begins to rise—the software is deteriorating due to change.

Another aspect of wear illustrates the difference between hardware and software. When a hardware component wears out, it is replaced by a spare part. There are no software spare parts. Every software failure indicates an error in design or in the process through which design was translated into machine executable code. Therefore, the software maintenance tasks that accommodate requests for change involve considerably more complexity than hardware maintenance.

3 In fact, from the moment that development begins and long before the first version is delivered, changes may be requested by a variety of different stakeholders.

1.1.2 Software Application Domains

Today, seven broad categories of computer software present continuing challenges for software engineers:

System software. A collection of programs written to service other programs. Some system software (e.g., compilers, editors, and file management utilities) processes complex, but determinate,[4] information structures. Other systems applications (e.g., operating system components, drivers, networking software, telecommunications processors) process largely indeterminate data.

Application software. Stand-alone programs that solve a specific business need. Applications in this area process business or technical data in a way that facilitates business operations or management/technical decision making.

Engineering/scientific software. A broad array of "number-crunching" or data science programs that range from astronomy to volcanology, from automotive stress analysis to orbital dynamics, from computer-aided design to consumer spending habits, and from genetic analysis to meteorology.

Embedded software. Resides within a product or system and is used to implement and control features and functions for the end user and for the system itself. Embedded software can perform limited and esoteric functions (e.g., key pad control for a microwave oven) or provide significant function and control capability (e.g., digital functions in an automobile such as fuel control, dashboard displays, and braking systems).

Product-line software. Composed of reusable components and designed to provide specific capabilities for use by many different customers. It may focus on a limited and esoteric marketplace (e.g., inventory control products) or attempt to address the mass consumer market.

Web/mobile applications. This network-centric software category spans a wide array of applications and encompasses browser-based apps, cloud computing, service-based computing, and software that resides on mobile devices.

Artificial intelligence software. Makes use of heuristics[5] to solve complex problems that are not amenable to regular computation or straightforward analysis. Applications within this area include robotics, decision-making systems, pattern recognition (image and voice), machine learning, theorem proving, and game playing.

Millions of software engineers worldwide are hard at work on software projects in one or more of these categories. In some cases, new systems are being built, but in many others, existing applications are being corrected, adapted, and enhanced. It is not uncommon for a young software engineer to work on a program that is older than she is! Past generations of software people have left a legacy in each of the categories we have discussed. Hopefully, the legacy to be left behind by this generation will ease the burden on future software engineers.

---

4 Software is *determinate* if the order and timing of inputs, processing, and outputs is predictable. Software is *indeterminate* if the order and timing of inputs, processing, and outputs cannot be predicted in advance.

5 The use of heuristics is an approach to problem solving that employs a practical method or "rule of thumb" not guaranteed to be perfect, but sufficient for the task at hand.

1.1.3 Legacy Software

Hundreds of thousands of computer programs fall into one of the seven broad application domains discussed in the preceding subsection. Some of these are state-of-the-art software. But other programs are older, in some cases *much* older.

These older programs—often referred to as *legacy software*—have been the focus of continuous attention and concern since the 1960s. Dayani-Fard and his colleagues [Day99] describe legacy software in the following way:

> Legacy software systems . . . were developed decades ago and have been continually modified to meet changes in business requirements and computing platforms. The proliferation of such systems is causing headaches for large organizations who find them costly to maintain and risky to evolve.

These changes may create an additional side effect that is often present in legacy software—*poor quality*.[6] Legacy systems sometimes have inextensible designs, convoluted code, poor or nonexistent documentation, test cases and results that were never archived, and a poorly managed change history. The list can be quite long. And yet, these systems often support "core functions and are indispensable to the business." What to do?

The only reasonable answer may be: *Do nothing,* at least until the legacy system must undergo some significant change. If the legacy software meets the needs of its users and runs reliably, it isn't broken and does not need to be fixed. However, as time passes, legacy systems often evolve for one or more of the following reasons:

- The software must be adapted to meet the needs of new computing environments or technology.
- The software must be enhanced to implement new business requirements.
- The software must be extended to make it work with other more modern systems or databases.
- The software must be re-architected to make it viable within an evolving computing environment.

When these modes of evolution occur, a legacy system must be reengineered so that it remains viable in the future. The goal of modern software engineering is to "devise methodologies that are founded on the notion of evolution; that is, the notion that software systems change continually, new software systems can be built from the old ones, and . . . all must interact and cooperate with each other" [Day99].

1.2 Defining the Discipline

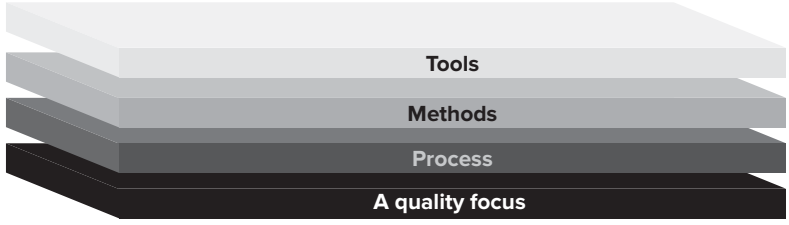
The IEEE [IEE17] has developed the following definition for software engineering:

> Software Engineering: The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.

6 In this case, quality is judged based on modern software engineering thinking—a somewhat unfair criterion since some modern software engineering concepts and principles may not have been well understood at the time that the legacy software was developed.

FIGURE 1.3

Software engineering layers



And yet, a "systematic, disciplined, and quantifiable" approach applied by one software team may be burdensome to another. We need discipline, but we also need adaptability and agility.

Software engineering is a layered technology. Referring to Figure 1.3, any engineering approach (including software engineering) must rest on an organizational commitment to quality. You may have heard of total quality management (TQM) or Six Sigma, and similar philosophies[7] that foster a culture of continuous process improvement. It is this culture that ultimately leads to more effective approaches to software engineering. The bedrock that supports software engineering is a quality focus.

The foundation for software engineering is the *process* layer. The software engineering process is the glue that holds the technology layers together and enables rational and timely development of computer software. Process defines a framework that must be established for effective delivery of software engineering technology. The software process forms the basis for management control of software projects and establishes the context in which technical methods are applied, work products (models, documents, data, reports, forms, etc.) are produced, milestones are established, quality is ensured, and change is properly managed.

Software engineering *methods* provide the technical how-to's for building software. Methods encompass a broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and support. Software engineering methods rely on a set of basic principles that govern each area of the technology and include modeling activities and other descriptive techniques.

Software engineering *tools* provide automated or semi-automated support for the process and the methods. When tools are integrated so that information created by one tool can be used by another, a system for the support of software development, called *computer-aided software engineering*, is established.

1.3 THE SOFTWARE PROCESS

A *process* is a collection of activities, actions, and tasks that are performed when some work product is to be created. An *activity* strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which

7 Quality management and related approaches are discussed throughout Part Three of this book.

software engineering is to be applied. An *action* (e.g., architectural design) encompasses a set of tasks that produce a major work product (e.g., an architectural model). A *task* focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.

In the context of software engineering, a process is *not* a rigid prescription for how to build computer software. Rather, it is an adaptable approach that enables the people doing the work (the software team) to pick and choose the appropriate set of work actions and tasks. The intent is always to deliver software in a timely manner and with sufficient quality to satisfy those who have sponsored its creation and those who will use it.

1.3.1 The Process Framework

A *process framework* establishes the foundation for a complete software engineering process by identifying a small number of *framework activities* that are applicable to all software projects, regardless of their size or complexity. In addition, the process framework encompasses a set of *umbrella activities* that are applicable across the entire software process. A generic process framework for software engineering encompasses five activities:

Communication. Before any technical work can commence, it is critically important to communicate and collaborate with the customer (and other stakeholders).[8] The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.

Planning. Any complicated journey can be simplified if a map exists. A software project is a complicated journey, and the planning activity creates a "map" that helps guide the team as it makes the journey. The map—called a *software project plan*—defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule.

Modeling. Whether you're a landscaper, a bridge builder, an aeronautical engineer, a carpenter, or an architect, you work with models every day. You create a "sketch" of the thing so that you'll understand the big picture—what it will look like architecturally, how the constituent parts fit together, and many other characteristics. If required, you refine the sketch into greater and greater detail in an effort to better understand the problem and how you're going to solve it. A software engineer does the same thing by creating models to better understand software requirements and the design that will achieve those requirements.

Construction. What you design must be built. This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code.

---

8 A *stakeholder* is anyone who has a stake in the successful outcome of the project—business managers, end users, software engineers, support people, and so forth. Rob Thomsett jokes that, "a stakeholder is a person holding a large and sharp stake. . . . If you don't look after your stakeholders, you know where the stake will end up."

Deployment. The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

These five generic framework activities can be used during the development of small, simple programs; the creation of Web applications; and for the engineering of large, complex computer-based systems. The details of the software process will be quite different in each case, but the framework activities remain the same.

For many software projects, framework activities are applied iteratively as a project progresses. That is, communication, planning, modeling, construction, and deployment are applied repeatedly through a number of project iterations. Each iteration produces a *software increment* that provides stakeholders with a subset of overall software features and functionality. As each increment is produced, the software becomes more and more complete.

1.3.2 Umbrella Activities

Software engineering process framework activities are complemented by a number of *umbrella activities.* In general, umbrella activities are applied throughout a software project and help a software team manage and control progress, quality, change, and risk. Typical umbrella activities include:

Software project tracking and control. Allows the software team to assess progress against the project plan and take any necessary action to maintain the schedule.

Risk management. Assesses risks that may affect the outcome of the project or the quality of the product.

Software quality assurance. Defines and conducts the activities required to ensure software quality.

Technical reviews. Assess software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity.

Measurement. Defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders' needs; can be used in conjunction with all other framework and umbrella activities.

Software configuration management. Manages the effects of change throughout the software process.

Reusability management. Defines criteria for work product reuse (including software components) and establishes mechanisms to achieve reusable components.

Work product preparation and production. Encompasses the activities required to create work products such as models, documents, logs, forms, and lists.

Each of these umbrella activities is discussed in detail later in this book.

1.3.3 Process Adaptation

Previously in this section, we noted that the software engineering process is not a rigid prescription that must be followed dogmatically by a software team. Rather, it should be agile and adaptable (to the problem, to the project, to the team, and to the organizational

culture). Therefore, a process adopted for one project might be significantly different than a process adopted for another project. Among the differences are:

- Overall flow of activities, actions, and tasks and the interdependencies among them
- Degree to which actions and tasks are defined within each framework activity
- Degree to which work products are identified and required
- Manner in which quality assurance activities are applied
- Manner in which project tracking and control activities are applied
- Overall degree of detail and rigor with which the process is described
- Degree to which the customer and other stakeholders are involved with the project
- Level of autonomy given to the software team
- Degree to which team organization and roles are prescribed

In Part One of this book, we examine software process in considerable detail.

1.4 SOFTWARE ENGINEERING PRACTICE

In Section 1.3, we introduced a generic software process model composed of a set of activities that establish a framework for software engineering practice. Generic framework activities—**communication, planning, modeling, construction,** and **deployment**—and umbrella activities establish a skeleton architecture for software engineering work. But how does the practice of software engineering fit in? In the sections that follow, you'll gain a basic understanding of the generic concepts and principles that apply to framework activities.[9]

1.4.1 The Essence of Practice

In the classic book *How to Solve It,* written before modern computers existed, George Polya [Pol45] outlined the essence of problem solving, and consequently, the essence of software engineering practice:

1. *Understand the problem* (communication and analysis).
2. *Plan a solution* (modeling and software design).
3. *Carry out the plan* (code generation).
4. *Examine the result for accuracy* (testing and quality assurance).

In the context of software engineering, these commonsense steps lead to a series of essential questions [adapted from Pol45]:

Understand the Problem. It's sometimes difficult to admit, but most of us suffer from hubris when we're presented with a problem. We listen for a few seconds and

9 You should revisit relevant sections within this chapter as we discuss specific software engineering methods and umbrella activities later in this book.

then think, *Oh yeah, I understand, let's get on with solving this thing*. Unfortunately, understanding isn't always that easy. It's worth spending a little time answering a few simple questions:

- *Who has a stake in the solution to the problem?* That is, who are the stakeholders?
- *What are the unknowns?* What data, functions, and features are required to properly solve the problem?
- *Can the problem be compartmentalized?* Is it possible to represent smaller problems that may be easier to understand?
- *Can the problem be represented graphically?* Can an analysis model be created?

Plan the Solution. Now you understand the problem (or so you think), and you can't wait to begin coding. Before you do, slow down just a bit and do a little design:

- *Have you seen similar problems before?* Are there patterns that are recognizable in a potential solution? Is there existing software that implements the data, functions, and features that are required?
- *Has a similar problem been solved?* If so, are elements of the solution reusable?
- *Can subproblems be defined?* If so, are solutions readily apparent for the subproblems?
- *Can you represent a solution in a manner that leads to effective implementation?* Can a design model be created?

Carry Out the Plan. The design you've created serves as a road map for the system you want to build. There may be unexpected detours, and it's possible that you'll discover an even better route as you go, but the "plan" will allow you to proceed without getting lost.

- *Does the solution conform to the plan?* Is source code traceable to the design model?
- *Is each component part of the solution provably correct?* Has the design and code been reviewed, or better, have correctness proofs been applied to the algorithm?

Examine the Result. You can't be sure that your solution is perfect, but you can be sure that you've designed a sufficient number of tests to uncover as many errors as possible.

- *Is it possible to test each component part of the solution?* Has a reasonable testing strategy been implemented?
- *Does the solution produce results that conform to the data, functions, and features that are required?* Has the software been validated against all stakeholder requirements?

It shouldn't surprise you that much of this approach is common sense. In fact, it's reasonable to state that a commonsense approach to software engineering will never lead you astray.

1.4.2 General Principles

The dictionary defines the word *principle* as "an important underlying law or assumption required in a system of thought." Throughout this book we'll discuss principles at many different levels of abstraction. Some focus on software engineering as a whole, others consider a specific generic framework activity (e.g., **communication**), and still others focus on software engineering actions (e.g., architectural design) or technical tasks (e.g., creating a usage scenario). Regardless of their level of focus, principles help you establish a mind-set for solid software engineering practice. They are important for that reason.

David Hooker [Hoo96] has proposed seven principles that focus on software engineering practice as a whole. They are reproduced in the following paragraphs:[10]

The First Principle: *The Reason It All Exists*

A software system exists for one reason: *to provide value to its users*. All decisions should be made with this in mind. Before specifying a system requirement, before noting a piece of system functionality, before determining the hardware platforms or development processes, ask yourself questions such as: "Does this add real value to the system?" If the answer is no, don't do it. All other principles support this one.

The Second Principle: *KISS (Keep It Simple, Stupid!)*

There are many factors to consider in any design effort. *All design should be as simple as possible, but no simpler*. This facilitates having a more easily understood and easily maintained system. This is not to say that features should be discarded in the name of simplicity. Indeed, the more elegant designs are usually the simpler ones. Simple does not mean "quick and dirty." It often takes a lot of thought and work over multiple iterations to simplify the design. The payoff is software that is more maintainable and less error-prone.

The Third Principle: *Maintain the Vision*

A clear vision is essential to the success of a software project. Without conceptual integrity, a system threatens to become a patchwork of incompatible designs, held together by the wrong kind of screws . . . Compromising the architectural vision of a software system weakens and will eventually break even the well-designed systems. Having an empowered architect who can hold the vision and enforce compliance helps ensure a very successful software project.

The Fourth Principle: *What You Produce, Others Will Consume*

Always specify, design, document, and implement knowing someone else will have to understand what you are doing. The audience for any product of software development is potentially large. Specify with an eye to the users. Design, keeping the implementers in mind. Code with concern for those that must maintain and extend the system. Someone may have to debug the code you write, and that makes them a user of your code. Making their job easier adds value to the system.

---

10 Reproduced with permission of the author [Hoo96]. Hooker defines patterns for these principles at http://c2.com/cgi/wiki?SevenPrinciplesOfSoftwareDevelopment.

The Fifth Principle: *Be Open to the Future*

In today's computing environments, where specifications change on a moment's notice and hardware platforms are obsolete just a few months old, software lifetimes are typically measured in months instead of years. However, true "industrial-strength" software systems must endure far longer. To do this, systems must be ready to adapt to these and other changes. Systems that do this successfully have been designed this way from the start. *Never design yourself into a corner.* Always ask "what if," and prepare for all possible answers by creating systems that solve the general problem, not just the specific one.[11]

The Sixth Principle: *Plan Ahead for Reuse*

Reuse saves time and effort.[12] Achieving a high level of reuse is arguably the hardest goal to accomplish in developing a software system. The reuse of code and designs has been proclaimed as a major benefit of using object-oriented technologies. However, the return on this investment is not automatic. *Planning ahead for reuse reduces the cost and increases the value of both the reusable components and the systems into which they are incorporated.*

The Seventh Principle: *Think!*

This last principle is probably the most overlooked. *Placing clear, complete thought before action almost always produces better results.* When you think about something, you are more likely to do it right. You also gain knowledge about how to do it right again. If you do think about something and still do it wrong, it becomes a valuable experience. A side effect of thinking is learning to recognize when you don't know something, at which point you can research the answer. When clear thought has gone into a system, value comes out. Applying the first six principles requires intense thought, for which the potential rewards are enormous.

If every software engineer and every software team simply followed Hooker's seven principles, many of the difficulties we experience in building complex computer-based systems would be eliminated.

1.5 How It All Starts

Every software project is precipitated by some business need—the need to correct a defect in an existing application; the need to adapt a "legacy system" to a changing business environment; the need to extend the functions and features of an existing application; or the need to create a new product, service, or system.

11 This advice can be dangerous if it is taken to extremes. Designing for the "general problem" sometimes requires performance compromises and can make specific solutions inefficient.

12 Although this is true for those who reuse the software on future projects, reuse can be expensive for those who must design and build reusable components. Studies indicate that designing and building reusable components can cost between 25 to 200 percent more than building targeted software. In some cases, the cost differential cannot be justified.

At the beginning of a software project, the business need is often expressed informally as part of a simple conversation. The conversation presented in the sidebar is typical.

SAFEHOME[13]



How a Project Starts

The scene: Meeting room at CPI Corporation, a (fictional) company that makes consumer products for home and commercial use.

The players: Mal Golden, senior manager, product development; Lisa Perez, marketing manager; Lee Warren, engineering manager; Joe Camalleri, executive vice president, business development

The conversation:

Joe: Okay, Lee, what's this I hear about your folks developing a what? A generic universal wireless box?

Lee: It's pretty cool . . . about the size of a small matchbook . . . we can attach it to sensors of all kinds, a digital camera, just about anything. Using the 802.11n wireless protocol. It allows us to access the device's output without wires. We think it'll lead to a whole new generation of products.

Joe: You agree, Mal?

Mal: I do. In fact, with sales as flat as they've been this year, we need something new. Lisa and I have been doing a little market research, and we think we've got a line of products that could be big.

Joe: How big . . . bottom line big?

Mal (avoiding a direct commitment): Tell him about our idea, Lisa.

Lisa: It's a whole new generation of what we call "home management products." We call 'em *SafeHome*. They use the new wireless interface, provide homeowners or small-businesspeople with a system that's controlled by their PC—home security, home surveillance, appliance and device control—you know, turn down the home air conditioner while you're driving home, that sort of thing.

Lee (jumping in): Engineering's done a technical feasibility study of this idea, Joe. It's doable at low manufacturing cost. Most hardware is off the shelf. Software is an issue, but it's nothing that we can't do.

Joe: Interesting. Now, I asked about the bottom line.

Mal: PCs and tablets have penetrated over 70 percent of all households in the USA. If we could price this thing right, it could be a killer app. Nobody else has our wireless box . . . it's proprietary. We'll have a 2-year jump on the competition. Revenue? Maybe as much as $30 to $40 million in the second year.

Joe (smiling): Let's take this to the next level. I'm interested.

With the exception of a passing reference, software was hardly mentioned as part of the conversation. And yet, software will make or break the *SafeHome* product line. The engineering effort will succeed only if *SafeHome* software succeeds. The market will accept the product only if the software embedded within it properly meets the customer's (as yet unstated) needs. We'll follow the progression of *SafeHome* software engineering in many of the chapters that follow.

13 *SafeHome* will be used throughout this book to illustrate the inner workings of project teams as they build a software product. The company, the project, and the people are purely fictitious, but the situations and problems are real.

1.6 SUMMARY

Software is the key element in the evolution of computer-based systems and products and one of the most important technologies on the world stage. Over the past 60 years, software has evolved from a specialized problem-solving and information analysis tool to an industry in itself. Yet we still have trouble developing high-quality software on time and within budget.

Software—programs, data, and descriptive information—addresses a wide array of technology and application areas. Legacy software continues to present special challenges to those who must maintain it.

Software engineering encompasses process, methods, and tools that enable complex computer-based systems to be built in a timely manner with quality. The software process incorporates five framework activities—communication, planning, modeling, construction, and deployment—that are applicable to all software projects. Software engineering practice is a problem-solving activity that follows a set of core principles. As you learn more about software engineering, you'll begin to understand why these principles should be considered when beginning any software project.

PROBLEMS AND POINTS TO PONDER

1.1. Provide at least five additional examples of how the law of unintended consequences applies to computer software.

1.2. Provide a number of examples (both positive and negative) that indicate the impact of software on our society.

1.3. Develop your own answers to the five questions asked at the beginning of Section 1.1. Discuss them with your fellow students.

1.4. Many modern applications change frequently—before they are presented to the end user and then after the first version has been put into use. Suggest a few ways to build software to stop deterioration due to change.

1.5. Consider the seven software categories presented in Section 1.1.2. Do you think that the same approach to software engineering can be applied for each? Explain your answer.

1.6. As software becomes more pervasive, risks to the public (due to faulty programs) become an increasingly significant concern. Develop a doomsday but realistic scenario in which the failure of a computer program could do great harm, either economic or human.

1.7. Describe a process framework in your own words. When we say that framework activities are applicable to all projects, does this mean that the same work tasks are applied for all projects, regardless of size and complexity? Explain.

1.8. Umbrella activities occur throughout the software process. Do you think they are applied evenly across the process, or are some concentrated in one or more framework activities?

Design element: Quick Look icon magnifying glass: © Roger Pressman

PART

One

THE SOFTWARE PROCESS

In this part of *Software Engineering: A Practitioner's Approach,* you'll learn about the process that provides a framework for software engineering practice. These questions are addressed in the chapters that follow:

- What is a software process?
- What are the generic framework activities that are present in every software process?
- How are processes modeled, and what are process patterns?
- What are the prescriptive process models, and what are their strengths and weaknesses?
- Why is *agility* a watchword in modern software engineering work?
- What is agile software development, and how does it differ from more traditional process models?

Once these questions are answered, you'll be better prepared to understand the context in which software engineering practice is applied.

CHAPTER

2 PROCESS MODELS

Building computer software is an iterative social learning process, and the outcome, something that Baetjer [Bae98] would call "software capital," is an embodiment of knowledge collected, distilled, and organized as the process is conducted.

But what exactly is a software process from a technical point of view? Within the context of this book, we define a *software process* as a framework for the activities, actions, and tasks required to build high-quality software. Is "process" synonymous with "software engineering"? The answer is yes and no. A software process defines the approach that is taken as software is engineered. But software engineering also encompasses technologies that populate the process—technical methods and automated tools.

More important, software engineering is performed by creative, knowledgeable people who should adapt a mature software process so that it is appropriate for the products that they build and the demands of their marketplace.

KEY CONCEPTS

evolutionary process model 29
generic process model 21
process assessment 24
process flow 24
process improvement 24
process patterns 24
prototyping 26
spiral model 29
task set 21
Unified Process 31
waterfall model 25



QUICK LOOK

What is it? When you work to build a product or system, it's important to follow a series of predictable steps (a road map) that helps you deliver a high-quality product on time. This road map is called a "software process."

Who does it? Software engineers adapt a process to their needs and then follow it. The people who have requested the software also have a role to play in the process of defining, building, and testing it.

Why is it important? A process provides stability, control, and organization to an activity so that it does not become chaotic. However, a modern software engineering process must be "agile." It must include only those activities, controls, and work products that are appropriate for the project team and the product that is to be produced.

What are the steps? The process that you adopt depends on the software that you're building. A process might be appropriate for creating software for an aircraft avionics system but may not work well for the creation of a mobile app or video game.

What is the work product? The work products are the programs, documents, and data produced by the engineering activities and tasks included in the process.

How do I ensure that I've done it right? The quality, timeliness, and long-term viability of the product built are the best indicators of the success of the process used.

2.1 A Generic Process Model

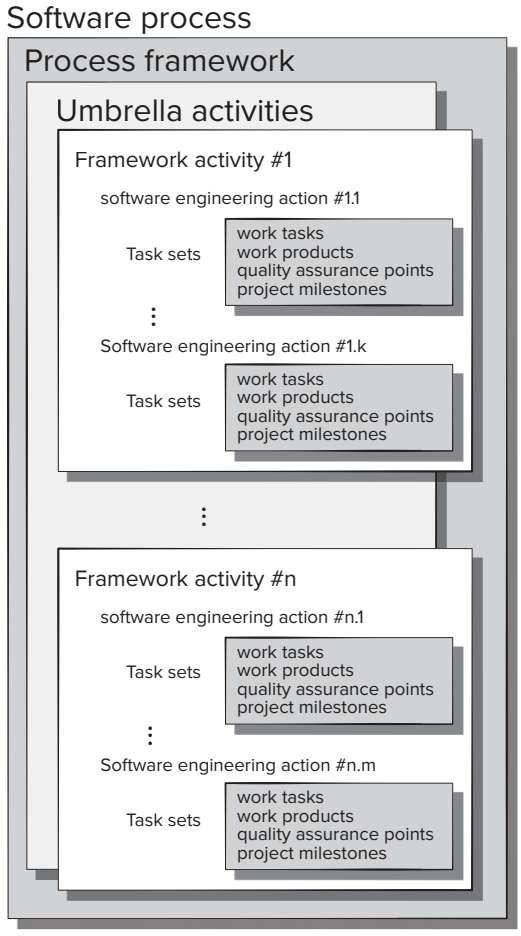
In Chapter 1, a process was defined as a collection of activities, actions, and tasks that are performed when some work product is to be created. Each of these activities, actions, and tasks resides within a framework or model that defines their relationship with the process and with one another.

The software process is represented schematically in Figure 2.1. Referring to the figure, each framework activity is populated by a set of software engineering actions. Each software engineering action is defined by a *task set* that identifies the work tasks that are to be completed, the work products that will be produced, the quality assurance points that will be required, and the milestones that will be used to indicate progress.

As we discussed in Chapter 1, a generic process framework for software engineering defines five framework activities—**communication, planning, modeling, construction,** and **deployment.** In addition, a set of umbrella activities—project tracking and control,

FIGURE 2.1

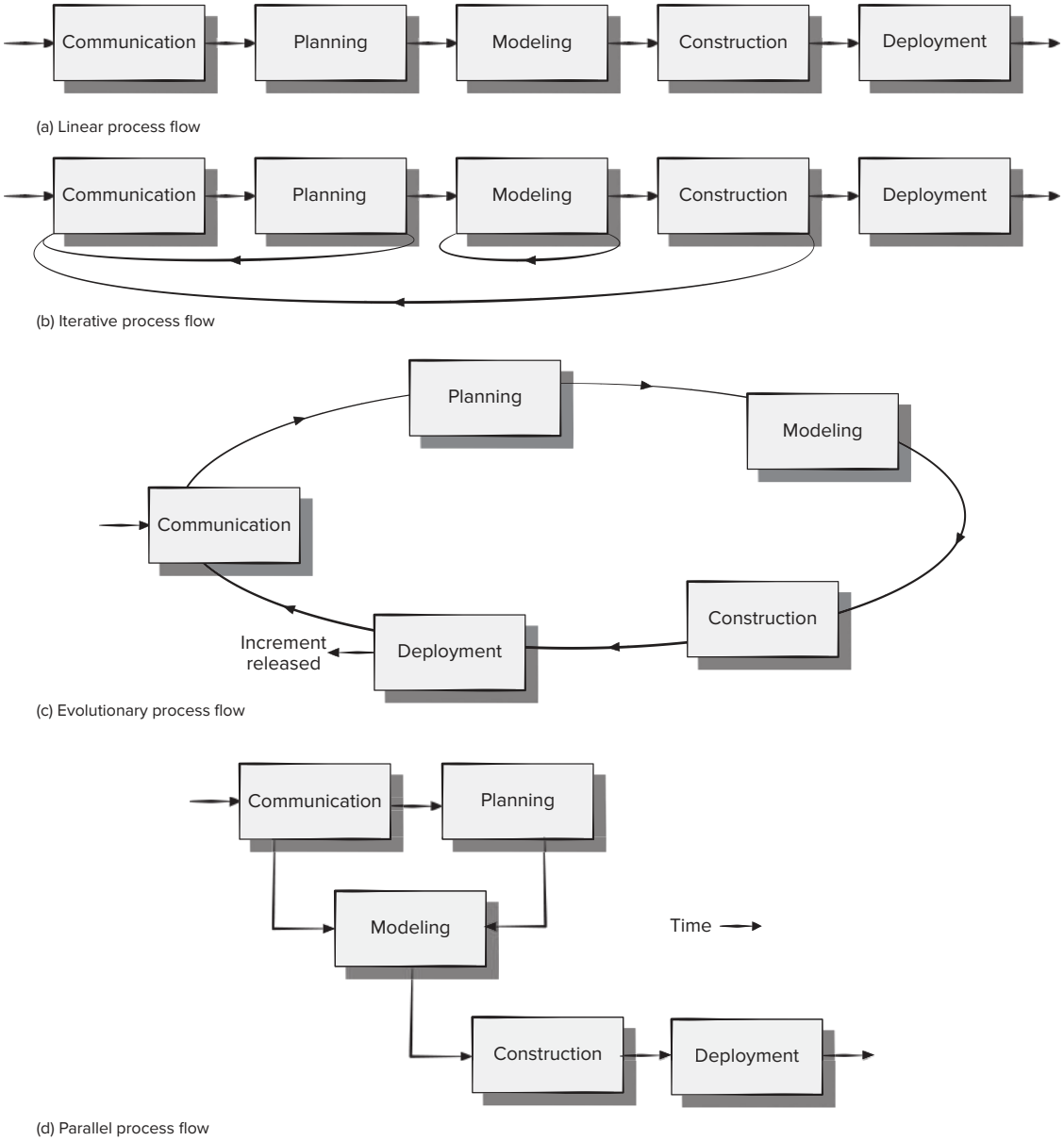
A software process framework



risk management, quality assurance, configuration management, technical reviews, and others—are applied throughout the process.

You should note that one important aspect of the software process has not been discussed yet. This aspect—called *process flow*—describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time. It is illustrated in Figure 2.2.

FIGURE 2.2 **Process flow**



A *linear process flow* executes each of the five framework activities in sequence, beginning with communication and culminating with deployment (Figure 2.2a). An *iterative process flow* repeats one or more of the activities before proceeding to the next (Figure 2.2b). An *evolutionary process flow* executes the activities in a "circular" manner. Each circuit through the five activities leads to a more complete version of the software (Figure 2.2c). A *parallel process flow* (Figure 2.2d) executes one or more activities in parallel with other activities (e.g., modeling for one aspect of the software might be executed in parallel with construction of another aspect of the software).

2.2 Defining a Framework Activity

Although we have described five framework activities and provided a basic definition of each in Chapter 1, a software team would need significantly more information before it could properly execute any one of these activities as part of the software process. Therefore, you are faced with a key question: *What actions are appropriate for a framework activity, given the nature of the problem to be solved, the characteristics of the people doing the work, and the stakeholders who are sponsoring the project?*

For a small software project requested by one person (at a remote location) with simple, straightforward requirements, the **communication** activity might encompass little more than a phone call or e-mail with the appropriate stakeholder. Therefore, the only necessary action is *phone conversation,* and the work tasks (the *task set*) that this action encompasses are:

1. Make contact with stakeholder via telephone.
2. Discuss requirements and develop notes.
3. Organize notes into a brief written statement of requirements.
4. E-mail to stakeholder for review and approval.

If the project was considerably more complex with lots of stakeholders, each with a different set of (sometimes conflicting) requirements, the communication activity might have six distinct actions: *inception, elicitation, elaboration, negotiation, specification,* and *validation.* Each of these software engineering actions might have many work tasks and in some cases a number of distinct work products.

2.3 Identifying a Task Set

Referring again to Figure 2.1, each software engineering action (e.g., *elicitation,* an action associated with the **communication** activity) can be represented by a number of different *task sets*—each a collection of software engineering work tasks, related work products, quality assurance points, and project milestones. Different projects demand different task sets. You should choose a task set that best accommodates the

TASK SET



A task set defines the actual work that needs to be done to accomplish the objectives of a software engineering action. For example, *elicitation* (more commonly called "requirements gathering") is an important software engineering action that occurs during the **communication** activity. The goal of requirements gathering is to understand what various stakeholders want from the software that is to be built.

For a small, relatively simple project, the task set for requirements gathering might look like this:

1. Make a list of stakeholders for the project.
2. Invite all stakeholders to an informal meeting.
3. Ask each stakeholder to make a list of features and functions required.
4. Discuss requirements and build a final list.
5. Prioritize requirements.
6. Note areas of uncertainty.

For a larger, more complex software project, a different task set would be required. It might encompass the following work tasks:

1. Make a list of stakeholders for the project.
2. Interview each stakeholder separately to determine overall wants and needs.
3. Build a preliminary list of functions and features based on stakeholder input.
4. Schedule a series of facilitated application specification meetings.
5. Conduct meetings.
6. Produce informal user scenarios as part of each meeting.
7. Refine user scenarios based on stakeholder feedback.
8. Build a revised list of stakeholder requirements.
9. Use quality function deployment techniques to prioritize requirements.
10. Package requirements so that they can be delivered incrementally.
11. Note constraints and restrictions that will be placed on the system.
12. Discuss methods for validating the system.

Both of these task sets achieve "requirements gathering," but they are quite different in their depth and level of formality. The software team chooses the task set that allows it to achieve the goal for each action and still maintain quality and agility.

needs of the project and the characteristics of your team. This implies that a software engineering action should be adapted to the specific needs of the software project and the characteristics of the project team.

2.4 PROCESS ASSESSMENT AND IMPROVEMENT

The existence of a software process is no guarantee that software will be delivered on time, that it will meet the customer's needs, or that it will exhibit the technical characteristics that will lead to long-term quality characteristics (Chapter 15). Process patterns must be coupled with solid software engineering practice (Part Two of this book). In addition, the process itself can be assessed to ensure that it meets a set of basic process criteria that have been shown to be essential for successful software engineering.[1]

The current thinking among most engineers is that software processes and activities should be assessed using numeric measures or software analytics (metrics). The

1 The SEI's CMMI-DEV [CMM07] describes the characteristics of a software process and the criteria for a successful process in voluminous detail.

progress you have made in a journey toward an effective software process will define the degree to which you can measure improvement in a meaningful way. The use of software process metrics to assess process quality is introduced in Chapter 17. A more detailed discussion of process assessment and improvement methods is presented in Chapter 28.

2.5 PRESCRIPTIVE PROCESS MODELS

Prescriptive process models define a predefined set of process elements and a predictable process work flow. Prescriptive process models[2] strive for structure and order in software development. Activities and tasks occur sequentially with defined guidelines for progress. But are prescriptive models appropriate for a software world that thrives on change? If we reject traditional process models (and the order they imply) and replace them with something less structured, do we make it impossible to achieve coordination and coherence in software work?

There are no easy answers to these questions, but there are alternatives available to software engineers. In the sections that follow, we provide an overview of the prescriptive process approach in which order and project consistency are dominant issues. We call them "prescriptive" because they prescribe a set of process elements—framework activities, software engineering actions, tasks, work products, quality assurance, and change control mechanisms for each project. Each process model also prescribes a process flow (also called a *work flow*)—that is, the manner in which the process elements are interrelated to one another.

All software process models can accommodate the generic framework activities described in Chapter 1, but each applies a different emphasis to these activities and defines a process flow that invokes each framework activity (as well as software engineering actions and tasks) in a different manner. In Chapters 3 and 4 we will discuss software engineering practices that strive to accommodate the changes that are inevitable during the development of many software projects.

2.5.1 The Waterfall Model

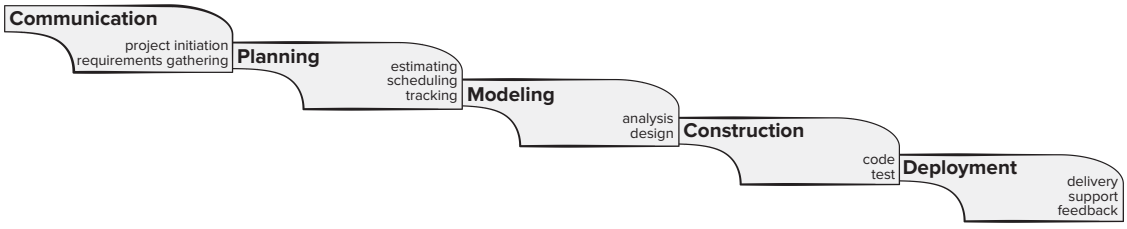
There are times when the requirements for a problem are well understood—when work flows from communication through deployment in a reasonably linear fashion. This situation is sometimes encountered when well-defined adaptations or enhancements to an existing system must be made (e.g., an adaptation to accounting software because it needs to accommodate changes to mandated government regulations). It may also occur in a limited number of new development efforts, but only when requirements are well defined and reasonably stable.

The *waterfall model,* sometimes called the *linear sequential model*, suggests a systematic, sequential approach[3] to software development that begins with customer

2 Prescriptive process models are sometimes referred to as "traditional" process models.

3 Although the original waterfall model proposed by Winston Royce [Roy70] made provision for "feedback loops," the vast majority of organizations that apply this process model treat it as if it were strictly linear.

FIGURE 2.3 The waterfall model



specification of requirements and progresses through planning, modeling, construction, and deployment, culminating in ongoing support of the completed software (Figure 2.3).

The waterfall model is the oldest paradigm for software engineering. However, over the past five decades, criticism of this process model has caused even ardent supporters to question its efficacy. Among the problems that are sometimes encountered when the waterfall model is applied are:

1. Real projects rarely follow the sequential work flow that the model proposes.
2. It is often difficult for the customer to state all requirements explicitly at the beginning of most projects.
3. The customer must have patience because a working version of the program(s) will not be available until late in the project time span.
4. Major blunders may not be detected until the working program is reviewed.

Today, software work is fast paced and subject to a never-ending stream of changes (to features, functions, and information content). The waterfall model is often inappropriate for such work.

2.5.2 Prototyping Process Model

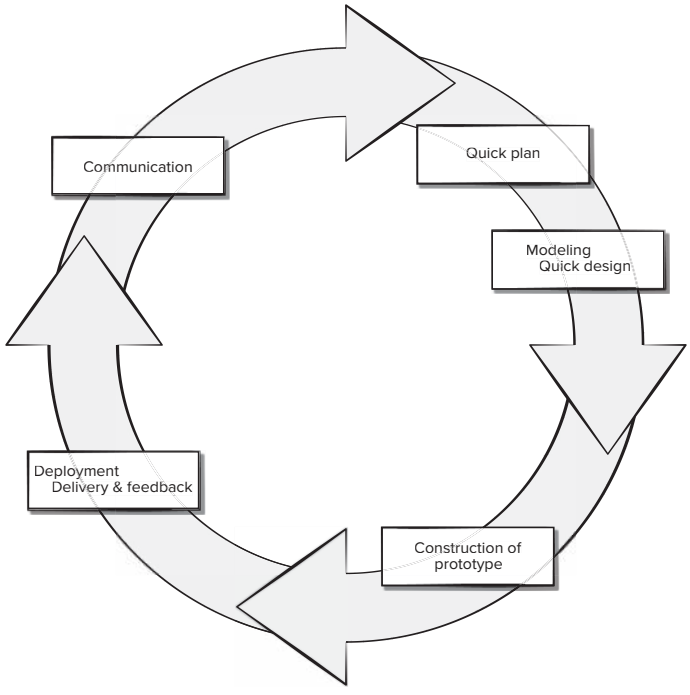
Often, a customer defines a set of general objectives for software but does not identify detailed requirements for functions and features. In other cases, the developer may be unsure of the efficiency of an algorithm, the adaptability of an operating system, or the form that human-machine interaction should take. In these, and many other situations, a *prototyping paradigm* may offer the best approach.

Although prototyping can be used as a stand-alone process model, it is more commonly used as a technique that can be implemented within the context of any one of the process models noted in this chapter. Regardless of the manner in which it is applied, the prototyping paradigm assists you and other stakeholders to better understand what is to be built when requirements are fuzzy.

For example, a fitness app developed using incremental prototypes might deliver the basic user interface screens needed to sync a mobile phone with the fitness device

FIGURE 2.4

The prototyping paradigm



and display the current data; the ability to set goals and store the fitness device data on the cloud might be included in the second prototype, creating and modifying the user interface screens based on customers feedback; and a third prototype might include social media integration to allow users to set fitness goals and share progress toward them with a set of friends.

The prototyping paradigm (Figure 2.4) begins with communication. You meet with other stakeholders to define the overall objectives for the software, identify whatever requirements are known, and outline areas where further definition is mandatory. A prototyping iteration is planned quickly, and modeling (in the form of a "quick design") occurs. A quick design focuses on a representation of those aspects of the software that will be visible to end users (e.g., human interface layout or output display formats). The quick design leads to the construction of a prototype. The prototype is deployed and evaluated by stakeholders, who provide feedback that is used to further refine requirements. Iteration occurs as the prototype is tuned to satisfy the needs of various stakeholders, while at the same time enabling you to better understand what needs to be done.

Ideally, the prototype serves as a mechanism for identifying software requirements. If a working prototype is to be built, you can make use of existing program fragments or apply tools that enable working programs to be generated quickly.

Both stakeholders and software engineers like the prototyping paradigm. Users get a feel for the actual system, and developers get to build something immediately. Yet, prototyping can be problematic for the following reasons:

1. Stakeholders see what appears to be a working version of the software. They may be unaware that the prototype architecture (program structure) is also evolving. This means that the developers may not have considered the overall software quality or long-term maintainability.
2. As a software engineer, you may be tempted to make implementation compromises to get a prototype working quickly. If you are not careful, these less-than-ideal choices have now become an integral part of the evolving system.

SafeHome



Selecting a Process Model, Part 1

The scene: Meeting room for the software engineering group at CPI Corporation, a (fictional) company that makes consumer products for home and commercial use.

The players: Lee Warren, engineering manager; Doug Miller, software engineering manager; Jamie Lazar, software team member; Vinod Raman, software team member; and Ed Robbins, software team member.

The conversation:

Lee: So let's recapitulate. I've spent some time discussing the *SafeHome* product line as we see it at the moment. No doubt, we've got a lot of work to do to simply define the thing, but I'd like you guys to begin thinking about how you're going to approach the software part of this project.

Doug: Seems like we've been pretty disorganized in our approach to software in the past.

Ed: I don't know, Doug, we always got product out the door.

Doug: True, but not without a lot of grief, and this project looks like it's bigger and more complex than anything we've done in the past.

Jamie: Doesn't look that hard, but I agree . . . our ad hoc approach to past projects won't work here, particularly if we have a very tight time line.

Doug (smiling): I want to be a bit more professional in our approach. I went to a short course last week and learned a lot about software engineering . . . good stuff. We need a process here.

Jamie (with a frown): My job is to build computer programs, not push paper around.

Doug: Give it a chance before you go negative on me. Here's what I mean. (Doug proceeds to describe the process framework described in Chapter 1 and the prescriptive process models presented to this point.)

Doug: So anyway, it seems to me that a linear model is not for us . . . assumes we have all requirements up front and, knowing this place, that's not likely.

Vinod: Yeah, and it sounds way too IT-oriented . . . probably good for building an inventory control system or something, but it's just not right for *SafeHome*.

Doug: I agree.

Ed: That prototyping approach seems okay. A lot like what we do here anyway.

Vinod: That's a problem. I'm worried that it doesn't provide us with enough structure.

Doug: Not to worry. We've got plenty of other options, and I want you guys to pick what's best for the team and best for the project.

Although problems can occur, prototyping can be an effective paradigm for software engineering. The key is to define the rules of the game at the beginning; that is, all stakeholders should agree that the prototype is built in part to serve as a mechanism for defining requirements. It is often desirable to design a prototype so it can be evolved into the final product. The reality is developers may need to discard (at least in part) a prototype to better meet the customer's evolving needs.

2.5.3 Evolutionary Process Model

Software, like all complex systems, evolves over time. Business and product requirements often change as development proceeds, making a straight-line path to an end product unrealistic. Tight market deadlines may make completion of a comprehensive software product impossible. It might be possible to create a limited version of a product to meet competitive or business pressure and release a refined version once all system features are better understood. In a situation like this you need a process model that has been explicitly designed to accommodate a product that grows and changes.

Originally proposed by Barry Boehm [Boe88], the *spiral model* is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model. It provides the potential for rapid development of increasingly more complete versions of the software.

Using the spiral model, software is developed in a series of evolutionary releases. During early iterations, the release might be a model or prototype. During later iterations, increasingly more complete versions of the engineered system are produced.

A spiral model is divided into a set of framework activities defined by the software engineering team. For illustrative purposes, we use the generic framework activities discussed earlier.[4] Each of the framework activities represent one segment of the spiral path illustrated in Figure 2.5. As this evolutionary process begins, the software team performs activities that are implied by a circuit around the spiral in a clockwise direction, beginning at the center. Risk (Chapter 26) is considered as each revolution is made. *Anchor point milestones*—a combination of work products and conditions that are attained along the path of the spiral—are noted for each evolutionary pass.

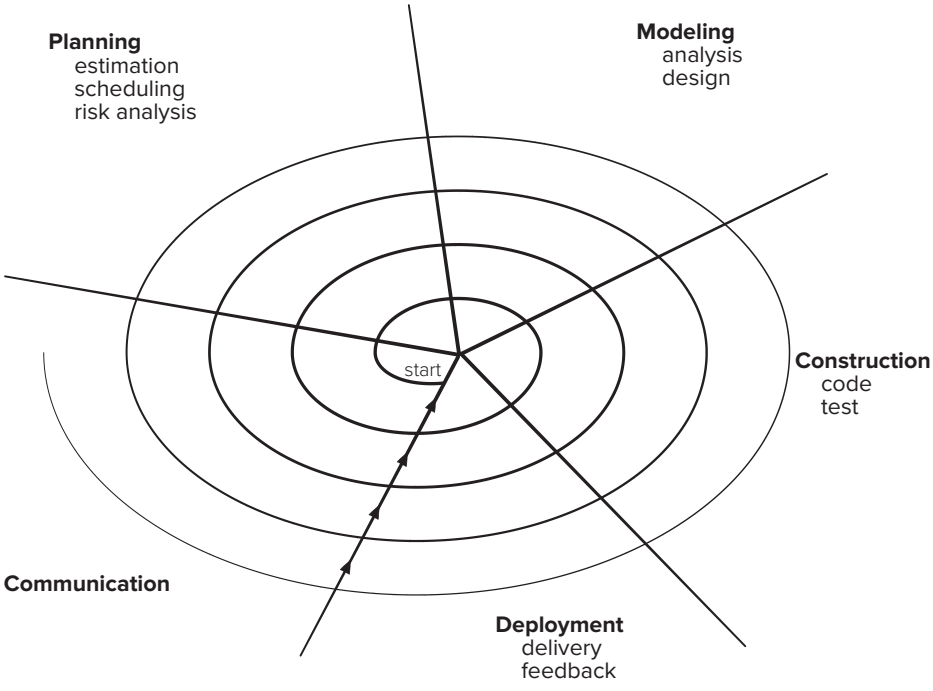
The first circuit around the spiral (beginning at the inside streamline nearest the center, as shown in Figure 2.5) might result in the development of a product specification; subsequent passes around the spiral might be used to develop a prototype and then progressively more sophisticated versions of the software. Each pass through the planning region results in adjustments to the project plan. Cost and schedule are adjusted based on feedback derived from the customer after delivery. In addition, the project manager adjusts the planned number of iterations required to complete the software.

Unlike other process models that end when software is delivered, the spiral model can be adapted to apply throughout the life of the computer software. The spiral model

4 The spiral model discussed in this section is a variation on the model proposed by Boehm. For further information on the original spiral model, see [Boe88]. More recent discussion of Boehm's spiral model can be found in [Boe98] and [Boe01a].

FIGURE 2.5

A typical spiral model



is a realistic approach to the development of large-scale systems and software. It uses prototyping as a risk reduction mechanism. The spiral model demands a direct consideration of technical risks at all stages of the project and, if properly applied, should reduce risks before they become problematic.

But like other paradigms, the spiral model is not a panacea. It may be difficult to convince customers (particularly in contract situations) that the evolutionary approach is controllable. It demands considerable risk assessment expertise and relies on this expertise for success. If a major risk is not uncovered and managed, problems will undoubtedly occur.

We have already noted that modern computer software is characterized by continual change, by very tight time lines, and by an emphatic need for customer-user satisfaction. In many cases, time to market is the most important management requirement. If a market window is missed, the software project itself may be meaningless.[5]

The intent of evolutionary models is to develop high-quality software[6] in an iterative or incremental manner. However, it is possible to use an evolutionary process to

5 It is important to note, however, that being the first to reach a market is no guarantee of success. In fact, many very successful software products have been second or even third to reach the market (learning from the mistakes of their predecessors).

6 In this context, software quality is defined quite broadly to encompass not only customer satisfaction, but also a variety of technical criteria discussed in Part Two of this book.

SAFEHOME



Selecting a Process Model, Part 2

The scene: Meeting room for the software engineering group at CPI Corporation, a company that makes consumer products for home and commercial use.

The players: Lee Warren, engineering manager; Doug Miller, software engineering manager; Vinod and Jamie, members of the software engineering team.

The conversation: (Doug describes evolutionary process options.)

Jamie: Now I see something I like. An incremental approach makes sense, and I really like the flow of that spiral model thing. That's keepin' it real.

Vinod: I agree. We deliver an increment, learn from customer feedback, re-plan, and then deliver another increment. It also fits into the nature of the product. We can have something on the market fast and then add functionality with each version, er, increment.

Lee: Wait a minute. Did you say that we regenerate the plan with each tour around the spiral, Doug? That's not so great; we need one plan, one schedule, and we've got to stick to it.

Doug: That's old-school thinking, Lee. Like the guys said, we've got to keep it real. I submit that it's better to tweak the plan as we learn more and as changes are requested. It's way more realistic. What's the point of a plan if it doesn't reflect reality?

Lee (frowning): I suppose so, but . . . senior management's not going to like this . . . they want a fixed plan.

Doug (smiling): Then you'll have to reeducate them, buddy.

emphasize flexibility, extensibility, and speed of development. The challenge for software teams and their managers is to establish a proper balance between these critical project and product parameters and customer satisfaction (the ultimate arbiter of software quality).

2.5.4 Unified Process Model

In some ways the Unified Process (UP) [Jac99] is an attempt to draw on the best features and characteristics of traditional software process models but characterize them in a way that implements many of the best principles of agile software development (Chapter 3). The Unified Process recognizes the importance of customer communication and streamlined methods for describing the customer's view of a system (the use case).[7] It emphasizes the important role of software architecture and "helps the architect focus on the right goals, such as understandability, reliance to future changes, and reuse" [Jac99]. It suggests a process flow that is iterative and incremental, providing the evolutionary feel that is essential in modern software development.

7 A *use case* (Chapter 7) is a text narrative or template that describes a system function or feature from the user's point of view. A use case is written by the user and serves as a basis for the creation of a more comprehensive analysis model.

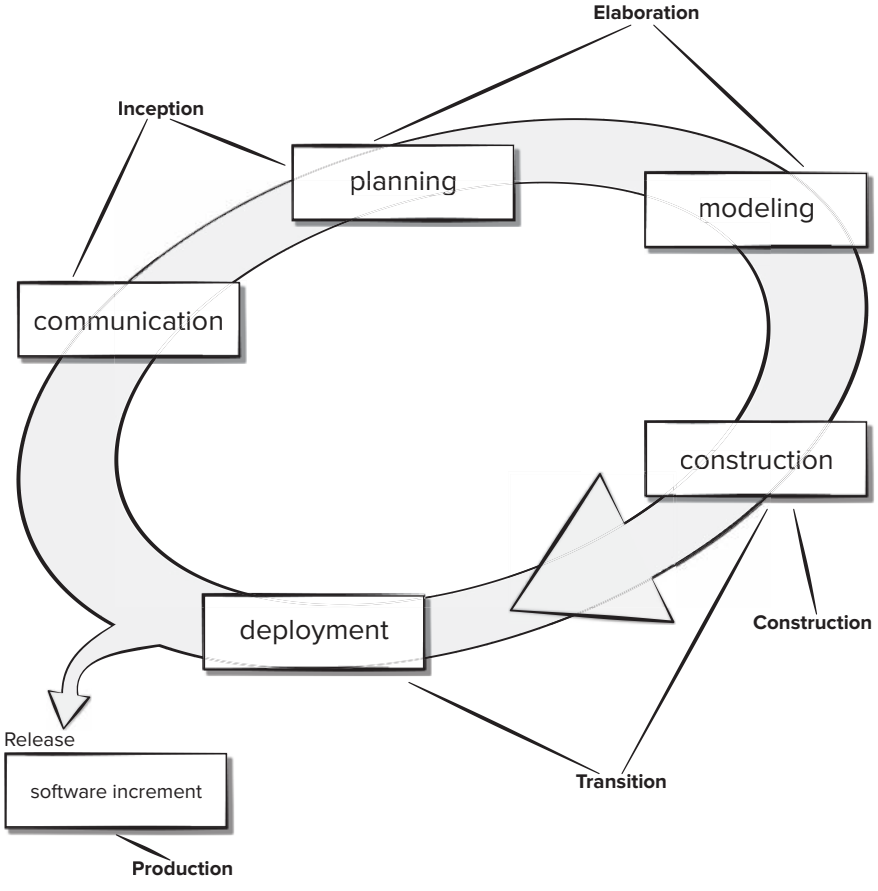
UML, the *unified modeling language,* was developed to support their work. UML contains a robust notation for the modeling and development of object-oriented systems and has became a de facto industry standard for modeling software of all types. UML is used throughout Part Two of this book to represent both requirements and design models. Appendix 1 presents an introductory tutorial and a list of recommended books for those who are unfamiliar with basic UML notation and modeling rules.

Figure 2.6 depicts the "phases" of the Unified Process and relates them to the generic activities that were discussed in Section 2.1.

The *inception phase* of the UP is where customer communication and planning takes place. Fundamental business requirements are described through a set of preliminary use cases (Chapter 7) that describe which features and functions each major class of users desires that will become realized in the software architecture. Planning identifies resources, assesses major risks, and defines a preliminary schedule for the software increments.

FIGURE 2.6

The Unified Process



The *elaboration phase* encompasses the planning and modeling activities of the generic process model (Figure 2.6). Elaboration refines and expands the preliminary use cases and creates an architectural baseline that includes five different views of the software—the use case model, the analysis model, the design model, the implementation model, and the deployment model.[8] Modifications to the plan are often made at this time.

The *construction phase* of the UP is identical to the construction activity defined for the generic software process. All necessary and required features and functions for the software increment (i.e., the release) are then implemented in source code. As components are being implemented, unit tests[9] are designed and executed for each. In addition, integration activities (component assembly and integration testing) are conducted. Use cases are used to derive a suite of acceptance tests that are executed prior to the initiation of the next UP phase.

The *transition phase* of the UP encompasses the latter stages of the generic construction activity and the first part of the generic deployment (delivery and feedback) activity. Software and supporting documentation is given to end users for beta testing, and user feedback reports both defects and necessary changes. At the conclusion of the transition phase, the software increment becomes a usable software release.

The *production phase* of the UP coincides with the deployment activity of the generic process. During this phase, the ongoing use of the software is monitored, support for the operating environment (infrastructure) is provided, and defect reports and requests for changes are submitted and evaluated.

It is likely that at the same time the construction, transition, and production phases are being conducted, work may have already begun on the next software increment. This means that the five UP phases do not occur in a sequence, but rather with staggered concurrency.

It should be noted that not every task identified for a UP workflow is conducted for every software project. The team adapts the process (actions, tasks, subtasks, and work products) to meet its needs.

2.6 PRODUCT AND PROCESS

Some of the strengths and weaknesses of the process models we have discussed are summarized in Table 2.1. In previous editions of this book we have discussed many others. The reality is that no process is perfect for every project. Usually the software team adapts one or more of the process models discussed in 2.5 or the agile process models discussed in Chapter 3 to meet their needs for the project at hand.

8 It is important to note that the architectural baseline is not a prototype in that it is not thrown away. Rather, the baseline is fleshed out during the next UP phase.

9 A comprehensive discussion of software testing (including *unit tests*) is presented in Chapters 19 through 21).

TABLE 2.1
Comparing process models

Waterfall pros	It is easy to understand and plan. It works for well-understood small projects. Analysis and testing are straightfoward.
Waterfall cons	It does not accommodate change well. Testing occurs late in the process. Customer approval is at the end.
Prototyping pros	There is a reduced impact of requirement changes. The customer is involved early and often. It works well for small projects. There is reduced likelihood of product rejection.
Prototyping cons	Customer involvement may cause delays. There may be a temptation to "ship" a prototype. Work is lost in a throwaway prototype. It is hard to plan and manage.
Spiral pros	There is continuous customer involvement. Development risks are managed. It is suitable for large, complex projects. It works well for extensible products.
Spiral cons	Risk analysis failures can doom the project. The project may be hard to manage. It requires an expert development team.
Unified Process pros	Quality documentation is emphasized. There is continuous customer involvement. It accommodates requirements changes. It works well for maintenance projects.
Unified Process cons	Use cases are not always precise. It has tricky software increment integration. Overlapping phases can cause problems. It requires an expert development team.

If the process is weak, the end product will undoubtedly suffer. But an obsessive overreliance on process is also dangerous. In a brief essay written many years ago, Margaret Davis [Dav95a] makes timeless comments on the duality of product and process:

> About every ten years give or take five, the software community redefines "the problem" by shifting its focus from product issues to process issues. . . .
>
> While the natural tendency of a pendulum is to come to rest at a point midway between two extremes, the software community's focus constantly shifts because new force is applied when the last swing fails. These swings are harmful in and of themselves because they confuse the average software practitioner by radically changing what it means to perform the job let alone perform it well. The swings also do not solve "the problem" for they are doomed to fail as long as product and process are treated as forming a dichotomy instead of a duality.
>
> . . . You can never derive or understand the full artifact, its context, use, meaning, and worth if you view it as only a process or only a product.
>
> All of human activity may be a process, but each of us derives a sense of self-worth from those activities that result in a representation or instance that can be used or

> appreciated either by more than one person, used over and over, or used in some other context not considered. That is, we derive feelings of satisfaction from reuse of our products by ourselves or others.
>
> Thus, while the rapid assimilation of reuse goals into software development potentially increases the satisfaction software practitioners derive from their work, it also increases the urgency for acceptance of the duality of product and process. . . .

People derive as much (or more) satisfaction from the creative process as they do from the end product. An artist enjoys the brush strokes as much as the framed result. A writer enjoys the search for the proper metaphor as much as the finished book. As a creative software professional, you should also derive as much satisfaction from the process as the end product. The duality of product and process is one important element in keeping creative people engaged as software engineering continues to evolve.

2.7 SUMMARY

A generic process model for software engineering encompasses a set of framework and umbrella activities, actions, and work tasks. Each of a variety of process models can be described by a different process flow—a description of how the framework activities, actions, and tasks are organized sequentially and chronologically. Process patterns can be used to solve common problems that are encountered as part of the software process.

Prescriptive process models have been applied for many years in an effort to bring order and structure to software development. Each of these models suggests a somewhat different process flow, but all perform the same set of generic framework activities: communication, planning, modeling, construction, and deployment.

Sequential process models, such as the waterfall model, are the oldest software engineering paradigms. They suggest a linear process flow that is often inconsistent with modern realities (e.g., continuous change, evolving systems, tight time lines) in the software world. They do, however, have applicability in situations where requirements are well defined and stable.

Incremental process models are iterative in nature and produce working versions of software quite rapidly. Evolutionary process models recognize the iterative, incremental nature of most software engineering projects and are designed to accommodate change. Evolutionary models, such as prototyping and the spiral model, produce incremental work products (or working versions of the software) quickly. These models can be adopted to apply across all software engineering activities—from concept development to long-term system maintenance.

The Unified Process is a "use case driven, architecture-centric, iterative and incremental" software process designed as a framework for UML methods and tools.

PROBLEMS AND POINTS TO PONDER

2.1. Baetjer [Bae98] notes: "The process provides interaction between users and designers, between users and evolving tools, and between designers and evolving tools [technology]." List five questions that (1) designers should ask users, (2) users should ask designers, (3) users should ask themselves about the software product that is to be built, and (4) designers should ask themselves about the software product that is to be built and the process that will be used to build it.

2.2. Discuss the differences among the various process flows described in Section 2.1. Identify the types of problems that might be applicable to each of the generic flows described.

2.3. Try to develop a set of actions for the communication activity. Select one action, and define a task set for it.

2.4. A common problem during communication occurs when you encounter two stakeholders who have conflicting ideas about what the software should be. That is, they have mutually conflicting requirements. Develop a process pattern that addresses this problem and suggest an effective approach to it.

2.5. Provide three examples of software projects that would be amenable to the waterfall model. Be specific.

2.6. Provide three examples of software projects that would be amenable to the prototyping model. Be specific.

2.7. As you move outward along the spiral process flow, what can you say about the software that is being developed or maintained?

2.8. Is it possible to combine process models? If so, provide an example.

2.9. What are the advantages and disadvantages of developing software in which quality is "good enough"? That is, what happens when we emphasize development speed over product quality?

2.10. It is possible to prove that a software component and even an entire program is correct? So why doesn't everyone do this?

2.11. Are the Unified Process and UML the same thing? Explain your answer.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

3 AGILITY AND PROCESS

In 2001, a group of noted software developers, writers, and consultants [Bec01] signed the "Manifesto for Agile Software Development" in which they argued in favor of "individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan."

KEY CONCEPTS

acceptance tests 48
Agile Alliance 40
agile process 40
agility 38
agility principles 40
cost of change 39
DevOps 50
Extreme Programming (XP) 46
Kanban 48
pair programming 48
politics of agile development 41
project velocity 47
refactoring 48
Scrum 42

QUICK LOOK

What is it? Agile software engineering combines a philosophy and a set of development guidelines. The philosophy encourages customer satisfaction and early incremental delivery of software; small, highly motivated project teams; informal methods; minimal software engineering work products; and overall development simplicity. The development guidelines stress delivery over analysis and design (although these activities are not discouraged).

Who does it? Software engineers and other project stakeholders (managers, customers, end users) work together on an agile team—a team that is self-organizing and in control of its own destiny. An agile team fosters communication and collaboration among all who serve on it.

Why is it important? Modern business environments that spawn computer-based systems and software products are fast paced and ever changing. Agile software engineering represents a reasonable alternative to conventional software engineering. It has been demonstrated to deliver successful systems quickly.

What are the steps? Agile development might best be termed "software engineering lite." The basic framework activities—communication, planning, modeling, construction, and deployment—remain. But they morph into a minimal task set that pushes the project team toward construction and delivery.

What is the work product? The most important work product is an operational "software increment" that is delivered to the customer on the appropriate commitment date. The most important documents created are the use stories and their associated test cases.

How do I ensure that I've done it right? If the agile team agrees that the process works, and the team produces deliverable software increments that satisfy the customer, you've done it right.

The underlying ideas that guide agile development led to the development of agile[1] methods designed to overcome perceived and actual weaknesses in conventional software engineering. Agile development can provide important benefits, but it may not be applicable to all projects, all products, all people, and all situations. It is also *not* antithetical to solid software engineering practice and can be applied as an overriding philosophy for all software work.

In the modern economy, it is often difficult or impossible to predict how a computer-based system (e.g., a mobile application) will evolve as time passes. Market conditions change rapidly, end-user needs evolve, and new competitive threats emerge without warning. In many situations, you won't be able to define requirements fully before the project begins. You must be agile enough to respond to a fluid business environment.

Fluidity implies change, and change is expensive—particularly if it is uncontrolled or poorly managed. One of the most compelling characteristics of the agile approach is its ability to reduce the costs of change through the software process.

In a thought-provoking book on agile software development, Alistair Cockburn [Coc02] argues that the prescriptive process models introduced in Chapter 2 have a major failing: *they forget the frailties of the people who build computer software.* Software engineers are not robots. They exhibit great variation in working styles and significant differences in skill level, creativity, orderliness, consistency, and spontaneity. Some communicate well in written form, others do not. If process models are to work, they must provide a realistic mechanism for encouraging the discipline that is necessary, or they must be characterized in a manner that shows "tolerance" for the people who do software engineering work.

3.1 What Is Agility?

Just what is agility in the context of software engineering work? Ivar Jacobson [Jac02a] argues that the pervasiveness of change is the primary driver for agility. Software engineers must be quick on their feet if they are to accommodate the rapid changes that Jacobson describes.

But agility is more than an effective response to change. It also encompasses the philosophy espoused in the manifesto noted at the beginning of this chapter. It encourages team structures and attitudes that make communication (among team members, between technologists and business people, and between software engineers and their managers) more facile. It emphasizes rapid delivery of operational software and deemphasizes the importance of intermediate work products (not always a good thing); it adopts the customer as a part of the development team and works to eliminate the "us and them" attitude that continues to pervade many software projects; it recognizes that planning in an uncertain world has its limits and that a project plan must be flexible.

Agility can be applied to any software process. However, to accomplish this, it is essential that the process be designed in a way that allows the project team to adapt tasks and to streamline them, conduct planning in a way that understands the fluidity of an

1 Agile methods are sometimes referred to as *light methods* or *lean methods.*

agile development approach, eliminate all but the most essential work products and keep them lean, and emphasize an incremental delivery strategy that gets working software to the customer as rapidly as feasible for the product type and operational environment.

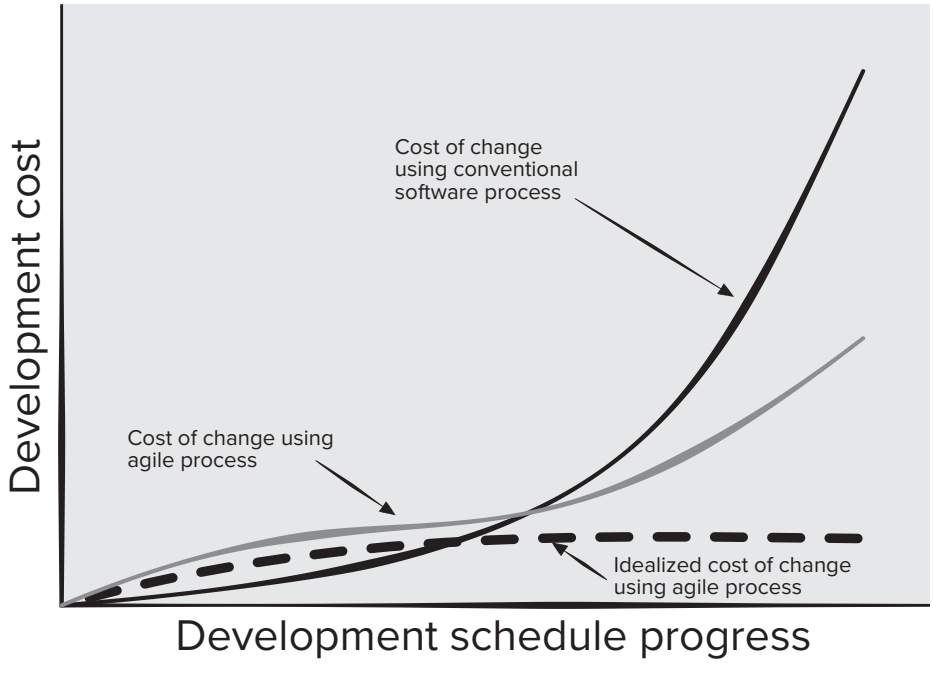
3.2 AGILITY AND THE COST OF CHANGE

The conventional wisdom in software development (supported by decades of experience) is that the cost of change increases nonlinearly as a project progresses (Figure 3.1, solid black curve). It is relatively easy to accommodate a change when a software team is gathering requirements (early in a project). A usage scenario might have to be modified, a list of functions may be extended, or a written specification can be edited. The costs of doing this work are minimal, and the time required will not adversely affect the outcome of the project. But what if we fast-forward a number of months? The team is in the middle of validation testing (something that occurs relatively late in the project), and an important stakeholder is requesting a major functional change. The change requires a modification to the architectural design of the software, the design and construction of three new components, modifications to another five components, the design of new tests, and so on. Costs escalate quickly, and the time and effort required to ensure that the change is made without unintended side effects are nontrivial.

Proponents of agility (e.g., [Bec99], [Amb04]) argue that a well-designed agile process "flattens" the cost of change curve (Figure 3.1, shaded, solid curve), allowing

FIGURE 3.1

Change costs as a function of time in development



a software team to accommodate changes late in a software project without dramatic cost and time impact. You've already learned that the agile process encompasses incremental delivery. When incremental delivery is coupled with other agile practices such as continuous unit testing and pair programming (discussed briefly in Section 3.5.1 and in more detail in Chapter 20), the cost of making a change is attenuated. Although debate about the degree to which the cost curve flattens is ongoing, there is evidence [Coc01a] to suggest that a significant reduction in the cost of change can be achieved.

3.3 What Is an Agile Process?

Any agile software process is characterized in a manner that addresses a number of key assumptions [Fow02] about the majority of software projects:

1. It is difficult to predict in advance which software requirements will persist and which will change. It is equally difficult to predict how customer priorities will change as the project proceeds.
2. For many types of software, design and construction are interleaved. That is, both activities should be performed in tandem so that design models are proven as they are created. It is difficult to predict how much design is necessary before construction is used to prove the design.
3. Analysis, design, construction, and testing are not as predictable (from a planning point of view) as we might like.

Given these three assumptions, an important question arises: How do we create a process that can manage *unpredictability*? The answer, as we have already noted, lies in process adaptability (to rapidly changing project and technical conditions). An agile process, therefore, must be *adaptable.*

But continual adaptation without forward progress accomplishes little. Therefore, an agile software process must adapt *incrementally.* To accomplish incremental adaptation, an agile team requires customer feedback (so that the appropriate adaptations can be made). An effective catalyst for customer feedback is an operational prototype or a portion of an operational system. Hence, an *incremental development strategy* should be instituted. *Software increments* (executable prototypes or portions of an operational system) must be delivered in short time periods so that adaptation keeps pace with change (unpredictability). This iterative approach enables the customer to evaluate the software increment regularly, provide necessary feedback to the software team, and influence the process adaptations that are made to accommodate the feedback.

3.3.1 Agility Principles

The Agile Alliance [Agi17][2] defines 12 principles for those software organizations that want to achieve agility. These principles are summarized in the paragraphs that follow.

Customer satisfaction is achieved by providing value through software that is delivered to the customer as rapidly as possible. To achieve this, agile developers

2 The Agile Alliance home page contains much useful information: https://www.agilealliance.org/.

recognize that requirements will change. They deliver software increments frequently and work together with all stakeholders so that feedback on their deliveries is rapid and meaningful.

An agile team is populated by motivated individuals, who communicate face-to face and work in an environment that is conducive to high quality software development. The team follows a process that encourages technical excellence and good design, emphasizing simplicity—"the art of maximized the amount of work not done" [Agi17]. Working software that meets customer needs is their primary goal, and the pace and direction of the team's work must be "sustainable," enabling them to work effectively for long periods of time.

An agile team is a "self-organizing team"—one that can develop well-structured architectures that lead to solid designs and customer satisfaction. Part of the team culture is to consider its work introspectively, always with the intent of improving the manner in which it addresses its primary goal.

Not every agile process model applies characteristics described in this section with equal weight, and some models choose to ignore (or at least downplay) the importance of one or more agile principles. However, these principles define an *agile spirit* that is maintained in each of the process models presented in this chapter.

3.3.2 The Politics of Agile Development

There is considerable debate (sometimes strident) about the benefits and applicability of agile software development as opposed to more conventional software engineering processes. Jim Highsmith [Hig02a] (facetiously) states the extremes when he characterizes the feeling of the pro-agility camp ("agilists"): "Traditional methodologists are a bunch of stick-in-the-muds who'd rather produce flawless documentation than a working system that meets business needs." As a counterpoint, he states (again, facetiously) the position of the traditional software engineering camp: "Lightweight, er, 'agile' methodologists are a bunch of glorified hackers who are going to be in for a heck of a surprise when they try to scale up their toys into enterprise-wide software."

Like all software technology arguments, this methodology debate risks degenerating into a religious war. If warfare breaks out, rational thought disappears and beliefs rather than facts guide decision making.

No one is against agility. The real question is: What is the best way to achieve it? Keep in mind that working software is important, but don't forget that it must also exhibit a variety of quality attributes including reliability, usability, and maintainability. How do you build software that meets customers' needs today and exhibits the quality characteristics that will enable it to be extended and scaled to meet customers' needs over the long term?

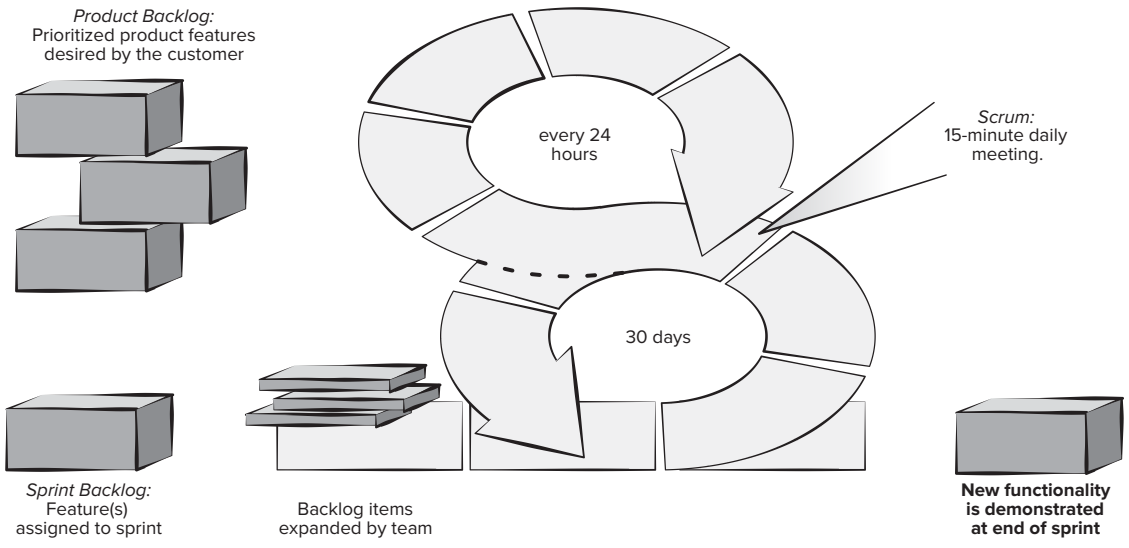
There are no absolute answers to either of these questions. Even within the agile school itself, there are many proposed framework models (Sections 3.4 and 3.5), each with a subtly different approach to the agility problem. Within each model there is a set of "ideas" (agilists are loath to call them "work tasks") that represent a significant departure from traditional software engineering. And yet, many agile concepts are simply adaptations of good software engineering concepts. The bottom line is there is much that can be gained by considering the best of both schools and virtually nothing to be gained by denigrating either approach.

3.4 SCRUM

Scrum (the name is derived from an activity that occurs during a rugby match)[3] is a very popular agile software development method that was conceived by Jeff Sutherland and his development team in the early 1990s. Further development on the Scrum methods was performed by Schwaber and Beedle [Sch01b].

Scrum principles are consistent with the agile manifesto and are used to guide development activities within a process that incorporates the following framework activities: requirements, analysis, design, evolution, and delivery. Within each framework activity, work tasks take place in a relatively short time-boxed[4] period called a *sprint*. The work conducted within a sprint (the number of sprints required for each framework activity will vary depending on size of the product and its complexity) is adapted to the problem at hand and is defined and often modified in real time by the Scrum team. The overall flow of the Scrum process is illustrated in Figure 3.2. Much of our description of the Scrum framework appears in Fowler and Sutherland [Fow16].[5]

FIGURE 3.2 **Scrum process flow**



3 A group of players forms around the ball, and the teammates work together (sometimes violently!) to move the ball downfield.

4 A *time-box* is a project management term (see Part Four of this book) that indicates a period of time that has been allocated to accomplish some task.

5 The Scrum Guide is available at: https://www.Scrum.org/resources/what-is-Scrum.

SAFEHOME



Considering Agile Software Development

The scene: Doug Miller's office.

The players: Doug Miller, software engineering manager; Jamie Lazar, software team member; Vinod Raman, software team member.

The conversation: (A knock on the door, Jamie and Vinod enter Doug's office.)

Jamie: Doug, you got a minute?

Doug: Sure Jamie, what's up?

Jamie: We've been thinking about our process discussion yesterday . . . you know, what process we're going to choose for this new *SafeHome* project.

Doug: And?

Vinod: I was talking to a friend at another company, and he was telling me about Scrum. It's an agile process model . . . heard of it?

Doug: Yeah, some good, some bad.

Jamie: Well, it sounds pretty good to us. Lets you develop software really fast, uses something called sprints to deliver software increments when the team decides the product is done . . . it's pretty cool, I think.

Doug: It does have a lot of really good ideas. I like the sprint concept, the emphasis on early test case creation, and the idea that the process owner should be part of the team.

Jamie: Huh? You mean that marketing will work on the project team with us?

Doug (nodding): They're stakeholders but not really the product owner. That would be Marg.

Jamie: Good. She will filter the changes marketing will want to send every 5 minutes.

Vinod: Even so, my friend said that there are ways to "embrace" changes during an agile project.

Doug: So you guys think we should use Scrum?

Jamie: It's definitely worth considering.

Doug: I agree. And even if we choose an incremental model as our approach, there's no reason why we can't incorporate much of what Scrum has to offer.

Vinod: Doug, before you said "some good, some bad." What was the bad?

Doug: The thing I don't like is the way Scrum downplays analysis and design . . . sort of says that writing code is where the action is . . .

(The team members look at one another and smile.)

Doug: So you agree with the Scrum approach?

Jamie (speaking for both): It can be adapted to fit our needs. Besides, writing code is what we do, Boss!

Doug (laughing): True, but I'd like to see you spend a little less time coding and then recoding and a little more time analyzing what needs to be done and designing a solution that works.

Vinod: Maybe we can have it both ways, agility with a little discipline.

Doug: I think we can, Vinod. In fact, I'm sure of it.

3.4.1 Scrum Teams and Artifacts

The Scrum team is a self-organizing interdisciplinary team consisting of a *product owner*, a *Scrum master,* and a small (three to six people) *development team.* The principle Scrum artifacts are the *product backlog*, the *sprint backlog*, and the code *increment*. Development proceeds by breaking the project into a series of incremental prototype development periods 2 to 4 weeks in length called *sprints*.

The product backlog is a prioritized list of product requirements or features that provide business value for the customer. Items can be added to the backlog at any time with the approval of the product owner and the consent of the development team. The product owner orders the items in the product backlog to meet the most important goals of all stakeholders. The product backlog is never complete while the product is evolving to meet stakeholder needs. The product owner is the only person who decides whether to end a sprint prematurely or extend the sprint if the increment is not accepted.

The sprint backlog is the subset of product backlog items selected by the product team to be completed as the code increment during the current active sprint. The increment is the union of all product backlog items completed in previous sprints and all backlog items to be completed in the current sprints. The development team creates a plan for delivering a software increment containing the selected features intended to meet an important goal as negotiated with the product owner in the current sprint. Most sprints are time-boxed to be completed in 3 to 4 weeks. How the development team completes the increment is left up to the team to decide. The development team also decides when the increment is done and ready to demonstrate to the product owner. No new features can be added to the sprint backlog unless the sprint is cancelled and restarted.

The Scrum master serves as facilitator to all members of the Scrum team. She runs the daily Scrum meeting and is responsible for removing obstacles identified by team members during the meeting. She coaches the development team members to help each other complete sprint tasks when they have time available. She helps the product owner find techniques for managing the product backlog items and helps ensure that backlog items are stated in clear and concise terms.

3.4.2 Sprint Planning Meeting

Prior to beginning, any development team works with the product owner and all other stakeholders to develop the items in the product backlog. Techniques for gathering these requirements are discussed in Chapter 7. The product owner and the development team rank the items in the product backlog by the importance of the owner's business needs and the complexity of the software engineering tasks (programming and testing) required to complete each of them. Sometimes this results in the identification of missing features needed to deliver the required functionality to the end users.

Prior to starting each sprint, the product owner states her development goal for the increment to be completed in the upcoming sprint. The Scrum master and the development team select the items to move from to the sprint backlog. The development team determines what can be delivered in the increment within the constraints of the time-box allocated for the sprint and, with the Scrum master, what work will be needed to deliver the increment. The development team decides which roles are needed and how they will need to be filled.

3.4.3 Daily Scrum Meeting

The daily Scrum meeting is a 15-minute event scheduled at the start of each workday to allow team members to synchronize their activities and make plans for the next

24 hours. The Scrum master and the development team always attend the daily Scrum. Some teams allow the product owner to attend occasionally.

Three key questions are asked and answered by all team members:

- What did you do since the last team meeting?
- What obstacles are you encountering?
- What do you plan to accomplish by the next team meeting?

The Scrum master leads the meeting and assesses the responses from each person. The Scrum meeting helps the team to uncover potential problems as early as possible. It is the Scrum master's task to clear obstacles presented before the next Scrum meeting if possible. These are not problem-solving meetings, those occur off-line and only involve the affected parties. Also, these daily meetings lead to "knowledge socialization" [Bee99] and thereby promote a self-organizing team structure.

Some teams use these meetings to declare sprint backlog items complete or done. When the team considers all sprint backlog items complete, the team may decide to schedule a demo and review of the completed increment with the product owner.

3.4.4 Sprint Review Meeting

The sprint review occurs at the end of the sprint when the development team has judged the increment complete. The sprint review is often time-boxed as a 4-hour meeting for a 4-week sprint. The Scrum master, the development team, the product owner, and selected stakeholders attend this review. The primary activity is a *demo* of the software increment completed during the sprint. It is important to note that the demo may not contain all planned functionality, but rather those functions that were to be delivered within the time-box defined for the sprint.

The product owner may accept the increment as complete or not. If it is not accepted, the product owner and the stakeholders provide feedback to allow a new round of sprint planning to take place. This is the time when new features may be added or removed from the product backlog. The new features may affect the nature of the increment developed in the next sprint.

3.4.5 Sprint Retrospective

Ideally, before beginning another sprint planning meeting, the Scrum master will schedule a 3-hour meeting (for a 4-week sprint) with the development team called a *sprint retrospective*. During this meeting the team discusses:

- What went well in the sprint
- What could be improved
- What the team will commit to improving in the next sprint

The Scrum master leads the meeting and encourages the team to improve its development practices to become more effective for the next sprint. The team plans ways to improve product quality by adapting its definition of "done." At the end of this meeting, the team should have a good idea about the improvements needed in the next sprint and be ready to plan the increment at the next sprint planning meeting.

3.5 Other Agile Frameworks

The history of software engineering is littered with dozens of obsolete process descriptions and methodologies, modeling methods and notations, tools, and technology. Each flared in notoriety and was then eclipsed by something new and (purportedly) better. With the introduction of a wide array of agile process frameworks—each contending for acceptance within the software development community—the agile movement has followed the same historical path.[6]

As we noted in the last section, one of the most widely used of all agile frameworks is Scrum. But many other agile frameworks have been proposed and are in use across the industry. In this section, we present a brief overview of three popular agile methods: Extreme Programming (XP), Kanban, and DevOps.

3.5.1 The XP Framework

In this section we provide a brief overview of *Extreme Programming* (XP), one of the most widely used approaches to agile software development. Kent Beck [Bec04a] wrote the seminal work on XP.

Extreme Programming encompasses a set of rules and practices that occur within the context of four framework activities: planning, design, coding, and testing. Figure 3.3 illustrates the XP process and notes some of the key ideas and tasks that are associated with each framework activity. The key XP activities are summarized in the paragraphs that follow.

Planning. The planning activity (also called the *planning game*) begins with a requirements activity called *listening*. Listening leads to the creation of a set of "stories" (also called *user stories*) that describe required output, features, and functionality for software to be built. Each *user story* (described in Chapter 7) is written by the customer and is placed on an index card. The customer assigns a *value* (i.e., a priority) to the story based on the overall business value of the feature or function.[7] Members of the XP team then assess each story and assign a *cost*—measured in development weeks—to it. It is important to note that new stories can be written at any time.

Customers and developers work together to decide how to group stories into the next release (the next software increment) to be developed by the XP team. Once a basic *commitment* (agreement on stories to be included, delivery date, and other project matters) is made for a release, the XP team orders the stories that will be developed in one of three ways: (1) all stories will be implemented immediately (within a few weeks), (2) the stories with highest value will be moved up in the schedule and implemented first, or (3) the riskiest stories will be moved up in the schedule and implemented first.

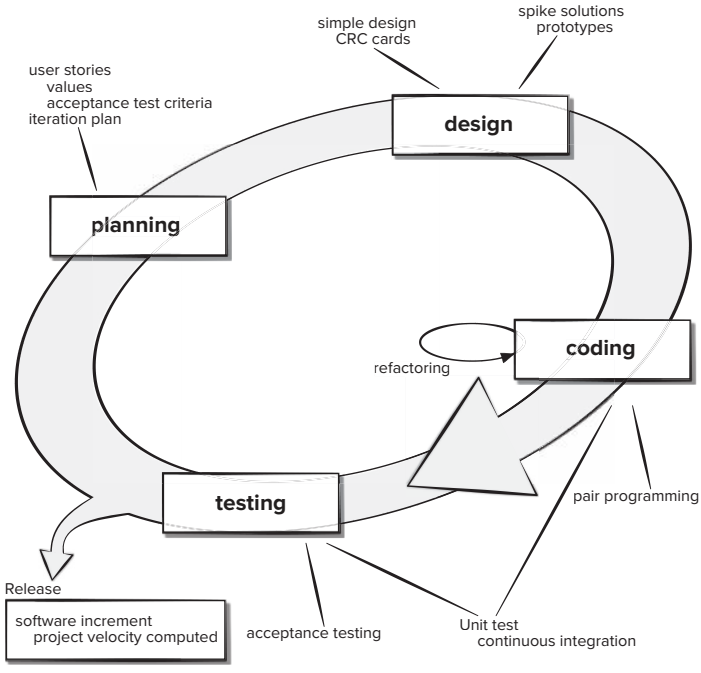
---

6 This is not a bad thing. Before one or more models or methods are accepted as a *de facto* standard, all must contend for the hearts and minds of software engineers. The "winners" evolve into best practice, while the "losers" either disappear or merge with the winning models.

7 The value of a story may also be dependent on the presence of another story.

FIGURE 3.3

The Extreme Programming process



After the first project release (also called a software increment) has been delivered, the XP team computes project velocity. Stated simply, *project velocity* is the number of customer stories implemented during the first release. Project velocity can then be used to help estimate delivery dates and schedule for subsequent releases. The XP team modifies its plans accordingly.

Design. XP design rigorously follows the KIS (keep it simple) principle. The design of extra functionality (because the developer assumes it will be required later) is discouraged.[8]

XP encourages the use of CRC cards (Chapter 8) as an effective mechanism for thinking about the software in an object-oriented context. CRC (class-responsibility-collaborator) cards identify and organize the object-oriented classes[9] that are relevant to the current software increment. CRC cards are the only design work product produced as part of the XP process.

If a difficult design problem is encountered as part of the design of a story, XP recommends the immediate creation of an operational prototype of that portion of the

8 These design guidelines should be followed in every software engineering method, although there are times when sophisticated design notation and terminology may get in the way of simplicity.

9 Object-oriented classes are discussed throughout Part Two of this book.

design. A central notion in XP is that design occurs both before *and after* coding commences. Refactoring—modifying/optimizing the code in a way that does not change the external behavior of the software [Fow00]—means that design occurs continuously as the system is constructed. In fact, the construction activity itself will provide the XP team with guidance on how to improve the design.

Coding. After user stories are developed and preliminary design work is done, the team does *not* move to code, but rather develops a series of unit tests that will exercise each of the stories that is to be included in the current release (software increment).[10] Once the unit test[11] has been created, the developer is better able to focus on what must be implemented to pass the test. Once the code is complete, it can be unit-tested immediately, thereby providing instantaneous feedback to the developers.

A key concept during the coding activity (and one of the most talked-about aspects of XP) is *pair programming.* XP recommends that two people work together at one computer to create code for a story. This provides a mechanism for real-time problem solving (two heads are often better than one) and real-time quality assurance (the code is reviewed as it is created).[12]

As pair programmers complete their work, the code they develop is integrated with the work of others. This "continuous integration" strategy helps uncover compatibility and interfacing errors early.

Testing. The unit tests that are created should be implemented using a framework that enables them to be automated (hence, they can be executed easily and repeatedly). This encourages implementing a regression testing strategy (Chapter 20) whenever code is modified (which is often, given the XP refactoring philosophy). XP *acceptance tests,* also called *customer tests,* are specified by the customer and focus on overall system features and functionality that are visible and reviewable by the customer. They are derived from user stories that have been implemented as part of a software release.

3.5.2 Kanban

The *Kanban* method [And16] is a lean methodology that describes methods for improving any process or workflow. Kanban is focused on change management and service delivery. Change management defines the process through which a requested change is integrated into a software-based system. Service delivery is encouraged by focusing on understanding customer needs and expectations. The team members manage the work and are given the freedom to organize themselves to complete it. Policies evolve as needed to improve outcomes.

---

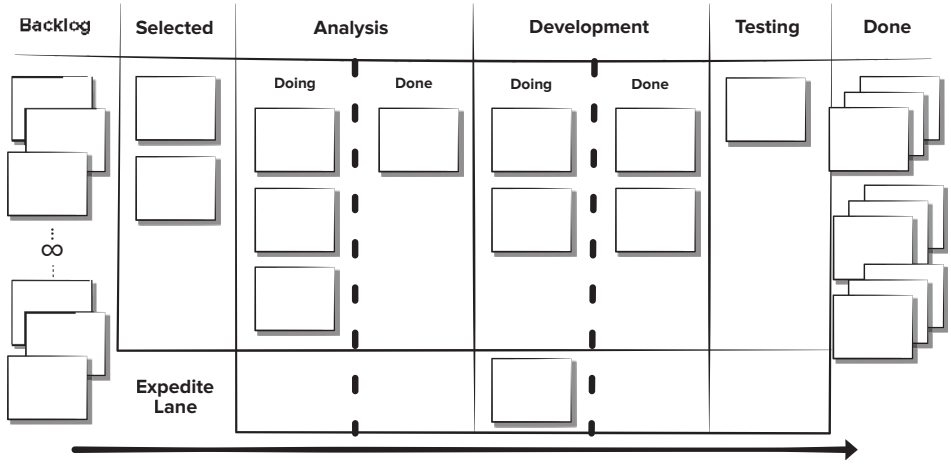
10 This approach is analogous to knowing the exam questions before you begin to study. It makes studying much easier by focusing attention only on the questions that will be asked.

11 Unit testing, discussed in detail in Chapter 20, focuses on an individual software component, exercising the component's interface, data structures, and functionality in an effort to uncover errors that are local to the component.

12 Pair programming has become so widespread throughout the software community that *The Wall Street Journal* [Wal12] ran a front-page story about the subject.

FIGURE 3.4

Kanban board



Kanban originated at Toyota as a set of industrial engineering practices and was adapted to software development by David Anderson [And16]. Kanban itself depends on six core practices:

1. Visualizing workflow using a Kanban board (an example is shown in Figure 3.4). The Kanban board is organized into columns representing the development stage for each element of software functionality. The cards on the board might contain single user stories or recently discovered defects on sticky notes and the team would advance them from "to do," to "doing," and "done" as the project progresses.
2. Limiting the amount of *work in progress* (WIP) at any given time. Developers are encouraged to complete their current task before starting another. This reduces lead time, improves work quality, and increases the team's ability to deliver software functionality frequently to their stakeholders.
3. Managing workflow to reduce waste by understanding the current value flow, analyzing places where it is stalled, defining changes, and then implementing the changes.
4. Making process policies explicit (e.g., write down your reasons for selecting items to work on and the criteria used to define "done").
5. Focusing on continuous improvement by creating feedback loops where changes are introduced based on process data and the effects of the change on the process are measured after the changes are made.[13]
6. Make process changes collaboratively and involve all team members and other stakeholders as needed.

13 The use of process metrics is discussed in Chapter 23.

The team meetings for Kanban are like those in the Scrum framework. If Kanban is being introduced to an existing project, not all items will start in the backlog column. Developers need to place their cards in the team process column by asking themselves: Where are they now? Where did they come from? Where are they going?

The basis of the daily Kanban standup meeting is a task called "walking the board." Leadership of this meeting rotates daily. The team members identify any items missing from the board that are being worked on and add them to the board. The team tries to advance any items they can to "done." The goal is to advance the high business value items first. The team looks at the flow and tries to identify any impediments to completion by looking at workload and risks.

During the weekly retrospective meeting, process measurements are examined. The team considers where process improvements may be needed and proposes changes to be implemented. Kanban can easily be combined with other agile development practices to add a little more process discipline.

3.5.3 DevOps

DevOps was created by Patrick DeBois [Kim16a] to combine Development and Operations. DevOps attempts to apply agile and lean development principles across the entire software supply chain. Figure 3.5 presents an overview of the DevOps workflow. The DevOps approach involves several stages that loop continuously until the desired product exists:

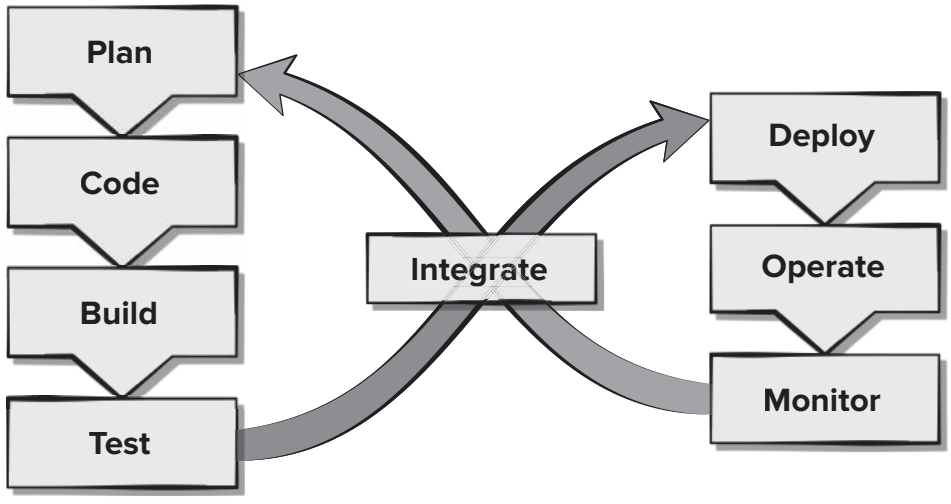
- **Continuous development.** Software deliverables are broken down and developed in multiple sprints with the increments delivered to the quality assurance[14] members of the development team for testing
- **Continuous testing.** Automated testing tools[15] are used to help team members test multiple code increments at the same time to ensure they are free of defects prior to integration.
- **Continuous integration.** The code pieces with new functionality are added to the existing code and to the run-time environment and then checked to ensure there are no errors after deployment.
- **Continuous deployment.** At this stage the integrated code is deployed (installed) to the production environment, which might include multiple sites globally that need to be prepared to receive the new functionality.
- **Continuous monitoring.** Operations staff who are members of the development team help to improve software quality by monitoring its performance in the production environment and proactively looking for possible problems before users find them.

DevOps enhances customers' experiences by reacting quickly to changes in their needs or desires. This can increase brand loyalty and increase market share. Lean approaches like DevOps can provide organizations with increased capacity to innovate

14 The quality assurance is discussed in Chapter 17.

15 Automated testing tools are discussed in Chapter 19.

FIGURE 3.5
DevOps



by reducing rework and allowing shifts to higher business value activities. Products do not make money until consumers have access to them, and DevOps can provide faster deployment time to production platforms [Sha17].

3.6 SUMMARY

In a modern economy, market conditions change rapidly, customer and end-user needs evolve, and new competitive threats emerge without warning. Practitioners must approach software engineering in a manner that allows them to remain agile—to define maneuverable, adaptive, lean processes that can accommodate the needs of modern business.

An agile philosophy for software engineering stresses four key issues: the importance of self-organizing teams that have control over the work they perform, communication and collaboration between team members and between practitioners and their customers, a recognition that change represents an opportunity, and an emphasis on rapid delivery of software that satisfies the customer. Agile process models have been designed to address each of these issues.

Some of the strengths and weaknesses of the agile methods we discussed are summarized in Table 3.1. In previous editions of this book we have discussed many others. The reality is that no agile method is perfect for every project. Agile developers work on self-directed teams and are empowered to create their own process models.

Scrum emphasizes the use of a set of software process patterns that have proven effective for projects with tight time lines, changing requirements, and business criticality. There is no reason why a Scrum team could not adopt the use of a Kanban chart to help organize its daily planning meeting.

TABLE 3.1
Comparing agile techniques

Scrum pros	The product owner sets priorities. The team owns decision making. Documentation is lightweight. It supports frequent updating.
Scrum cons	It is difficult to control the cost of changes. It may not be suitable for large teams. It requires expert team members.
XP pros	It emphasizes customer involvement. It establishes rational plans and schedules. There is high developer commitment to the project. There is reduced likelihood of product rejection.
XP cons	There is temptation to "ship" a prototype. It requires frequent meetings about increasing costs. It may allow for excessive changes. There is a dependence on highly skilled team members.
Kanban pros	It has lower budget and time requirements. It allows for early product delivery. Process policies are written down. There is continuous process improvement.
Kanban cons	Team collaboration skills determine success. Poor business analysis can doom the project. Flexibility can cause developers to lose focus. Developer reluctance to use measurement.
DevOps pros	There is reduced time to code deployment. The team has developers and operations staff. The team has end-to-end project ownership. There is proactive monitoring of deployed product.
DevOps cons	There is pressure to work on both old and new code. There is heavy reliance on automated tools to be effective. Deployment may affect the production environment. It requires an expert development team.

Extreme programming (XP) is organized around four framework activities—**planning, design, coding,** and **testing**—XP suggests a number of innovative and powerful techniques that allow an agile team to create frequent software releases that deliver features and functionality that have been described and then prioritized by stakeholders. There is nothing preventing them from using DevOps techniques to decrease their time to deployment.

PROBLEMS AND POINTS TO PONDER

3.1. Read the "Manifesto of Agile Software Development" [Bec01] noted at the beginning of this chapter. Can you think of a situation in which one or more of the four "values" could get a software team into trouble?

3.2. Describe agility (for software projects) in your own words.

3.3. Why does an iterative process make it easier to manage change? Is every agile process discussed in this chapter iterative? Is it possible to complete a project in just one iteration and still be agile? Explain your answers.

3.4. Try to come up with one more "agility principle" that would help a software engineering team become even more maneuverable.

3.5. Why do requirements change so much? After all, don't people know what they want?

3.6. Most agile process models recommend face-to-face communication. Yet today, members of a software team and their customers may be geographically separated from one another. Do you think this implies that geographical separation is something to avoid? Can you think of ways to overcome this problem?

3.7. Write a user story that describes the "favorite places" or "favorites" feature available on most Web browsers.

3.8. Describe the XP concepts of refactoring and pair programming in your own words.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

4 RECOMMENDED PROCESS MODEL

In Chapters 2 and 3, we provided brief descriptions of several software process models and software engineering frameworks. Every project is different, and every team is different. There is no single software engineering framework that is appropriate for every software product. In this chapter, we'll share our thoughts on using an adaptable process that can be tailored to fit the needs of software developers working on many types of products.

KEY CONCEPTS

defining requirements 57
estimating resources 60
evaluating prototype. 64
go, no-go decision. 65
maintain. 69
preliminary architectural design 59
prototype construction 61
prototype evolution. 67
release candidate 68
risk assessment. 66
scope definition. 67
test and evaluate. 63

QUICK LOOK

What is it? Every software product needs a "road map" or "generic software process" of some kind. It doesn't have to be complete before you start, but you need to know where you're headed before you begin. Any road map or generic process should be based on best industry practices.

Who does it? Software engineers and their product stakeholders collaborate to adapt a generic software process model to meet the needs of team and then either follow it directly, or more likely, adapt as needed. Every software team should be disciplined but flexible and self-empowered when needed.

Why is it important? Software development can easily become chaotic without the control and organization offered by a defined process. As we stated in Chapter 3, a modern software engineering approach must be "agile" and embrace changes that are needed to satisfy the stakeholders' requirements. It is important not to be too focused on documents and rituals. The process should only include those activities, controls, and work products that are appropriate for the project team and the product that is to be produced.

What are the steps? Even if a generic process must be adapted to meet the needs of the specific products being built, you need to be sure that all stakeholders have a role to play in defining, building, and testing the evolving software. There is likely to be substantial overlap among the basic framework activities (communication, planning, modeling, construction, and deployment). *Design a little, build a little, test a little, repeat* is a better approach than creating rigid project plans and documents for most software projects.

What is the work product? From the point of view of a software team, the work products are working program increments, useful documents, and data that are produced by the process activities.

How do I ensure that I've done it right? The timeliness, levels of stakeholder satisfaction, overall quality, and long-term viability of the product increments built are the best indicators that your process is working.

A paper by Rajagoplan [Raj14] reviews the general weaknesses of prescriptive software life-cycle approaches (e.g., the waterfall model) and contains several suggestions that should be considered when organizing a modern software development project.

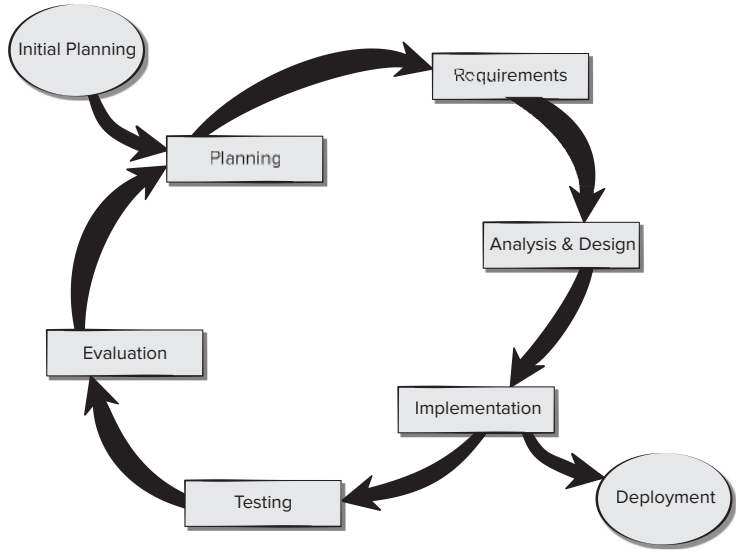
1. It is risky to use a linear process model without ample feedback.
2. It is never possible nor desirable to plan big up-front requirements gathering.
3. Up-front requirements gathering may not reduce costs or prevent time slippage.
4. Appropriate project management is integral to software development.
5. Documents should evolve with the software and should not delay the start of construction.
6. Involve stakeholders early and frequently in the development process.
7. Testers need to become involved in the process prior to software construction.

In Section 2.6, we listed the pros and cons of several prescriptive process models. The waterfall model is not amenable to changes that may need to be introduced once developers start coding. Stakeholder feedback is therefore limited to the beginning and end of the project. Part of the reason for this is the waterfall model suggests that all analysis and design work products be completed before any programming or testing occurs. This makes it hard to adapt to projects with evolving requirements.

One temptation is to switch to an incremental model (Figure 4.1) like the prototyping model or Scrum. Incremental process models involve customers early and often and therefore reduce the risk of creating product that is not accepted by the customers. There is a temptation to encourage lots of changes as stakeholders view each prototype and realize that functions and features they now realize they need are missing. Often, developers do not plan for prototype evolution and create throwaway prototypes.

FIGURE 4.1

Incremental model for prototype development



Recall that the goal of software engineering is to reduce unnecessary effort, so prototypes need to be designed with reuse in mind. Incremental models do provide a better basis for creating an adaptable process if changes can be managed wisely.

In Section 3.5, we discussed the pros and cons of several agile process models other than Scrum. Agile process models are very good at accommodating the uncertain knowledge about the stakeholders' real needs and problems. Key characteristics of agile process models are:

- Prototypes created are designed to be extended in future software increments.
- Stakeholders are involved throughout the development process.
- Documentation requirements are lightweight, and documentation should evolve along with the software.
- Testing is planned and executed early.

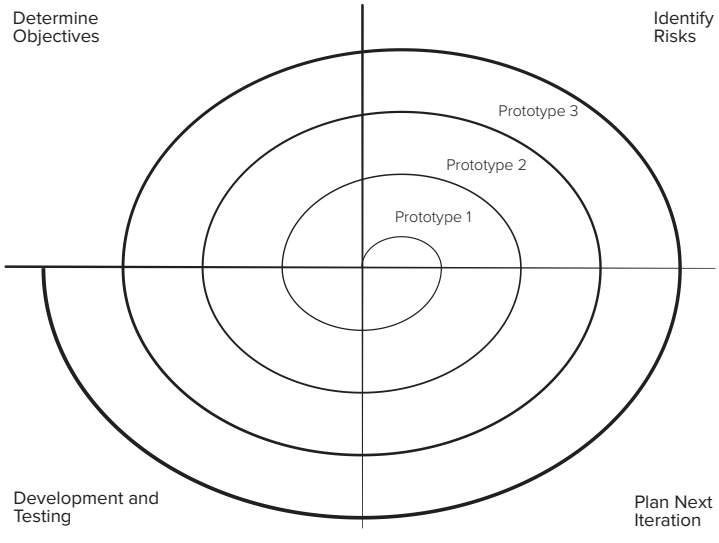
Scrum and Kanban extend these characteristics. Scrum is sometimes criticized for requiring too many meetings. But daily meetings make it hard for developers to stray too far from building products that stakeholders find useful. Kanban (Section 3.5.2) provides a good lightweight tracking system for managing the status and priorities of user stories.

Both Scrum and Kanban allow for controlled introduction of new requirements (user stories). Agile teams are small by design and may not be suitable for projects requiring large numbers of developers, unless the project can be partitioned into small and independently assignable components. Still, agile process models offer many good features that can be incorporated into an adaptable process model.

The spiral model (Figure 4.2) can be thought of as an evolutionary prototyping model with a risk assessment element. The spiral model relies on moderate stakeholder

FIGURE 4.2

Spiral model for prototype development



involvement and was designed for large teams and large projects. Its goal is to create extensible prototypes each time the process is iterated. Early testing is essential. Documentation evolves with the creation of each new prototype. The spiral model is somewhat unique in that formal risk assessment is built in and used as the basis for deciding whether to invest the resources needed to create the next prototype. Some people argue that it may be hard to manage a project using the spiral model, because the project scope may not be known at the start of the project. This is typical of most incremental process models. The spiral is a good basis for building an adaptable process model.

How do agile process models compare to evolutionary models? We've summarized some of the key characteristics in a sidebar.

Characteristics of Agile Models

Agile

1. Not suitable for large high-risk or mission critical projects.
2. Minimal rules and minimal documentation
3. Continuous involvement of testers
4. Easy to accommodate product changes
5. Depends heavily on stakeholder interaction
6. Easy to manage
7. Early delivery of partial solutions
8. Informal risk management
9. Built-in continuous process improvement

Spiral

1. Not suitable for small, low-risk projects
2. Several steps required, along with documentation done up front
3. Early involvement of testers (might be done by outside team)
4. Hard to accommodate product changes until prototype completed
5. Continuous stakeholder involvement in planning and risk assessment
6. Requires formal project management and coordination
7. Project end not always obvious
8. Good risk management
9. Process improvement handled at end of project

Creative, knowledgeable people perform software engineering. They adapt software processes to make them appropriate for the products that they build and to meet the demands of the marketplace. We think that using a spiral-like approach that has agility built into every cycle is a good place to start for many software projects. Developers learn many things as they proceed in the development process. That's why it is important for developers to be able to adapt their process as quickly as practical to accommodate this new knowledge.

4.1 Requirements Definition

Every software project begins with the team trying to understand the problem to be solved and determining what outcomes are important to the stakeholders. This includes understanding the business needs motivating the project and the technical issues which constrain it. This process is called *requirements engineering* and will be discussed in

more detail in Chapter 7. Teams that fail to spend a reasonable amount of time on this task will find that their project contains expensive rework, cost overruns, poor product quality, late delivery times, dissatisfied customers, and poor team morale. Requirements engineering cannot be neglected, nor can it be allowed to iterate endlessly before proceeding to product construction.

It's reasonable to ask what best practices should be followed to achieve thorough and agile requirements engineering. Scott Ambler [Amb12] suggests several best practices for agile requirements definition:

1. Encourage active stakeholder participation by matching their availability and valuing their input.
2. Use simple models (e.g., Post-it notes, fast sketches, user stories) to reduce barriers to participation.
3. Take time to explain your requirement representation techniques before using them.
4. Adopt stakeholder terminology, and avoid technical jargon whenever possible.
5. Use a breadth-first approach to get the big picture of the project done before getting bogged down in details.
6. Allow the development team to refine (with stakeholder input) requirement details "just in time" as user stories are scheduled to be implemented.
7. Treat the list of features to be implemented like a prioritized list, and implement the most important user stories first.
8. Collaborate closely with your stakeholders and only document requirements at a level that is useful to all when creating the next prototype.
9. Question the need to maintain models and documents that will not be referred to in the future.
10. Make sure you have management support to ensure stakeholder and resource availability during requirements definition.

It is important to recognize two realities: (1) it is impossible for stakeholders to describe an entire system before seeing the working software, and (2) it is difficult for stakeholders to describe quality requirements needed for the software before seeing it in action. Developers must recognize that requirements will be added and refined as the software increments are created. Capturing stakeholders' descriptions about what the system needs to do in their own words in a user story is a good place to begin.

If you can get stakeholders to define acceptance criteria for each user story, your team is off to a great start. It is likely that stakeholders will need to see a user story coded and running to know whether it has been implemented correctly or not. Therefore, requirements definition needs to be done iteratively and include the development of prototypes for stakeholder review.

Prototypes are tangible realizations of project plans that can be easily referenced by stakeholders when trying to describe desired changes. Stakeholders are motivated to discuss requirements changes in more concrete terms, which improves communication. It's important to recognize that prototypes allow developers to focus on short-term goals by only focusing on users' visible behaviors. It will be important to review

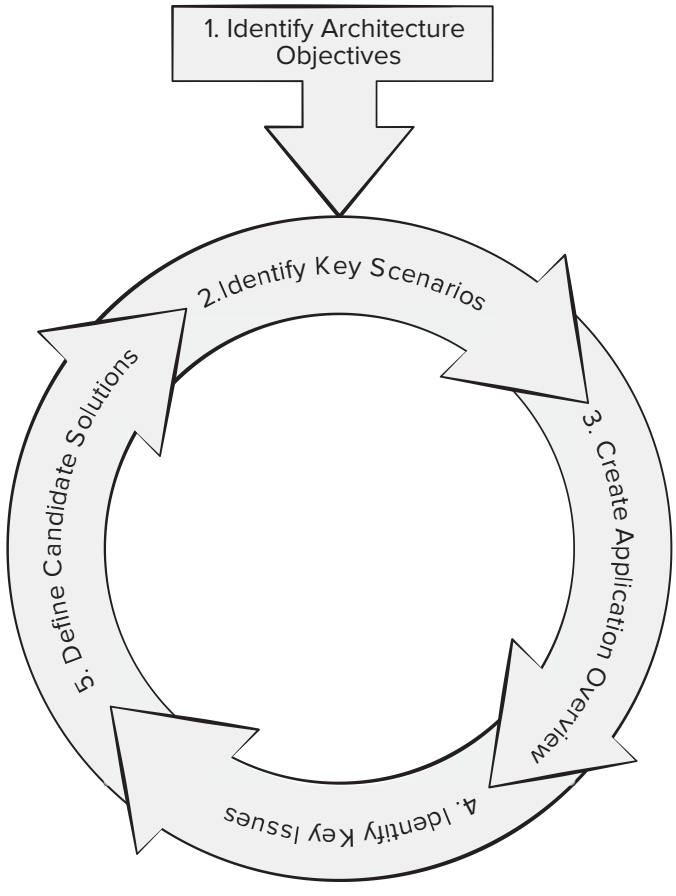
prototypes with an eye to the quality. Developers need to be aware that using prototypes may increase the volatility of the requirements if stakeholders are not focused on getting things right the first time. There is also the risk that creating prototypes before the software architectural requirements are well understood may result in prototypes that must be discarded, wasting time and resources [Kap15].

4.2 Preliminary Architectural Design

The decisions required to develop a solid architectural design are discussed in Chapter 10, but preliminary design decisions must often be made as requirements are defined. As shown in Figure 4.3, at some point in time, architectural decisions will need to be allocated to product increments. According to Bellomo and her colleagues [Bel14], early understanding of requirements and architecture choices is key to managing the development of large or complex software products.

FIGURE 4.3

Architectural design for prototype development



Requirements can be used to inform architecture design. Exploring the architecture as the prototype is developed facilitates the process of detailing the requirements. It is best to conduct these activities concurrently to achieve the right balance. There are four key elements to agile architectural design:

1. Focus on key quality attributes, and incorporate them into prototypes as they are constructed.
2. When planning prototypes, keep in mind that successful software products combine customer-visible features and the infrastructure to enable them.
3. Recognize that an agile architecture enables code maintainability and evolvability if sufficient attention is paid to architectural decisions and related quality issues.
4. Continuously managing and synchronizing dependencies between the functional and architectural requirements is needed to ensure the evolving architectural foundation will be ready just in time for future increments.

Software architecture decision making is critical to the success of a software system. The architecture of a software system determines its qualities and impacts the system throughout its life cycle. Dasanayake et al. [Das15] found that software architects are prone to making errors when their decisions are made under levels of uncertainty. Architects make fewer bad decisions if they can reduce this uncertainty through better architectural knowledge management. Despite the fact that agile approaches discourage heavy documentation, failing to record design decisions and their rationale early in the design process makes it hard to revisit them when creating future prototypes. Documenting the right things can assist with process improvement activities. Documenting your lessons learned is one of the reasons that retrospectives should be conducted after evaluating the delivered prototype and before beginning the next program increment. Reuse of previously successful solutions to architectural problems is also helpful and will be discussed in Chapter 14.

4.3 Resource Estimation

One of the more controversial aspects of using spiral or agile prototyping is estimating the time it will take to complete a project when it cannot be defined completely. It is important to understand before you begin whether you have a reasonable chance of delivering software products on time and with acceptable costs before you agree to take on the project. Early estimates run the risk of being incorrect because the project scope is not well defined and is likely to change once development starts. Estimates made when the project is almost finished do not provide any project management guidance. The trick is to estimate the software development time early based on what is known at the time and revise your estimates on a regular basis as requirements are added or after software increments are delivered. We discuss methods of estimating project scope in Chapter 25.

Let's examine how an experienced software project manager might estimate a project using the agile spiral model we have proposed. The estimates produced by this

method would need to be adjusted for the number of developers and the number of user stories that can be completed simultaneously.

1. Use historic data (Chapter 23), and work as a team to develop an estimate of how many days it will take to complete each of the user stories known at the start of the project.
2. Loosely organize the user stories into the sets that will make up each sprint[1] (Section 3.4) planned to complete a prototype.
3. Sum the number of days to complete each sprint to provide an estimate for the duration of the total project.
4. Revise the estimate as requirements are added to the project or prototypes are delivered and accepted by the stakeholders.

Keep in mind that doubling the number of developers almost never cuts the development time in half.

Rosa and Wallshein [Ros17] found that knowing initial software requirements at the start of a project provides an adequate but not always accurate estimate of project completion times. To get more accurate estimates, it is also important to know the type of project and the experience of the team. We will describe more detailed estimation techniques (e.g., function points or use case points) in Part Four of this book.

4.4 First Prototype Construction

In Section 2.5.2 we described the creation of prototypes as a means of helping the stakeholders move from statements of general objectives and user stories to the level of detail that developers will need to implement this functionality. Developers may use the first prototype to prove that their initial architectural design is a feasible approach to delivering the required functionality while satisfying the customer's performance constraints. To create an operational prototype suggests that requirements engineering, software design, and construction all proceed in parallel. This process is shown in Figure 4.1. This section describes steps that will be used to create the first prototypes. Details of best practices for software design and construction appear later in this book.

Your first task is to identify the features and functions that are most important to the stakeholders. These will help define the objectives for the first prototype. If the stakeholders and developers have created a prioritized list of user stories, it should be easy to confirm which are the most important.

Next, decide how much time will be allowed to create the first prototype. Some teams may choose a fixed time, such as a 4-week sprint, to deliver each prototype. In this case, the developers will look at their time and resource estimate and determine which of the high-priority user stories can be finished in 4 weeks. The team would then confirm with the stakeholders that the selected user stories are the best ones to include in the first prototype. An alternative approach would be to have the stakeholders

1 Sprint was described (Section 3.4) as a time period in which a subset of the system user stories will be delivered to the product owner.

and developers jointly choose a small number of high-priority user stories to include in the first prototype and use their time and resource estimates to develop the schedule to complete the first prototype.

The engineers working at National Instruments published a white paper that outlines their process for creation of a first functional prototype [Nat15]. These steps can be applied to a variety of software projects:

1. Transition from paper prototype to software design
2. Prototype a user interface
3. Create a virtual prototype
4. Add input and output to your prototype
5. Engineer your algorithms
6. Test your prototype
7. Prototype with deployment in mind

Referring to these seven steps, creating a *paper prototype* for a system is very inexpensive and can be done early in the development process. Customers and stakeholders are not usually experienced developers. Nontechnical users can often recognize what they like or do not like about a user interface very quickly once they see it sketched out. Communications between people are often filled with misunderstandings. People forget to tell each other what they really need to know or assume that everyone has the same understanding. Creating a paper prototype and reviewing it with the customer before doing any programming can help avoid wasted time building the wrong prototype. We will talk about several diagrams that can be used to model a system in Chapter 8.

Creating a *prototype user interface* as part of the first functional prototype is a wise idea. Many systems are implemented on the Web or as mobile applications and rely heavily on touch user interfaces. Computer games and virtual reality applications require a great deal of communication with end users to operate correctly. If customers find a software product easy to learn and use, they are more likely to use it.

Many misunderstandings between developers and stakeholders can be alleviated by beginning with a paper prototype of the user interface. Sometimes stakeholders need to see the basics of the user interface in action to be able to explain what they really like and dislike about it. It is less expensive to throw away an early user interface design than to finish the prototype and try to put a new user interface on top of it. Designing user interfaces that provide good user experiences is discussed in Chapter 12.

Adding input and output to your user interface prototype provides an easy way to begin testing the evolving prototype. Testing software component interfaces should be accomplished before testing the code that makes up the component's algorithms. To test the algorithms themselves, developers often use a "test frame" to ensure their implemented algorithms are working as intended. Creating a separate test frame and throwing it away is often a poor use of project resources. If properly designed, the user interface can serve as the test frame for component algorithms, thereby eliminating the effort required to build separate test frames.

Engineering your algorithms refers to the process of transforming your ideas and sketches into programming language code. You need to consider both the functional

requirements stated in the user story and the performance constraints (both explicit and implicit) when designing the necessary algorithms. This is the point where additional support functionality is likely to be identified and added to the project scope, if it does not already exist in a code library.

Testing your prototype demonstrates required functionality and identifies yet undiscovered defects before demonstrating it to the customer. Sometimes it is wise to involve the customer in the testing process before the prototype is finished to avoid developing the wrong functionality. The best time to create test cases is during requirements gathering or when use cases have been selected for implementation. Testing strategies and tactics are discussed in Chapters 19 through 21.

Prototyping with deployment in mind is very important because it helps you to avoid taking shortcuts that lead to creating software that will be hard to maintain in the future. This is not saying that every line of code will make it to the final software product. Like many creative tasks, developing a prototype is iterative. Drafts and revisions are to be expected.

As prototype development occurs, you should carefully consider the software architectural choices you make. It is relatively inexpensive to change a few lines of code if you catch errors before deployment. It is very expensive to change the architecture of a software application once it has been released to end users around the globe.

SafeHome



Room Designer
Considering First Prototype

The scene: Doug Miller's office.

The players: Doug Miller, software engineering manager; Jamie Lazar, software team member; Vinod Raman, software team member.

The conversation: (A knock on the door. Jamie and Vinod enter Doug's office.)

Jamie: Doug, you got a minute?

Doug: Sure, Jamie, what's up?

Jamie: We've been thinking about the scope of this *SafeHome* room design tool.

Doug: And?

Vinod: There's a lot of work that needs to be done on the back end of the project before people can start dropping alarm sensors and trying furniture layouts.

Jamie: We don't want to work on the back end for months and then have the project cancelled when the marketing folks decide they hate the product.

Doug: Have you tried to work out a paper prototype and review it with the marketing group?

Vinod: Well, no. We thought it was important to get a working computer prototype quickly and did not want to take the time to do one.

Doug: My experience is that people need to see something before they know whether they like it or not.

Jamie: Maybe we should step back and create a paper prototype of the user interface and get them to work with it and see if they like the concept at all.

Vinod: I suppose it wouldn't be too hard to program an executable user interface using the game engine we were considering using for the virtual reality version of the app.

Doug: Sounds like a plan. Try that approach, and see if you have the confidence you need to start evolving your prototype.

4.5 Prototype Evaluation

Testing is conducted by the developers as the prototype is being built and becomes an important part of prototype evaluation. Testing demonstrates that prototype components are operational, but it's unlikely that test cases will have found all the defects. In the spiral model, the results of evaluation allow the stakeholders and developers to assess whether it is desirable to continue development and create the next prototype. Part of this decision is based on user and stakeholder satisfaction, and part is derived from an assessment of the risks of cost overruns and failure to deliver a working product when the project is finished. Dam and Siang [Dam17] suggest several best-practice tips for gathering feedback on your prototype.

1. Provide scaffolding when asking for prototype feedback.
2. Test your prototype on the right people.
3. Ask the right questions.
4. Be neutral when presenting alternatives to users.
5. Adapt while testing.
6. Allow the user to contribute ideas.

Providing scaffolding is a mechanism for allowing the user to offer feedback that is not confrontational. Users are often reluctant to tell developers that they hate the product they are using. To avoid this, it is often easier to ask the user to provide feedback using a framework such as "I like, I wish, What if" as a means of providing open and honest feedback. *I like* statements encourage users to provide positive feedback on the prototype. *I wish* statements prompt users to share ideas about how the prototype can be improved. These statements can provide negative feedback and constructive criticism. *What if* statements encourage users to suggest ideas for your team to explore when creating prototypes in future iterations.

Getting the right people to evaluate the prototype is essential to reduce the risk of developing the wrong product. Having development team members do all the testing is not wise because they are not likely to be the representative of the intended user population. It's important to have the right mix of users (e.g., novice, typical, and advanced) to give you feedback on the prototype.

Asking the right questions implies that all stakeholders agree on prototype objectives. As a developer, it's important to keep an open mind and do your best to convince users that their feedback is valuable. Feedback drives the prototyping process as you plan for future product development activities. In addition to general feedback, try to ask specific questions about any new features included in the prototype.

Be neutral when presenting alternatives allows the software team to avoid making users feel they are being "sold" on one way to do things. If you want honest feedback, let the users know that you have not already made up your mind that there is only one right way to do things. *Egoless programming* is a development philosophy that focuses on producing the best product the team can create for the intended users. Although it is not desirable to create throwaway prototypes, egoless programming suggests that things that are not working need to be fixed or

discarded. So try not to become too attached to your ideas when creating early prototypes.

Adapt while testing means that you need a flexible mind-set while users are working with the prototype. This might mean altering your test plan or making quick changes to the prototype and then restarting the testing. The goal is to get the best feedback you can from users, including direct observation of them as they interact with the prototype. The important thing is that you get the feedback you need to help decide whether to build the next prototype or not.

Allow users to contribute ideas means what it says. Make sure you have a way of recording their suggestions and questions (electronically or otherwise). We will discuss additional ways of conducting user testing in Chapter 12.

SafeHome



Room Designer
Evaluating First Prototype

The scene: Doug Miller's office.

The players: Doug Miller, software engineering manager; Jamie Lazar, software team member; Vinod Raman, software team member.

The conversation: (A knock on the door. Jamie and Vinod enter Doug's office.)

Jamie: Doug, you got a minute?

Doug: Sure, Jamie, what's up?

Jamie: We completed the evaluation of the *SafeHome* room design tool working with our marketing stakeholders.

Doug: How did things go?

Vinod: We mostly focused on the user interface that will allow users to place alarm sensors in the room.

Jamie: I am glad we let them review a paper prototype before we created the PC prototype.

Doug: Why is that?

Vinod: We made some changes, and the marketing people liked the new design better, so that's the design we used when we started programming it.

Doug: Good. What's the next step?

Jamie: We completed the risk analysis and since we did not pick up any new user stories, we think it is reasonable to work on creating the next incremental prototype since we are still on time and within budget.

Vinod: So, if you agree, we'll get the developers and stakeholders together and begin planning the next software increment.

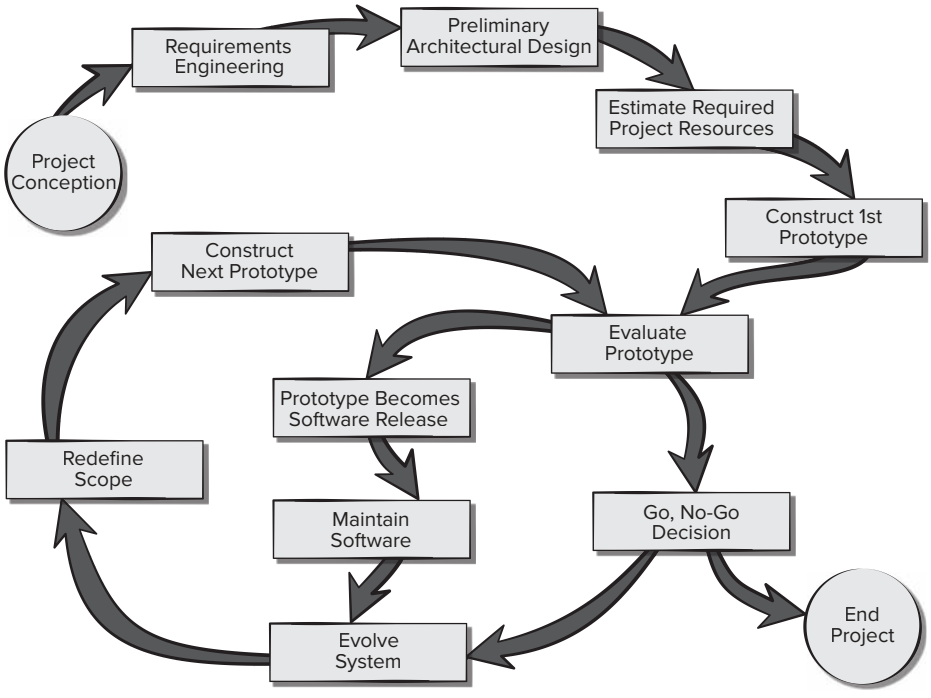
Doug: I agree. Keep me in the loop and try to keep the development time on the next prototype to 6 weeks or less.

4.6 Go, No-Go Decision

After the prototype is evaluated, project stakeholders decide whether to continue development of the software product. If you refer to Figure 4.4, a slightly different decision based on the prototype evaluation might be to release the prototype to the

FIGURE 4.4

Recommended software process model



end users and begin the maintenance process. Sections 4.8 and 4.9 discuss this decision in more detail. We discussed the use of risk assessment in the spiral model as part of the prototype assessment process in Section 2.5.3. The first circuit around the spiral might be used to solidify project requirements. But it really should be more. In the method we are proposing here, each trip around the spiral develops a meaningful increment of the final software product. You can work with the project user story or feature backlog to identify an important subset of the final product to include in the first prototype and repeat this for each subsequent prototype.

A pass through the planning region follows the evaluation process. Revised cost estimates and schedule changes are proposed based on what was discovered when evaluating the current software prototype. This may involve adding new user stories or features to the project backlog as the prototype is evaluated. The risk of exceeding the budget and missing the project delivery date is assessed by comparing new cost and time estimates against old ones. The risk of failing to satisfy user expectations is also considered and discussed with the stakeholders and sometimes senior management before deciding to create another prototype.

The goal of the risk assessment process is to get the commitment of all stakeholders and company management to provide the resources needed to create the next prototype. If the commitment is not there because the risk of project failure is too great, then the project can be terminated. In the Scrum framework (Section 3.4), the go, no-go decision might be made during the Scrum retrospective

meeting held between the prototype demonstration and the new sprint planning meeting. In all cases, the development team lays out the case for the product owners and lets them decide whether to continue product development or not. A more detailed discussion of software risk assessment methods is presented in Chapter 26.

4.7 PROTOTYPE EVOLUTION

Once a prototype has been developed and reviewed by the development team and other stakeholders, it's time to consider the development of the next prototype. The first step is to collect all feedback and data from the evaluation of the current prototype. The developers and stakeholders then begin negotiations to plan the creation of another prototype. Once the desired features for the new prototype have been agreed upon, consideration is given to any known time and budget constraints as well as the technical feasibility of implementing the prototype. If the development risks are deemed to be acceptable, the work continues.

The evolutionary prototyping process model is used to accommodate changes that inevitably occur as software is developed. Each prototype should be designed to allow for future changes to avoid throwing it away and creating the next prototype from scratch. This suggests favoring both well-understood and important features when setting the goals for each prototype. As always, the customer's needs should be given great importance in this process.

4.7.1 New Prototype Scope

The process of determining the scope of a new prototype is like the process of determining the scope of the initial prototype. Developers would either: (1) select features to develop within the time allocated to a sprint, or (2) allocate sufficient time to implement the features needed to satisfy the goals set by the developers with stakeholder input. Either approach requires the developers to maintain a prioritized list of features or user stories. The priorities used to order the list should be determined by the goals set for the prototype by the stakeholders and developers.

In XP (Section 3.5.1), the stakeholders and developers work together to group the most important user stories into a prototype that will become the next release of the software and determine its completion date. In Kanban (Section 3.5.2), developers and stakeholders make use of a board that allows them to focus on the completion status of each user story. This is a visual reference that can be used to assist developers using any incremental prototype process model to plan and monitor the progress of the software development. Stakeholders can easily see the feature backlog and help the developers order it to identify the most useful stories needed in the next prototype. It is probably easier to estimate the time required to complete the selected user stories than to find the user stories that need to fit into a fixed time block. But the advice to keep prototype development time to 4 to 6 weeks should be followed to ensure adequate stakeholder involvement and feedback.

4.7.2 Constructing New Prototypes

A user story should contain both a description of how the customer plans to interact with the system to achieve a specific goal and a description of what the customer's definition of acceptance is. The development team's task is to create additional software components to implement the user stories selected for inclusion in the new prototype along with the necessary test cases. Developers need to continue communication with all stakeholders as they create the new prototype.

What makes this new prototype trickier to build is that software components created to implement new features in the evolving prototype need to work with the components used to implement the features included in the previous prototype. It gets even trickier if developers need to remove components or modify those that were included in the previous prototype because requirements have changed. Strategies for managing these types of software changes are discussed in Chapter 22.

It is important for the developers to make design decisions that will make the software prototype more easily extensible in the future. Developers need to document design decisions in a way that will make it easier to understand the software when making the next prototype. The goal is to be agile in both development and documentation. Developers need to resist the temptation to overdesign the software to accommodate features that may or may not be included in the final product. They also should limit their documentation to that which they need to refer to during development or when changes need to be made in the future.

4.7.3 Testing New Prototypes

Testing the new prototype should be relatively straightforward if the development team created test cases as part of the design process before the programming was completed. Each user story should have had acceptance criteria attached to it as it was created. These acceptance statements should guide the creation of the test cases intended to help verify that the prototype meets customer needs. The prototype will need to be tested for defects and performance issues as well.

One additional testing concern for evolutionary prototypes is to ensure that adding new features does not accidentally break features that were working correctly in the previous prototype. *Regression testing* is the process of verifying that software that was previously developed and tested still performs the same way after it has been changed. It is important to use your testing time wisely and make use of the test cases that are designed to detect defects in the components most likely to be affected by the new features. Regression testing is discussed in more detail in Chapter 20.

4.8 Prototype Release

When an evolutionary prototyping process is applied, it can be difficult for developers to know when a product is finished and ready for release to the customers. Software developers do not want to release a buggy software product to the end users and have them decide the software has poor quality. A prototype being considered as a release candidate must be subjected to user acceptance testing in addition to functional and nonfunctional (performance) testing that would have been conducted during prototype construction.

User acceptance tests are based on the agreed-upon acceptance criteria that were recorded as each user story was created and added to the product backlog. This allows

user representatives to verify that the software behaves as expected and collect suggestions for future improvements. David Nielsen [Nie10] has several suggestions for conducting prototype testing in industrial settings.

When testing a release candidate, functional and nonfunctional tests should be repeated using the test cases that were developed during the construction phases of the incremental prototypes. Additional nonfunctional tests should be created to verify that the performance of the prototype is consistent with the agreed-upon benchmarks for the final product. Typical performance benchmarks may deal with system response time, data capacity, or usability. One of the most important nonfunctional requirements to verify is ensuring that the release candidate will run in all planned run-time environments and on all targeted devices. The process should be focused on testing limited to the acceptance criteria established before the prototype was created. Testing cannot prove a software product is bug free, only that the test cases ran correctly.

User feedback during acceptance testing should be organized by user-visible functions as portrayed via the user interface. Developers should examine the device in question and make changes to the user interface screen if implementing these changes will not delay the release of the prototype. If changes are made, they need to be verified in a second round of testing before moving on. You should not plan for more than two iterations of user acceptance testing.

It is important, even for projects using agile process models, to use an issue tracking or bug reporting system (e.g., Bugzilla[2] or Jira[3]) to capture the testing results. This allows developers to record test failures and makes it easier to identify the test cases that will need to be run again to verify that a repair properly corrects the problem that was uncovered. In each case the developers need to assess whether the changes can be made to the software without causing a cost overrun or late product delivery. The implications of not fixing a problem need to be documented and shared with both the customer and senior managers who may decide to cancel the project rather than committing the resources needed to deliver the final project.

The issues and lessons learned from creating the release candidate should be documented and considered by the developers and stakeholders as part of the project postmortem. This information should be considered before deciding to undertake future development of a software product following its release to the user community. The lessons learned from the current product can help developers make better cost and time estimates for similar projects in the future.

Techniques for conducting user acceptance testing are discussed in Chapters 12 and 20. A more detailed discussion on software quality assurance is presented in Chapter 17.

4.9 Maintain Release Software

Maintenance is defined as the activities needed to keep software operational after it has been accepted and delivered (released) in the end-user environment. Maintenance will continue for the life of the software product. Some software engineers believe that the majority of the money spent on a software product will be spent on maintenance activities. *Corrective maintenance* is the reactive modification of software to repair problems

2 https://www.bugzilla.org/.

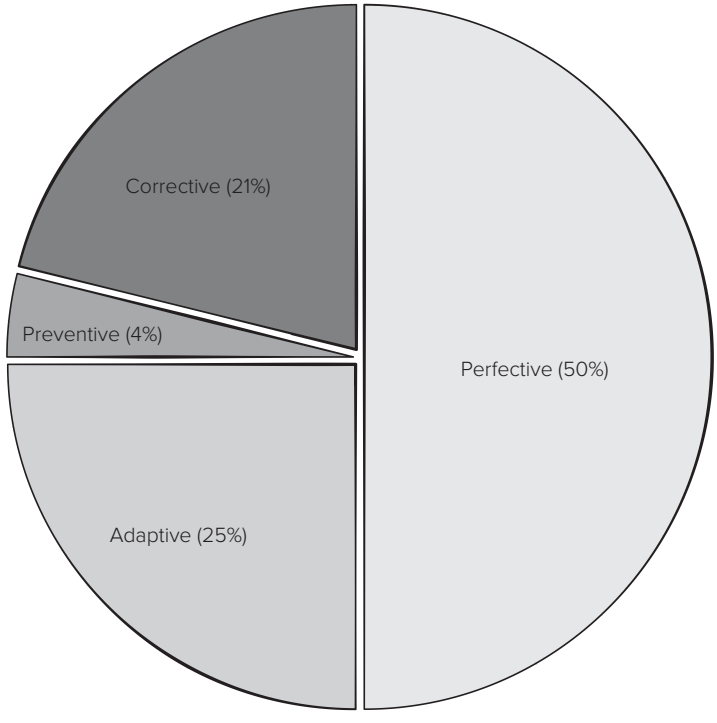
3 https://www.atlassian.com/software/jira.

discovered after the software has been delivered to a customer's end user. *Adaptive maintenance* is reactive modification of software after delivery to keep the software usable in a changing end-user environment. *Perfective maintenance* is the proactive modification of the software after delivery to provide new user features, better program code structure, or improved documentation. *Preventive maintenance* is the proactive modification of the software after delivery to detect and correct product faults before they are discovered by users in the field [SWEBOK[4]]. Proactive maintenance can be scheduled and planned for. Reactive maintenance is often described as *firefighting* because it cannot be planned for and must be attended immediately for software systems that are critical to the success of the end-users' activities. Figure 4.5 shows that only 21 percent of the developers' time is typically spent on corrective maintenance.

For an agile evolutionary process model like the one described in this chapter, developers release working partial solutions with the creation of each incremental prototype. Much of the engineering work done is preventive or perfective maintenance as new features are added to the evolving software system. It is tempting to think that maintenance is handled simply by planning to make another trip around the spiral. But software problems cannot always be anticipated, so repairs may need to be made quickly and developers may be tempted to cut corners when trying to repair the broken

FIGURE 4.5

Distribution of maintenance effort



4 SWEBOK refers to the Software Engineering Body of Knowledge, which can be accessed using the following link: https://www.computer.org/web/swebok/v3.

software. Developers may not want to spend time on risk assessment or planning. Yet, developers cannot afford to make changes to software without considering the possibility that the changes required to fix one problem will cause new problems to other portions of the program.

It is important to understand the program code before making changes to it. If developers have documented the code, it will be easier to understand if other people need to do the maintenance work. If the software is designed to be extended, maintenance may also be accomplished more easily, other than emergency defect repairs. It is essential to test the modified software carefully to ensure software changes have their intended effect and do not break the software in other places.

The task of creating software products that are easy to support and easy to maintain requires careful and thoughtful engineering. A more detailed discussion on the task of maintaining and supporting software after delivery is presented in Chapter 27.

Recommended Software Process Steps

1. Requirements engineering
   - Gather user stories from all stakeholders.
   - Have stakeholders describe acceptance criteria user stories.
2. Preliminary architectural design
   - Make use of paper prototypes and models.
   - Assess alternatives using nonfunctional requirements.
   - Document architecture design decisions.
3. Estimate required project resources
   - Use historic data to estimate time to complete each user story.
   - Organize the user stories into sprints.
   - Determine the number of sprints needed to complete the product.
   - Revise the time estimates as use stories are added or deleted.
4. Construct first prototype
   - Select subset of user stories most important to stakeholders.
   - Create paper prototype as part of the design process.
   - Design a user interface prototype with inputs and outputs.
   - Engineer the algorithms needed for first prototypes.
   - Prototype with deployment in mind.
5. Evaluate prototype
   - Create test cases while prototype is being designed.
   - Test prototype using appropriate users.
   - Capture stakeholder feedback for use in revision process.
6. Go, no-go decision
   - Determine the quality of the current prototype.
   - Revise time and cost estimates for completing development.
   - Determine the risk of failing to meet stakeholder expectations.
   - Get commitment to continue development.
7. Evolve system
   - Define new prototype scope.
   - Construct new prototype.
   - Evaluate new prototype and include regression testing.
   - Assess risks associated with continuing evolution.
8. Release prototype
   - Perform acceptance testing.
   - Document defects identified.
   - Share quality risks with management.
9. Maintain software
   - Understand code before making changes.
   - Test software after making changes.
   - Document changes.
   - Communicate known defects and risks to all stakeholders.

4.10 Summary

Every project is unique, and every development team is made up of unique individuals. Every software project needs a road map, and the process of developing software requires a predictable set of basic tasks (communication, planning, modeling, construction, and deployment). However, these tasks should not be performed in isolation and may need to be adapted to meet the needs of each new project. In this chapter, we suggested the use of a highly interactive, incremental prototyping process. We think this is better than producing rigid product plans and large documents prior to doing any programming. Requirements change. Stakeholder input and feedback should occur early and often in the development process to ensure the delivery of useful product.

We suggest the use of an evolutionary process model that emphasizes frequent stakeholder involvement in the creation and evaluation of incremental software prototypes. Limiting requirements engineering artifacts to the set of minimal useful documents and models allows the early production of prototypes and test cases. Planning to create evolutionary prototypes reduces the time lost repeating the work needed to create throwaway prototypes. Making use of paper prototypes early in the design process can also help to avoid programming products that do not satisfy customer expectations. Getting the architectural design right before beginning actual development is also important to avoiding schedule slippage and cost overruns.

Planning is important but should be done expeditiously to avoid delaying the start of development. Developers should have a general idea about how long a project will take to complete, but they need to recognize that they are not likely to know all the project requirements until the software products are delivered. Developers would be wise to avoid detailed planning that extends beyond planning the current prototype. The developers and stakeholders should adopt a process for adding features to be implemented in future prototypes and to assess the impact of these changes on the project schedule and budget.

Risk assessment and acceptance testing are an important part of the prototype assessment process. Having an agile philosophy about managing requirements and adding new features to the final product is important as well. The biggest challenges developers have with evolutionary process models is managing scope creep while delivering a product that meets customer expectations and doing all this while delivering the product on time and within budget. That's what makes software engineering so challenging and rewarding.

Problems and Points to Ponder

4.1. How does the Extreme Programming (XP) model differ from the spiral model in its treatment of incremental prototypes?

4.2. Write the acceptance criteria for the user story that describe the use of the "favorite places" or "favorites" feature found on most Web browsers that you wrote for Problem 3.7 in Chapter 3.

4.3. How would you create a preliminary architectural design for the first prototype for a mobile app that lets you create and save a shopping list on your device?

4.4. Where would you get the historic date needed to estimate the development time for the user stories in a prototype before it is written?

4.5. Create a series of sketches representing the key screens for a paper prototype for the shopping list app you created in Problem 4.3.

4.6. How can you test the viability of the paper prototype you created for Problem 4.5?

4.7. What data points are needed to make the go, no-go decision during the assessment of an evolutionary prototype?

4.8. What is the difference between reactive and proactive maintenance?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

5 HUMAN ASPECTS OF SOFTWARE ENGINEERING

In a special issue of *IEEE Software*, the guest editors [deS09] make the following observation:

> Software engineering has an abundance of techniques, tools, and methods designed to improve both the software development process and the final product. However, software isn't simply a product of the appropriate technical solutions applied in appropriate technical ways. Software is developed by people, used by people, and supports interaction among people. As such, human characteristics, behavior, and cooperation are central to practical software development.
>
> Without a team of skilled and motivated people, success is unlikely.

KEY CONCEPTS

agile team 78
global teams 80
jelled team 76
psychology, software engineering 75
social media 79
team attributes 77
team structures 78
team toxicity 77
traits, software engineer 75

QUICK LOOK

What is it? At the end of the day, *people* build computer software. The human aspects of software engineering often have as much to do with the success of a project as the latest and greatest technology.

Who does it? Individuals and teams do software engineering work. In some cases, one person has much of the responsibility, but in most industry-grade software efforts, a team of people does the work.

Why is it important? A software team will be successful only if the dynamics of the team are right. It is essential for software engineers on a team to play well with their colleagues and with other product stakeholders.

What are the steps? First, you need to try to emulate personal characteristics of successful software engineers. Next, you should appreciate the complex psychology of software engineering work so that you can navigate your way through a project without peril. Then, you need to understand the structure and dynamics of software teams. Finally, you should appreciate the impact of social media, the cloud, and other collaborative tools.

What is the work product? Better insight into the people, the process, and the product.

How do I ensure that I've done it right? Spend the time to observe how successful software engineers do their work, and tune your approach to take advantage of the strengths they project.

5.1 CHARACTERISTICS OF A SOFTWARE ENGINEER

So you want to be a software engineer? Obviously, you need to master the technical stuff, learn the skills required to understand the problem, design an effective solution, build the software, and test it in an effort to develop the highest-quality products possible. You need to manage change, communicate with stakeholders, and use appropriate tools as needed. We will discuss these things at great length later in this book.

But there are other things that are equally important—the human aspects that will make you effective as a software engineer. Erasmus [Era09] identifies seven traits that are present when a software engineer exhibits "superprofessional" behavior.

An effective software engineer has a sense of *individual responsibility*. This implies a drive to deliver on her promises to peers, stakeholders, and her management. It implies that she will do what needs to be done, when it needs to be done in an overriding effort to achieve a successful outcome.

An effective software engineer has an *acute awareness* of the needs of other team members, the stakeholders requesting changes to an existing software solution, and the managers who have overall control of the project. He observes the environment in which people work and adapts his behavior to take both into account.

An effective software engineer is *brutally honest*. If she sees a flawed design, she points out the flaws in a constructive but honest manner. If asked to distort facts about schedules, features, performance, or other product or project characteristics, she opts to be realistic and truthful.

An effective software engineer exhibits *resilience under pressure.* Software engineering is always on the edge of chaos. Pressure comes in many forms—changes in requirements and priorities, demanding stakeholders, and overbearing managers. An effective software engineer manages pressure so that his performance does not suffer.

An effective software engineer has a *heightened sense of fairness.* She gladly shares credit with her colleagues. She tries to avoid conflicts of interest and never acts to sabotage the work of others.

An effective software engineer exhibits *attention to detail.* This does not imply an obsession with perfection. He carefully considers the broader criteria (e.g., performance, cost, quality) that have been established for the product and the project in making his daily technical decisions.

Finally, an effective software engineer is pragmatic. She recognizes that software engineering is not a religion in which dogmatic rules must be followed, but rather a discipline that can be adapted based on the circumstances at hand.

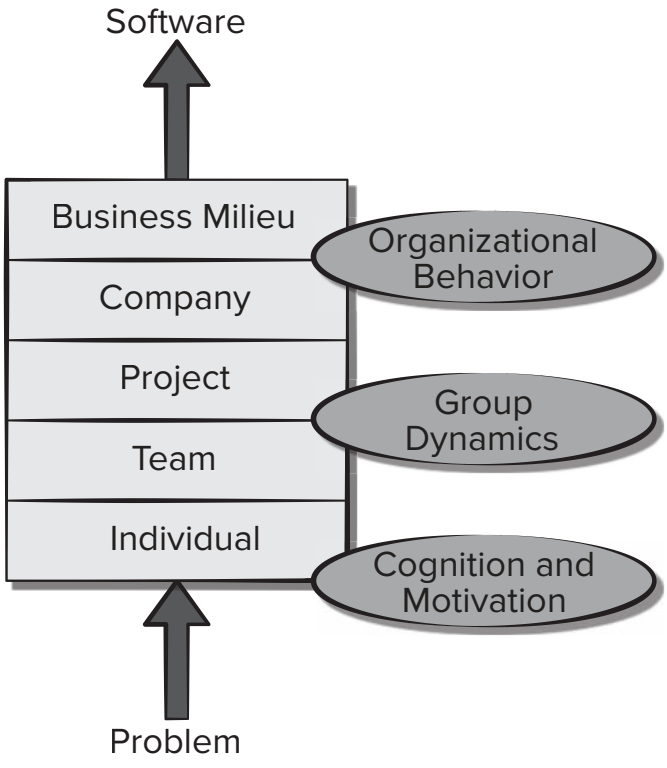
5.2 THE PSYCHOLOGY OF SOFTWARE ENGINEERING

In a seminal paper on the psychology of software engineering, Bill Curtis and Diane Walz [Cur90] suggest a layered behavioral model for software development (Figure 5.1). At an individual level, software engineering psychology focuses on recognition of the problem to be solved, the problem-solving skills required to solve it, and the motivation to complete the solution within the constraints established by outer layers in the model. At the team and project levels, group dynamics becomes the

FIGURE 5.1

A layered behavioral model for software engineering (adapted from [Cur90])

Source: Adapted from Curtis, Bill, and Walz, Diane, "The Psychology of Programming in the Large: Team and Organizational Behavior," Psychology of Programming, Academic Press, 1990.



dominating factor. Here, team structure and social factors govern success. Group communication, collaboration, and coordination are as important as the skills of an individual team member. At the outer layers, organizational behavior governs the actions of the company and its response to the business milieu.

5.3 THE SOFTWARE TEAM

In their classic book *Peopleware,* Tom DeMarco and Tim Lister [DeM98] discuss the cohesiveness of a software team:

> We tend to use the word *team* loosely in the business world, calling any group of people assigned to work together a "team." But many of these groups just don't behave like teams. They may not have a common definition of success or any identifiable team spirit. What is missing is a phenomenon that we call *jell.*
>
> A jelled team is a group of people so strongly knit that the whole is greater than the sum of the parts. . . .
>
> Once a team begins to jell, the probability of success goes way up. The team can become unstoppable, a juggernaut for success. . . . They don't need to be managed in the traditional way, and they certainly don't need to be motivated. They've got momentum.

DeMarco and Lister contend that members of jelled teams are significantly more productive and more motivated than average. They share a common goal, a common culture, and in many cases, a "sense of eliteness" that makes them unique.

There is no foolproof method for creating a jelled team. But there are attributes that are normally found in effective software teams.[1] Miguel Carrasco [Car08] suggests that an effective software team must establish a *sense of purpose*. An effective team must also inculcate a *sense of involvement* that allows every member to feel that his skill set and contributions are valued.

An effective team should foster a *sense of trust.* Software engineers on the team should trust the skills and competence of their peers and their managers. The team should encourage a *sense of improvement,* by periodically reflecting on its approach to software engineering and looking for ways to improve their work.

The most effective software teams are diverse in the sense that they combine a variety of different skill sets. Highly skilled technologists are complemented by members who may have less technical background but are more empathetic to the needs of stakeholders.

But not all teams are effective and not all teams jell. In fact, many teams suffer from what Jackman [Jac98] calls "team toxicity." She defines five factors that "foster a potentially toxic team environment": (1) a frenzied work atmosphere, (2) high frustration that causes friction among team members, (3) a "fragmented or poorly coordinated" software process, (4) an unclear definition of roles on the software team, and (5) "continuous and repeated exposure to failure."

To avoid a frenzied work environment, the team should have access to all information required to do the job. Major goals and objectives, once defined, should not be modified unless absolutely necessary. A software team can avoid frustration if it is given as much responsibility for decision making as possible. An inappropriate process (e.g., unnecessary or burdensome work tasks or poorly chosen work products) can be avoided by understanding the product to be built and the people doing the work and by allowing the team to select the process model. The team itself should establish its own mechanisms for accountability (technical reviews[2] are an excellent way to accomplish this) and define a series of corrective approaches when a member of the team fails to perform. And finally, the key to avoiding an atmosphere of failure is to establish team-based techniques for feedback and problem solving.

In addition to the five toxins described by Jackman, a software team often struggles with the differing human traits of its members. Some people gather information intuitively, distilling broad concepts from disparate facts. Others process information linearly, collecting and organizing minute details from the data provided. Some team members are comfortable making decisions only when a logical, orderly argument is presented. Others are intuitive, willing to make a decision based on "feel." Some work

1 Bruce Tuckman observes that successful teams go through four phases (forming, storming, norming, and performing) on their way to becoming productive (http://en.wikipedia.org/wiki/Tuckman%27s_stages_of_group_development).

2 Technical reviews are discussed in detail in Chapter 16.

hard to get things done long before a milestone date, thereby avoiding stress as the date approaches, while others are energized by the rush to make a last-minute deadline. Recognition of human differences, along with other guidelines presented in this section, provide a higher likelihood of creating teams that jell.

5.4 TEAM STRUCTURES

The "best" team structure depends on the management style of your organization, the number of people who will populate the team and their skill levels, and the overall problem difficulty. Mantei [Man81] describes a number of project factors that should be considered when planning the structure of software engineering teams: (1) difficulty of the problem to be solved, (2) "size" of the resultant program(s) in lines of code or function points,[3] (3) time that the team will stay together (team lifetime), (4) degree to which the problem can be modularized, (5) required quality and reliability of the system to be built, (6) rigidity of the delivery date, and (7) degree of sociability (communication) required for the project.

Over the past decade, agile software development (Chapter 3) has been suggested as an antidote to many of the problems that have plagued software project work. The agile philosophy encourages customer satisfaction and early incremental delivery of software, small highly motivated project teams, informal methods, minimal software engineering work products, and overall development simplicity.

A small, highly motivated project team, also called an *agile team,* adopts many of the characteristics of successful software project teams discussed in Section 5.3 and avoids many of the toxins that create problems. However, the agile philosophy stresses individual (team member) competency, coupled with group collaboration as critical success factors for the team. Channeling creative energy into a high-performance team must be a central goal of a software engineering organization. Cockburn and Highsmith [Coc01a] suggest that "good" software people can work within the framework of any software process, and weak performers will struggle regardless. The bottom line, they contend, is that "people trump process" but that even good people can be hampered by an ill-defined process and poor resource support. We agree.

To make effective use of the competencies of each team member and to foster effective collaboration through a software project, agile teams are *self-organizing.* A self-organizing team does not necessarily maintain a single team structure. The team makes the changes needed to its structure to respond to the changes in the development environment or changes in the evolving engineering problem solution.

Communication between team members and all project stakeholders is essential. Agile teams often have customer representatives as team members. This fosters respect among the developers and stakeholders, as well as providing avenues for timely and frequent feedback on the evolving products.

3 Lines of code (LOC) and function points are measures of the size of a computer program and are discussed in Chapter 24.

SAFEHOME



Team Structure

The scene: Doug Miller's office prior to the initiation of the *SafeHome* software project.

The players: Doug Miller (manager of the *SafeHome* software engineering team), Vinod Raman, Jamie Lazar, and other members of the product software engineering team.

The conversation:

Doug: Have you guys had a chance to look over the preliminary info on *SafeHome* that marketing has prepared?

Vinod (nodding and looking at his teammates): Yes. But we have a bunch of questions.

Doug: Let's hold on to that for a moment. I'd like to talk about how we're going to structure the team, who's responsible for what . . .

Jamie: I'm really into the agile philosophy, Doug. I think we should be a self-organizing team.

Vinod: I agree. Given the tight time line and some of the uncertainty, and the fact that we're all really competent [laughs], that seems like the right way to go.

Doug: That's okay with me, but you guys know the drill.

Jamie (smiling and talking as if she was reciting something): We make tactical decisions, about who does what and when, but it's our responsibility to get product out the door on time.

Vinod: And with quality.

Doug: Exactly. But remember there are constraints. Marketing defines the software increments to be produced—in consultation with us, of course.

5.5 THE IMPACT OF SOCIAL MEDIA

E-mail, texting, and videoconferencing have become ubiquitous activities in software engineering work. But these communication mechanisms are really nothing more than modern substitutes or supplements for the face-to-face contact. Social media is different.

Begel [Beg10] and his colleagues address the growth and application of social media in software engineering when they write:

> The social processes around software development are . . . highly dependent on engineers' abilities to find and connect with individuals who share similar goals and complementary skills, to harmonize each team member's communication and teaming preferences, to collaborate and coordinate during the entire software lifecycle, and advocate for their product's success in the marketplace.

In some ways, this "connection" can be as important as face-to-face communication. The value of social media grows as team size increases and is magnified further when the team is geographically dispersed.

Social networking tools (e.g., Facebook, LinkedIn, Slack, Twitter) allow degrees-of-separation connections among software developers and related technologists. This allows "friends" on a social networking site to learn about friends of friends who may have knowledge or expertise related to the application domain or problem to be solved. Specialized private networks built on the social networking paradigm can be used within an organization.

It is very important to note that privacy and security issues should not be overlooked when using social media for software engineering work. Much of the work performed by software engineers may be proprietary to their employer and disclosure could be very harmful. For that reason, the distinct benefits of social media must be weighed against the threat of uncontrolled disclosure of private information.

5.6 Global Teams

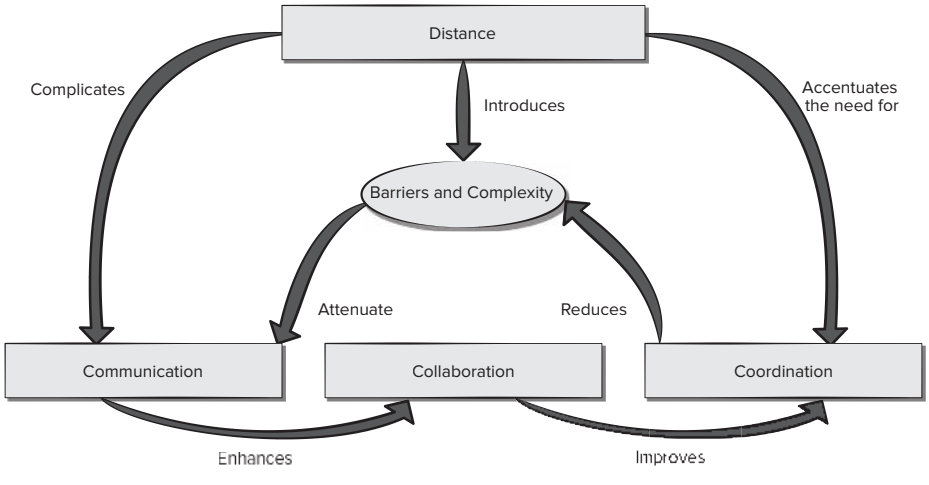
In the software domain, globalization implies more than the transfer of goods and services across international boundaries. For the past few decades, an increasing number of major software products have been built by software teams that are often located in different countries. These global software development (GSD) teams have unique challenges that include coordination, collaboration, communication, and specialized decision making. Approaches to coordination, collaboration, and communication are influenced by the team structure that has been established. Decision making on all software teams is complicated by four factors [Gar10a]:

- Complexity of the problem.
- Uncertainty and risk associated with the decision.
- The law of unintended consequences (i.e., a work-associated decision has an unintended effect on another project objective).
- Different views of the problem that lead to different conclusions about the way forward.

For a GSD team, the challenges associated with coordination, collaboration, and communication can have a profound effect on decision making. Figure 5.2 illustrates the impact of distance on the challenges that face a GSD team. Distance complicates

FIGURE 5.2

Factors affecting a GSD team



communication, but, at the same time, accentuates the need for coordination. Distance also introduces barriers and complexity that can be driven by cultural differences. Barriers and complexity attenuate communication (i.e., the signal-to-noise ratio decreases). The problems inherent in this dynamic can result in a project that becomes unstable.

5.7 SUMMARY

A successful software engineer must have technical skills. But, in addition, he must take responsibility for his commitments, be aware of the needs of his peers, be honest in his assessment of the product and the project, be resilient under pressure, treat his peers fairly, and exhibit attention to detail.

The psychology of software engineering includes individual cognition and motivation, the group dynamics of a software team, and the organizational behavior of the company. A successful ("jelled") software team is more productive and motivated than average. To be effective, a software team must have a sense of purpose, a sense of involvement, a sense of trust, and a sense of improvement. In addition, the team must avoid "toxicity" that is characterized by a frenzied and frustrating work atmosphere, an inappropriate software process, an unclear definition of roles on the software team, and continuous exposure to failure.

Agile teams subscribe to the agile philosophy and generally have more autonomy than more conventional software teams with rigid member roles and external management control. Agile teams emphasize communication, simplicity, feedback, courage, and respect.

Social media tools are becoming an integral part of many software projects, providing services that enhance communication and collaboration for a software team. Social media and electronic communication are particularly useful for global software development where geographic separation can precipitate barriers to successful software engineering.

PROBLEMS AND POINTS TO PONDER

5.1. Based on your personal observation of people who are excellent software developers, name three personality traits that appear to be common among them.

5.2. How can you be "brutally honest" and still not be perceived (by others) as insulting or aggressive?

5.3. How does a software team construct "artificial boundaries" that reduce their ability to communicate with others?

5.4. Write a scenario in which the *SafeHome* team members make use of one or more forms of social media as part of their software project.

5.5. Referring to Figure 5.2, why does distance complicate communication? Why does distance accentuate the need for coordination? What types of barriers and complexities are introduced by distance?

Design element: Quick Look icon magnifying glass: © Roger Pressman

PART

Two

MODELING

In this part of *Software Engineering: A Practitioner's Approach,* you'll learn about the principles, concepts, and methods that are used to create high-quality requirements and design models. These questions are addressed in the chapters that follow:

- What concepts and principles guide software engineering practice?
- What is requirements engineering, and what are the underlying concepts that lead to good requirements analysis?
- How is the requirements model created, and what are its elements?
- What are the elements of a good design?
- How does architectural design establish a framework for all other design actions, and what models are used?
- How do we design high-quality software components?
- What concepts, models, and methods are applied as the user experience is designed?
- What is pattern-based design?
- What specialized strategies and methods are used to design mobile apps?

Once these questions are answered, you'll be better prepared to apply software engineering practice.

CHAPTER

6 Principles That Guide Practice

Software engineers are often depicted as working long hours by themselves to meet impossible deadlines without connecting to other people. This is a dark image of software engineering practice to be sure. Yet, as we discussed in the previous chapters, most software engineers work on teams and frequently interact with their stakeholders. If you search for surveys of technical professionals on the Internet, you will see software engineers listed among those experiencing the greatest satisfaction from their jobs.

Key Concepts

coding principles 96
communication principles 88
core principles 85
deployment principles 98
modeling principles 92
planning principles 91
practice 85
process 85
testing principles 96

Quick Look

What is it? Software engineering practice is a broad array of principles, concepts, methods, and tools that you must consider as software is planned and developed. Principles that guide practice establish a foundation from which software engineering is conducted.

Who does it? Practitioners (software engineers) and their managers conduct a variety of software engineering tasks.

Why is it important? Software process provides everyone involved in the creation of a computer-based system or product with a road map for getting to a successful destination. Software practice provides you with the details you'll need to drive along the road. It tells you where the bridges, the roadblocks, and the forks are located. It helps you understand the concepts and principles that must be understood and followed to drive safely and rapidly. It instructs you on how to drive, where to slow down, and where to speed up. In the context of software engineering, practice is what you do day in and day out as software evolves from an idea to a reality.

What are the steps? Four elements of practice apply regardless of the process model that is chosen. They are principles, concepts, methods, and tools. Tools support the application of methods.

What is the work product? Practice encompasses the technical activities that produce all work products that are defined by the software process model that has been chosen.

How do I ensure that I've done it right? First, have a firm understanding of the principles that apply to the work (e.g., design) that you're doing at the moment. Then, be certain that you've chosen an appropriate method for the work, be sure that you understand how to apply the method, use automated tools when they're appropriate for the task, and be adamant about the need for techniques to ensure the quality of work products that are produced. You also need to be agile enough to make changes to your plans and methods as needed.

People who create computer software practice the art or craft or discipline[1] that is software engineering. But what is software engineering "practice"? In a generic sense, *practice* is a collection of concepts, principles, methods, and tools that a software engineer calls upon on a daily basis. Practice allows managers to manage software projects and software engineers to build computer programs. Practice populates a software process model with the necessary technical and management how-to's to get the job done. Practice transforms a haphazard unfocused approach into something that is more organized, more effective, and more likely to achieve success.

Various aspects of software engineering practice will be examined throughout the remainder of this book. In this chapter, our focus is on principles and concepts that guide software engineering practice in general.

6.1 Core Principles

Software engineering is guided by a collection of core principles that help in the application of a meaningful software process and the execution of effective software engineering methods. At the process level, core principles establish a philosophical foundation that guides a software team as it performs the framework and umbrella activities and produces a set of software engineering work products. At the practice level, core principles establish a collection of values and rules that can guide you in analyzing a problem, designing a solution, implementing and testing the solution, and ultimately deploying the software so stakeholders can use it.

6.1.1 Principles That Guide Process

In Part One of this book we discussed the importance of the software process and described several process models that have been proposed for software engineering work. Every project is unique, and every team is unique. That means you must adapt your process to best fit your needs. Regardless of the process model your team adopts, it contains elements of the generic process framework we described in Chapter 1. The following set of core principles can be applied to this framework and to every software process. A simplified view of this framework is shown in Figure 6.1.

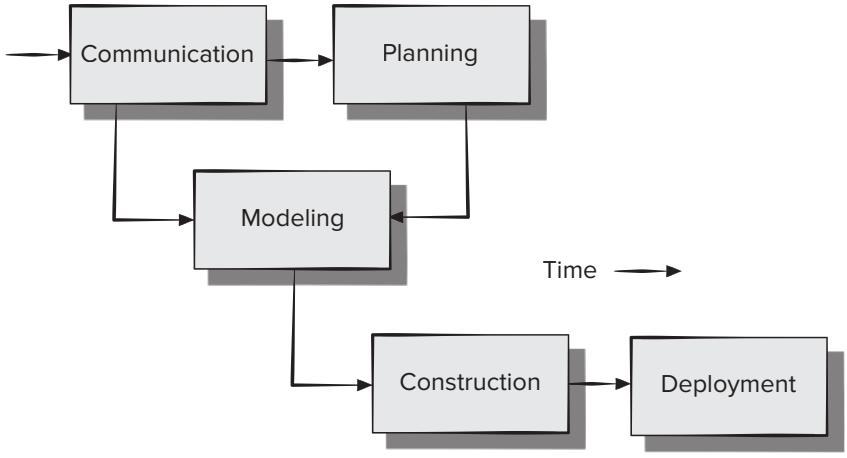
Principle 1. ***Be agile.*** Whether the process model you choose is prescriptive or agile, the basic tenets of agile development should govern your approach. Every aspect of the work you do should emphasize economy of action—keep your technical approach as simple as possible, keep the work products you produce as concise as possible, and make decisions locally whenever possible.

Principle 2. ***Focus on quality at every step.*** The exit condition for every process activity, action, and task should focus on the quality of the work product that has been produced.

Principle 3. ***Be ready to adapt.*** Process is not a religious experience, and dogma has no place in it. When necessary, adapt your approach to constraints imposed by the problem, the people, and the project itself.

1 Some writers argue for one of these terms to the exclusion of the others. In reality, software engineering is all three.

FIGURE 6.1
Simplified process framework



Principle 4. ***Build an effective team.*** Software engineering process and practice are important, but the bottom line is people. Build a self-organizing team that has mutual trust and respect.[2]

Principle 5. ***Establish mechanisms for communication and coordination.*** Projects fail because important information falls into the cracks and/or stakeholders fail to coordinate their efforts to create a successful end product. These are management issues, and they must be addressed.

Principle 6. ***Manage change.*** The approach may be either formal or informal, but mechanisms must be established to manage the way changes are requested, assessed, approved, and implemented.

Principle 7. ***Assess risk.*** Lots of things can go wrong as software is being developed. It's essential that you establish contingency plans. Some of these contingency plans will form the basis for security engineering tasks (Chapter 18).

Principle 8. ***Create work products that provide value for others.*** Create only those work products that provide value for other process activities, actions, or tasks. Every work product that is produced as part of software engineering practice will be passed on to someone else. Be sure that the work product imparts the necessary information without ambiguity or omission.

Part Four of this book focuses on project and process management issues and considers various aspects of each of these principles in some detail.

6.1.2 Principles That Guide Practice

Software engineering practice has a single overriding goal: *to deliver on-time, high-quality, operational software that contains functions and features that meet the needs of all stakeholders*. To achieve this goal, you should adopt a set of core principles that guide your technical work. These principles have merit regardless of the analysis

2 The characteristics of effective software teams were discussed in Chapter 5.

and design methods that you apply, the construction techniques (e.g., programming language, automated tools) that you use, or the verification and validation approach that you choose. The following set of core principles is fundamental to the practice of software engineering:

Principle 1. ***Divide and conquer.*** Stated in a more technical manner, analysis and design should always emphasize *separation of concerns* (SoCs). A large problem is easier to solve if it is subdivided into a collection of elements (or *concerns*).

Principle 2. ***Understand the use of abstraction.*** At its core, an abstraction is a simplification of some complex element of a system used to communicate meaning in a single phrase. When we use the abstraction *spreadsheet*, it is assumed that you understand what a spreadsheet is, the general structure of content that a spreadsheet presents, and the typical functions that can be applied to it. In software engineering practice, you use many different levels of abstraction, each imparting or implying meaning that must be communicated. In analysis and design work, a software team normally begins with models that represent high levels of abstraction (e.g., a spreadsheet) and slowly refines those models into lower levels of abstraction (e.g., a *column* or the *SUM* function).

Principle 3. ***Strive for consistency.*** Whether it's creating an analysis model, developing a software design, generating source code, or creating test cases, the principle of consistency suggests that a familiar context makes software easier to use. As an example, consider the design of a user interface for a mobile app. Consistent placement of menu options, the use of a consistent color scheme, and the consistent use of recognizable icons help to create a highly effective user experience.

Principle 4. ***Focus on the transfer of information.*** Software is about information transfer—from a database to an end user, from a legacy system to a WebApp, from an end user into a graphic user interface (GUI), from an operating system to an application, from one software component to another—the list is almost endless. In every case, information flows across an interface, and this means there are opportunities for errors, omissions, or ambiguity. The implication of this principle is that you must pay special attention to the analysis, design, construction, and testing of interfaces.

Principle 5. ***Build software that exhibits effective modularity.*** Separation of concerns (Principle 1) establishes a philosophy for software. *Modularity* provides a mechanism for realizing the philosophy. Any complex system can be divided into modules (components), but good software engineering practice demands more. Modularity must be *effective*. That is, each module should focus exclusively on one well-constrained aspect of the system. Additionally, modules should be interconnected in a relatively simple manner to other modules, to data sources, and to other environmental aspects.

Principle 6. ***Look for patterns.*** Software engineers use patterns as a means of cataloging and reusing solutions to problems they have encountered in the past. The use of these design patterns can be applied to wider systems engineering and systems integration problems, by allowing components in complex systems to evolve independently. Patterns will be discussed further in Chapter 14.

Principle 7. ***When possible, represent the problem and its solution from several different perspectives.*** When a problem and its solution are examined from different

perspectives, it is more likely that greater insight will be achieved and that errors and omissions will be uncovered. The unified modeling language (UML) provides a means of describing a problem solution from multiple viewpoints, as described in Appendix 1.

Principle 8. ***Remember that someone will maintain the software.*** Over the long term, software will be corrected as defects are uncovered, adapted as its environment changes, and enhanced as stakeholders request more capabilities. These maintenance activities can be facilitated if solid software engineering practice is applied throughout the software process.

These principles are not all you'll need to build high-quality software, but they do establish a foundation for every software engineering method discussed in this book.

6.2 Principles That Guide Each Framework Activity

In the sections that follow, we consider principles that have a strong bearing on the success of each generic framework activity defined as part of the software process. In many cases, the principles that are discussed for each of the framework activities are a refinement of the principles presented in Section 6.1. They are simply core principles stated at a lower level of abstraction.

6.2.1 Communication Principles

Before customer requirements can be analyzed, modeled, or specified, they must be gathered through the communication activity. A customer has a problem that may be amenable to a computer-based solution. You respond to the customer's request for help. Communication has begun. But the road from communication to understanding is often full of potholes.

Effective communication (among technical peers, with the customer and other stakeholders, and with project managers) is among the most challenging activities that you will confront. There are many ways to communicate, but it's important to recognize that not all are equal in richness or effectiveness (Figure 6.2). In this context, we discuss communication principles as they apply to customer communication. However, many of the principles apply equally to all forms of communication that occur within a software project.

INFO



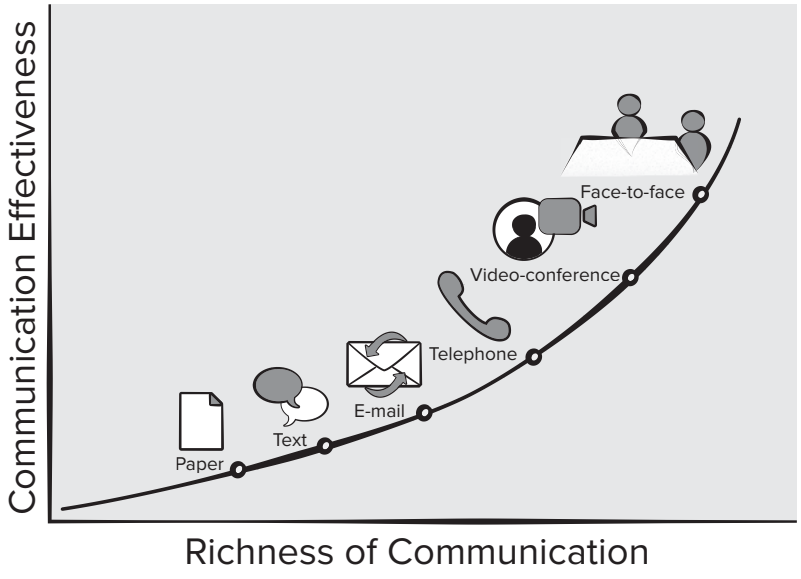
The Difference Between Customers and End Users

A *customer* is the person or group who (1) originally requested the software to be built, (2) defines overall business objectives for the software, (3) provides basic product requirements, and (4) coordinates funding for the project. In a product or system business, the customer is often the marketing group. In an information technology (IT) environment, the customer might be a business component or department.

An *end user* is the person or group who (1) will actually use the software that is built to achieve some business purpose and (2) will define operational details of the software so the business purpose can be achieved. In some cases, the customer and the end user may be one and the same, but for many projects that is not the case.

FIGURE 6.2

Effectiveness of communication modes



Principle 1. ***Listen.*** Before communicating, be sure you understand the point of view of the other party, know a bit about his or her needs, and then listen. Try to focus on the speaker's words, rather than formulating your response to those words. Ask for clarification if something is unclear, and avoid constant interruptions. *Never* become contentious in your words or actions (e.g., rolling your eyes or shaking your head) as a person is talking.

Principle 2. ***Prepare before you communicate.*** Spend the time to understand the problem before you meet with others. If necessary, do some research to understand business domain jargon. If you have responsibility for conducting a meeting, prepare an agenda in advance of the meeting.

Principle 3. ***Someone should facilitate the activity.*** Every communication meeting should have a leader (a facilitator) to keep the conversation moving in a productive direction, (2) to mediate any conflict that does occur, and (3) to ensure that other principles are followed.

Principle 4. ***Face-to-face communication is best.*** However, this communication usually works better when some other representation of the relevant information is present. For example, a participant may create a drawing or a "strawman" document that serves as a focus for discussion.

Principle 5. ***Take notes and document decisions.*** Things have a way of falling into the cracks. Someone participating in the communication should serve as a "recorder" and write down all important points and decisions.

Principle 6. ***Strive for collaboration.*** Collaboration and consensus occur when the collective knowledge of members of the team is used to describe product or system functions or features. Each small collaboration serves to build trust among team members and creates a common goal for the team.

Principle 7. ***Stay focused; modularize your discussion.*** The more people are involved in any communication, the more likely that discussion will bounce from one topic to the next. The facilitator should keep the conversation modular, leaving one topic only after it has been resolved (however, see Principle 9).

Principle 8. ***If something is unclear, draw a picture.*** Verbal communication goes only so far. A sketch or drawing can often provide clarity when words fail to do the job.

Principle 9. ***(a) Once you agree to something, move on. (b) If you can't agree to something, move on. (c) If a feature or function is unclear and cannot be clarified right now, move on.*** Communication, like any software engineering activity, takes time. Rather than iterating endlessly, the people who participate should recognize that many topics require discussion (see Principle 2) and that "moving on" is sometimes the best way to achieve communication agility.

Principle 10. ***Negotiation is not a contest or a game. It works best when both parties win.*** There are many instances in which you and other stakeholders must negotiate functions and features, priorities, and delivery dates. If the team has collaborated well, all parties have a common goal. Still, negotiation will demand compromise from all parties.

SAFEHOME



Communication Mistakes

The scene: Software engineering team workspace

The players: Jamie Lazar, software team member; Vinod Raman, software team member; Ed Robbins, software team member.

The conversation:

Ed: What have you heard about this *SafeHome* project?

Vinod: The kickoff meeting is scheduled for next week.

Jamie: I've already done a little bit of investigation, but it didn't go well.

Ed: What do you mean?

Jamie: Well, I gave Lisa Perez a call. She's the marketing honcho on this thing.

Vinod: And . . . ?

Jamie: I wanted her to tell me about *SafeHome* features and functions . . . that sort of thing. Instead, she began asking me questions about security systems, surveillance systems . . . I'm no expert.

Vinod: What does that tell you?

(Jamie shrugs)

Vinod: That marketing will need us to act as consultants and that we'd better do some homework on this product area before our kickoff meeting. Doug said that he wanted us to "collaborate" with our customer, so we'd better learn how to do that.

Ed: Probably would have been better to stop by her office. Phone calls just don't work as well for this sort of thing.

Jamie: You're both right. We've got to get our act together or our early communications will be a struggle.

Vinod: I saw Doug reading a book on "requirements engineering." I'll bet that lists some principles of good communication. I'm going to borrow it from him.

Jamie: Good idea . . . then you can teach us.

Vinod (smiling): Yeah, right.

6.2.2 Planning Principles

The planning activity encompasses a set of management and technical practices that enable the software team to define a road map as it travels toward its strategic goal and tactical objectives.

Try as we might, it's impossible to predict exactly how a software project will evolve. There is no easy way to determine what unforeseen technical problems will be encountered, what important information will remain undiscovered until late in the project, what misunderstandings will occur, or what business issues will change. And yet, a good software team must plan its approach. Often planning is iterative (Figure 6.3).

There are different planning philosophies.[3] Some people are "minimalists," arguing that change often obviates the need for a detailed plan. Others are "traditionalists," arguing that the plan provides an effective road map and the more detail it has, the less likely the team will become lost.

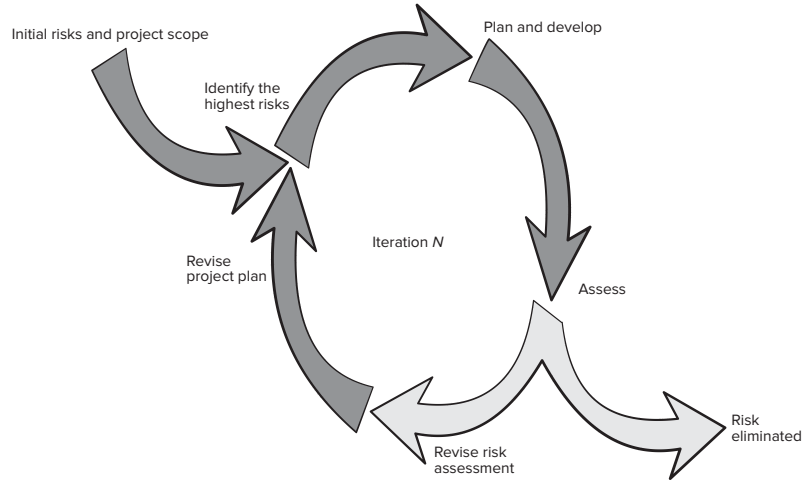
What to do? On many projects, overplanning is time consuming and fruitless (too many things change), but underplanning is a recipe for chaos. Like most things in life, planning should be agile and conducted in moderation, enough to provide useful guidance for the team—no more, no less. Regardless of the rigor with which planning is conducted, the following principles always apply:

Principle 1. ***Understand the scope of the project.*** It's impossible to use a road map if you don't know where you're going. Scope provides the software team with a destination.

Principle 2. ***Involve stakeholders in the planning activity.*** Stakeholders define priorities and establish project constraints. To accommodate these realities, software engineers must often negotiate order of delivery, time lines, and other project-related issues.

FIGURE 6.3

Iterative planning



3 A detailed discussion of software project planning and management is presented in Part Four of this book.

Principle 3. ***Recognize that planning is iterative.*** A project plan is never engraved in stone. As work begins, it is very likely that things will change. The plan will need to be adjusted. Iterative, incremental process models include time for revising plans after the delivery of each software increment based on feedback received from users.

Principle 4. ***Estimate based on what you know.*** The intent of estimation is to provide an indication of effort, cost, and task duration, based on the team's current understanding of the work to be done. If information is vague or unreliable, estimates will be equally unreliable.

Principle 5. ***Consider risk as you define the plan.*** If you have identified risks that have high impact and high probability, contingency planning is necessary. The project plan (including the schedule) should be adjusted to accommodate the likelihood that one or more of these risks will occur.

Principle 6. ***Be realistic.*** People don't work 100 percent of every day. Changes will occur. Even the best software engineers make mistakes. These and other realities should be considered as a project plan is established.

Principle 7. ***Adjust granularity as you define the plan.*** The term *granularity* refers to the detail with which some element of planning is represented or conducted. A *high-granularity* plan provides significant work task detail that is planned over relatively short time increments (so that tracking and control occur frequently). A *low-granularity* plan provides broader work tasks that are planned over longer time periods. In general, granularity moves from high to low as the project time line moves away from the current date. Activities that won't occur for many months do not require high granularity (too much can change).

Principle 8. ***Define how you intend to ensure quality.*** The plan should identify how the software team intends to ensure quality. If technical reviews[4] are to be conducted, they should be scheduled. If pair programming (Chapter 3) is to be used during construction, it should be explicitly defined within the plan.

Principle 9. ***Describe how you intend to accommodate change.*** Uncontrolled change can obviate even the best planning. You should identify how changes are to be accommodated as software engineering work proceeds. For example, can the customer request a change at any time? If a change is requested, is the team obliged to implement it immediately? How is the impact and cost of the change assessed?

Principle 10. ***Track the plan frequently, and make adjustments as required.*** Software projects fall behind schedule one day at a time. Therefore, it makes sense to track progress daily, looking for problem areas and situations in which scheduled work does not conform to actual work conducted. When slippage is encountered, the plan is adjusted accordingly.

To be most effective, everyone on the software team should participate in the planning activity. Only then will team members "sign up" to the plan.

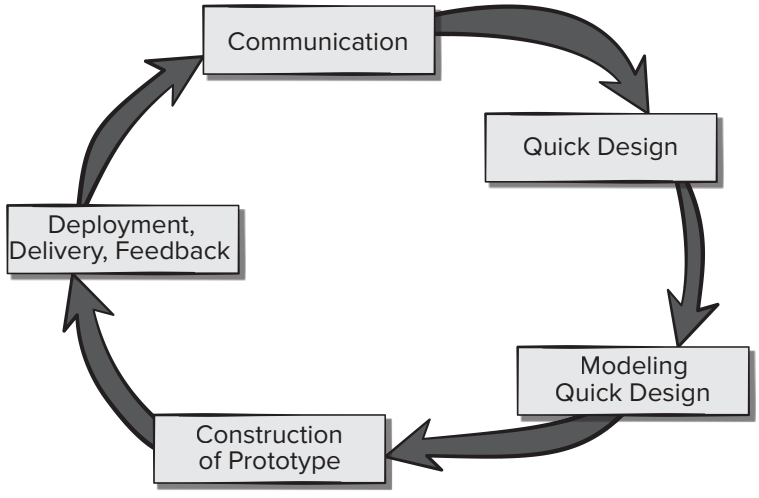
6.2.3 Modeling Principles

We create models to gain a better understanding of the actual entity to be built. When the entity is a physical thing (e.g., a building, a plane, a machine), we can build a

4 Technical reviews are discussed in Chapter 16.

FIGURE 6.4

Role of software modeling



three-dimensional (3D) model that is identical in form and shape but smaller in scale. However, when the entity to be built is software, our model must take a different form. It must be capable of representing the information that software transforms, the architecture and functions that enable the transformation to occur, the features that users desire, and the behavior of the system as the transformation is taking place. Models must accomplish these objectives at different levels of abstraction—first depicting the software from the customer's viewpoint and later representing the software at a more technical level. Figure 6.4 shows how modeling may be used in agile software design.

In software engineering work, two classes of models can be created: requirements models and design models. *Requirements models* (also called *analysis models*) represent customer requirements by depicting the software in three different domains: the information domain, the functional domain, and the behavioral domain (Chapter 8). *Design models* represent characteristics of the software that help practitioners to construct it effectively: the architecture, the user interface, and component-level detail (Chapters 9 through 12).

In their book on agile modeling, Scott Ambler and Ron Jeffries [Amb02] define a set of modeling principles[5] that are intended for those who use an agile process model (Chapter 3) but are appropriate for all software engineers who perform modeling action and tasks:

Principle 1. ***The primary goal of the software team is to build software, not create models.*** Agility means getting software to the customer in the fastest possible time. Models that make this happen are worth creating, but models that slow the process down or provide little new insight should be avoided.

5 The principles noted in this section have been abbreviated and rephrased for the purposes of this book.

Principle 2. ***Travel light—don't create more models than you need.*** Every model that is created must be kept up to date as changes occur. More importantly, every new model takes time that might otherwise be spent on construction (coding and testing). Therefore, create only those models that make it easier and faster to construct the software.

Principle 3. ***Strive to produce the simplest model that will describe the problem or the software.*** Don't overbuild the software [Amb02]. By keeping models simple, the resultant software will also be simple. The result is software that is easier to integrate, easier to test, and easier to maintain (to change). In addition, simple models are easier for members of the software team to understand and critique, resulting in an ongoing form of feedback that optimizes the end result.

Principle 4. ***Build models in a way that makes them amenable to change.*** Assume that your models will change, but in making this assumption don't get sloppy. The problem with this attitude is that you may not create a reasonably complete requirements model, which means you'll create a design (design model) that will invariably miss important functions and features.

Principle 5. ***Be able to state an explicit purpose for each model that is created.*** Every time you create a model, ask yourself why you're doing so. If you can't provide solid justification for the existence of the model, don't spend time on it.

Principle 6. ***Adapt the models you develop to the system at hand.*** It may be necessary to adapt model notation or rules to the application; for example, a video game application might require a different modeling technique than real-time, embedded software for adaptive cruise control in an automobile.

Principle 7. ***Try to build useful models, but forget about building perfect models.*** When building requirements and design models, a software engineer reaches a point of diminishing returns. That is, the effort required to make a model that is complete and internally consistent may not be worth the benefits of these properties. Iterating endlessly to make a model "perfect" does not serve the need for agility.

Principle 8. ***Don't become dogmatic about the syntax of the model. If it communicates content successfully, representation is secondary.*** Although everyone on a software team should try to use consistent notation during modeling, the most important characteristic of the model is to communicate information that enables the next software engineering task. If a model does this successfully, incorrect syntax can be forgiven.

Principle 9. ***If your instincts tell you a model isn't right even though it seems okay on paper, you probably have reason to be concerned.*** If you are an experienced software engineer, trust your instincts. Software work teaches many lessons—some of them on a subconscious level. If something tells you that a design model is doomed to fail (even though you can't prove it explicitly), you have reason to spend additional time examining the model or developing a different one.

Principle 10. ***Get feedback as soon as you can.*** The intent of any model is to communicate information. It should stand on its own. Assume that you won't be there to explain the model. Every model should be reviewed by members of the software team. The intent of these reviews is to provide feedback that can be used to correct modeling mistakes, change misinterpretations, and add features or functions that were inadvertently omitted.

6.2.4 Construction Principles

The construction activity encompasses a set of coding and testing tasks that lead to operational software that is ready for delivery to the customer or end user.

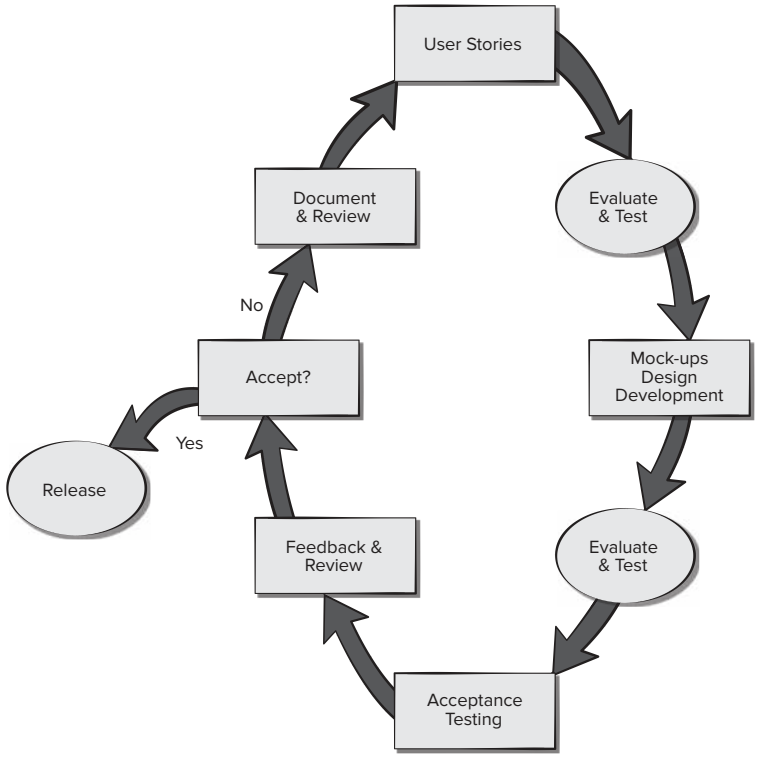
In modern software engineering work, coding may be: (1) the direct creation of programming language source code, (2) the automatic generation of source code using an intermediate design like representation of the component to be built, or (3) the automatic generation of executable code using a fourth-generation programming language (e.g., Unreal4 *Blueprints*).[6]

The initial focus of testing is at the component level, often called *unit testing*. Other levels of testing include (1) *integration testing* (conducted as the system is constructed), (2) *validation testing* that assesses whether requirements have been met for the complete system (or software increment), and (3) *acceptance testing* that is conducted by the customer in an effort to exercise all required features and functions. Figure 6.5 shows where testing and test case design is placed in agile processes.

Testing is considered in detail in Chapters 19 through 21. The following set of fundamental principles and concepts are applicable to coding and testing.

FIGURE 6.5

Testing in agile processes



6 *Blueprints* is a visual scripting tool created by Epic Games (https://docs.unrealengine.com/latest/INT/Engine/Blueprints/).

Coding Principles. The principles that guide the coding task are closely aligned with programming style, programming languages, and programming methods. There are some fundamental principles that might be followed:

Preparation Principles: Before you write one line of code, be sure you:

Principle 1. ***Understand the problem you're trying to solve.***

Principle 2. ***Understand basic design principles and concepts.***

Principle 3. ***Pick a programming language that meets the needs of the software to be built and the environment in which it will operate.***

Principle 4. ***Select a programming environment that provides tools that will make your work easier.***

Principle 5. ***Create a set of unit tests that will be applied once the component you code is completed.***

Coding Principles: As you begin writing code, be sure you:

Principle 6. ***Constrain your algorithms by following structured programming [Boh00] practice.***

Principle 7. ***Consider the use of pair programming.***

Principle 8. ***Select data structures that will meet the needs of the design.***

Principle 9. ***Understand the software architecture and create interfaces that are consistent with it.***

Validation Principles: After you've completed your first coding pass, be sure you:

Principle 10. ***Conduct a code walkthrough when appropriate.***

Principle 11. ***Perform unit tests and correct errors you've uncovered.***

Principle 12. ***Refactor the code to improve its quality.***

Testing Principles. In a classic book on software testing, Glen Myers [Mye79] states a number of rules that can serve well as testing objectives:

1. Testing is a process of executing a program with the intent of finding an error.
2. A good test case is one that has a high probability of finding an as-yet-undiscovered error.
3. A successful test is one that uncovers an as-yet-undiscovered error.

These objectives imply a dramatic change in viewpoint for some software developers. They move counter to the commonly held view that a successful test is one in which no errors are found. Your objective is to design tests that systematically uncover different classes of errors and to do so with a minimum amount of time and effort.

As a secondary benefit, testing demonstrates that software functions appear to be working according to specification and that behavioral and performance requirements appear to have been met. In addition, the data collected as testing is conducted provide a good indication of software reliability and some indication of software quality.

FIGURE 6.6

Testing is never complete



Testing cannot show the absence of errors and defects; it can show only that software errors and defects are present (Figure 6.6). It is important to keep this (rather gloomy) statement in mind as testing is being conducted.

Davis [Dav95b] suggests a set of testing principles[7] that have been adapted for use in this book. In addition, Everett and Meyer [Eve09] suggest additional principles:

Principle 1. ***All tests should be traceable to customer requirements.***[8] The objective of software testing is to uncover errors. It follows that the most severe defects (from the customer's point of view) are those that cause the program to fail to meet its requirements.

Principle 2. ***Tests should be planned long before testing begins.*** Test planning (Chapter 19) can begin as soon as the requirements model is complete. Detailed definition of test cases can begin as soon as the design model has been solidified. Therefore, all tests can be planned and designed before any code has been generated.

Principle 3. ***The Pareto principle applies to software testing.*** In this context, the Pareto principle implies that 80 percent of all errors uncovered during testing will

7 Only a small subset of Davis's testing principles are noted here. For more information, see [Dav95b].

8 This principle refers to *functional tests*, that is, tests that focus on requirements. *Structural tests* (tests that focus on architectural or logical detail) may not address specific requirements directly.

likely be traceable to 20 percent of all program components. The problem, of course, is to isolate these suspect components and to thoroughly test them.

Principle 4. ***Testing should begin "in the small" and progress toward testing "in the large."*** The first tests planned and executed generally focus on individual components. As testing progresses, focus shifts to looking for errors in integrated clusters of components and ultimately in the entire system.

Principle 5. ***Exhaustive testing is not possible.*** The number of path permutations for even a moderately sized program is exceptionally large. For this reason, it is impossible to execute every combination of paths during testing. It is possible, however, to adequately cover program logic and to ensure that all conditions in the component-level design have been exercised.

Principle 6. ***Apply to each module in the system a testing effort commensurate with its expected fault density.*** These are often the newest modules or the ones that are least understood by the developers.

Principle 7. ***Static testing techniques can yield high results.*** More than 85 percent of software defects originate in the software documentation (requirements, specifications, code walk-throughs, and user manuals) [Jon91]. There may be value in testing the system documentation.

Principle 8. ***Track defects and look for patterns in defects uncovered by testing.*** The total defects uncovered is a good indicator of software quality. The types of defects uncovered can be a good measure of software stability. Patterns of defects found over time can forecast numbers of expected defects.

Principle 9. ***Include test cases that demonstrate software is behaving correctly.*** As software components are being maintained or adapted, unexpected interactions cause unintended side effects in other components. It is important to have a set of regression test cases (Chapter 19) ready to check system behavior after changes are made to a software product.

6.2.5 Deployment Principles

As we noted in Part One of this book, the deployment activity encompasses three actions: delivery, support, and feedback. Because modern software process models are evolutionary or incremental in nature, deployment happens not once, but several times as software moves toward completion. Each delivery cycle provides the customer and end users with an operational software increment that provides usable functions and features. Each support cycle provides documentation and human assistance for all functions and features introduced during all deployment cycles to date. Each feedback cycle provides the software team with important guidance that results in modifications to the functions, features, and approach taken for the next increment. Typical deployment actions are illustrated in Figure 6.7.

The delivery of a software increment represents an important milestone for any software project. Some key principles should be followed as the team prepares to deliver an increment:

Principle 1. ***Customer expectations for the software must be managed.*** Too often, the customer expects more than the team has promised to deliver, and disappointment

FIGURE 6.7

Deployment actions



occurs immediately. This results in feedback that is not productive and ruins team morale. In her book on managing expectations, Naomi Karten [Kar94] states: "The starting point for managing expectations is to become more conscientious about what you communicate and how." She suggests that a software engineer must be careful about sending the customer conflicting messages (e.g., promising more than you can reasonably deliver in the time period provided or delivering more than you promise for one software increment and then less than promised for the next).

Principle 2. ***A complete delivery package should be assembled and tested.*** All executable software, support data files, support documents, and other relevant information should be assembled and thoroughly beta-tested with actual users. All installation scripts and other operational features should be thoroughly exercised in all possible computing configurations (i.e., hardware, operating systems, peripheral devices, networking arrangements).

Principle 3. ***A support regimen must be established before the software is delivered.*** An end user expects responsiveness and accurate information when a question or problem arises. If support is ad hoc, or worse, nonexistent, the customer will become dissatisfied immediately. Support should be planned, support materials should be prepared, and appropriate record-keeping mechanisms should be established so that the software team can conduct a categorical assessment of the kinds of support requested.

Principle 4. ***Appropriate instructional materials must be provided to end users.*** The software team delivers more than the software itself. Appropriate training aids (if required) should be developed; troubleshooting guidelines should be provided,

and when necessary, a "what's different about this software increment" description should be published.[9]

Principle 5. ***Buggy software should be fixed first, delivered later.*** Under time pressure, some software organizations deliver low-quality increments with a warning to the customer that bugs "will be fixed in the next release." This is a mistake. There's a saying in the software business: "Customers will forget you delivered a high-quality product a few days late, but they will never forget the problems that a low-quality product caused them. The software reminds them every day."

6.3 SUMMARY

Software engineering practice encompasses principles, concepts, methods, and tools that software engineers apply throughout the software process. Every software engineering project is different. Yet, a set of generic principles applies to the process and to the practice of each framework activity regardless of the project or the product.

A set of core principles helps in the application of a meaningful software process and the execution of effective software engineering methods. At the process level, core principles establish a philosophical foundation that guides a software team as it navigates through the software process. At the practice level, core principles establish a collection of values and rules that serve as a guide as you analyze a problem, design a solution, implement and test the solution, and ultimately deploy the software in the user community.

Communication principles focus on the need to reduce noise and improve bandwidth as the conversation between developer and customer progresses. Both parties must collaborate for the best communication to occur.

Planning principles provide guidelines for constructing the best map for the journey to a completed system or product. The plan may be designed solely for a single software increment, or it may be defined for the entire project. Regardless, it must address what will be done, who will do it, and when the work will be completed.

Modeling principles serve as a foundation for the methods and notation that are used to create representations of the software. Modeling encompasses both analysis and design, describing representations of the software that progressively become more detailed. The intent of the models is to solidify understanding of the work to be done and to provide technical guidance to those who will implement the software.

Construction incorporates a coding and testing cycle in which source code for a component is generated and tested. Coding principles define generic actions that should occur before code is written, while it is being created, and after it has been completed. Although there are many testing principles, only one is dominant: Testing is a process of executing a program with the intent of finding an error.

Deployment occurs as each software increment is presented to the customer and encompasses delivery, support, and feedback. Key principles for delivery consider managing customer expectations and providing the customer with appropriate support

9 During the communication activity, the software team should determine what types of help materials users want.

information for the software. Support demands advance preparation. Feedback allows the customer to suggest changes that have business value and provide the developer with input for the next iterative software engineering cycle.

PROBLEMS AND POINTS TO PONDER

6.1. Because a focus on quality demands resources and time, is it possible to be agile and still maintain a quality focus?

6.2. Of the eight core principles that guide process (discussed in Section 6.1.1), which do you believe is most important?

6.3. Describe the concept of *separation of concerns* in your own words.

6.4. Why is it necessary to "move on"?

6.5. Do some research on "negotiation" for the communication activity, and prepare a set of guidelines that focus solely on negotiation.

6.6. Why are models important in software engineering work? Are they always necessary? Are there qualifiers to your answer about necessity?

6.7. What is a successful test?

6.8. Why is feedback important to the software team?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

7 UNDERSTANDING REQUIREMENTS

Understanding problem requirements is among the most difficult tasks facing a software engineer. When you first think about it, developing a clear understanding of the requirements doesn't seem that hard. After all, doesn't the customer know what is required? Shouldn't the end users have a good understanding of the features and functions that they need? Surprisingly, in many instances, the answer to these questions is no. And even if customers and end users can state their needs explicitly, those needs will change throughout the project.

KEY CONCEPTS

analysis model 118
analysis patterns 122
collaboration 108
elaboration 104
elicitation 104
inception 104
negotiation 105
requirements engineering 103
requirements gathering 110
requirements management 106
requirements monitoring 123
specification 105
stakeholders 107
use cases 113
validating requirements 123
validation 105
viewpoints 107
work products 114

QUICK LOOK



What is it? Before you begin any technical work, it's a good idea to create a set of requirements for the engineering tasks. By establishing a set of requirements, you'll gain an understanding of what the business impact of the software will be, what the customer wants, and how end users will interact with the software.

Who does it? Software engineers and other project stakeholders (managers, customers, and end users) all participate in requirements engineering.

Why is it important? To understand what the customer wants before you begin to design and build a computer-based system. Building an elegant computer program that solves the wrong problem helps no one.

What are the steps? Requirements engineering begins with inception (a task that defines the scope and nature of the problem to be solved). It moves onward to elicitation (a task that helps stakeholders define what is required), and then elaboration (where basic requirements are refined and modified). As stakeholders define the problem, negotiation occurs (what are the priorities, what is essential, when is it required?). Finally, the problem is specified in some manner and then reviewed or validated to ensure that your understanding of the problem and the stakeholders' understanding of the problem coincide.

What is the work product? Requirements engineering provides all parties with a written understanding of the problem. The work products may include: usage scenarios, function and feature lists, and requirements models.

How do I ensure that I've done it right? Requirements engineering work products are reviewed with stakeholders to ensure that everyone is on the same page. A word of warning: Even after all parties agree, things will change, and they will continue to change throughout the project.

In the forward to a book by Ralph Young [You01] on effective requirements practices, one of us [RSP] wrote:

> It's your worst nightmare. A customer walks into your office, sits down, looks you straight in the eye, and says, "I know you think you understand what I said, but what you don't understand is what I said is not what I meant." Invariably, this happens late in the project, after deadline commitments have been made, reputations are on the line, and serious money is at stake.
>
> All of us who have worked in the systems and software business for more than a few years have lived this nightmare, and yet, few of us have learned to make it go away. We struggle when we try to elicit requirements from our customers. We have trouble understanding the information that we do acquire. We often record requirements in a disorganized manner, and we spend far too little time verifying what we do record. We allow change to control us, rather than establishing mechanisms to control change. In short, we fail to establish a solid foundation for the system or software. Each of these problems is challenging. When they are combined, the outlook is daunting for even the most experienced managers and practitioners. But solutions do exist.

It's reasonable to argue that the techniques we'll discuss in this chapter are not a true "solution" to the challenges just noted. But they do provide a solid approach for addressing these challenges.

7.1 REQUIREMENTS ENGINEERING

Designing and building computer software is challenging, creative, and just plain fun. In fact, building software is so compelling that many software developers want to jump right in before they have a clear understanding of what is needed. They argue that things will become clear as they build, that project stakeholders will be able to understand need only after examining early iterations of the software, that things change so rapidly that any attempt to understand requirements in detail is a waste of time, that the bottom line is producing a working program, and that all else is secondary. What makes these arguments seductive is that they contain elements of truth. But each argument is flawed and can lead to a failed software project.

Requirements engineering is the term for the broad spectrum of tasks and techniques that lead to an understanding of requirements. From a software process perspective, requirements engineering is a major software engineering action that begins during the communication activity and continues into the modeling activity. Requirements engineering establishes a solid base for design and construction. Without it, the resulting software has a high probability of not meeting customer's needs. It must be adapted to the needs of the process, the project, the product, and the people doing the work. It is important to realize that each of these tasks is done iteratively as the project team and the stakeholders continue to share information about their respective concerns.

Requirements engineering builds a bridge to design and construction. But where does the bridge originate? One could argue that it begins with the project stakeholders (e.g., managers, customers, and end users), where business needs are defined, user scenarios are described, functions and features are delineated, and project constraints are identified. Others might suggest that it begins with a broader system definition,

where software is but one component of the larger system domain. But regardless of the starting point, the journey across the bridge takes you high above the project, allowing you to examine the context of the software work to be performed; the specific needs that design and construction must address; the priorities that guide the order in which work is to be completed; and the information, functions, and behaviors that will have a profound impact on the resulting design. Requirements engineering encompasses seven tasks with sometimes muddy boundaries: *inception, elicitation, elaboration, negotiation, specification, validation,* and *management.* It is important to note that some of these tasks occur in parallel and all are adapted to the needs of the project. Expect to do a bit of design during requirements work and a bit of requirements work during design.

7.1.1 Inception

How does a software project get started? In general, most projects begin with an identified business need or when a potential new market or service is discovered. At project inception, you establish a basic understanding of the problem, the people who want a solution, and the nature of the solution that is desired. Communication between all stakeholders and the software team needs to be established during this task to begin an effective collaboration.

7.1.2 Elicitation

It certainly seems simple enough—ask the customer, the users, and others what the objectives for the system or product are, what is to be accomplished, how the system or product fits into the needs of the business, and finally, how the system or product is to be used on a day-to-day basis. But it isn't simple—it's very hard.

An important part of elicitation is to understand the business goals [Cle10]. A *goal* is a long-term aim that a system or product must achieve. Goals may deal with either functional or nonfunctional (e.g., reliability, security, usability) concerns [Lam09].

Goals are often a good way to explain requirements to stakeholders and, once established, can be used to manage conflicts among stakeholders. Goals should be specified precisely and serve as the basis for requirements elaboration, verification and validation, conflict management, negotiation, explanation, and evolution.

Your job is to engage stakeholders and to encourage them to share their goals honestly. Once the goals are captured, you establish a prioritization mechanism and create a design rationale for a potential architecture (that meets stakeholder goals).

Agility is an important aspect of requirements engineering. The intent of elicitation is to transfer ideas from stakeholders to the software team smoothly and without delay. It is highly likely that new requirements will continue to emerge as iterative product development occurs.

7.1.3 Elaboration

The elaboration task focuses on developing a refined requirements model that identifies various aspects of software function, behavior, and information (Chapter 8). Elaboration is driven by the creation and refinement of user scenarios obtained during elicitation. These scenarios describe how the end users (and other actors) will interact with the system. Each user scenario is parsed to extract *analysis classes*—business domain

entities that are visible to the end user. The attributes of each analysis class are defined, and the services that are required by each class are identified. The relationships and collaboration between classes are identified. Elaboration is a good thing, but you need to know when to stop. The key is to describe the problem in a way that establishes a firm base for design and then move on. Do not obsess over unnecessary details.

7.1.4 Negotiation

It isn't unusual for customers and users to ask for more than can be achieved, given limited business resources. It's also relatively common for different customers or users to propose conflicting requirements, arguing that their version is "essential for our special needs."

These conflicts need to be reconciled through the process of negotiation. Customers, users, and other stakeholders are asked to rank requirements and then discuss conflicts in priority. There should be no winner and no loser in an effective negotiation. Both sides win, because a "deal" that both can live with is solidified. You should use an iterative approach that prioritizes requirements, assesses their cost and risk, and addresses internal conflicts. In this way, requirements are eliminated, combined, and/or modified so that each party achieves some measure of satisfaction.

7.1.5 Specification

In the context of computer-based systems (and software), the term *specification* means different things to different people. A specification can be a written document, a set of graphical models, a formal mathematical model, a collection of usage scenarios, a prototype, or any combination of these.

Some suggest that a "standard template" [Som97] should be developed and used for a specification, arguing that this leads to requirements that are presented in a consistent and therefore more understandable manner. However, it is sometimes necessary to remain flexible when a specification is to be developed. The formality and format of a specification varies with the size and the complexity of the software to be built. For large systems, a written document, combining natural language descriptions and graphical models, may be the best approach. A template for a formal software requirements specification document can be downloaded from: https://web.cs.dal.ca/~hawkey/3130/srs_template-ieee.doc. However, usage scenarios may be all that are required for smaller products or systems that reside within well-understood technical environments.

7.1.6 Validation

The work products produced during requirements engineering are assessed for quality during a validation step. A key concern during requirements validation is consistency. Use the analysis model to ensure that requirements have been consistently stated. Requirements validation examines the specification to ensure that all software requirements have been stated unambiguously; that inconsistencies, omissions, and errors have been detected and corrected; and that the work products conform to the standards established for the process, the project, and the product.

The primary requirements validation mechanism is the *technical review* (Chapter 16). The review team that validates requirements includes software engineers, customers, users, and other stakeholders who examine the specification looking for errors in

content or interpretation, areas where clarification may be required, missing information, inconsistencies (a major problem when large products or systems are engineered), conflicting requirements, or unrealistic (unachievable) requirements.

To illustrate some of the problems that occur during requirements validation, consider two seemingly innocuous requirements:

- The software should be user friendly.
- The probability of a successful unauthorized database intrusion should be less than 0.0001.

The first requirement is too vague for developers to test or assess. What exactly does "user friendly" mean? To validate it, it must be quantified or qualified in some manner.

The second requirement has a quantitative element ("less than 0.0001"), but intrusion testing will be difficult and time consuming. Is this level of security even warranted for the application? Can other complementary requirements associated with security (e.g., password protection, specialized handshaking) replace the quantitative requirement noted?

7.1.7 Requirements Management

Requirements for computer-based systems change, and the desire to change requirements persists throughout the life of the system. Requirements management is a set of activities that help the project team identify, control, and track requirements and changes to requirements at any time as the project proceeds. Many of these activities are identical to the software configuration management (SCM) techniques discussed in Chapter 22.

INFO



Requirements Validation Checklist

It is often useful to examine each requirement against a set of checklist questions. Here is a small subset of those that might be asked:

1. Are requirements stated clearly? Can they be misinterpreted?
2. Is the source (e.g., a person, a regulation, a document) of the requirement identified? Has the final statement of the requirement been examined by or against the original source?
3. Is the requirement bounded in quantitative terms?
4. What other requirements relate to this requirement? Are they clearly noted via a cross-reference matrix or other mechanism?
5. Does the requirement violate any system domain constraints?
6. Is the requirement testable? If so, can we specify tests (sometimes called validation criteria) to exercise the requirement?
7. Is the requirement traceable to any system model that has been created?
8. Is the requirement traceable to overall system and product objectives?
9. Is the specification structured in a way that leads to easy understanding, easy reference, and easy translation into more technical work products?
10. Has an index for the specification been created?
11. Have requirements associated with performance, behavior, and operational characteristics been clearly stated? What requirements appear to be implicit?

7.2 Establishing the Groundwork

In an ideal setting, stakeholders and software engineers work together on the same team. In such cases, requirements engineering is simply a matter of conducting meaningful conversations with colleagues who are well-known members of the team. But reality is often quite different.

Customer(s) or end users may reside in different cities or countries, may have only a vague idea of what is required, may have conflicting opinions about the system to be built, may have limited technical knowledge, and may have limited time to interact with the requirements engineer. None of these things are desirable, but all are common, and you are often forced to work within the constraints imposed by this situation.

In the sections that follow, we discuss the steps required to establish the groundwork for an understanding of software requirements—to get the project started in a way that will keep it moving forward toward a successful solution.

7.2.1 Identifying Stakeholders

Sommerville and Sawyer [Som97] define a *stakeholder* as "anyone who benefits in a direct or indirect way from the system which is being developed." We have already identified the usual suspects: business operations managers, product managers, marketing people, internal and external customers, end users, consultants, product engineers, software engineers, support and maintenance engineers, and others. Each stakeholder has a different view of the system, achieves different benefits when the system is successfully developed, and is open to different risks if the development effort should fail.

At inception, you should create a list of people who will contribute input as requirements are elicited (Section 7.3). The initial list will grow as stakeholders are contacted because every stakeholder will be asked: "Whom else do you think I should talk to?"

7.2.2 Recognizing Multiple Viewpoints

Because many different stakeholders exist, the requirements of the system will be explored from many different points of view. For example, the marketing group is interested in features that will excite the potential market, making the new system easy to sell. Business managers are interested in a feature set that can be built within budget and that will be ready to meet defined market windows. End users may want features that are familiar to them and that are easy to learn and use. Software engineers may be concerned with functions that are invisible to nontechnical stakeholders but that enable an infrastructure that supports more marketable functions and features. Support engineers may focus on the maintainability of the software.

Each of these constituencies (and others) will contribute information to the requirements engineering process. As information from multiple viewpoints is collected, emerging requirements may be inconsistent or may conflict with one another. You should categorize all stakeholder information (including inconsistent and conflicting requirements) in a way that will allow decision makers to choose an internally consistent set of requirements for the system.

Several things can make it hard to elicit requirements for software that satisfies its users: project goals are unclear, stakeholders' priorities differ, people have unspoken assumptions, stakeholders interpret meanings differently, and requirements are stated in a way that makes them difficult to verify [Ale11]. The goal of effective requirements engineering is to eliminate or at least reduce these problems.

7.2.3 Working Toward Collaboration

If five stakeholders are involved in a software project, you may have five (or more) different opinions about the proper set of requirements. Throughout earlier chapters, we have noted that customers (and other stakeholders) should collaborate among themselves (avoiding petty turf battles) and with software engineering practitioners if a successful system is to result. But how is this collaboration accomplished?

The job of a requirements engineer is to identify areas of commonality (i.e., requirements on which all stakeholders agree) and areas of conflict or inconsistency (i.e., requirements that are desired by one stakeholder but conflict with the needs of another stakeholder). It is, of course, the latter category that presents a challenge.

Collaboration does not necessarily mean that requirements are "defined by committee." In many cases, stakeholders collaborate by providing their view of requirements, but a strong "project champion" (e.g., a business manager or a senior technologist) may make the final decision about which requirements make the cut.

INFO



Using "Planning Poker"

One way of resolving conflicting requirements and at the same time better understanding the relative importance of all requirements is to use a "voting" scheme based on *priority points*. All stakeholders are provided with some number of priority points that can be "spent" on any number of requirements. A list of requirements is presented, and each stakeholder indicates the relative importance of each (from his viewpoint) by spending one or more priority points on it. Points spent cannot be reused. Once a stakeholder's priority points are exhausted, no further action on requirements can be taken by that person. Overall points spent on each requirement by all stakeholders provide an indication of the overall importance of each requirement.

7.2.4 Asking the First Questions

Questions asked at the inception of the project should be "context free" [Gau89]. The first set of context-free questions focuses on the customer and other stakeholders and the overall project goals and benefits. For example, you might ask:

- Who is behind the request for this work?
- Who will use the solution?
- What will be the economic benefit of a successful solution?
- Is there another source for the solution that you need?

These questions help to identify all stakeholders who will have interest in the software to be built. In addition, the questions identify the measurable benefit of a successful implementation and possible alternatives to custom software development.

The next set of questions enables you to gain a better understanding of the problem and allows the customer to voice her perceptions about a solution:

- How would you characterize "good" output that would be generated by a successful solution?
- What problem(s) will this solution address?

- Can you show me (or describe) the business environment in which the solution will be used?
- Will special performance issues or constraints affect the way the solution is approached?

The final set of questions focuses on the effectiveness of the communication activity itself. Gause and Weinberg [Gau89] call these "meta-questions" and propose the following (abbreviated) list:

- Are you the right person to answer these questions? Are your answers "official"?
- Are my questions relevant to the problem that you have?
- Am I asking too many questions?
- Can anyone else provide additional information?
- Should I be asking you anything else?

These questions (and others) will help to "break the ice" and initiate the communication that is essential to successful elicitation. But a question-and-answer (Q&A) meeting format is not an approach that has been overwhelmingly successful. In fact, the Q&A session should be used for the first encounter only and then replaced by a requirements elicitation format that combines elements of problem solving, negotiation, and specification. An approach of this type is presented in Section 7.3.

7.2.5 Nonfunctional Requirements

A *nonfunctional requirement* (NFR) can be described as a quality attribute, a performance attribute, a security attribute, or a general constraint on a system. These are often not easy for stakeholders to articulate. Chung [Chu09] suggests that there is a lopsided emphasis on functionality of the software, yet the software may not be useful or usable without the necessary nonfunctional characteristics.

It is possible to define a two-phase approach [Hne11] that can assist a software team and other stakeholders in identifying nonfunctional requirements. During the first phase, a set of software engineering guidelines is established for the system to be built. These include guidelines for best practice, but also address architectural style (Chapter 10) and the use of design patterns (Chapter 14). A list of NFRs (e.g., requirements that address usability, testability, security, or maintainability) is then developed. A simple table lists NFRs as *column labels* and software engineering guidelines as *row labels*. A relationship matrix compares each guideline to all others, helping the team to assess whether each pair of guidelines is *complementary*, *overlapping*, *conflicting*, or *independent*.

In the second phase, the team prioritizes each nonfunctional requirement by creating a homogeneous set of nonfunctional requirements using a set of decision rules that establish which guidelines to implement and which to reject.

7.2.6 Traceability

Traceability is a software engineering term that refers to documented links between software engineering work products (e.g., requirements and test cases). A *traceability matrix* allows a requirements engineer to represent the relationship between requirements and other software engineering work products. Rows of the traceability

matrix are labeled using requirement names, and columns can be labeled with the name of a software engineering work product (e.g., a design element or a test case). A matrix cell is marked to indicate the presence of a link between the two.

The traceability matrices can support a variety of engineering development activities. They can provide continuity for developers as a project moves from one project phase to another, regardless of the process model being used. Traceability matrices often can be used to ensure the engineering work products have taken all requirements into account.

As the number of requirements and the number of work products grows, it becomes increasingly difficult to keep the traceability matrix up to date. Nonetheless, it is important to create some means for tracking the impact and evolution of the product requirements [Got11].

7.3 REQUIREMENTS GATHERING

Requirements gathering combines elements of problem solving, elaboration, negotiation, and specification. To encourage a collaborative, team-oriented approach to requirements gathering, stakeholders work together to identify the problem, propose elements of the solution, negotiate different approaches, and specify a preliminary set of solution requirements [Zah90].

7.3.1 Collaborative Requirements Gathering

Many different approaches to collaborative requirements gathering have been proposed. Each makes use of a slightly different scenario, but all apply some variation on the following basic guidelines:

- Meetings (either real or virtual) are conducted and attended by both software engineers and other stakeholders.
- Rules for preparation and participation are established.
- An agenda is suggested that is formal enough to cover all important points but informal enough to encourage the free flow of ideas.
- A "facilitator" (can be a customer, a developer, or an outsider) controls the meeting.
- A "definition mechanism" (can be worksheets, flip charts, or wall stickers or an electronic bulletin board, chat room, or virtual forum) is used.

The goal is to identify the problem, propose elements of the solution, negotiate different approaches, and specify a preliminary set of solution requirements.

A one- or two-page "product request" is generated during inception (Section 7.2). A meeting place, time, and date are selected; a facilitator is chosen; and attendees from the software team and other stakeholder organizations are invited to participate. If a system or product will serve many users, be absolutely certain that requirements are elicited from a representative cross section of users. If only one user defines all requirements, acceptance risk is high (meaning there may be several other stakeholders who will not accept the product). The product request is distributed to all attendees before the meeting date.

As an example, consider an excerpt from a product request written by a marketing person involved in the *SafeHome* project. This person writes the following narrative about the home security function that is to be part of *SafeHome:*

> Our research indicates that the market for home management systems is growing at a rate of 40 percent per year. The first *SafeHome* function we bring to market should be the home security function. Most people are familiar with "alarm systems," so this would be an easy sell. We might also consider using voice control of the system using some technology like Alexa.
>
> The home security function would protect against and/or recognize a variety of undesirable "situations" such as illegal entry, fire, flooding, carbon monoxide levels, and others. It'll use our wireless sensors to detect each situation, can be programmed by the homeowner, and will automatically contact a monitoring agency and the owner's cell phone when a situation is detected.

In reality, others would contribute to this narrative during the requirements gathering meeting and considerably more information would be available. But even with additional information, ambiguity is present, omissions are likely to exist, and errors might occur. For now, the preceding "functional description" will suffice.

While reviewing the product request in the days before the meeting, each attendee is asked to make a list of objects that are part of the environment that surrounds the system, other objects that are to be produced by the system, and objects that are used by the system to perform its functions. In addition, each attendee is asked to make another list of services (processes or functions) that manipulate or interact with the objects. Finally, lists of constraints (e.g., cost, size, business rules) and performance criteria (e.g., speed, accuracy, security) are also developed. The attendees are informed that the lists are not expected to be exhaustive but are expected to reflect each person's perception of the system.

Objects described for *SafeHome* might include the control panel, smoke detectors, window and door sensors, motion detectors, an alarm, an event (a sensor has been activated), a display, a tablet, telephone numbers, a telephone call, and so on. The list of services might include *configuring* the system, *setting* the alarm, *monitoring* the sensors, *dialing* the phone using a wireless router, *programming* the control panel, and *reading* the display (note that services act on objects). In a similar fashion, each attendee will develop lists of constraints (e.g., the system must recognize when sensors are not operating, must be user friendly, must interface directly to a standard phone line) and performance criteria (e.g., a sensor event should be recognized within 1 second, and an event priority scheme should be implemented).

The lists of objects can be pinned to the walls of the room using large sheets of paper, stuck to the walls using adhesive-backed sheets, or written on a wall board. Alternatively, the lists may have been posted on a group forum or at an internal website or posed in a social networking environment for review prior to the meeting. Ideally, each listed entry should be capable of being manipulated separately so that lists can be combined, entries can be deleted, and additions can be made. At this stage, critique and debate are strictly prohibited. Avoid the impulse to shoot down a customer's idea as "too costly" or "impractical." The idea here is to negotiate a list that is acceptable to all. To do this, you must keep an open mind.

After individual lists are presented in one topic area, the group creates a combined list by eliminating redundant entries, adding any new ideas that come up during the

discussion, but not deleting anything. After you create combined lists for all topic areas, discussion—coordinated by the facilitator—ensues. The combined list is shortened, lengthened, or reworded to properly reflect the product or system to be developed. The objective is to develop a consensus list of objects, services, constraints, and performance for the system to be built.

In many cases, an object or service described on a list will require further explanation. To accomplish this, stakeholders develop *mini-specifications* for entries on the lists or by creating a use case (Section 7.4) that involves the object or service. For example, the mini-spec for the *SafeHome* object **Control Panel** might be:

> The control panel is a wall-mounted unit that is approximately 230 × 130 mm in size. The control panel has wireless connectivity to sensors and a tablet. User interaction occurs through a keypad containing 12 keys. A 75 × 75 mm OLED color display provides user feedback. Software provides interactive prompts, echo, and similar functions.

The mini-specs are presented to all stakeholders for discussion. Additions, deletions, and further elaboration are made. In some cases, the development of mini-specs will uncover new objects, services, constraints, or performance requirements that will be added to the original lists. During all discussions, the team may raise an issue that cannot be resolved during the meeting. An *issues list* is maintained so that these ideas will be acted on later.

SAFEHOME



Case Study Example
Conducting a Requirements Gathering Meeting

The scene: A meeting room. The first requirements gathering meeting is in progress.

The players: Jamie Lazar, software team member; Vinod Raman, software team member; Ed Robbins, software team member; Doug Miller, software engineering manager; three members of marketing; a product engineering representative; and a facilitator.

The conversation:

Facilitator (pointing at whiteboard): So that's the current list of objects and services for the home security function.

Marketing person: That about covers it from our point of view.

Vinod: Didn't someone mention that they wanted all *SafeHome* functionality to be accessible via the Internet? That would include the home security function, no?

Marketing person: Yes, that's right . . . we'll have to add that functionality and the appropriate objects.

Facilitator: Does that also add some constraints?

Jamie: It does, both technical and legal.

Production rep: Meaning?

Jamie: We better make sure an outsider can't hack into the system, disarm it, and rob the place or worse. Heavy liability on our part.

Doug: Very true.

Marketing: But we still need that . . . just be sure to stop an outsider from getting in.

Ed: That's easier said than done and . . .

Facilitator (interrupting): I don't want to debate this issue now. Let's note it as an action item and proceed.

(Doug, serving as the recorder for the meeting, makes an appropriate note.)

Facilitator: I have a feeling there's still more to consider here.

(The group spends the next 20 minutes refining and expanding the details of the home security function.)

Many stakeholder concerns (e.g., accuracy, data accessibility, security) are the basis for nonfunctional system requirements (Section 7.2). As stakeholders enunciate these concerns, software engineers must consider them within the context of the system to be built. The questions that must be answered [Lag10] are:

- Can we build the system?
- Will this development process allow us to beat our competitors to market?
- Do adequate resources exist to build and maintain the proposed system?
- Will the system performance meet the needs of our customers?

7.3.2 Usage Scenarios

As requirements are gathered, an overall vision of system functions and features begin to materialize. However, it is difficult to move into more technical software engineering activities until you understand how the features will be used by different classes of end users. To accomplish this, developers and users can create a set of scenarios that identify a thread of usage for the system to be constructed. The scenarios, often called *use cases* [Jac92], provide a description of how the system will be used. Use cases are discussed in greater detail in Section 7.4.

SAFEHOME



Developing a Preliminary User Scenario

The scene: A meeting room, continuing the first requirements gathering meeting.

The players: Jamie Lazar, software team member; Vinod Raman, software team member; Ed Robbins, software team member; Doug Miller, software engineering manager; three members of marketing; a product engineering representative; and a facilitator.

The conversation:

Facilitator: We've been talking about security for access to *SafeHome* functionality that will be accessible via the Internet. I'd like to try something. Let's develop a usage scenario for access to the home security function.

Jamie: How?

Facilitator: We can do it a couple of different ways, but for now, I'd like to keep things really informal. Tell us (he points at a marketing person) how you envision accessing the system.

Marketing person: Um . . . well, this is the kind of thing I'd do if I was away from home and I had to let someone into the house, say a housekeeper or repair guy, who didn't have the security code.

Facilitator (smiling): That's the reason you'd do it . . . tell me how you'd actually do this.

Marketing person: Um . . . the first thing I'd need is a PC. I'd log on to a website we'd maintain for all users of *SafeHome*. I'd provide my user ID and . . .

Vinod (interrupting): The Web page would have to be secure, encrypted, to guarantee that we're safe and . . .

Facilitator (interrupting): That's good information, Vinod, but it's technical. Let's just focus on how the end user will use this capability. OK?

Vinod: No problem.

Marketing person: So as I was saying, I'd log on to a website and provide my user ID and two levels of passwords.

Jamie: What if I forget my password?

Facilitator (interrupting): Good point, Jamie, but let's not address that now. We'll make a note of that and call it an *exception*. I'm sure there'll be others.

Marketing person: After I enter the passwords, a screen representing all *SafeHome* functions will appear. I'd select the home security function. The system might request that I verify who I am, say, by asking for my address or phone number or something. It would then display a picture of the security system control panel along with a list of functions that I can perform—arm the system, disarm the system, disarm one or more sensors. I suppose it might also allow me to reconfigure security zones and other things like that, but I'm not sure.

(As the marketing person continues talking, Doug takes copious notes; these form the basis for the first informal usage scenario. Alternatively, the marketing person could have been asked to write the scenario, but this would be done outside the meeting.)

7.3.3 Elicitation Work Products

The work products produced during requirements elicitation will vary depending on the size of the system or product to be built. For large systems, the work products may include: (1) a statement of need and feasibility; (2) a bounded statement of scope for the system or product; (3) a list of customers, users, and other stakeholders who participated in requirements elicitation; (4) a description of the system's technical environment; (5) a list of requirements (preferably organized by function) and the domain constraints that apply to each; and (6) a set of usage scenarios that provide insight into the use of the system or product under different operating conditions. Each of these work products is reviewed by all people who have participated in requirements elicitation.

7.4 Developing Use Cases

A use case tells a stylized story about how an end user (playing one of several possible roles) interacts with the system under a specific set of circumstances. The story may be narrative text (a *user story*), an outline of tasks or interactions, a template-based description, or a diagrammatic representation. Regardless of its form, a use case depicts the software or system from the end user's point of view.

The first step in writing a use case is to define the set of "actors" that will be involved in the story. *Actors* are the different people (or devices) that use the system or product within the context of the function and behavior that is to be described. Actors will represent the roles that people (or devices) play as the system operates. Defined somewhat more formally, an actor is anything that communicates with the system or product and that is external to the system itself. Every actor has one or more goals when using the system.

It is important to note that an actor and an end user are not necessarily the same thing. A typical user may play several different roles when using a system, whereas an actor represents a class of external entities (often, but not always, people) that play just one role in the context of the use case. As an example, consider a user who interacts with the program that allows experimenting with alarm sensor configuration

in a virtual building. After careful review of requirements, the software for the control computer requires four different modes (roles) for interaction: placement mode, testing mode, monitoring mode, and troubleshooting mode. Therefore, four actors can be defined: editor, tester, monitor, and troubleshooter. In some cases, the user can play all the roles. In others, different people may play the role of each actor.

Because requirements elicitation is an evolutionary activity, not all actors are identified during the first iteration. It is possible to identify primary actors [Jac92] during the first iteration and secondary actors as more is learned about the system. *Primary actors* interact to achieve required system function and derive the intended benefit from the system. They work directly and frequently with the software. *Secondary actors* support the system so that primary actors can do their work.

Once actors have been identified, use cases can be developed. Jacobson [Jac92] suggests questions that should be answered by a use case:

1. Who is the primary actor, the secondary actor(s)?
2. What are the actor's goals?
3. What preconditions should exist before the story begins?
4. What main tasks or functions are performed by the actor?
5. What exceptions might be considered as the story is described?
6. What variations in the actor's interaction are possible?
7. What system information will the actor acquire, produce, or change?
8. Will the actor have to inform the system about changes in the external environment?
9. What information does the actor desire from the system?
10. Does the actor wish to be informed about unexpected changes?

Recalling basic *SafeHome* requirements, we define four actors: **homeowner** (a user), **setup manager** (likely the same person as **homeowner,** but playing a different role), **sensors** (devices attached to the system), and the **monitoring and response subsystem** (the central station that monitors the *SafeHome* home security function). For the purposes of this example, we consider only the **homeowner** actor. The **homeowner** actor interacts with the home security function in different ways using either the alarm control panel, a tablet, or a cell phone.

The homeowner:

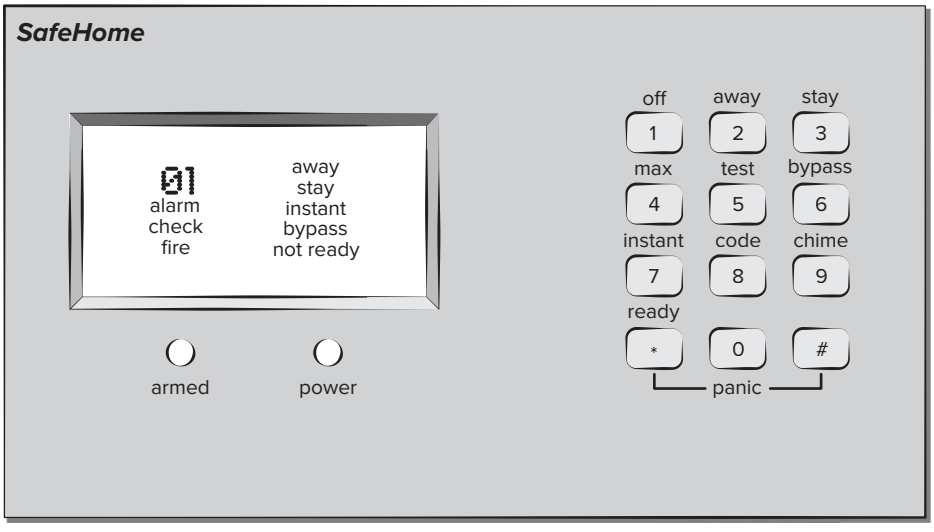
1. Enters a password to allow all other interactions
2. Inquires about the status of a security zone
3. Inquires about the status of a sensor
4. Presses the panic button in an emergency
5. Activates and deactivates the security system

Considering the situation in which the homeowner uses the control panel, the basic use case for system activation follows:

1. The homeowner observes the *SafeHome* control panel (Figure 7.1) to determine if the system is ready for input. If the system is not ready, a *not ready*

FIGURE 7.1

***SafeHome* control panel**



message is displayed on the LCD display, and the homeowner must physically close windows or doors so that the *not ready* message disappears. (A *not ready* message implies that a sensor is open, i.e., that a door or window is open.)

2. The homeowner uses the keypad to key in a four-digit password. The password is compared with the valid password stored in the system. If the password is incorrect, the control panel will beep once and reset itself for additional input. If the password is correct, the control panel awaits further action.
3. The homeowner selects and keys in "stay" or "away" (see Figure 7.1) to activate the system. *Stay* activates only perimeter sensors (inside motion detecting sensors are deactivated). *Away* activates all sensors.
4. When activation occurs, a red alarm light can be observed by the homeowner.

The basic use case presents a high-level user story that describes the interaction between the actor and the system.

In many instances, uses cases are further elaborated to provide considerably more detail about the interaction. For example, Cockburn [Coc01b] suggests the following template for detailed descriptions of use cases:

Use case:	*InitiateMonitoring*
Primary actor:	Homeowner.
Goal in context:	To set the system to monitor sensors when the homeowner leaves the house or remains inside.
Preconditions:	System has been programmed for a password and to recognize various sensors.
Trigger:	The homeowner decides to "set" the system, that is, to turn on the alarm functions.

Scenario:

1. Homeowner observes control panel.
2. Homeowner enters password.
3. Homeowner selects "stay" or "away."
4. Homeowner observes red alarm light to indicate that *SafeHome* has been armed.

Exceptions:

1. Control panel is *not ready:* Homeowner checks all sensors to determine which are open and then closes them.
2. Password is incorrect (control panel beeps once): Homeowner reenters correct password.
3. Password not recognized: Monitoring and response subsystem must be contacted to reprogram password.
4. *Stay* is selected: Control panel beeps twice, and a *stay* light is lit; perimeter sensors are activated.
5. *Away* is selected: Control panel beeps three times, and an *away* light is lit; all sensors are activated.

Priority: Essential, must be implemented
When available: First increment
Frequency of use: Many times per day
Channel to actor: Via control panel interface
Secondary actors: Support technician, sensors

Channels to secondary actors:

Support technician: phone line

Sensors: hardwired and radio frequency interfaces

Open issues:

1. Should there be a way to activate the system without the use of a password or with an abbreviated password?
2. Should the control panel display additional text messages?
3. How much time does the homeowner have to enter the password from the time the first key is pressed?
4. Is there a way to deactivate the system before it actually activates?

Use cases for other **homeowner** interactions would be developed in a similar manner. It is important to review each use case with care. If some element of the interaction is ambiguous, it is likely that a review of the use case will indicate a problem. Use cases are often written informally as user stories. However, using the template shown here helps to ensure that you've addressed all key issues. This is very important for systems where user safety or security is a stakeholder concern.

SAFEHOME



Developing a High-Level Use Case Diagram

The scene: A meeting room, continuing the requirements gathering meeting.

The players: Jamie Lazar, software team member; Vinod Raman, software team member; Ed Robbins, software team member; Doug Miller, software engineering manager; three members of marketing; a product engineering representative; and a facilitator.

The conversation:

Facilitator: We've spent a fair amount of time talking about *SafeHome* home security functionality. During the break I sketched a use case diagram to summarize the important scenarios that are part of this function. Take a look.

(All attendees look at Figure 7.2.)

Jamie: I'm just beginning to learn UML notation. So the home security function is represented by the big box with the ovals inside it? And the ovals represent use cases that we've written in text?

Facilitator: Yep. And the stick figures represent actors—the people or things that interact with the system as described by the use case . . . oh, I use the labeled square to represent an actor that's not a person . . . in this case, sensors.

Doug: Is that legal in UML?

Facilitator: Legality isn't the issue. The point is to communicate information. I view the use of a humanlike stick figure for representing a device to be misleading. So I've adapted things a bit. I don't think it creates a problem.

Vinod: Okay, so we have use case narratives for each of the ovals. Do we need to develop the more detailed template-based narratives I've read about?

Facilitator: Probably, but that can wait until we've considered other *SafeHome* functions.

Marketing person: Wait, I've been looking at this diagram and all of a sudden I realize we missed something.

Facilitator: Oh really. Tell me what we've missed.

(The meeting continues.)

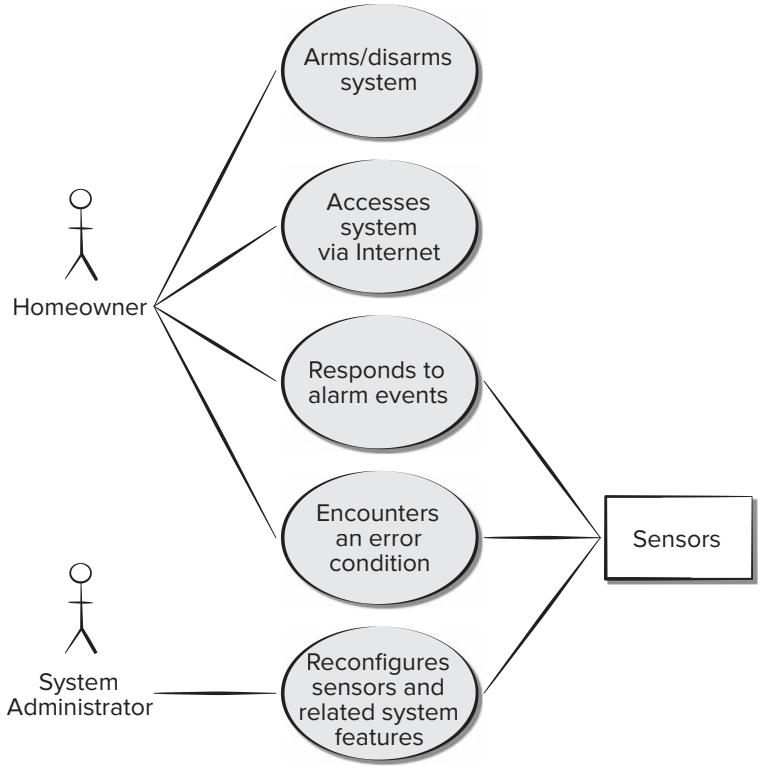
7.5 BUILDING THE ANALYSIS MODEL

The intent of the analysis model is to provide a description of the required informational, functional, and behavioral domains for a computer-based system. The model changes dynamically as you learn more about the system to be built, and as stakeholders understand more about what they really require. For that reason, the analysis model is a snapshot of requirements at any given time. You should expect it to change.

As the analysis model evolves, certain elements will become relatively stable, providing a solid foundation for the design tasks that follow. However, other elements of the model may be more volatile, indicating that stakeholders do not yet fully understand requirements for the system. If your team finds that it does not use certain elements of the analysis model as the project moves to design and construction, those elements should not be created in the future and should not be maintained as the requirements change in the current project. The analysis model and the methods that are used to build it are presented in detail in Chapter 8. We present a brief overview in the sections that follow.

FIGURE 7.2

UML use case diagram for *SafeHome* home security function



7.5.1 Elements of the Analysis Model

There are many ways to look at the requirements for a computer-based system. Some software people argue that it's best to select one mode of representation (e.g., the use case) and apply it to the exclusion of all other modes. Other practitioners believe that it's worthwhile to use several different modes of representation to depict the analysis model. Using different modes of representation forces you to consider requirements from different viewpoints—an approach that has a higher probability of uncovering omissions, inconsistencies, and ambiguity. It is always a good idea to get stakeholders involved. One of the best ways to do this is to have each stakeholder write use cases that describe how the software will be used. A set of generic elements common to most analysis models is introduced in this chapter.

Scenario-Based Elements. Scenario-based elements of the requirements model are often the first part of the model that is developed. They describe the system from the user's point of view. For example, basic user stories (Section 7.4) and their corresponding use case diagrams (Figure 7.2) may evolve into more elaborate template-based use cases (Section 7.4). As such, they serve as input for the creation of other modeling elements. It is always a good idea to get stakeholders involved. One of the best ways to do this is to have each stakeholder write use cases that describe how the software will be used.

Class-Based Elements. Each usage scenario implies a set of objects that are manipulated as an actor interacts with the system. These objects are categorized into classes—a collection of things that have similar attributes and common behaviors. For example, a UML class diagram can be used to depict a **Sensor** class for the *SafeHome* security function (Figure 7.3).

Note that the diagram lists the attributes of sensors (e.g., name, type) and the operations (e.g., *identify, enable*) that can be applied to modify these attributes. Other analysis modeling elements depict how classes collaborate with one another and the relationships and interactions among classes. One way to isolate classes is to look for descriptive nouns in a use case script. At least some of the nouns will be candidate classes. The verbs found in the use case script may be considered candidate methods for these classes. These and other techniques are discussed in more detail in Chapter 8.

Behavioral Elements. The behavior of a computer-based system can have a profound effect on the design that is chosen and the implementation approach that is applied. Therefore, the requirements model must provide modeling elements that depict behavior.

The *state diagram* is one method for representing the behavior of a system by depicting its states and the events that cause the system to change state. A *state* is any externally observable mode of behavior. In addition, the state diagram indicates what actions (e.g., process activations) are taken when events occur. External stimuli (events) cause transitions between states.

FIGURE 7.3
Class diagram for sensor

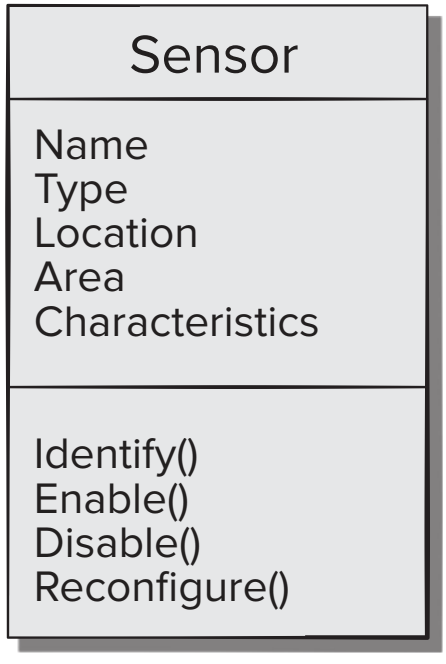
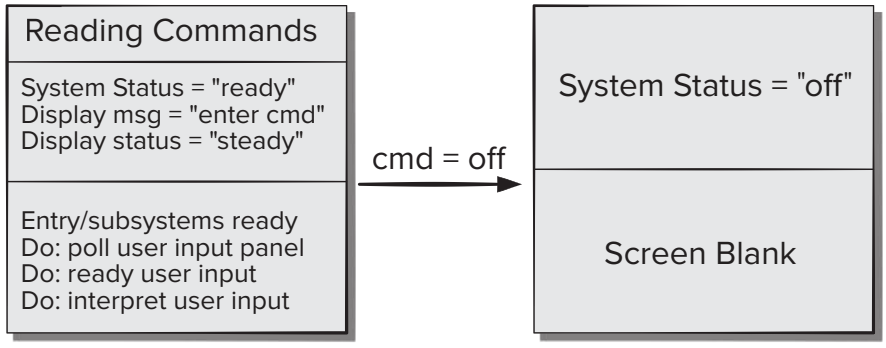


FIGURE 7.4

UML state diagram notation



To illustrate the use of a state diagram, consider software embedded within the *SafeHome* control panel that is responsible for reading user input. An example of UML state diagram notation is shown in Figure 7.4. Further discussion of behavioral modeling is presented in Chapter 8.

SAFEHOME



Preliminary Behavioral Modeling

The scene: A meeting room, continuing the requirements meeting.

The players: Jamie Lazar, software team member; Vinod Raman, software team member; Ed Robbins, software team member; Doug Miller, software engineering manager; three members of marketing; a product engineering representative; and a facilitator.

The conversation:

Facilitator: We've just about finished talking about *SafeHome* home security functionality. But before we do, I want to discuss the behavior of the function.

Marketing person: I don't understand what you mean by behavior.

Ed (smiling): That's when you give the product a "timeout" if it misbehaves.

Facilitator: Not exactly. Let me explain.

(The facilitator explains the basics of behavioral modeling to the requirements gathering team.)

Marketing person: This seems a little technical. I'm not sure I can help here.

Facilitator: Sure you can. What behavior do you observe from the user's point of view?

Marketing person: Uh . . . well, the system will be *monitoring* the sensors. It'll be *reading commands* from the homeowner. It'll be *displaying* its status.

Facilitator: See, you can do it.

Jamie: It'll also be *polling* the PC to determine if there is any input from it, for example, Internet-based access or configuration information.

Vinod: Yeah, in fact, *configuring the system* is a state in its own right.

Doug: You guys are rolling. Let's give this a bit more thought . . . is there a way to diagram this stuff?

Facilitator: There is, but let's postpone that until after the meeting.

7.5.2 Analysis Patterns

Anyone who has done requirements engineering on more than a few software projects notices that certain problems reoccur across all projects within a specific application domain. These *analysis patterns* [Fow97] suggest solutions (e.g., a class, a function, a behavior) within the application domain that can be reused when modeling many applications.

Analysis patterns are integrated into the analysis model by reference to the pattern name. They are also stored in a repository so that requirements engineers can use search facilities to find and reuse them. Information about an analysis pattern (and other types of patterns) is presented in a standard template [Gey01] that is discussed in more detail in Chapter 14. Examples of analysis patterns and further discussion of this topic are presented in Chapter 8.

7.6 NEGOTIATING REQUIREMENTS

In an ideal world, the requirements engineering tasks (inception, elicitation, and elaboration) determine customer requirements in sufficient detail to proceed to subsequent software engineering activities. Unfortunately, this rarely happens. You may have to enter into *negotiations* with one or more stakeholders. In most cases, stakeholders are asked to balance functionality, performance, and other product or system characteristics against cost and time to market. The intent of these negotiations is to develop a project plan that meets stakeholder needs while at the same time reflecting the real-world constraints (e.g., time, people, budget) that have been placed on the software team.

The best negotiations strive for a "win-win" result. That is, stakeholders win by getting the system or product that satisfies most their needs, and you (as a member of the software team) win by working to realistic and achievable budgets and deadlines.

Fricker [Fri10] and his colleagues suggest replacing the traditional handoff of requirements specifications to software teams with a bidirectional communication process called handshaking. Handshaking might be one way to accomplish a win-win result. In *handshaking*, the software team proposes solutions to requirements, describes their impact, and communicates their intentions to customer representatives. The customer representatives review the proposed solutions, focusing on missing features and seeking clarification of novel requirements. Requirements are determined to be *good enough* if the customers accept the proposed solution. Handshaking tends to improve identification, analysis, and selection of variants and promotes win-win negotiation.

SAFEHOME



The Start of a Negotiation

The scene: Lisa Perez's office, after the first requirements gathering meeting.

The players: Doug Miller, software engineering manager, and Lisa Perez, marketing manager.

The conversation:

Lisa: So, I hear the first meeting went really well.

Doug: Actually, it did. You sent some good people to the meeting . . . they really contributed.

Lisa (smiling): Yeah, they actually told me they got into it, and it wasn't a "propeller head activity."

Doug (laughing): I'll be sure to take off my techie beanie the next time I visit . . . Look, Lisa, I think we may have a problem with getting all of the functionality for the home security system out by the dates your management is talking about. It's early, I know, but I've already been doing a little back-of-the-envelope planning and . . .

Lisa (frowning): We've got to have it by that date, Doug. What functionality are you talking about?

Doug: I figure we can get full home security functionality out by the drop-dead date, but we'll have to delay Internet access 'til the second release.

Lisa: Doug, it's the Internet access that gives *SafeHome* "gee whiz" appeal. We're going to build our entire marketing campaign around it. We've gotta have it!

Doug: I understand your situation, I really do. The problem is that in order to give you Internet access, we'll have to have a fully secure website up and running. That takes time and people. We'll also have to build a lot of additional functionality into the first release . . . I don't think we can do it with the resources we've got.

Lisa (still frowning): I see, but you've got to figure out a way to get it done. It's pivotal to home security functions and to other functions as well . . . those can wait until the next releases . . . I'll agree to that.

Lisa and Doug appear to be at an impasse, and yet they must negotiate a solution to this problem. Can they both "win" here? Playing the role of a mediator, what would you suggest?

7.7 REQUIREMENTS MONITORING

Incremental development is commonplace. This means that use cases evolve, new test cases are developed for each new software increment, and continuous integration of source code occurs throughout a project. *Requirements monitoring* can be extremely useful when incremental development is used. It encompasses five tasks: (1) *distributed debugging* uncovers errors and determines their cause, (2) *run-time verification* determines whether software matches its specification, (3) *run-time validation* assesses whether the evolving software meets user goals, (4) *business activity monitoring* evaluates whether a system satisfies business goals, and (5) *evolution and codesign* provides information to stakeholders as the system evolves.

Incremental development implies the need for incremental validation. Requirements monitoring supports continuous validation by analyzing user goal models against the system in use. For example, a monitoring system might continuously assess user satisfaction and use feedback to guide incremental improvements [Rob10].

7.8 VALIDATING REQUIREMENTS

As each element of the requirements model is created, it is examined for inconsistency, omissions, and ambiguity. This is true even for agile process models where requirements tend to be written as user stories and/or test cases. The requirements represented

by the model are prioritized by stakeholders and grouped within requirements packages that will be implemented as software increments. A review of the requirements model addresses the following questions:

1. Is each requirement consistent with the overall objectives for the system or product?
2. Have all requirements been specified at the proper level of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?
3. Is the requirement really necessary or does it represent an add-on feature that may not be essential to the objective of the system?
4. Is each requirement bounded and unambiguous?
5. Does each requirement have attribution? That is, is a source (generally, a specific individual) noted for each requirement?
6. Do any requirements conflict with other requirements?
7. Is each requirement achievable in the technical environment that will house the system or product?
8. Is each requirement testable, once implemented?
9. Does the requirements model properly reflect the information, function, and behavior of the system to be built?
10. Has the requirements model been "partitioned" in a way that exposes progressively more detailed information about the system?
11. Have requirements patterns been used to simplify the requirements model? Have all patterns been properly validated? Are all patterns consistent with customer requirements?

These and other questions should be asked and answered to ensure that the requirements model is an accurate reflection of stakeholder needs and that it provides a solid foundation for design.

7.9 SUMMARY

Requirements engineering tasks are conducted to establish a solid foundation for design and construction. Requirements engineering occurs during the communication and modeling activities that have been defined for the generic software process. Seven requirements engineering activities—inception, elicitation, elaboration, negotiation, specification, validation, and management—are conducted by members of the software team and product stakeholders.

At project inception, stakeholders establish basic problem requirements, define overriding project constraints, and address major features and functions that must be present for the system to meet its objectives. This information is refined and expanded during elicitation—a requirements gathering activity that makes use of facilitated meetings and the development of usage scenarios (user stories).

Elaboration further expands requirements in a model—a collection of scenario-based, activity-based, class-based, and behavioral elements. The model may reference

analysis patterns, characteristics of the problem domain that have been seen to reoccur across different applications.

As requirements are identified and the requirements model is being created, the software team and other project stakeholders negotiate the priority, availability, and relative cost of each requirement. The intent of this negotiation is to develop a realistic project plan. Each requirement needs to be validated against customer needs to ensure that the right system is to be built.

PROBLEMS AND POINTS TO PONDER

7.1. Why is it that many software developers don't pay enough attention to requirements engineering? Are there ever circumstances where you can skip it?

7.2. You have been given the responsibility to elicit requirements from a customer who tells you he is too busy to meet with you. What should you do?

7.3. Discuss some of the problems that occur when requirements must be elicited from three or four different customers.

7.4. Your instructor will divide the class into groups of four or six students. Half of the group will play the role of the marketing department and half will take on the role of software engineering. Your job is to define requirements for the *SafeHome* security function described in this chapter. Conduct a requirements gathering meeting using the guidelines presented in this chapter.

7.5. Develop a complete use case for one of the following activities:

a. Making a withdrawal at an ATM
b. Using your charge card for a meal at a restaurant
c. Searching for books (on a specific topic) using an online bookstore

7.6. Write a user story for one of the activities listed in Problem 7.5.

7.7. Consider the use case you created in Problem 7.5, and write a nonfunctional requirement for the application.

7.8. Using the template presented in Section 7.5.2, suggest one or more analysis patterns for the following application domains:

a. E-mail software.
b. Internet browsers.
c. Mobile app creation software.

7.9. What does *win-win* mean in the context of negotiation during the requirements engineering activity?

7.10. What do you think happens when requirement validation uncovers an error? Who is involved in correcting the error?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

8 REQUIREMENTS MODELING—A RECOMMENDED APPROACH

The written word is a wonderful vehicle for communication, but it is not necessarily the best way to represent the requirements for computer software. At a technical level, software engineering begins with a series of modeling tasks that lead to a specification of requirements and a design representation for the software to be built. The requirements model is actually a set of models that make up the first technical representation of a system. Software engineers often prefer to include graphical representations of complex model relationships.

KEY CONCEPTS

activity diagram 146
analysis classes 137
attributes 140
behavioral model 149
class-based modeling 127
collaborations 144
CRC modeling 144
events 149
formal use case 135
functional model 146
grammatical parse 137
operations 140
procedural view 146
requirements analysis 127
requirements modeling 129
responsibilities 144
scenario-based modeling 128
sequence diagrams 148
state diagrams 150
swimlane diagram 151
UML models 130
use cases 131
  documentation 135
  exception 134



QUICK LOOK

What is it? Requirements modeling uses a combination of text and diagrammatic forms to depict requirements in a way that is relatively easy to understand, and more important, straightforward to review for correctness, completeness, and consistency.

Who does it? A software engineer (sometimes called an analyst) builds these models from requirements elicited from various stakeholders.

Why is it important? Requirements models can be readily evaluated by all stakeholders, resulting in useful feedback at the earliest possible time. Later, as the model is refined, it becomes the basis for software design.

What are the steps? Requirements modeling combines three steps: scenario-based modeling, class modeling, and behavioral modeling.

What is the work product? Usage scenarios, called use cases, describe software functions and usage. In addition, a series of UML diagrams can be used to represent system behavior and other aspects.

How do I ensure that I've done it right? Requirements modeling work products must be reviewed for correctness, completeness, and consistency.

For some types of software, a user story (Section 7.3.2) may be the only requirements modeling representation that is required. For others, formal use cases (Section 7.4) and class-based models (Section 8.3) may be developed. Class-based modeling represents the objects that the system will manipulate, the operations (also called *methods* or *services*) that will be applied to the objects to effect the manipulation, relationships (some hierarchical) between the objects, and the collaborations that occur between the classes that are defined. Class-based methods can be used to create a representation of an application that can be understood by nontechnical stakeholders.

In still other situations, complex application requirements may demand an examination of how an application behaves in reaction to either internal or external events. These behaviors need to be modeled (Section 8.5) as well. UML diagrams have become a standard software engineering means of modeling analysis model element relationships and behaviors graphically. As the requirements model is refined and expanding, it evolves into a specification that can be used by software engineers in the creation of the software design.

The important thing to keep in mind when modeling requirements is to only create the models that will be used by the development team. If models developed early in a requirements analysis phase of a project are not referred to during the design and implementation phases, they may not be worth updating. The sections that follow present a series of informal guidelines that will assist in the creation and representation of requirements models.

8.1 REQUIREMENTS ANALYSIS

Requirements analysis results in the specification of software's operational characteristics, indicates software's interface with other system elements, and establishes constraints that software must meet. Requirements analysis allows you (regardless of whether you're called a *software engineer,* an *analyst,* or a *modeler*) to elaborate on basic requirements established during the inception, elicitation, and negotiation tasks that are part of requirements engineering (Chapter 7).

The requirements modeling action results in one or more of the following types of models:

- *Scenario-based models* of requirements from the point of view of various system "actors."
- *Class-oriented models* that represent object-oriented classes (attributes and operations) and how classes collaborate to achieve system requirements.
- *Behavioral models* that depict how the software reacts to internal or external "events."
- *Data models* that depict the information domain for the problem.
- *Flow-oriented models* that represent the functional elements of the system and how they transform data as they move through the system.

These models provide a software designer with information that can be translated to architectural-, interface-, and component-level designs. Finally, the requirements

model (and the software requirements specification) provides the developer and the customer with the means to assess quality once software is built.

In this section, we focus on *scenario-based modeling*—a technique that is very popular throughout the software engineering community. In Sections 8.3 and 8.5 we consider class-based modeling and behavioral modeling. Over the past decade, flow and data modeling have become less commonly used, while scenario and class-based methods, supplemented with behavioral approaches have grown in popularity.[1]

8.1.1 Overall Objectives and Philosophy

Throughout analysis modeling, your primary focus is on *what,* not *how.* What user interaction occurs, what objects does the system manipulate, what functions must the system perform, what behaviors does the system exhibit, what interfaces are defined, and what constraints apply?[2]

In previous chapters, we noted that complete specification of requirements may not be possible at this stage. The customer may be unsure of precisely what is required for certain aspects of the system. The developer may be unsure that a specific approach will properly accomplish function and performance. These realities mitigate in favor of an iterative approach to requirements analysis and modeling. The analyst should model what is known and use that model as the basis for design of the software increment.[3]

The requirements model must achieve three primary objectives: (1) to describe what the customer requires, (2) to establish a basis for the creation of a software design, and (3) to define a set of requirements that can be validated once the software is built. The analysis model bridges the gap between a system-level description that describes overall system or business functionality (software, hardware, data, human elements) and a software design (Chapters 9 through 14) that describes the software's application architecture, user interface, and component-level structure. This relationship is illustrated in Figure 8.1.

It is important to note that all elements of the requirements model will be directly traceable to parts of the design model. A clear division between analysis and design modeling tasks is not always possible. Some design invariably occurs as part of analysis, and some analysis will be conducted during design.

8.1.2 Analysis Rules of Thumb

Several rules of thumb [Arl02] are worth considering when creating an analysis model. First, focus on the problem or business domain while keeping the level of abstraction high. Second, recognize that an analysis model should provide insight into

---

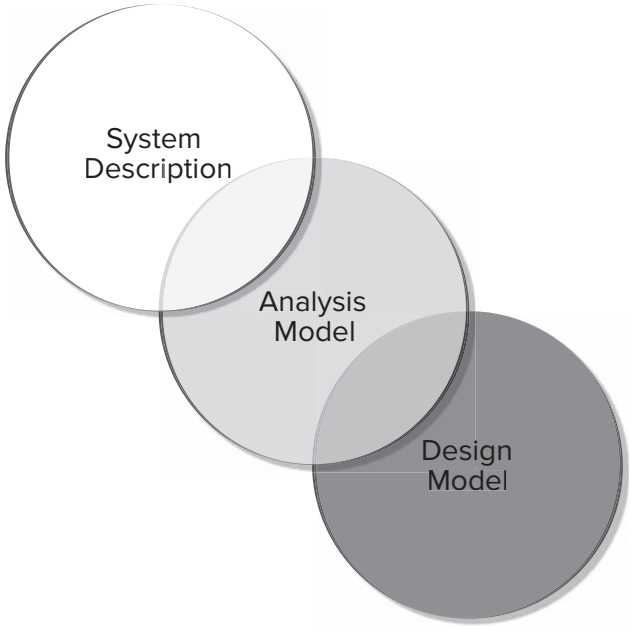
1 A presentation of flow-oriented modeling and data modeling is no longer included in this chapter. However, copious information about these older requirements modeling methods can be found on the Web. If you have interest, use the search phrase "structured analysis."

2 It should be noted that as customers become more technologically sophisticated, there is a trend toward the specification of *how* as well as *what.* However, the primary focus should remain on *what.*

3 Alternatively, the software team may choose to create a prototype (Chapter 4) to better understand requirements for the system.

FIGURE 8.1

The analysis model as a bridge between the system description and the design model



the information domain, the function, and the behavior of the software. Third, delay a consideration of software architecture and nonfunctional details until later in the modeling activity. Also, it's important to be aware of the ways in which elements of the software are interconnnected with other elements (we call this *system coupling*).

The analysis model must be structured in a way that provides value to all stakeholders and should be kept as simple as possible without sacrificing clarity.

8.1.3 Requirements Modeling Principles

Over the past four decades, several requirements modeling methods have been developed. Investigators have identified requirements analysis problems and their causes and have developed a variety of modeling notations and corresponding sets of heuristics to overcome them. Each analysis method has a unique point of view. A set of operational principles relates analysis methods:

Principle 1. ***The information domain of a problem must be represented and understood.*** The *information domain* encompasses the data that flow into the system (from end users, other systems, or external devices), the data that flow out of the system (via the user interface, network interfaces, reports, graphics, and other means), and the data stores that collect and organize the data that are maintained permanently.

Principle 2. ***The functions that the software performs must be defined.*** Software functions provide direct benefit to end users and those that provide internal support for those features that are user visible. Some functions transform data that

flow into the system. In other cases, functions effect some level of control over internal software processing or external system elements.

Principle 3. ***The behavior of the software (as a consequence of external events) must be represented.*** The behavior of computer software is driven by its interaction with the external environment. Input provided by end users, control data provided by an external system, or monitoring data collected over a network all cause the software to behave in a specific way.

Principle 4. ***The models that depict information, function, and behavior must be partitioned in a manner that uncovers detail in a layered (or hierarchical) fashion.*** Requirements modeling is the first step in software engineering problem solving. It allows you to better understand the problem and establishes a basis for the solution (design). Complex problems are difficult to solve in their entirety. For this reason, you should use a divide-and-conquer strategy. A large, complex problem is divided into subproblems until each subproblem is relatively easy to understand. This concept is called *partitioning* or *separation of concerns,* and it is a key strategy in requirements modeling.

Principle 5. ***The analysis task should move from essential information toward implementation detail.*** Analysis modeling begins by describing the problem from the end-user's perspective. The "essence" of the problem is described without any consideration of how a solution will be implemented. For example, a video game requires that the player "instruct" its protagonist on what direction to proceed as she moves into a dangerous maze. That is the essence of the problem. Implementation detail (normally described as part of the design model) indicates how the essence will be implemented. For the video game, voice input might be used. Alternatively, a keyboard command might be typed, a game pad joystick (or mouse) might be pointed in a specific direction, a motion-sensitive device might be waved in the air, or a device that reads the player's body or eye movements directly can be used.

By applying these principles, a software engineer approaches a problem systematically. But how are these principles applied in practice? This question will be answered in the remainder of this chapter.

8.2 SCENARIO-BASED MODELING

Although the success of a computer-based system or product is measured in many ways, user satisfaction resides at the top of the list. If you understand how end users (and other actors) want to interact with a system, your software team will be better able to properly characterize requirements and build meaningful analysis and design models. Using UML[4] to model requirements begins with the creation of scenarios in the form of use case diagrams, activity diagrams, and sequence diagrams.

4 UML will be used as the modeling notation throughout this book. Appendix 1 provides a brief tutorial for those readers who may be unfamiliar with basic UML notation.

8.2.1 Actors and User Profiles

A UML *actor* models an entity that interacts with a system object. Actors may represent roles played by human stakeholders or external hardware as they interact with system objects by exchanging information. A single physical entity may be portrayed by several actors if the physical entity takes on several roles that are relevant to realizing different system functions.

A UML *profile* provides a way of extending an existing model to other domains or platforms. This might allow you to revise the model of a Web-based system and model the system for various mobile platforms. Profiles might also be used to model the system from the viewpoints of different users. For example, system administrators may have a different view of the functionality of an automated teller machine than end users.

8.2.2 Creating Use Cases

In Chapter 7, we discussed user stories as a way of summarizing the stakeholders' perspective on how they will interact with the proposed system. However, they are written in plain English or the language used by the stakeholders. Developers need more precise means of describing this interaction before beginning to create the software. Alistair Cockburn characterizes a *use case* as a "contract for behavior" [Coc01b]. As we discussed in Chapter 7, the "contract" defines the way in which an actor[5] uses a computer-based system to accomplish some goal. In other words, a use case captures the interactions that occur between producers and consumers of information within the system itself. In this section, we examine how preliminary use cases are developed as part of the analysis modeling activity.[6]

In Chapter 7, we noted that a use case describes a specific usage scenario in straightforward language from the point of view of a defined actor. But how do you know (1) what to write about, (2) how much to write about it, (3) how detailed to make your description, and (4) how to organize the description? These are the questions that must be answered if use cases are to provide value as a modeling tool.

What to Write About? The first two requirements engineering tasks—inception and elicitation—provide you with the information you'll need to begin writing use cases. Requirements gathering meetings and other requirements engineering mechanisms are used to identify stakeholders, define the scope of the problem, specify overall operational goals, establish priorities, outline all known functional requirements, and describe the things (objects) that will be manipulated by the system.

To begin developing a set of use cases, list the functions or activities performed by a specific actor. You can obtain these from a list of required system functions, through conversations with stakeholders, or by an evaluation of activity diagrams (Section 8.4) developed as part of requirements modeling.

---

5 An actor is not a specific person, but rather a role that a person (or a device) plays within a specific context. An actor "calls on the system to deliver one of its services" [Coc01b].

6 Use cases are a particularly important part of analysis modeling for user interfaces. Interface analysis and design is discussed in detail in Chapter 12.

SAFEHOME



Developing Another Preliminary Use Case

The scene: A meeting room, during the second requirements gathering meeting.

The players: Jamie Lazar, software team member; Ed Robbins, software team member; Doug Miller, software engineering manager; three members of marketing; a product engineering representative; and a facilitator.

The conversation:

Facilitator: It's time that we begin talking about the *SafeHome* surveillance function. Let's develop a user scenario for access to the surveillance function.

Jamie: Who plays the role of the actor on this?

Facilitator: I think Meredith (a marketing person) has been working on that functionality. Why don't you play the role?

Meredith: You want to do it the same way we did it last time, right?

Facilitator: Right . . . same way.

Meredith: Well, obviously the reason for surveillance is to allow the homeowner to check out the house while he or she is away, to record and play back video that is captured . . . that sort of thing.

Ed: Will we use compression to store the video?

Facilitator: Good question, Ed, but let's postpone implementation issues for now. Meredith?

Meredith: Okay, so basically there are two parts to the surveillance function . . . the first configures the system including laying out a floor plan—we need to have the AR/VR tools to help the homeowner do this—and the second part is the actual surveillance function itself. Since the layout is part of the configuration activity, I'll focus on the surveillance function.

Facilitator (smiling): Took the words right out of my mouth.

Meredith: Um . . . I want to gain access to the surveillance function either via a mobile device or via the Internet. My feeling is that the Internet access would be more frequently used. Anyway, I want to be able to display camera views on a mobile device or PC and control pan and zoom for a specific camera. I specify the camera by selecting it from the house floor plan. I want to selectively record camera output and replay camera output. I also want to be able to block access to one or more cameras with a specific password. I also want the option of seeing small windows that show views from all cameras and then be able to pick the one I want enlarged.

Jamie: Those are called thumbnail views.

Meredith: Okay, then I want thumbnail views of all the cameras. I also want the interface for the surveillance function to have the same look and feel as all other *SafeHome* interfaces. I want it to be intuitive, meaning I don't want to have to read a manual to use it.

Facilitator: Good job. Now, let's go into this function in a bit more detail . . .

The *SafeHome* home surveillance function (subsystem) discussed in the sidebar identifies the following functions (an abbreviated list) that are performed by the **homeowner** actor:

- Select camera to view.
- Request thumbnails from all cameras.
- Display camera views in a device window.
- Control pan and zoom for a specific camera.

- Selectively record camera output.
- Replay camera output.
- Access camera surveillance via the Internet.

As further conversations with the stakeholder (who plays the role of a homeowner) progress, the requirements gathering team develops use cases for each of the functions noted. In general, use cases are written first in an informal narrative fashion. If more formality is required, the same use case is rewritten using a structured format like the one proposed in Chapter 7.

To illustrate, consider the function *access camera surveillance via the Internet—display camera views* (**ACS-DCV**). The stakeholder who takes on the role of the **homeowner** actor might write the following user story:

Use case: Access camera surveillance via the Internet—display camera views (ACS-DCV)

Actor: homeowner

If I'm at a remote location, I can use any mobile device with appropriate browser software to log on to the *SafeHome Products* website. I enter my user ID and two levels of passwords and once I'm validated, I have access to all functionality for my installed *SafeHome* system. To access a specific camera view, I select "surveillance" from the major function buttons displayed. I then select "pick a camera" and the floor plan of the house is displayed. I then select the camera that I'm interested in. Alternatively, I can look at thumbnail snapshots from all cameras simultaneously by selecting "all cameras" as my viewing choice. Once I choose a camera, I select "view" and a one-frame-per-second view appears in a viewing window that is identified by the camera ID. If I want to switch cameras, I select "pick a camera," the original viewing window disappears, and the floor plan of the house is displayed again. I then select the camera that I'm interested in. A new viewing window appears.

A variation of a narrative use case presents the interaction as an ordered sequence of user actions. Each action is represented as a declarative sentence. Revisiting the **ACS-DCV** function, you would write:

Use case: Access camera surveillance via the Internet—display camera views (ACS-DCV)

Actor: homeowner

1. The homeowner logs onto the *SafeHome Products* website.
2. The homeowner enters his or her user ID.
3. The homeowner enters two passwords (each at least eight characters in length).
4. The system displays all major function buttons.
5. The homeowner selects the "surveillance" from the major function buttons.
6. The homeowner selects "pick a camera."
7. The system displays the floor plan of the house.
8. The homeowner selects a camera icon from the floor plan.
9. The homeowner selects the "view" button.
10. The system displays a viewing window that is identified by the camera ID.
11. The system displays video output within the viewing window at one frame per second.

It is important to note that this sequential presentation does not consider any alternative interactions (the narrative is free flowing and did represent a few alternatives). Use cases of this type are sometimes referred to as *primary scenarios* [Sch98].

A description of alternative interactions is essential for a complete understanding of the function that is being described by a use case. Therefore, each step in the primary scenario is evaluated by asking the following questions [Sch98]:

- *Can the actor take some other action at this point?*
- *Is it possible that the actor will encounter some error condition at this point?* If so, what might it be?
- *Is it possible that the actor will encounter some other behavior at this point (e.g., behavior that is invoked by some event outside the actor's control)?* If so, what might it be?

Answers to these questions result in the creation of a set of *secondary scenarios* that are part of the original use case but represent alternative behavior. For example, consider steps 6 and 7 in the primary scenario presented earlier:

6. The homeowner selects "pick a camera."

7. The system displays the floor plan of the house.

Can the actor take some other action at this point? The answer is yes. Referring to the free-flowing narrative, the actor may choose to view thumbnail snapshots of all cameras simultaneously. Hence, one secondary scenario might be "View thumbnail snapshots for all cameras."

Is it possible that the actor will encounter some error condition at this point? Any number of error conditions can occur as a computer-based system operates. In this context, we consider only error conditions that are likely as a direct result of the action described in step 6 or step 7. Again, the answer to the question is yes. A floor plan with camera icons may have never been configured. Hence, selecting "pick a camera" results in an error condition: "No floor plan configured for this house."[7] This error condition becomes a secondary scenario.

Is it possible that the actor will encounter some other behavior at this point? Again, the answer to the question is yes. As steps 6 and 7 occur, the system may encounter an alarm condition. This would result in the system displaying a special alarm notification (type, location, system action) and providing the actor with several options relevant to the nature of the alarm. Because this secondary scenario can occur at any time for virtually all interactions, it will not become part of the **ACS-DCV** use case. Rather, a separate use case—**Alarm condition encountered**—would be developed and referenced from other use cases as required.

Each of the situations described in the preceding paragraphs is characterized as a use case exception. An *exception* describes a situation (either a failure condition or

7 In this case, another actor, the **system administrator,** would have to configure the floor plan, install (e.g., assign an equipment ID) all cameras and initialize them, and test each camera to be certain that it is accessible via the system and through the floor plan.

an alternative chosen by the actor) that causes the system to exhibit somewhat different behavior.

Cockburn [Coc01b] recommends a "brainstorming" session to derive a reasonably complete set of exceptions for each use case. In addition to the three generic questions suggested earlier in this section, the following issues should also be explored:

- *Are there cases in which some "validation function" occurs during this use case?* This implies that the validation function is invoked, and a potential error condition might occur.
- *Are there cases in which a supporting function (or actor) will fail to respond appropriately?* For example, a user action awaits a response but the function that is to respond times out.
- *Can poor system performance result in unexpected or improper user actions?* For example, a Web-based or mobile interface responds too slowly, resulting in a user making multiple selects on a processing button. These selects queue inappropriately and ultimately generate an error condition.

The list of extensions developed by asking and answering these questions should be "rationalized" [Coc01b] using the following criteria: An exception should be noted within the use case if the software can detect the condition described and then handle the condition once it has been detected. In some cases, an exception will precipitate the development of another use case (to handle the condition noted).

8.2.3 Documenting Use Cases

The informal use cases presented in Section 8.2.2 are sometimes sufficient for requirements modeling. However, when a use case involves a critical activity or describes a complex set of steps with a significant number of exceptions, a more formal approach may be desirable.

The **ACS-DCV** use case shown in the sidebar follows a typical outline for formal use cases. The *goal in context* identifies the overall scope of the use case. The *precondition* describes what is known to be true before the use case is initiated. The *trigger* identifies the event or condition that "gets the use case started" [Coc01b]. The *scenario* lists the specific actions that are required by the actor and the appropriate system responses. *Exceptions* identify the situations uncovered as the preliminary use case is refined (Section 8.2.2). Additional headings may or may not be included and are reasonably self-explanatory.

Most developers like to create graphical representation as they create use cases out of user stories. A diagrammatic representation can facilitate better understanding of the problem by all stakeholders, particularly when the scenario is complex. As we noted earlier in this book, UML provides use case diagramming capability. Figure 8.2 depicts a use case diagram for the *SafeHome* product. The use case diagram helps to show the relations among the use cases in the usage scenario. Each use case is represented by an oval. Only the **ACS-DCV** use case has been discussed in this section.

Each modeling notation has limitations, and the UML use case is no exception. Like any other form of written description, a use case is only as good as its author(s). If the

SafeHome



Use Case Template for Surveillance

Use case: Access camera surveillance via the Internet—display camera views (ACS-DCV)

Iteration: 2, last modification: January 14 by V. Raman.

Primary actor: Homeowner.

Goal in context: To view output of cameras placed throughout the house from any remote location via the Internet.

Preconditions: System must be fully configured; appropriate user ID and passwords must be obtained.

Trigger: The homeowner decides to take a look inside the house while away.

Scenario:

1. The homeowner logs onto the *SafeHome Products* website.
2. The homeowner enters his or her user ID.
3. The homeowner enters two passwords (each at least eight characters in length).
4. The system displays all major function buttons.
5. The homeowner selects the "surveillance" button from the major function buttons.
6. The homeowner selects "pick a camera."
7. The system displays the floor plan of the house.
8. The homeowner selects a camera icon from the floor plan.
9. The homeowner selects the "view" button.
10. The system displays a viewing window that is identified by the camera ID.
11. The system displays video output within the viewing window at one frame per second.

Exceptions:

1. ID or passwords are incorrect or not recognized—see use case **Validate ID and passwords.**
2. Surveillance function not configured for this system—system displays appropriate error message; see use case **Configure surveillance function.**
3. Homeowner selects "View thumbnail snapshots for all camera"—see use case **View thumbnail snapshots for all cameras.**
4. A floor plan is not available or has not been configured—display appropriate error message and see use case **Configure floor plan.**
5. An alarm condition is encountered—see use case **Alarm condition encountered.**

Priority: Moderate priority, to be implemented after basic functions.

When available: Third increment.

Frequency of use: Infrequent.

Channel to actor: Via PC-based browser and Internet connection.

Secondary actors: System administrator, cameras.

Channels to secondary actors:

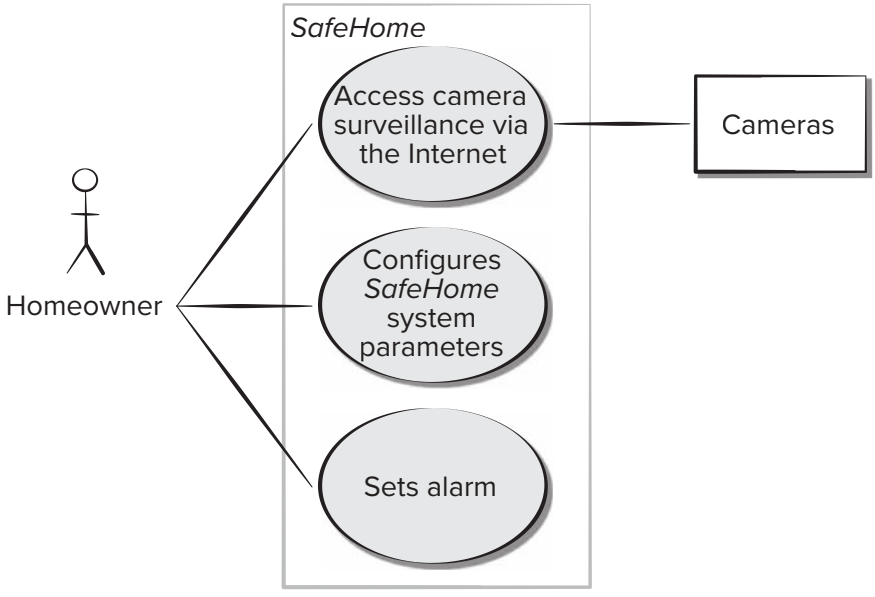
1. System administrator: PC-based system.
2. Cameras: wireless connectivity.

Open issues:

1. What mechanisms protect unauthorized use of this capability by employees of *SafeHome Products*?
2. Is security sufficient? Hacking into this feature would represent a major invasion of privacy.
3. Will system response via the Internet be acceptable given the bandwidth required for camera views?
4. Will we develop a capability to provide video at a higher frames-per-second rate when high-bandwidth connections are available?

FIGURE 8.2

Preliminary use case diagram for the *SafeHome* system



description is unclear, the use case can be misleading or ambiguous. A use case focuses on function and behavioral requirements and is generally inappropriate for nonfunctional requirements. For situations in which the requirements model must have significant detail and precision (e.g., safety critical systems), a use case may not be sufficient.

However, scenario-based modeling is appropriate for a significant majority of all situations that you will encounter as a software engineer. If developed properly, the use case can provide substantial benefit as a modeling tool.

8.3 CLASS-BASED MODELING

If you look around a room, there is a set of physical objects that can be easily identified, classified, and defined (in terms of attributes and operations). But when you "look around" the problem space of a software application, the classes (and objects) may be more difficult to comprehend.

8.3.1 Identifying Analysis Classes

We can begin to identify classes by examining the usage scenarios developed as part of the requirements model (Section 8.2) and performing a "grammatical parse" [Abb83] on the use cases developed for the system to be built. Classes are determined by underlining each noun or noun phrase and entering it into a simple table. Synonyms should be noted. If the class (noun) is required to implement a solution, then it is part of the solution space; otherwise, if a class is necessary only to describe a solution, it is part of the problem space.

But what should we look for once all the nouns have been isolated? *Analysis classes* manifest themselves in one of the following ways:

- *External entities* (e.g., other systems, devices, people) that produce or consume information to be used by a computer-based system.
- *Things* (e.g., reports, displays, letters, signals) that are part of the information domain for the problem.
- *Occurrences or events* (e.g., a property transfer or the completion of a series of robot movements) that occur within the context of system operation.
- *Roles* (e.g., manager, engineer, salesperson) played by people who interact with the system.
- *Organizational units* (e.g., division, group, team) that are relevant to an application.
- *Places* (e.g., manufacturing floor or loading dock) that establish the context of the problem and the overall function of the system.
- *Structures* (e.g., sensors, four-wheeled vehicles, or computers) that define a class of objects or related classes of objects.

This categorization is but one of many that have been proposed in the literature.[8] For example, Budd [Bud96] suggests a taxonomy of classes that includes *producers* (sources) and *consumers* (sinks) of data, *data managers*, *view* or *observer classes*, and *helper classes*.

To illustrate how analysis classes might be defined during the early stages of modeling, consider a grammatical parse (nouns are underlined, verbs italicized) for a processing narrative[9] for the *SafeHome* security function.

> The SafeHome security function *enables* the homeowner to *configure* the security system when it is *installed, monitors* all sensors *connected* to the security system, and *interacts* with the homeowner through the Internet, a PC or a control panel.
>
> During installation, the SafeHome PC is used to *program* and *configure* the system. Each sensor is assigned a number and type, a master password is programmed for *arming* and *disarming* the system, and telephone number(s) are *input* for *dialing* when a sensor event occurs.
>
> When a sensor event is *recognized*, the software *invokes* an audible alarm attached to the system. After a delay time that is *specified* by the homeowner during system configuration activities, the software dials a telephone number of a monitoring service, *provides* information about the location, *reporting* the nature of the event that has been detected. The telephone number will be *redialed* every 20 seconds until telephone connection is *obtained.*
>
> The homeowner *receives* security information via a control panel, the PC, or a browser, collectively called an interface. The interface *displays* prompting messages and system status information on the control panel, the PC, or the browser window. Homeowner interaction takes the following form . . .

8 Another important categorization, defining entity, boundary, and controller classes, is discussed in Section 10.3.

9 A processing narrative is similar to the use case in style but somewhat different in purpose. The processing narrative provides an overall description of the function to be developed. It is not a scenario written from one actor's point of view. It is important to note, however, that a grammatical parse can also be used for every use case developed as part of requirements gathering (elicitation).

Extracting the nouns, we can propose several potential classes:

Potential Class	General Classification
homeowner	role or external entity
sensor	external entity
control panel	external entity
installation	occurrence
system (alias security system)	thing
number, type	not objects, attributes of sensor
master password	thing
telephone number	thing
sensor event	occurrence
audible alarm	external entity
monitoring service	organizational unit or external entity

The list would be continued until all nouns in the processing narrative have been considered. Note that we call each entry in the list a "potential" object. We must consider each further before a final decision is made.

Coad and Yourdon [Coa91] suggest six selection characteristics that should be used as you consider each potential class for inclusion in the analysis model:

1. **Retained information.** The potential class will be useful during analysis only if information about it must be remembered so that the system can function.
2. **Needed services.** The potential class must have a set of identifiable operations that can change the value of its attributes in some way.
3. **Multiple attributes.** During requirement analysis, the focus should be on "major" information; a class with a single attribute may, in fact, be useful during design but is probably better represented as an attribute of another class during the analysis activity.
4. **Common attributes.** A set of attributes can be defined for the potential class, and these attributes apply to all instances of the class.
5. **Common operations.** A set of operations can be defined for the potential class, and these operations apply to all instances of the class.
6. **Essential requirements.** External entities that appear in the problem space and produce or consume information essential to the operation of any solution for the system will almost always be defined as classes in the requirements model.

To be considered a legitimate class for inclusion in the requirements model, a potential object should satisfy most of these characteristics. The decision for inclusion of potential classes in the analysis model is somewhat subjective, and later evaluation may cause an object to be discarded or reinstated. However, the first step of class-based modeling is the definition of classes, and decisions (even subjective ones) must

be made. You should apply the selection characteristics to the list of potential *SafeHome* classes:

Potential Class	Characteristic Number That Applies
homeowner	rejected: 1, 2 fail even though 6 applies
sensor	accepted: all apply
control panel	accepted: all apply
installation	rejected
system (alias security function)	accepted: all apply
number, type	rejected: 3 fails, attributes of sensor
master password	rejected: 3 fails
telephone number	rejected: 3 fails
sensor event	accepted: all apply
audible alarm	accepted: 2, 3, 4, 5, 6 apply
monitoring service	rejected: 1, 2 fail even though 6 applies

It should be noted that: (1) the preceding list is not all-inclusive, so additional classes would have to be added to complete the model; (2) some of the rejected potential classes will become attributes for those classes that were accepted (e.g., number and type are attributes of **Sensor,** and master password and telephone number may become attributes of **System**); and (3) different statements of the problem might cause different "accept or reject" decisions to be made (e.g., if each homeowner had an individual password or was identified by voice print, the **Homeowner** class would satisfy characteristics 1 and 2 and would have been accepted).

8.3.2 Defining Attributes and Operations

Attributes describe a class that has been selected for inclusion in the analysis model. It is the attributes that define the class—that clarify what is meant by the class in the context of the problem space.

To develop a meaningful set of attributes for an analysis class, you should study each use case and select those "things" that reasonably "belong" to the class. In addition, the following question should be answered for each class: *What data items (composite and/or elementary) fully define this class in the context of the problem at hand?*

To illustrate, we consider the **System** class defined for *SafeHome*. A homeowner can configure the security function to reflect sensor information, alarm response information, activation and deactivation information, identification information, and so forth. We can represent these composite data items in the following manner:

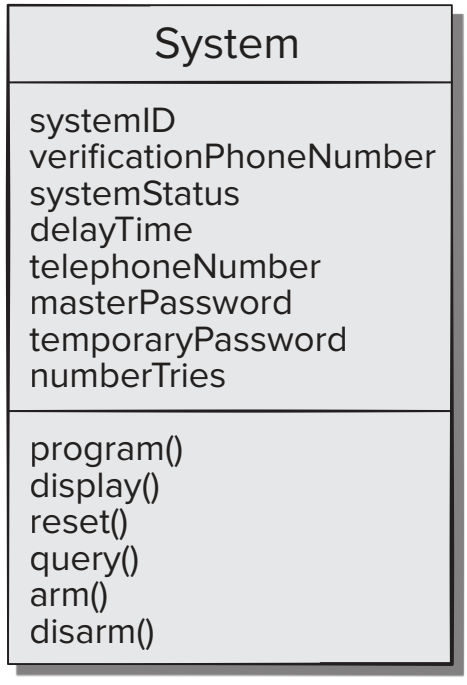
identification information = system ID + verification phone number + system status

alarm response information = delay time + telephone number

activation/deactivation information = master password + number of allowable tries + temporary password

FIGURE 8.3

Class diagram for the System class



Each of the data items to the right of the equal sign could be further defined to an elementary level, but for our purposes, they constitute a reasonable list of attributes for the **System** class (Figure 8.3).

Sensors are part of the overall *SafeHome* system, and yet they are not listed as data items or as attributes in Figure 8.3. **Sensor** has already been defined as a class, and multiple **Sensor** objects will be associated with the **System** class. In general, we avoid defining an item as an attribute if more than one of the items is to be associated with the class.

Operations define the behavior of an object. Although many different types of operations exist, they can generally be divided into four broad categories: (1) operations that manipulate data in some way (e.g., adding, deleting, reformatting, selecting), (2) operations that perform a computation, (3) operations that inquire about the state of an object, and (4) operations that monitor an object for the occurrence of a controlling event. These functions are accomplished by operating on attributes and/or associations (Section 8.3.3). Therefore, an operation must have "knowledge" of the nature of the class attributes and associations.

8.3.3 UML Class Models

As a first iteration at deriving a set of operations for an analysis class, you can again study a processing narrative (or use case) and select those operations that reasonably belong to the class. To accomplish this, the grammatical parse is again studied, and verbs are isolated. Some of these verbs will be legitimate operations and can be easily

connected to a specific class. For example, from the *SafeHome* processing narrative presented earlier in this chapter, we see that "sensor is *assigned* a number and type" or "a master password is *programmed* for *arming and disarming* the system." These phrases indicate several things:

- That an *assign()* operation is relevant for the **Sensor** class.
- That a *program()* operation will be applied to the **System** class.
- That *arm()* and *disarm()* are operations that apply to **System** class.

SAFEHOME



Class Models

The scene: Ed's cubicle, as analysis modeling begins.

The players: Jamie, Vinod, and Ed, all members of the *SafeHome* software engineering team.

The conversation:

[Ed has been working to extract classes from the use case template for ACS-DCV (presented in an earlier sidebar in this chapter) and is presenting the classes he has extracted to his colleagues.]

Ed: So, when the homeowner wants to pick a camera, he or she must pick it from a floor plan. I've defined a **FloorPlan** class. Here's the diagram.

(They look at Figure 8.4.)

Jamie: So, **FloorPlan** is an object that is put together with walls, doors, windows, and cameras. That's what those labeled lines mean, right?

Ed: Yeah, they're called "associations." One class is associated with another according to the associations I've shown. [Associations are discussed in Section 8.3.3.]

Vinod: So, the actual floor plan is made up of walls and contains cameras and sensors that are placed within those walls. How does the floor plan know where to put those objects?

Ed: It doesn't, but the other classes do. See the attributes under, say, **WallSegment,** which is used to build a wall. The wall segment has start and stop coordinates and the *draw()* operation does the rest.

Jamie: And the same goes for windows and doors. Looks like camera has a few extra attributes.

Ed: Yeah, I need them to provide pan and zoom info.

Vinod: I have a question. Why does the camera have an ID, but the others don't? I notice you have an attribute called **nextWall.** How will **WallSegment** know what the next wall will be?

Ed: Good question, but as they say, that's a design decision, so I'm going to delay that until . . .

Jamie: Give me a break . . . I'll bet you've already figured it out.

Ed (smiling sheepishly): True, I'm gonna use a list structure which I'll model when we get to design. If you get religious about separating analysis and design, the level of detail I have right here could be suspect.

Jamie: Looks pretty good to me, but I have a few more questions.

(Jamie asks questions which result in minor modifications.)

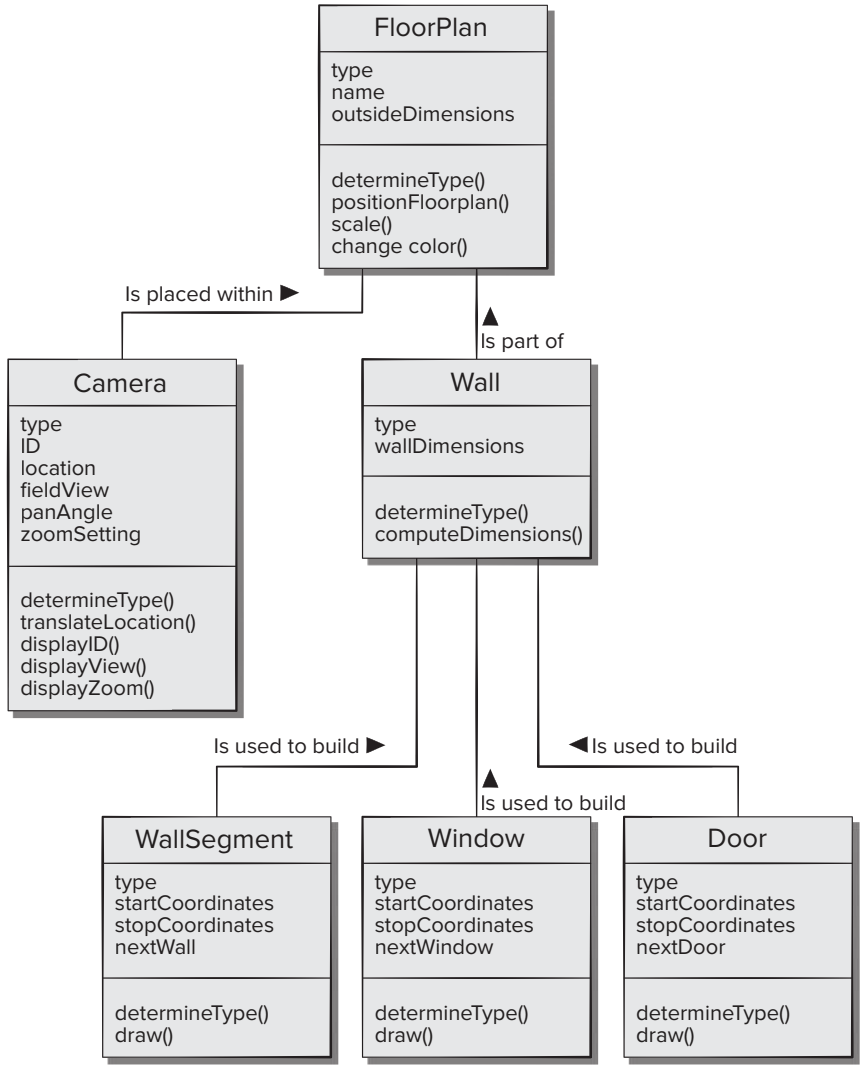
Vinod: Do you have CRC cards for each of the objects? If so, we ought to role-play through them, just to make sure nothing has been omitted.

Ed: I'm not quite sure how to do them.

Vinod: It's not hard and they really pay off. I'll show you.

FIGURE 8.4

Class diagram for FloorPlan (see sidebar discussion)



Upon further investigation, it is likely that the operation *program()* will be divided into several more specific suboperations required to configure the system. For example, *program()* implies specifying phone numbers, configuring system characteristics (e.g., creating the sensor table, entering alarm characteristics), and entering password(s). But for now, we specify *program()* as a single operation.

In addition to the grammatical parse, you can gain additional insight into other operations by considering the communication that occurs between objects. Objects communicate by passing messages to one another. Before continuing with the specification of operations, we explore this matter in a bit more detail.

In many instances, two analysis classes are related to one another in some fashion. In UML, these relationships are called *associations.* Referring to Figure 8.4, the **FloorPlan** class is defined by identifying a set of associations between **FloorPlan** and two other classes, **Camera** and **Wall.** The class **Wall** is associated with three classes that allow a wall to be constructed, **WallSegment, Window,** and **Door.**

8.3.4 Class-Responsibility-Collaborator Modeling

Class-responsibility-collaborator (CRC) modeling [Wir90] provides a simple means for identifying and organizing the classes that are relevant to system or product requirements. A CRC model can be viewed as a collection of index cards. Each index card contains a list of responsibilities on the left side and the corresponding collaborations that enable the responsibilities to be fulfilled on the right side (Figure 8.5). *Responsibilities* are the attributes and operations that are relevant for the class. *Collaborators* are those classes that provide a class with the information needed or action required to complete a responsibility. A simple CRC index card for the **FloorPlan** class is illustrated in Figure 8.5.

The list of responsibilities shown on the CRC card is preliminary and is subject to additions or modification. The classes **Wall** and **Camera** are noted next to the responsibility that requires their collaboration.

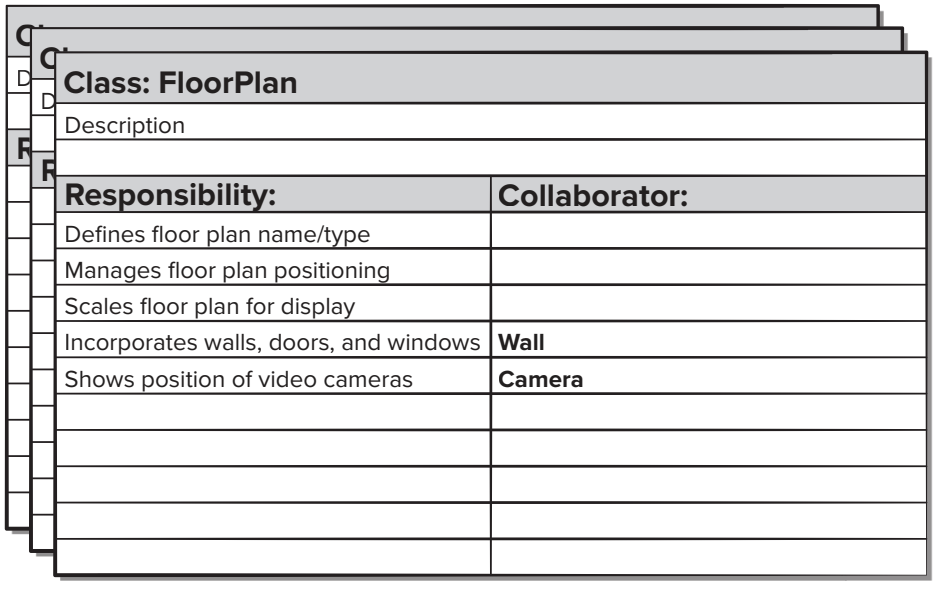
Classes. Basic guidelines for identifying classes and objects were presented earlier in Section 8.3.1.

Responsibilities. Basic guidelines for identifying responsibilities (attributes and operations) were presented in Section 8.3.2.

Collaborations. Classes fulfill their responsibilities in one of two ways: (1) A class can use its own operations to manipulate its own attributes, thereby fulfilling a

FIGURE 8.5

A CRC model index card



responsibility itself, or (2) a class can collaborate with other classes. Collaborations are identified by determining whether a class can fulfill each responsibility itself. If it cannot, then it needs to interact with another class.

As an example, consider the *SafeHome* security function. As part of the activation procedure, the **ControlPanel** object must determine whether any sensors are open. A responsibility named *determine-sensor-status()* is defined. If sensors are open, **ControlPanel** must set a status attribute to "not ready." Sensor information can be acquired from each **Sensor** object. The responsibility *determine-sensor-status()* can be fulfilled only if **ControlPanel** works in collaboration with **Sensor.**

Once a complete CRC model has been developed, the representatives from the stakeholders can review the model using the following approach [Amb95]:

1. All participants in the review (of the CRC model) are given a subset of the CRC model index cards. No reviewer should have two cards that collaborate.
2. The review leader reads the use case deliberately. As the review leader comes to a named object, she passes a token to the person holding the corresponding class index card.
3. When the token is passed, the holder of the class card is asked to describe the responsibilities noted on the card. The group determines whether one (or more) of the responsibilities satisfies the use case requirement.
4. If an error is found, modifications are made to the cards. This may include the definition of new classes (and corresponding CRC index cards) or revising lists of responsibilities or collaborations on existing cards.

SAFEHOME



CRC Models

The scene: Ed's cubicle, as requirements modeling begins.

The players: Vinod and Ed, members of the *SafeHome* software engineering team.

The conversation:

(Vinod has decided to show Ed how to develop CRC cards by showing him an example.)

Vinod: While you've been working on surveillance and Jamie has been tied up with security, I've been working on the home management function.

Ed: What's the status of that? Marketing kept changing its mind.

Vinod: Here's the first-cut use case for the whole function . . . we've refined it a bit, but it should give you an overall view . . .

Use case: *SafeHome* home management function.

Narrative: We want to use the home management interface on a mobile device or an Internet connection to control electronic devices that have wireless interface controllers. The system should allow me to turn specific lights on and off, to control appliances that are connected to a wireless interface, and to set my heating and air-conditioning system to temperatures that I define. To do this, I want to select the devices from a floor plan of the house. Each device must be identified on the floor plan. As an optional feature, I want to control all audiovisual devices—audio, television, DVD, digital recorders, and so forth.

With a single selection, I want to be able to set the entire house for various situations. One is home, another is away, a third is overnight travel, and a fourth is extended travel. All these situations will have settings that will be applied to all devices. In the overnight travel and extended travel states, the system should turn lights on and off at random intervals (to make it look like someone is home) and control the heating and air-conditioning system. I should be able to override these setting via the Internet with appropriate password protection . . .

Ed: Do the hardware guys have all the wireless interfacing figured out?

Vinod (smiling): They're working on it; say it's no problem. Anyway, I extracted a bunch of classes for home management, and we can use one as an example. Let's use the **HomeManagementInterface** class.

Ed: Okay . . . so the responsibilities are what . . . the attributes and operations for the class and the collaborations are the classes that the responsibilities point to.

Vinod: I thought you didn't understand CRC.

Ed: So, looking at the **HomeManagementInterface** card, when the operation *accessFloorplan()* is invoked, it collaborates with the **FloorPlan** object just like the one we developed for surveillance. Wait, I have a description of it here. (They look at Figure 8.4.)

Vinod: Exactly. And if we wanted to review the entire class model, we could start with this index card, then go to the collaborator's index card, and from there to one of the collaborator's collaborators, and so on.

Ed: Good way to find omissions or errors.

Vinod: Yep.

8.4 FUNCTIONAL MODELING

The *functional model* addresses two application processing elements, each representing a different level of procedural abstraction: (1) user-observable functionality that is delivered by the app to end users, and (2) the operations contained within analysis classes that implement behaviors associated with the class.

User-observable functionality encompasses any processing functions that are initiated directly by the user. For example, a financial mobile app might implement a variety of financial functions (e.g., computation of mortgage payment). These functions may be implemented using operations within analysis classes, but from the point of view of the end user, the function (more correctly, the data provided by the function) is the visible outcome.

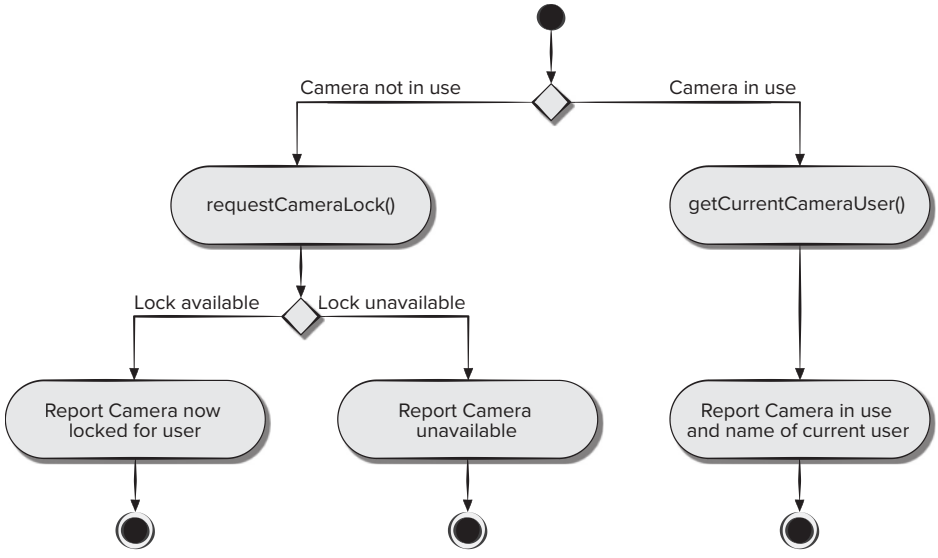
At a lower level of procedural abstraction, the requirements model describes the processing to be performed by analysis class operations. These operations manipulate class attributes and are involved as classes collaborate with one another to accomplish some required behavior.

8.4.1 A Procedural View

Regardless of the level of procedural abstraction, the UML activity diagram can be used to represent processing details. At the analysis level, activity diagrams should be used only where the functionality is relatively complex. Much of the complexity of mobile apps occurs not in the functionality provided, but rather with the nature of the information that can be accessed and the ways in which this can be manipulated.

FIGURE 8.6

Activity diagram for the *takeControlOfCamera()* operation



The UML activity diagram supplements the use case by providing a graphical representation of the flow of interaction within a specific scenario. An activity diagram is like a flowchart. The activity diagram (Figure 8.6) uses rounded rectangles to imply a specific system function, arrows to represent flow through the system, decision diamonds to depict a branching decision (each arrow emanating from the diamond is labeled), and solid horizontal lines to indicate that parallel activities are occurring.

An example of a relatively complex functionality for **SafeHomeAssured.com** is addressed by a use case entitled *Get recommendations for sensor layout for my space.* The user has already developed a layout for the space to be monitored, and in this use case, selects that layout and requests recommended locations for sensors within the layout. **SafeHomeAssured.com** responds with a graphical representation of the layout with additional information on the recommended locations for sensors. The interaction is quite simple and the content is somewhat more complex, but the underlying functionality is very sophisticated. The system must undertake a relatively complex analysis of the floor layout to determine the optimal set of sensors. It must examine room dimensions and the location of doors and windows and coordinate these with sensor capabilities and specifications. No small task! A set of activity diagrams can be used to describe processing for this use case.

The second example is the use case *Control cameras.* In this use case, the interaction is relatively simple, but there is the potential for complex functionality, given that this "simple" operation requires complex communication with devices located remotely and accessible across the Internet. A further possible complication relates to negotiation of control when multiple authorized people attempt to monitor and/or control a single sensor at the same time.

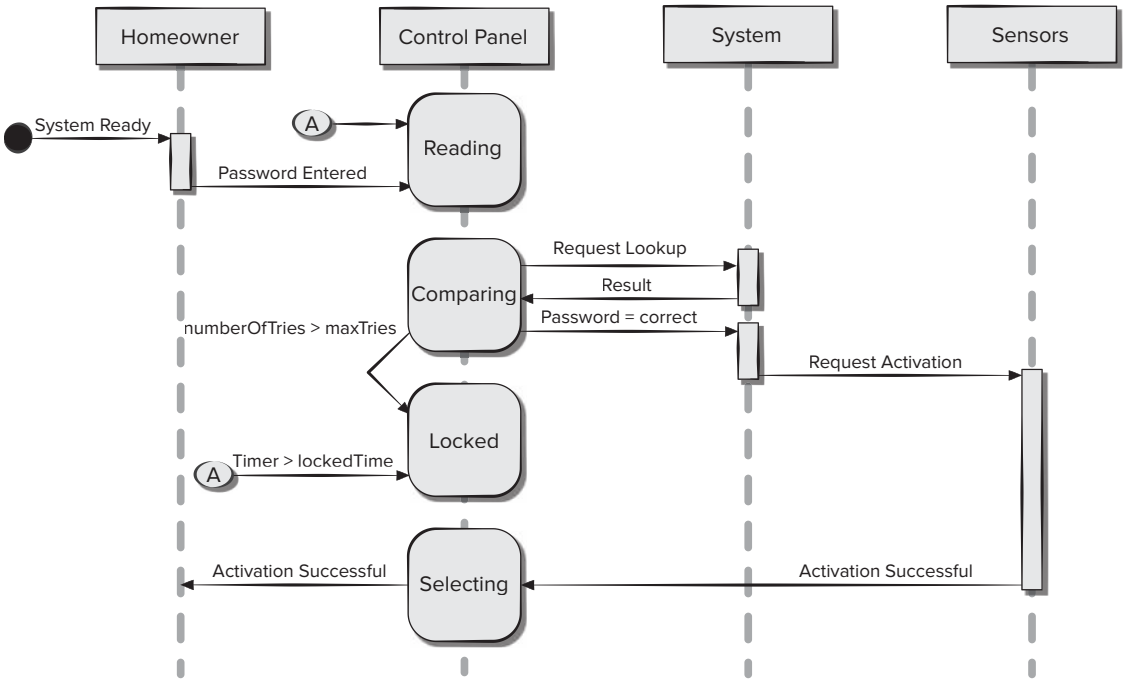
Figure 8.6 depicts an activity diagram for the *takeControlOfCamera()* operation that is part of the **Camera** analysis class used within the *Control cameras* use case. It should be noted that two additional operations are invoked with the procedural flow: *requestCameraLock(),* which tries to lock the camera for this user, and *getCurrentCameraUser(),* which retrieves the name of the user who is currently controlling the camera. The construction details indicating how these operations are invoked and the interface details for each operation are not considered until software design commences.

8.4.2 UML Sequence Diagrams

The UML *sequence diagram* can be used for behavioral modeling. Sequence diagrams can also be used to show how events cause transitions from object to object. Once events have been identified by examining a use case, the modeler creates a sequence diagram—a representation of how events cause flow from one object to another as a function of time. The sequence diagram is a shorthand version of the use case. It represents key classes and the events that cause behavior to flow from class to class.

Figure 8.7 illustrates a partial sequence diagram for the *SafeHome* security function. Each of the arrows represents an event (derived from a use case) and indicates how the event channels behavior between *SafeHome* objects. Time is measured vertically (downward), and the narrow vertical rectangles represent time spent in processing an activity. States may be shown along a vertical time line.

FIGURE 8.7 **Sequence diagram (partial) for the *SafeHome* security function**



The first event, *system ready*, is derived from the external environment and channels behavior to the **Homeowner** object. The homeowner enters a password. A *request lookup* event is passed to **System,** which looks up the password in a simple database and returns a *result* (*found* or *not found*) to **ControlPanel** (now in the *comparing* state). A valid password results in a *password=correct* event to **System,** which activates **Sensors** with a *request activation* event. Ultimately, control is passed back to the homeowner with the *activation successful* event.

Once a complete sequence diagram has been developed, all the events that cause transitions between system objects can be collated into a set of input events and output events (from an object). This information is useful in the creation of an effective design for the system to be built.

8.5 BEHAVIORAL MODELING

A *behavioral model* indicates how software will respond to internal or external events or stimuli. This information is useful in the creation of an effective design for the system to be built. UML activity diagrams can be used to model how system elements respond to internal events. UML state diagrams can be used to model how system elements respond to external events.

To create the model, you should perform the following steps: (1) evaluate all use cases to fully understand the sequence of interaction within the system, (2) identify events that drive the interaction sequence and understand how these events relate to specific objects, (3) create a sequence for each use case, (4) build a state diagram for the system, and (5) review the behavioral model to verify accuracy and consistency. Each of these steps is discussed in the sections that follow.

8.5.1 Identifying Events with the Use Case

In Section 8.3.3, you learned that the use case represents a sequence of activities that involves actors and the system. In general, an event occurs whenever the system and an actor exchange information. An event is *not* the information that has been exchanged, but rather the fact that information has been exchanged.

A use case is examined for points of information exchange. To illustrate, reconsider the use case for a portion of the *SafeHome* security function.

> The homeowner uses the keypad to key in a four-digit password. The password is compared with the valid password stored in the system. If the password is incorrect, the control panel will beep once and reset itself for additional input. If the password is correct, the control panel awaits further action.

The underlined portions of the use case scenario indicate events. An actor should be identified for each event; the information that is exchanged should be noted, and any conditions or constraints should be listed.

As an example of a typical event, consider the underlined use case phrase "homeowner uses the keypad to key in a four-digit password." In the context of the requirements model, the object, **Homeowner,**[10] transmits an event to the object **ControlPanel.**

10 In this example, we assume that each user (homeowner) that interacts with *SafeHome* has an identifying password and is therefore a legitimate object.

The event might be called *password entered*. The information transferred is the four digits that constitute the password, but this is not an essential part of the behavioral model. It is important to note that some events have an explicit impact on the flow of control of the use case, while others have no direct impact on the flow of control. For example, the event *password entered* does not explicitly change the flow of control of the use case, but the results of the event *password compared* (derived from the interaction "password is compared with the valid password stored in the system") will have an explicit impact on the information and control flow of the *SafeHome* software.

Once all events have been identified, they are allocated to the objects involved. Objects can be responsible for generating events (e.g., **Homeowner** generates the *password entered* event) or recognizing events that have occurred elsewhere (e.g., **ControlPanel** recognizes the binary result of the *password compared* event).

8.5.2 UML State Diagrams

In the context of behavioral modeling, two different characterizations of states must be considered: (1) the state of each class as the system performs its function and (2) the state of the system as observed from the outside as the system performs its function.

The state of a class takes on both passive and active characteristics [Cha93]. A *passive state* is simply the current values assigned to an object's attributes. The *active state* of an object indicates the status of the object as it undergoes a continuing transformation or processing. An *event* (sometimes called a *trigger*) must occur to force an object to make a transition from one active state to another.

State Diagrams for Analysis Classes. One component of a behavioral model is a UML state diagram[11] that represents active states for each class and the events (triggers) that cause changes between these active states. Figure 8.8 illustrates a state diagram for the **ControlPanel** object in the *SafeHome* security function.

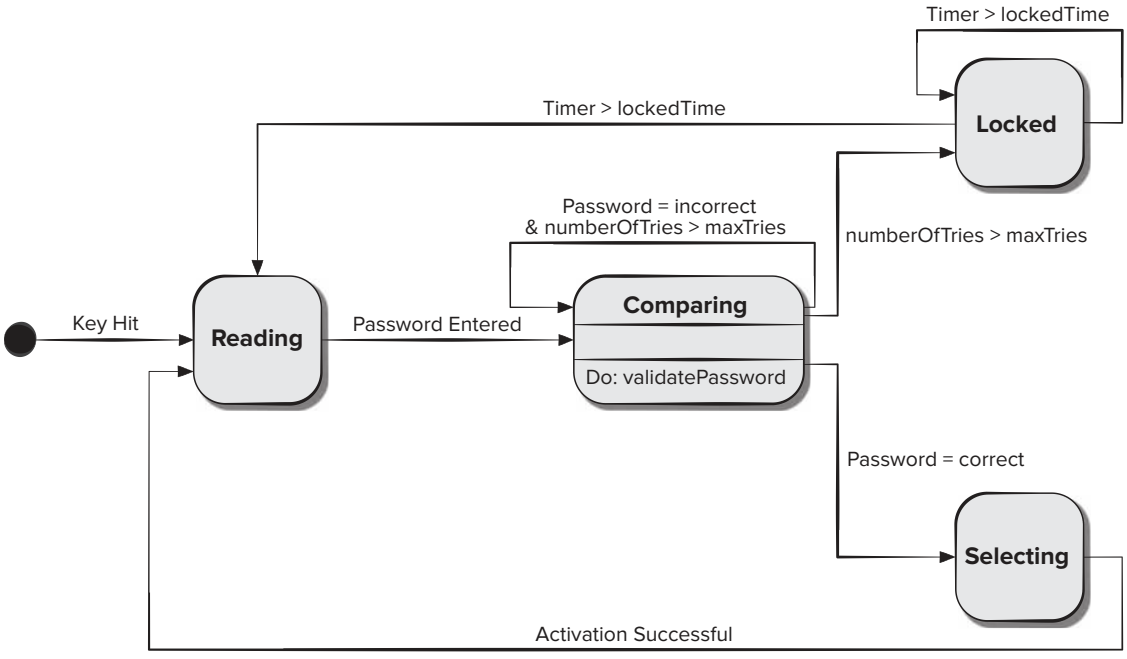
Each arrow shown in Figure 8.8 represents a transition from one active state of an object to another. The labels shown for each arrow represent the event that triggers the transition. Although the active state model provides useful insight into the "life history" of an object, it is possible to specify additional information to provide more depth in understanding the behavior of an object. In addition to specifying the event that causes the transition to occur, you can specify a guard and an action [Cha93]. A *guard* is a Boolean condition that must be satisfied for the transition to occur. For example, the guard for the transition from the "reading" state to the "comparing" state in Figure 8.8 can be determined by examining the use case:

```
if (password input = 4 digits) then compare to stored password
```

In general, the guard for a transition usually depends upon the value of one or more attributes of an object. In other words, the guard depends on the passive state of the object.

11 If you are unfamiliar with UML, a brief introduction to this important modeling notation is presented in Appendix 1.

FIGURE 8.8 State diagram for the ControlPanel class



An *action* occurs concurrently with the state transition or because of it and generally involves one or more operations (responsibilities) of the object. For example, the action connected to the *password entered* event (Figure 8.8) is an operation named *validatePassword()* that accesses a **password** object and performs a digit-by-digit comparison to validate the entered password.

8.5.3 UML Activity Diagrams

The UML activity diagram supplements the use case by providing a graphical representation of the flow of interaction within a specific scenario. Many software engineers like to describe activity diagrams as a way of representing how a system reacts to internal events.

An activity diagram for the **ACS-DCV** use case is shown in Figure 8.9. It should be noted that the activity diagram adds additional detail not directly mentioned (but implied) by the use case. For example, a user may only attempt to enter **userID** and **password** a limited number of times. A decision diamond represents this below: "Prompt for reentry."

The UML swimlane diagram is a useful variation of the activity diagram and allows you to represent the flow of activities described by the use case and at the same time indicate which actor (if there are multiple actors involved in a specific use case) or analysis class (Section 8.3.1) has responsibility for the action described by an activity rectangle. Responsibilities are represented as parallel segments that divide the diagram vertically, like the lanes in a swimming pool.

FIGURE 8.9

Activity diagram for Access camera surveillance via the Internet—display camera views function

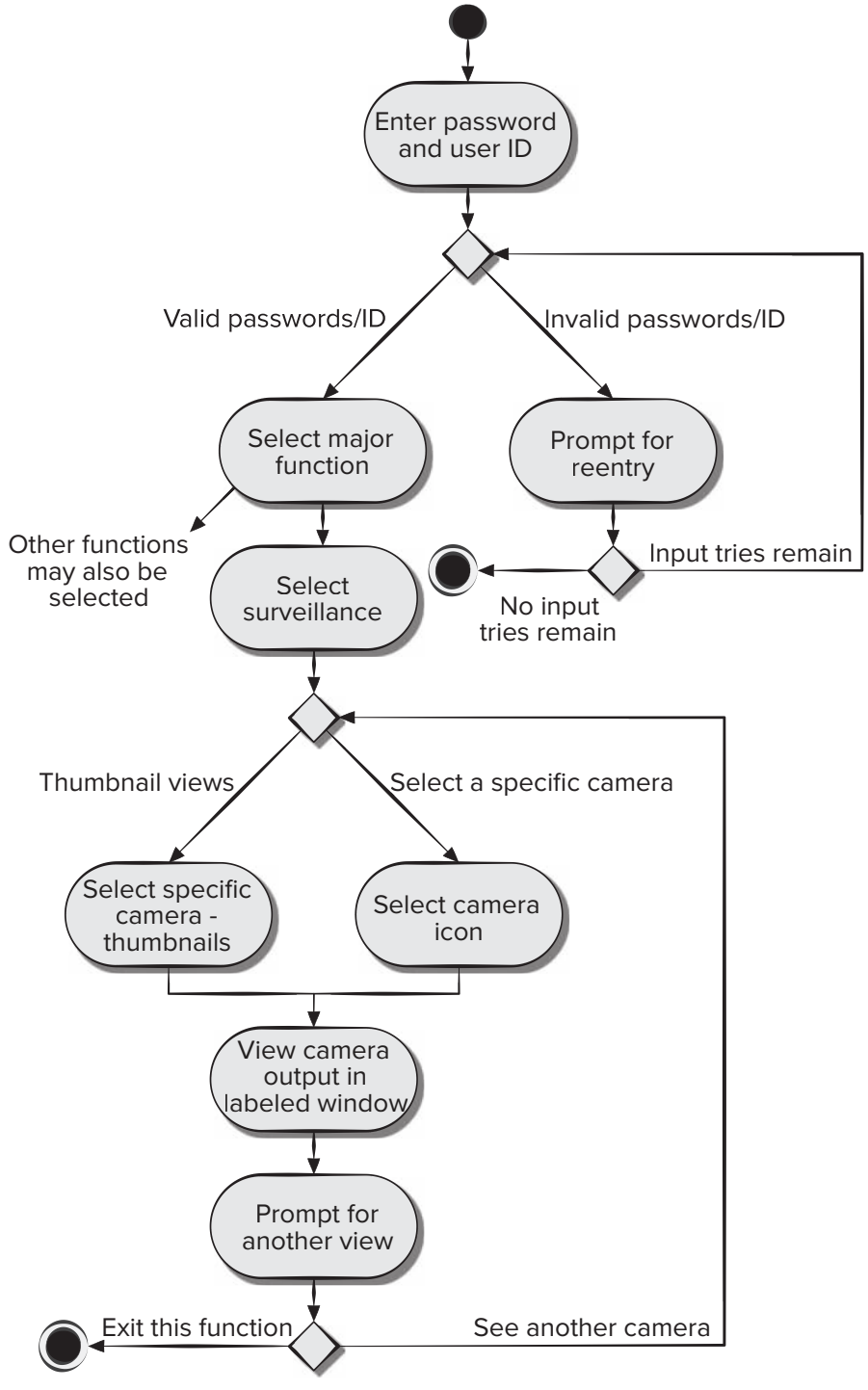
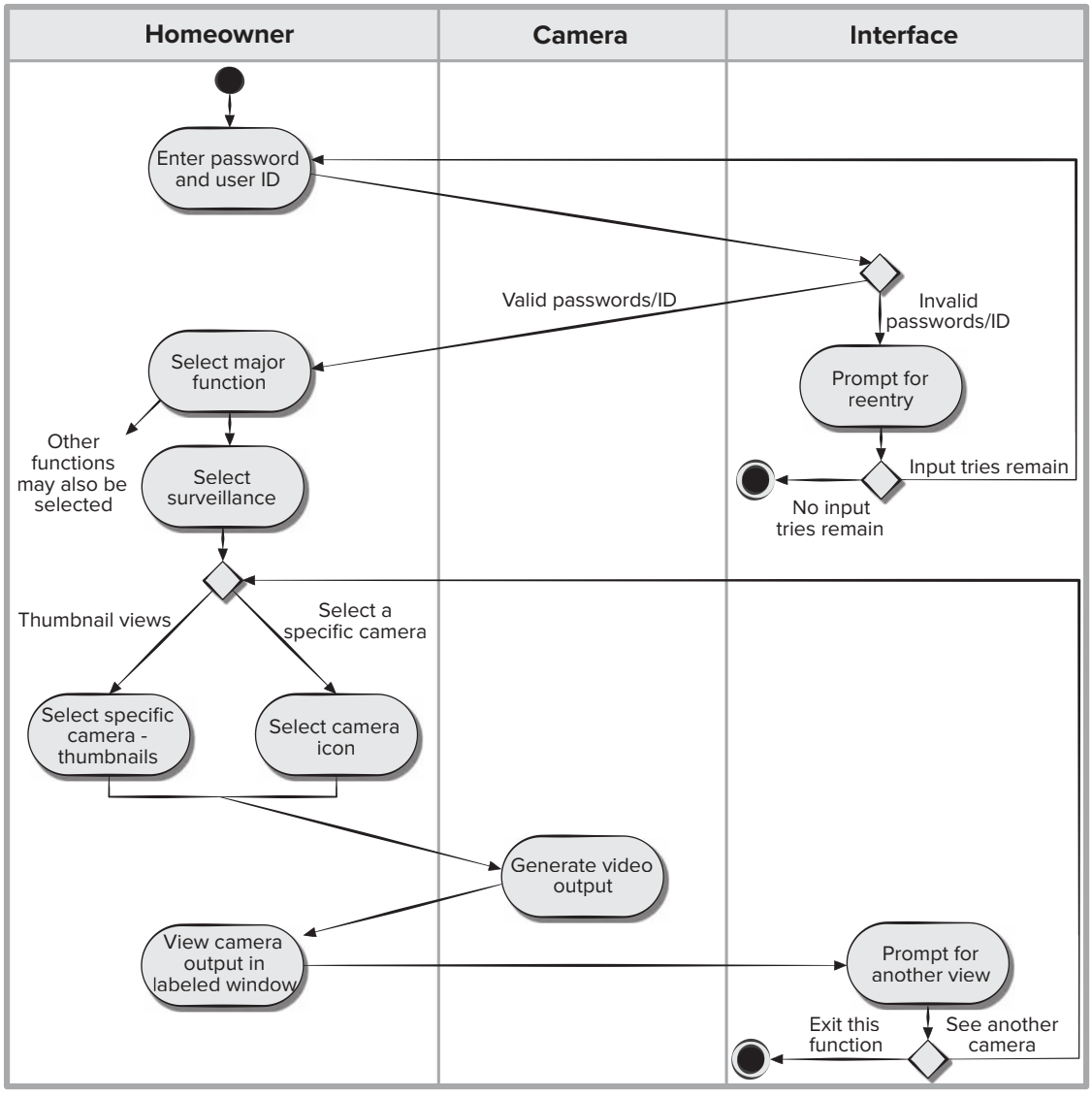


FIGURE 8.10 **Swimlane diagram for Access camera surveillance via the Internet—display camera views function**



Three analysis classes—**Homeowner, Camera,** and **Interface**—have direct or indirect responsibilities in the context of the activity diagram represented in Figure 8.9. Referring to Figure 8.10, the activity diagram is rearranged so that activities associated with an analysis class fall inside the swimlane for that class. For example, the **Interface** class represents the user interface as seen by the

homeowner. The activity diagram notes two prompts that are the responsibility of the interface—"prompt for reentry" and "prompt for another view." These prompts and the decisions associated with them fall within the Interface swimlane. However, arrows lead from that swimlane back to the **Homeowner** swimlane, where homeowner actions occur.

Use cases, along with the activity and swimlane diagrams, are procedurally oriented. Taken together they can be used to represent the way various actors invoke specific functions (or other procedural steps) to meet the requirements of the system.

8.6 SUMMARY

The objective of requirements modeling is to create a variety of representations that describe what the customer requires, establish a basis for the creation of a software design, and define a set of requirements that can be validated once the software is built. The requirements model bridges the gap between a system-level description that describes overall system and business functionality and a software design that describes the software's application architecture, user interface, and component-level structure.

Scenario-based models depict software requirements from the user's point of view. The use case—a narrative or template-driven description of an interaction between an actor and the software—is the primary modeling element. Derived during requirements elicitation, the use case defines the key steps for a specific function or interaction. The degree of use case formality and detail varies, but they can provide necessary input to all other analysis modeling activities. Scenarios can also be described using an activity diagram—a graphical representation that depicts the processing flow within a specific scenario. Temporal relations in a use case can be modeled using sequence diagrams.

Class-based modeling uses information derived from use cases and other written application descriptions to identify analysis classes. A grammatical parse may be used to extract candidate classes, attributes, and operations from text-based narratives. Criteria for the definition of a class are defined using the parse results.

A set of class-responsibility-collaborator index cards can be used to define relationships between classes. In addition, a variety of UML modeling notation can be applied to define hierarchies, relationships, associations, aggregations, and dependencies among classes.

Behavioral modeling during requirements analysis depicts dynamic behavior of the software. The behavioral model uses input from scenario-based or class-based elements to represent the states of analysis classes. To accomplish this, states are identified, the events that cause a class (or the system) to make a transition from one state to another are defined, and the actions that occur as transition is accomplished are also identified. UML state diagrams, activity diagrams, swim lane diagrams, and sequence diagrams can be used for behavioral modeling.

PROBLEMS AND POINTS TO PONDER

8.1. Is it possible to begin coding immediately after a requirements model has been created? Explain your answer, and then argue the counterpoint.

8.2. An analysis rule of thumb is that the model "should focus on requirements that are visible within the problem or business domain." What types of requirements are *not* visible in these domains? Provide a few examples.

8.3. The department of public works for a large city has decided to develop a Web-based pothole tracking and repair system (PHTRS). A description follows:

> Citizens can log onto a website and report the location and severity of potholes. As potholes are reported they are logged within a "public works department repair system" and are assigned an identifying number, stored by street address, size (on a scale of 1 to 10), location (middle, curb, etc.), district (determined from street address), and repair priority (determined from the size of the pothole). Work order data are associated with each pothole and include pothole location and size, repair crew identifying number, number of people on crew, equipment assigned, hours applied to repair, hole status (work in progress, repaired, temporary repair, not repaired), amount of filler material used, and cost of repair (computed from hours applied, number of people, material and equipment used). Finally, a damage file is created to hold information about reported damage due to the pothole and includes citizen's name, address, phone number, type of damage, and dollar amount of damage. PHTRS is an online system; all queries are to be made interactively.

Draw a UML use case diagram PHTRS system. You'll have to make a number of assumptions about the manner in which a user interacts with this system.

8.4. Write two or three use cases that describe the roles of various actors in the PHTRS described in Problem 8.3.

8.5. Develop an activity diagram for one aspect of PHTRS.

8.6. Develop a swimlane diagram for one or more aspects of PHTRS.

8.7. Develop a class model for the PHTRS system presented in Problem 8.3.

8.8. Develop a complete set of CRC model index cards on the product or system you chose as part of Problem 8.3.

8.9. Conduct a review of the CRC index cards with your colleagues. How many additional classes, responsibilities, and collaborators were added as a consequence of the review?

8.10. How does a sequence diagram differ from a state diagram? How are they similar?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

9 DESIGN CONCEPTS

Software design encompasses the set of principles, concepts, and practices that lead to the development of a high-quality system or product. *Design principles* establish an overriding philosophy that guides the design work you must perform. *Design concepts* must be understood before the mechanics of design practice are applied, and *design practice* leads to the creation of various representations of the software that serve as a guide for the construction activity that follows.

KEY CONCEPTS

abstraction 163
architecture 163
cohesion 167
data design 174
design modeling principles 173
design process 159
functional independence 167
good design 160
information hiding 166
modularity 165
patterns 164
quality attributes 160
quality guidelines 160
refactoring 168
separation of concerns 165
software design 157
stepwise refinement 167
technical debt 157

QUICK LOOK

What is it? Design is what almost every engineer wants to do. It is the place where creativity rules—where requirements and technical considerations come together in the formulation of a product or system. Design creates a representation or model of the software and provides detail about software architecture, data structures, interfaces, and components that are necessary to implement the system.

Who does it? Software engineers conduct each of the design tasks while continuing communication with the stakeholders.

Why is it important? During the design phase, you model the system or product that needs to be built. The design model can be assessed for quality and improved before code is generated, tests are conducted, and end users become involved in large numbers.

What are the steps? Design makes use of several different representations of the software. First, the architecture of the system or product must be modeled. Then, the interfaces that connect the software to end users, to other systems and devices, and to its own constituent components are represented. Finally, the software components that are used to construct the system are designed.

What is the work product? A design model that encompasses architectural, interface, component-level, and deployment representations is the primary work product that is produced during software design.

How do I ensure that I've done it right? The design model is assessed by the software team (including relevant stakeholders) in an effort to determine whether it contains errors, inconsistencies, or omissions; whether better alternatives exist; and whether the model can be implemented within the constraints, schedule, and cost that have been established.

Design is pivotal to successful software engineering. Some developers are tempted to begin programming once the use cases have been created, without regard to how the software components needed to implement the use cases relate to one another. It is possible to do analysis, design, and implementation iteratively by creating several software increments. It is a bad idea to ignore the design considerations needed to create an appropriate architecture for the evolving software product. *Technical debt* is a concept in software development that refers to costs associated with rework caused by choosing a "quick and dirty" solution right now instead of using a better approach that would take more time. It is impossible to avoid creating technical debt when building a software product incrementally. However, a good development team must try to pay down this technical debt by refactoring (Section 9.3.9) the software on a regular basis. Just like taking out a loan, you can wait until the loan is due and pay a lot of interest or you can pay the loan off a little at a time and pay less interest overall.

One strategy to keep technical debt in check without delaying coding is to make use of the design practices of diversification and convergence. *Diversification* is the practice of identifying possible design alternatives suggested by the elements of the requirements model. *Convergence* is the process of evaluating and rejecting design alternatives that do not meet the constraints imposed by the nonfunctional requirements defined for the software solution. Diversification and convergence combine (1) intuition and judgment based on experience in building similar entities, (2) a set of principles and/or heuristics that guide the way in which the model evolves, (3) a set of criteria that enables quality to be judged, and (4) a process of iteration that ultimately leads to a final design representation. Once a viable design alternative is identified this way, the developers are in a good position to create a software increment that is not likely to be a throwaway prototype.

Software design changes continually as new methods, better analysis, and broader understanding evolve.[1] Even today, most software design methodologies lack the depth, flexibility, and quantitative nature that are normally associated with more classical engineering design disciplines. However, methods for software design do exist, criteria for design quality are available, and design notation can be applied.

In this chapter, we explore the fundamental concepts and principles that are applicable to all software design, the elements of the design model, and the impact of patterns on the design process. In Chapters 10 through 14 we'll present a variety of software design methods as they are applied to architectural, interface, and component-level design as well as pattern-based, mobile, and user experience design approaches.

9.1 DESIGN WITHIN THE CONTEXT OF SOFTWARE ENGINEERING

Software design sits at the technical kernel of software engineering and is applied regardless of the software process model that is used. Beginning once software requirements have been analyzed and modeled, software design is the last software engineering action within the modeling activity and sets the stage for **construction** (code generation and testing).

1 Those readers with further interest in the philosophy of software design might have interest in Philippe Kruchen's intriguing discussion of "postmodern" design [Kru05].

Each of the elements of the requirements model (Chapter 8) provide information that is necessary to create the four design models required for a complete specification of design. The flow of information during software design is illustrated in Figure 9.1. The requirements model, manifested by scenario-based, class-based, and behavioral elements, feed the design task. Using design notation and design methods discussed in later chapters, design produces a data/class design, an architectural design, an interface design, and a component design.

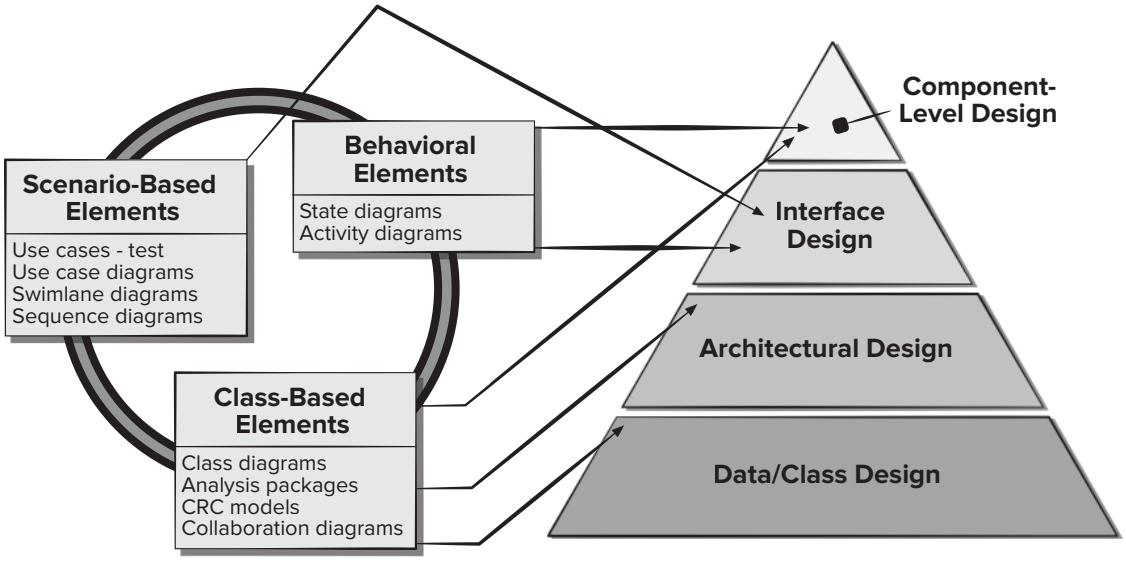
The data/class design transforms class models (Chapter 8) into design class realizations and the requisite data structures required to implement the software. The objects and relationships defined in the CRC model and the detailed data content depicted by class attributes and other notation provide the basis for the data design activity. Part of class design may occur in conjunction with the design of software architecture. More detailed class design occurs as each software component is designed.

The architectural design defines the relationship between major structural elements of the software, the architectural style, and patterns (Chapter 14) that can be used to achieve the requirements defined for the system, and the constraints that affect the way in which architecture can be implemented [Sha15]. The architectural design representation—the framework of a computer-based system—is derived from the requirements model.

The interface design describes how the software communicates with systems that interoperate with it, and with humans who use it. An interface implies a flow of information (e.g., data and/or control) and a specific type of behavior. Therefore, usage scenarios and behavioral models provide much of the information required for interface design.

The component-level design transforms structural elements of the software architecture into a procedural description of software components. Information obtained from the class-based models and behavioral models serve as the basis for component design.

FIGURE 9.1 **Translating the requirements model into the design model**



During design you make decisions that will ultimately affect the success of software construction and, just as important, the ease with which software can be maintained. But why is design so important?

The importance of software design can be stated with a single word—*quality*. Design is the place where quality is fostered in software engineering. It provides you with representations of software that can be assessed for quality. Design is the only way that you can accurately translate stakeholders' requirements into a finished software product or system. Software design serves as the foundation for all the software engineering and software support activities that follow. Without design, you risk building an unstable system—one that will fail when small changes are made; one that may be difficult to test; one whose quality cannot be assessed until late in the software process. Late in the project is when time is short, and many budgeted dollars have already been spent.

SAFEHOME



Design Versus Coding

The scene: Jamie's cubicle, as the team prepares to translate requirements into design.

The players: Jamie, Vinod, and Ed, all members of the *SafeHome* software engineering team.

The conversation:

Jamie: You know, Doug [the team manager] is obsessed with design. I gotta be honest, what I really love doing is coding. Give me C++ or Java, and I'm happy.

Ed: Nah . . . you like to design.

Jamie: You're not listening—coding is where it's at.

Vinod: I think what Ed means is that you don't really like coding; you like to design and express it in code. Code is the language you use to represent the design.

Jamie: And what's wrong with that?

Vinod: Level of abstraction.

Jamie: Huh?

Ed: A programming language is good for representing details like data structures and algorithms, but it's not so good for representing architecture or component-to-component collaboration . . . stuff like that.

Vinod: And a screwed-up architecture can ruin even the best code.

Jamie (thinking for a minute): So, you're saying that I can't represent architecture in code . . . that's not true.

Vinod: You can certainly imply architecture in code, but in most programming languages, it's difficult to get a quick, big-picture read on architecture by examining the code.

Ed: And that's what we want before we begin coding.

Jamie: Okay, maybe design and coding are different, but I still like coding better.

9.2 THE DESIGN PROCESS

Software design is an iterative process through which requirements are translated into a "blueprint" for constructing the software. Initially, the blueprint depicts a holistic view of software. That is, the design is represented at a high level of abstraction—a level that can be directly traced to the specific system objective and more detailed data, functional, and behavioral requirements. As design iterations occur, subsequent

refinement leads to design representations at much lower levels of abstraction. These can still be traced to requirements, but the connections may not be obvious at these lower levels of abstraction.

9.2.1 Software Quality Guidelines and Attributes

Throughout the design process, the quality of the evolving design is assessed with a series of technical reviews discussed in Chapter 16. McGlaughlin [McG91] suggests three characteristics that serve as a guide for the evaluation of a good design:

- The design should implement all explicit requirements contained in the requirements model, and it must accommodate all the implicit requirements desired by stakeholders.
- The design should be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.
- The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.

Each of these characteristics is a goal of the design process. But how is each of these goals achieved?

Quality Guidelines. To evaluate the quality of a design representation, you and other members of the software team must establish technical criteria for good design. In Section 9.3, we discuss design concepts that also serve as software quality criteria. For the time being, consider the following guidelines:

1. A design should exhibit an architecture that (a) has been created using recognizable architectural styles or patterns, (b) is composed of components that exhibit good design characteristics (these are discussed later in this chapter), and (c) can be implemented in an evolutionary fashion,[2] thereby facilitating implementation and testing.
2. A design should be modular; that is, the software should be logically partitioned into elements or subsystems.
3. A design should contain distinct representations of data, architecture, interfaces, and components.
4. A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.
5. A design should lead to components that exhibit independent functional characteristics.
6. A design should lead to interfaces that reduce the complexity of connections between components and with the external environment.
7. A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis.
8. A design should be represented using a notation that effectively communicates its meaning.

2 For smaller systems, design can sometimes be developed linearly.

Chance alone will not achieve these design guidelines. They are achieved through the application of fundamental design principles, systematic methodology, and thorough review.

INFO



Assessing Design Quality—The Technical Review

Design is important because it allows a software team to assess the quality[3] of the software before it is implemented—at a time when errors, omissions, or inconsistencies are easy and inexpensive to correct. But how do we assess quality during design? The software can't be tested, because there is no executable software to test. What to do?

During design, quality is assessed by conducting a series of technical reviews (TRs). TRs are discussed in detail in Chapter 16,[4] but it's worth providing a summary of the technique at this point. A technical review is a meeting conducted by members of the software team. Usually two, three, or four people participate depending on the scope of the design information to be reviewed. Each person plays a role. The *review leader* plans the meeting, sets an agenda, and runs the meeting; the *recorder* takes notes so that nothing is missed; and the *producer* is the person whose work product (e.g., the design of a software component) is being reviewed. Prior to the meeting, each person on the review team is given a copy of the design work product and is asked to read it, looking for errors, omissions, or ambiguity. When the meeting commences, the intent is to note all problems with the work product so that they can be corrected before implementation begins. The TR typically lasts between 60 to 90 minutes. After the TR concludes, the review team determines whether further actions are required from the producer before the design work product can be approved as part of the final design model.

9.2.2 The Evolution of Software Design

The evolution of software design is a continuing process that has now spanned more than six decades. Early design work concentrated on criteria for the development of modular programs [Den73] and methods for refining software structures in a top-down "structured" manner ([Wir71], [Dah72], [Mil72]). Newer design approaches (e.g., [Jac92], [Gam95]) proposed an object-oriented approach to design derivation. More recent emphasis in software design has been on software architecture [Kru06] and the design patterns that can be used to implement software architectures and lower levels of design abstractions (e.g., [Hol06], [Sha05]). There is a growing emphasis on aspect-oriented methods (e.g., [Cla05], [Jac04]), model-driven development [Sch06], and test-driven development [Ast04], which focus on techniques for achieving more effective modularity and architectural structure in the designs that are created.

In the past 10 years, Search-Based Software Engineering (SBSE) techniques have been applied to all phases of the software engineering life cycle, including design [Har12]. SBSE attempts to solve software engineering problems using automated search techniques augmented by operations research and machine learning algorithms to provide design recommendations to software developers. Many modern software systems must accommodate a high degree of variability, both in their deployment environments and

3 The quality factors discussed in Chapter 23 can assist the review team as it assesses quality.

4 You might consider looking ahead to Chapter 16 at this time. Technical reviews are a critical part of the design process and are an important mechanism for achieving design quality.

the number of usage scenarios they expected to satisfy. Design of *variability-intensive systems*[5] requires developers to anticipate future changes in the features to be modified in future versions of the product being designed today [Gal16]. A detailed discussion of the design of variability-intensive systems is beyond the scope of this book.

Several design methods, growing out of the work just noted, are being applied throughout the industry. Like the analysis methods presented in Chapter 8, each software design method introduces unique heuristics and notation, as well as a somewhat parochial view of what characterizes design quality. Yet, each of these methods has common characteristics: (1) a mechanism for the translation of the requirements model into a design representation, (2) a notation for representing functional components and their interfaces, (3) heuristics for refinement and partitioning, and (4) guidelines for quality assessment.

Regardless of the design method that is used, you should apply a set of basic concepts to data, architectural, interface, and component-level design. These concepts are considered in the sections that follow.

TASK SET



Generic Task Set for Design

Please note: These tasks are often performed iteratively and in parallel. They are rarely completed sequentially and in isolation from one another unless you are following the waterfall process model.

1. Examine the information model, and design appropriate data structures for data objects and their attributes.
2. Using the analysis model, select an architectural style (pattern) that is appropriate for the software.
3. Partition the analysis model into design subsystems and allocate these subsystems within the architecture:
   - Be certain that each subsystem is functionally cohesive.
   - Design subsystem interfaces.
   - Allocate analysis classes or functions to each subsystem.
4. Create a set of design classes or components:
   - Translate an analysis class description into a design class.
   - Check each design class against design criteria; consider inheritance issues.
   - Define methods and messages associated with each design class.
   - Evaluate and select design patterns for a design class or a subsystem.
   - Review design classes and revise as required.
5. Design any interface required with external systems or devices.
6. Design the user interface:
   - Review results of task analysis.
   - Specify action sequence based on user scenarios.
   - Create a behavioral model of the interface.
   - Define interface objects and control mechanisms.
   - Review the interface design, and revise as required.
7. Conduct component-level design. Specify all algorithms at a relatively low level of abstraction.
   - Refine the interface of each component.
   - Define component-level data structures.
   - Review each component, and correct all errors uncovered.
8. Develop a deployment model.

5 Variability-intensive systems refers to systems that may be required to be self-modifying based on changes in the run-time environment or families of software products resulting from product line engineering practices for building specialized product variants out of existing software products.

9.3 Design Concepts

Several fundamental software design concepts have evolved over the history of software engineering. Although the degree of interest in these concepts has varied over the years, each has stood the test of time. Each provides the software designer with a foundation from which more sophisticated design methods can be applied. Each helps you define criteria that can be used to partition software into individual components, separate out data structure detail from a conceptual representation of the software, and establish uniform criteria that define the technical quality of a software design. These concepts help developers design the software actually needed, rather than simply focusing on creating any old working program.

9.3.1 Abstraction

When you consider a modular solution to any problem, many levels of abstraction can be posed. At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment (e.g., a user story). At lower levels of abstraction, a more detailed description of the solution is provided. Problem-oriented terminology is coupled with implementation-oriented terminology to state a solution (e.g., use case). Finally, at the lowest level of abstraction, the solution is stated in a manner that can be directly implemented (e.g., pseudocode).

As different levels of abstraction are developed, you work to create both procedural and data abstractions. A *procedural abstraction* refers to a sequence of instructions that have a specific and limited function. The name of a procedural abstraction implies these functions, but specific details are suppressed. An example of a procedural abstraction would be the word *use* for a camera in the *SafeHome* system. *Use* implies a long sequence of procedural steps (e.g., activate the *SafeHome* system on a mobile device, log on to the *SafeHome* system, select a camera to preview, locate the camera controls on mobile app user interface, etc.).[6]

A *data abstraction* is a named collection of data that describes a data object. In the context of the procedural abstraction *open,* we can define a data abstraction called **camera.** Like any data object, the data abstraction for **camera** would encompass a set of attributes that describe the camera (e.g., camera ID, location, field view, pan angle, zoom). It follows that the procedural abstraction *use* would make use of information contained in the attributes of the data abstraction **camera.**

9.3.2 Architecture

Software architecture alludes to "the overall structure of the software and the ways in which that structure provides conceptual integrity for a system" [Sha15]. In its simplest form, architecture is the structure or organization of program components (modules), the ways in which these components interact, and the structure of data that are

6 It should be noted, however, that one set of operations can be replaced with another, if the function implied by the procedural abstraction remains the same. Therefore, the steps required to implement *use* would change dramatically if the camera were automatic and attached to a sensor that automatically triggered an alert on your mobile device.

used by the components. In a broader sense, however, components can be generalized to represent major system elements and their interactions.

One goal of software design is to derive an architectural rendering of a system. This rendering serves as a framework from which more detailed design activities are conducted. A set of architectural patterns enables a software engineer to reuse design-level concepts.

Shaw and Garlan [Sha15] describe a set of properties that should be specified as part of an architectural design. *Structural properties* define "the components of a system (e.g., modules, objects, filters) and the manner in which those components are packaged and interact with one another." *Extra-functional properties* address "how the design architecture achieves requirements for performance, capacity, reliability, security, adaptability, and other system characteristics (e.g., nonfunctional system requirements)." *Families of related systems* "draw upon repeatable patterns that are commonly encountered in the design of families of similar systems."[7]

Given the specification of these properties, the architectural design can be represented using one or more of several different models [Gar95]. *Structural models* represent architecture as an organized collection of program components. *Framework models* increase the level of design abstraction by attempting to identify repeatable architectural design frameworks (patterns) that are encountered in similar types of applications. *Dynamic models* address the behavioral aspects of the program architecture, indicating how the structure or system configuration may change as a function of external events. *Process models* focus on the design of the business or technical process that the system must accommodate. Finally, *functional models* can be used to represent the functional hierarchy of a system.

Several different *architectural description languages* (ADLs) have been developed to represent these models [Sha15]. Although many different ADLs have been proposed, the majority provide mechanisms for describing system components and the ways in which they are connected to one another.

You should note that there is some debate about the role of architecture in design. Some researchers argue that the derivation of software architecture should be separated from design and occurs between requirements engineering actions and more conventional design actions. Others believe that the derivation of architecture is an integral part of the design process. The ways in which software architecture is characterized and its role in design are discussed in Chapter 10.

9.3.3 Patterns

Brad Appleton defines a *design pattern* in the following manner: "A pattern is a named nugget of insight which conveys the essence of a proven solution to a recurring problem within a certain context amidst competing concerns" [App00]. Stated in another way, a design pattern describes a design structure that solves a well-defined design problem within a specific context and amid "forces" that may have an impact on the manner in which the pattern is applied and used.

---

7 These families of related software products sharing common features are called *software product lines*.

The intent of each design pattern is to provide a description that enables a designer to determine (1) whether the pattern is applicable to the current work, (2) whether the pattern can be reused (hence, saving design time), and (3) whether the pattern can serve as a guide for developing a similar, but functionally or structurally different, pattern. Design patterns are discussed in detail in Chapter 14.

9.3.4 Separation of Concerns

Separation of concerns is a design concept [Dij82] that suggests that any complex problem can be more easily handled if it is subdivided into pieces that can each be solved and/or optimized independently. A *concern* is a feature or behavior that is specified as part of the requirements model for the software. By separating concerns into smaller and therefore more manageable pieces, a problem takes less effort and time to solve.

It follows that the perceived complexity of two problems when they are combined is often greater than the sum of the perceived complexity when each is taken separately. This leads to a divide-and-conquer strategy—it's easier to solve a complex problem when you break it into manageable pieces. This has important implications for software modularity.

Separation of concerns is manifested in other related design concepts: modularity, functional independence, and refinement. Each will be discussed in the subsections that follow.

9.3.5 Modularity

Modularity is the most common manifestation of separation of concerns. Software is divided into separately named and addressable components, sometimes called *modules,* that are integrated to satisfy problem requirements.

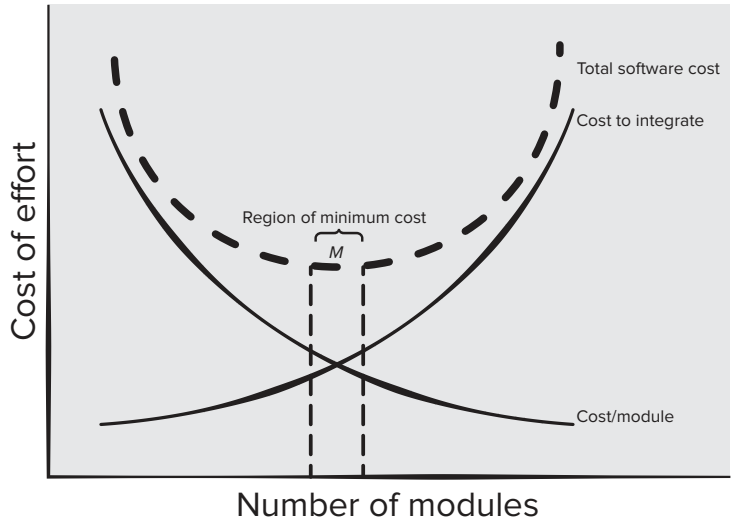
It has been stated that "modularity is the single attribute of software that allows a program to be intellectually manageable" [Mye78]. Monolithic software (i.e., a large program composed of a single module) cannot be easily grasped by a software engineer. The number of control paths, span of reference, number of variables, and overall complexity would make understanding close to impossible. In almost all instances, you should break the design into many modules, hoping to make understanding easier and reduce the cost required to build the software.

Recalling our discussion of separation of concerns, it is possible to conclude that if you subdivide software indefinitely the effort required to develop it will become negligibly small! Unfortunately, other forces come into play, causing this conclusion to be (sadly) invalid. Referring to Figure 9.2, the effort (cost) to develop an individual software module tends to decrease as the total number of modules increases.

Given the same set of requirements, the more modules used in your program means smaller individual sizes. However, as the number of modules grows, the effort (cost) associated with integrating modules with each other grows. These characteristics lead to a total cost or effort curve, shown in Figure 9.2. There is a number, *M,* of modules that would result in minimum development cost, but we do not have the necessary sophistication to predict *M* with assurance.

The curves shown in Figure 9.2 do provide useful qualitative guidance when modularity is considered. You should modularize, but care should be taken to stay in the vicinity of *M*. Using too few modules or too many modules should be avoided.

FIGURE 9.2
Modularity and software cost



But how do you know the vicinity of *M*? How modular should you make software? The answers to these questions require an understanding of other design concepts considered in Sections 9.3.6 through 9.3.9.

You modularize a design (and the resulting program) so that development can be more easily planned, software increments can be defined and delivered, changes can be more easily accommodated, testing and debugging can be conducted more efficiently, and long-term maintenance can be conducted without serious side effects.

9.3.6 Information Hiding

The concept of modularity leads you to a fundamental question: "How do I decompose a software solution to obtain the best set of modules?" The principle of *information hiding* [Par72] suggests that modules should be "characterized by design decisions that (each) hides from all others." In other words, modules should be specified and designed so that information (algorithms and data) contained within a module is inaccessible to other modules that have no need for such information.

Hiding implies that effective modularity can be achieved by defining a set of independent modules that communicate with one another only that information necessary to achieve software function. Abstraction helps to define the procedural (or informational) entities that make up the software. Hiding defines and enforces access constraints to both procedural detail within a module and any local data structure used by the module [Ros75].

The use of information hiding as a design criterion for modular systems provides the greatest benefits when modifications are required during testing and later during software maintenance. Because most data and procedural detail are hidden from other parts of the software, inadvertent errors introduced during modification are less likely to propagate to other locations within the software.

9.3.7 Functional Independence

The concept of functional independence is a direct outgrowth of separation of concerns, modularity, and the concepts of abstraction and information hiding. In landmark papers on software design, Wirth [Wir71] and Parnas [Par72] each allude to refinement techniques that enhance module independence. Later work by Stevens, Myers, and Constantine [Ste74] solidified the concept.

Functional independence is achieved by developing modules with "single-minded" function and an "aversion" to excessive interaction with other modules. Stated another way, you should design software so that each module addresses a specific subset of requirements and has a simple interface when viewed from other parts of the program structure.

It is fair to ask why independence is important. Software with effective modularity, that is, independent modules, is easier to develop because function can be compartmentalized and interfaces are simplified (consider the ramifications when development is conducted by a team). Independent modules are easier to maintain (and test) because secondary effects caused by design or code modification are limited, error propagation is reduced, and reusable modules are possible. To summarize, functional independence is a key to good design, and design is the key to software quality. Evaluation of your CRC card model (Chapter 8) can help you spot problems with functional independence. User stories that contain many instances of words such as *and* or *except* are not likely to encourage you to design modules that are "single-minded" system functions.

Independence is assessed using two qualitative criteria: cohesion and coupling. *Cohesion* is an indication of the relative functional strength of a module. *Coupling* is an indication of the relative interdependence among modules.

Cohesion is a natural extension of the information-hiding concept described in Section 9.3.6. A cohesive module performs a single task, requiring little interaction with other components in other parts of a program. Stated simply, a cohesive module should (ideally) do just one thing. Although you should always strive for high cohesion (i.e., single-mindedness), it is often necessary and advisable to have a software component perform multiple functions. However, "schizophrenic" components (modules that perform many unrelated functions) are to be avoided if a good design is to be achieved.

Coupling is an indication of interconnections among modules in a software structure. Coupling depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface. In software design, you should strive for the lowest possible coupling. Simple connectivity among modules results in software that is easier to understand and less likely to propagate errors found in one module to other system modules.

9.3.8 Stepwise Refinement

Stepwise refinement is a top-down design strategy originally proposed by Niklaus Wirth [Wir71]. Successively refining levels of procedural detail is a good way to develop an application. A hierarchy is developed by decomposing a macroscopic statement of function (a procedural abstraction) in a stepwise fashion until programming language statements are reached.

Refinement is a process of *elaboration.* You begin with a statement of function (or description of information) that is defined at a high level of abstraction. That is, the

statement describes function or information conceptually but provides no indication of the internal workings of the function or the internal structure of the information. You then elaborate on the original statement, providing more detail as each successive refinement (elaboration) occurs.

Abstraction and refinement are complementary concepts. Abstraction enables you to specify procedure and data internally but suppress the need for "outsiders" to have knowledge of low-level details. Refinement helps you to reveal low-level details as design progresses. Both concepts allow you to create a complete design model as the design evolves.

9.3.9 Refactoring

An important design activity suggested for many agile methods (Chapter 3), *refactoring* is a reorganization technique that simplifies the design (or code) of a component without changing its function or behavior. Fowler [Fow00] defines refactoring in the following manner: "Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code [design] yet improves its internal structure."

When software is refactored, the existing design is examined for redundancy, unused design elements, inefficient or unnecessary algorithms, poorly constructed or inappropriate data structures, or any other design failure that can be corrected to yield a better design. For example, a first design iteration might yield a large component that exhibits low cohesion (i.e., it performs three functions that have only a limited relationship to one another). After careful consideration, you may decide that the component should be refactored into three separate components, each of which exhibits high cohesion. The result will be software that is easier to integrate, easier to test, and easier to maintain.

Although the intent of refactoring is to modify the code in a manner that does not alter its external behavior, inadvertent side effects can and do occur. Refactoring tools [Soa10] are sometimes used to analyze code changes automatically and to "generate a test suite suitable for detecting behavioral changes."

SAFEHOME



Design Concepts

The scene: Vinod's cubicle, as design modeling begins.

The players: Vinod, Jamie, and Ed—members of the *SafeHome* software engineering team. Also, Shakira, a new member of the team.

The conversation:

(All four team members have just returned from a morning seminar entitled "Applying Basic Design Concepts," offered by a local computer science professor.)

Vinod: Did you get anything out of the seminar?

Ed: Knew most of the stuff, but it's not a bad idea to hear it again, I suppose.

Jamie: When I was an undergrad CS major, I never really understood why information hiding was as important as they say it is.

Vinod: Because . . . bottom line . . . it's a technique for reducing error propagation in a program. Actually, functional independence also accomplishes the same thing.

Shakira: I wasn't an SE grad, so a lot of the stuff the instructor mentioned is new to me. I can generate good code and fast. I don't see why this stuff is so important.

Jamie: I've seen your work, Shak, and you know what, you do a lot of this stuff naturally . . . that's why your designs and code work.

Shakira (smiling): Well, I always do try to partition the code, keep it focused on one thing, keep interfaces simple and constrained, reuse code whenever I can . . . that sort of thing.

Ed: Modularity, functional independence, hiding, patterns . . . see.

Jamie: I still remember the very first programming course I took . . . they taught us to refine the code iteratively.

Vinod: Same thing can be applied to design, you know.

Jamie: The only concepts I hadn't heard of before were "design classes" and "refactoring."

Shakira: Refactoring is used in Extreme Programming, I think she said.

Ed: Yep. It's not a whole lot different than refinement, only you do it after the design or code is completed. Kind of like an optimization pass through the software, if you ask me.

Jamie: Let's get back to the *SafeHome* design. I think we should put these concepts on our review checklist as we develop the design model for *SafeHome*.

Vinod: I agree. But as important, let's all commit to think about them as we develop the design.

9.3.10 Design Classes

The analysis model defines a set of analysis classes (Chapter 8). Each of these classes describes some element of the problem domain, focusing on aspects of the problem that are user visible. The level of abstraction of an analysis class is relatively high.

As the design model evolves, you will define a set of *design classes* that refine the analysis classes by providing design detail that will enable the classes to be implemented and to create a software infrastructure that supports the business solution.

As the software architecture forms, the level of abstraction is reduced as each analysis class (Chapter 8) is transformed into a design representation. That is, analysis classes represent data objects and associated services that are applied to them. Design classes present significantly more technical detail as a guide for implementation.

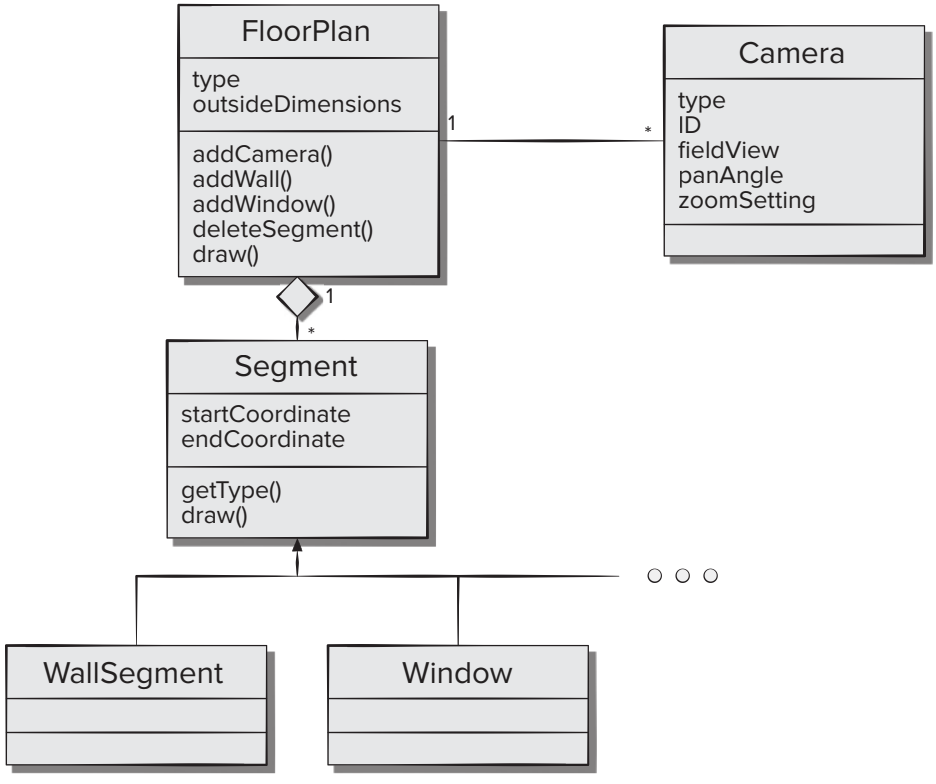
Arlow and Neustadt [Arl02] suggest that each design class be reviewed to ensure that it is "well-formed." They define four characteristics of a well-formed design class:

Complete and sufficient. A design class should be the complete encapsulation of all attributes and methods that can reasonably be expected (based on a knowledgeable interpretation of the class name) to exist for the class. For example, the class **FloorPlan** (Figure 9.3) defined for the *SafeHome* room layout software is complete only if it contains all attributes and methods that can reasonably be associated with the creation of a floor plan. Sufficiency ensures that the design class contains only those methods that are sufficient to achieve the intent of the class, no more and no less.

Primitiveness. Methods associated with a design class should be focused on accomplishing one service for the class. Once the service has been implemented with a method, the class should not provide another way to accomplish the same thing. For example, the class **Segment** (Figure 9.3) for use by the room layout

FIGURE 9.3

Design class for FloorPlan and composite aggregation for the class (see sidebar discussion)



software might have attributes `startCoordinate` and `endCoordinate` to indicate the start and end points of the segment to be drawn. The method *setCoordinates()* provides the only means for establishing start and end points for the segment.

High cohesion. A cohesive design class has a small, focused set of responsibilities and single-mindedly applies attributes and methods to implement those responsibilities. For example, the class **FloorPlan** (Figure 9.3) might contain a set of methods for editing the house floor plan. As long as each method focuses solely on attributes associated with the floor plan, cohesion is maintained.

Low coupling. Within the design model, it is necessary for design classes to collaborate with one another. However, collaboration should be kept to an acceptable minimum. If a design model is highly coupled (all design classes collaborate with all other design classes), the system is difficult to implement, to test, and to maintain over time. In general, design classes within a subsystem should have only limited knowledge of other classes. This restriction, called the *Law of Demeter* [Lie03], suggests that a method should only send messages to methods in neighboring classes.[8]

8 A less formal way of stating the Law of Demeter is "Each unit should only talk to its friends; don't talk to strangers."

SAFEHOME



Refining an Analysis Class into a Design Class

The scene: Ed's cubicle, as design modeling begins.

The players: Vinod and Ed, members of the *SafeHome* software engineering team.

The conversation:

[Ed is working on the **FloorPlan** class (see sidebar discussion in Section 8.3.3 and Figure 8.4) and has refined it for the design model.]

Ed: So you remember the **FloorPlan** class, right? It's used as part of the surveillance and home management functions.

Vinod (nodding): Yeah, I seem to recall that we used it as part of our CRC discussions for home management.

Ed: We did. Anyway, I'm refining it for design. I want to show how we'll actually implement the **FloorPlan** class. My idea is to implement it as a set of linked lists [a specific data structure]. So . . . I had to refine the analysis class **FloorPlan** (Figure 8.4) and actually, sort of simplify it.

Vinod: The analysis class showed only things in the problem domain, well, actually on the computer screen, that were visible to the end user, right?

Ed: Yep, but for the **FloorPlan** design class, I've got to add some things that are implementation specific. I needed to show that **FloorPlan** is an aggregation of segments—hence the **Segment** class—and that the **Segment** class is composed of lists for wall segments, windows, doors, and so on. The class **Camera** collaborates with **FloorPlan,** and obviously, there can be many cameras in the floor plan.

Vinod: Phew, let's see a picture of this new **FloorPlan** design class.

(Ed shows Vinod the drawing shown in Figure 9.3.)

Vinod: Okay, I see what you're trying to do. This allows you to modify the floor plan easily because new items can be added to or deleted from the list—the aggregation—without any problems.

Ed (nodding): Yeah, I think it'll work.

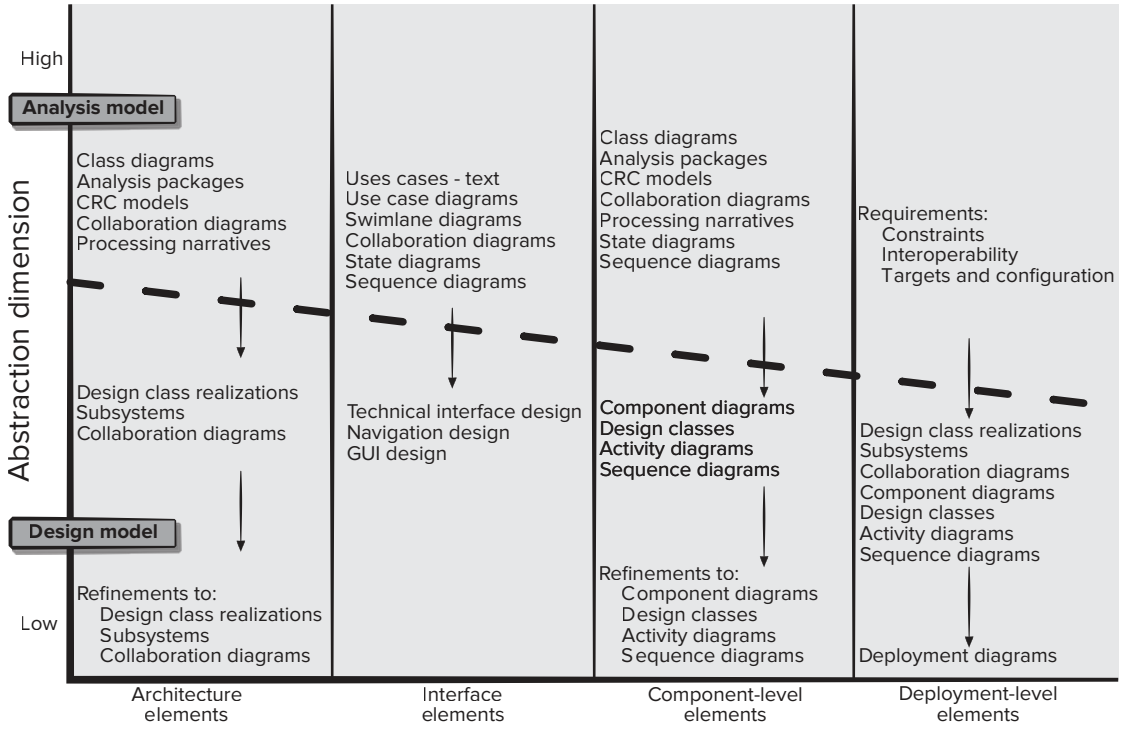
Vinod: So do I.

9.4 THE DESIGN MODEL

The software design model is the equivalent of an architect's plans for a house. It begins by representing the totality of the thing to be built (e.g., a three-dimensional rendering of the house) and slowly refines the thing to provide guidance for constructing each detail (e.g., the plumbing layout). Similarly, the design model that is created for software provides a variety of different views of the system.

The design model can be viewed in two different dimensions, as illustrated in Figure 9.4. The *process dimension* indicates the evolution of the design model as design tasks are executed as part of the software process. The *abstraction dimension* represents the level of detail as each element of the analysis model is transformed into a design equivalent and then refined iteratively. Referring to the figure, the dashed line indicates the boundary between the analysis and design models. In some cases, a clear distinction between the analysis and design models is possible. In other cases, the analysis model slowly blends into the design and a clear distinction is less obvious.

FIGURE 9.4 Dimensions of the design model



The elements of the design model use many of the same UML diagrams[9] that were used in the analysis model. The difference is that these diagrams are refined and elaborated as part of design; more implementation-specific detail is provided, and architectural structure and style, components that reside within the architecture, and interfaces between the components and with the outside world are all emphasized.

You should note, however, that model elements indicated along the horizontal axis are not always developed in a sequential fashion. In most cases, preliminary architectural design sets the stage and is followed by interface design and component-level design, which often occur in parallel. The deployment model is usually delayed until the design has been fully developed.

You can apply design patterns (Chapter 14) at any point during design. These patterns enable you to apply design knowledge to domain-specific problems that have been encountered and solved by others.

9 Appendix 1 provides a tutorial on basic UML concepts and notation.

9.4.1 Design Modeling Principles

There is no shortage of methods for deriving the various elements of a software design model. Some methods are data driven, allowing the data structure to dictate the program architecture and the resultant processing components. Others are pattern driven, using information about the problem domain (the requirements model) to develop architectural styles and processing patterns. Still others are object oriented, using problem domain objects as the driver for the creation of data structures and the methods that manipulate them. Yet all embrace a set of design principles that can be applied, regardless of the method that is used:

Principle 1. ***Design should be traceable to the requirements model.*** The requirements model describes the information domain of the problem, user-visible functions, system behavior, and a set of requirements classes that package business objects with the methods that service them. The design model translates this information into an architecture, a set of subsystems that implement major functions, and a set of components that are the realization of requirements classes. The elements of the design model should be traceable to the requirements model.

Principle 2. ***Always consider the architecture of the system to be built.*** Software architecture (Chapter 10) is the skeleton of the system to be built. It affects interfaces, data structures, program control flow and behavior, the manner in which testing can be conducted, the maintainability of the resultant system, and much more. For all these reasons, design should start with architectural considerations. Only after the architecture has been established should component-level issues be considered.

Principle 3. ***Design of data is as important as design of processing functions.*** Data design is an essential element of architectural design. The ways in which data objects are realized within the design cannot be left to chance. A well-structured data design helps to simplify program flow, makes the design and implementation of software components easier, and makes overall processing more efficient.

Principle 4. ***Interfaces (both internal and external) must be designed with care.*** The ways in which data flows between the components of a system has much to do with processing efficiency, error propagation, and design simplicity. A well-designed interface makes integration easier and assists the tester in validating component functions.

Principle 5. ***User interface design should be tuned to the needs of the end user. However, in every case, it should stress ease of use.*** The user interface is the visible manifestation of the software. No matter how sophisticated its internal functions, no matter how comprehensive its data structures, no matter how well designed its architecture, a poor interface design often leads to the perception that the software is "bad."

Principle 6. ***Component-level design should be functionally independent.*** Functional independence is a measure of the "single-mindedness" of a software component. The functionality that is delivered by a component should be cohesive—that is, it should focus on one and only one function.

Principle 7. ***Components should be loosely coupled to one another and to the external environment.*** Coupling is achieved in many ways—via a component interface, by messaging, and through global data. As the level of coupling increases, the likelihood of error propagation also increases and the overall maintainability of the software decreases. Therefore, component coupling should be kept as low as is reasonable.

Principle 8. ***Design representations (models) should be easily understandable.*** The purpose of design is to communicate information to practitioners who will generate code, to those who will test the software, and to others who may maintain the software in the future. If the design is difficult to understand, it will not serve as an effective communication medium.

Principle 9. ***The design should be developed iteratively.*** With each iteration, the designer should strive for greater simplicity. Like almost all creative activities, design occurs iteratively. The first iterations work to refine the design and correct errors, but later iterations should strive to make the design as simple as is possible.

Principle 10. ***Creation of a design model does not preclude an agile approach.*** Some proponents of agile software development (Chapter 3) insist that the code is the only design documentation that is needed. Yet the purpose of a design model is to help others who must maintain and evolve the system. It is extremely difficult to understand either the higher-level purpose of a code fragment or its interactions with other modules in a modern multithreaded run-time environment.

Agile design documentation should be kept in sync with the design and development, so that at the end of the project the design is documented at a level that allows the code to be understood and maintained. The design model provides benefit because it is created at a level of abstraction that is stripped of unnecessary technical detail and is closely coupled to the application concepts and requirements. Complementary design information can incorporate a design rationale, including the descriptions of rejected architectural design alternatives.

9.4.2 Data Design Elements

Like other software engineering activities, data design (sometimes referred to as *data architecting*) creates a model of data and/or information that is represented at a high level of abstraction (the customer or user's view of data). This data model is then refined into progressively more implementation-specific representations that can be processed by the computer-based system. In many software applications, the architecture of the data will have a profound influence on the architecture of the software that must process it.

The structure of data has always been an important part of software design. At the program-component level, the design of data structures and the associated algorithms required to manipulate them is essential to the creation of high-quality applications. At the application level, the translation of a data model (derived as part of requirements engineering) into a database is pivotal to achieving the business objectives of a system. At the business level, the collection of information stored in disparate databases and reorganized into a "data warehouse" enables data mining or knowledge discovery that can have an impact on the success of the business itself. In every case, data design plays an important role. Data design is discussed in more detail in Chapter 10.

9.4.3 Architectural Design Elements

The *architectural design* for software is the equivalent to the floor plan of a house. The floor plan depicts the overall layout of the rooms; their size, shape, and relationship to one another; and the doors and windows that allow movement into and out of the rooms. The floor plan gives us an overall view of the house. Architectural design elements give us an overall view of the software.

The architectural model [Sha15] is derived from three sources: (1) information about the application domain for the software to be built, (2) specific requirements model elements such as use cases or analysis classes, their relationships, and collaborations for the problem at hand, and (3) the availability of architectural styles (Chapter 10) and patterns (Chapter 14).

The architectural design element is usually depicted as a set of interconnected subsystems, often derived from analysis packages within the requirements model. Each subsystem may have its own architecture (e.g., a graphical user interface might be structured according to a preexisting architectural style for user interfaces). Techniques for deriving specific elements of the architectural model are presented in Chapter 10.

9.4.4 Interface Design Elements

The interface design for software is analogous to a set of detailed drawings (and specifications) for the doors, windows, and external utilities of a house. The detailed drawings (and specifications) for the doors, windows, and external utilities tell us how things and information flow into and out of the house and within the rooms that are part of the floor plan. The interface design elements for software depict information flows into and out of a system and how it is communicated among the components defined as part of the architecture.

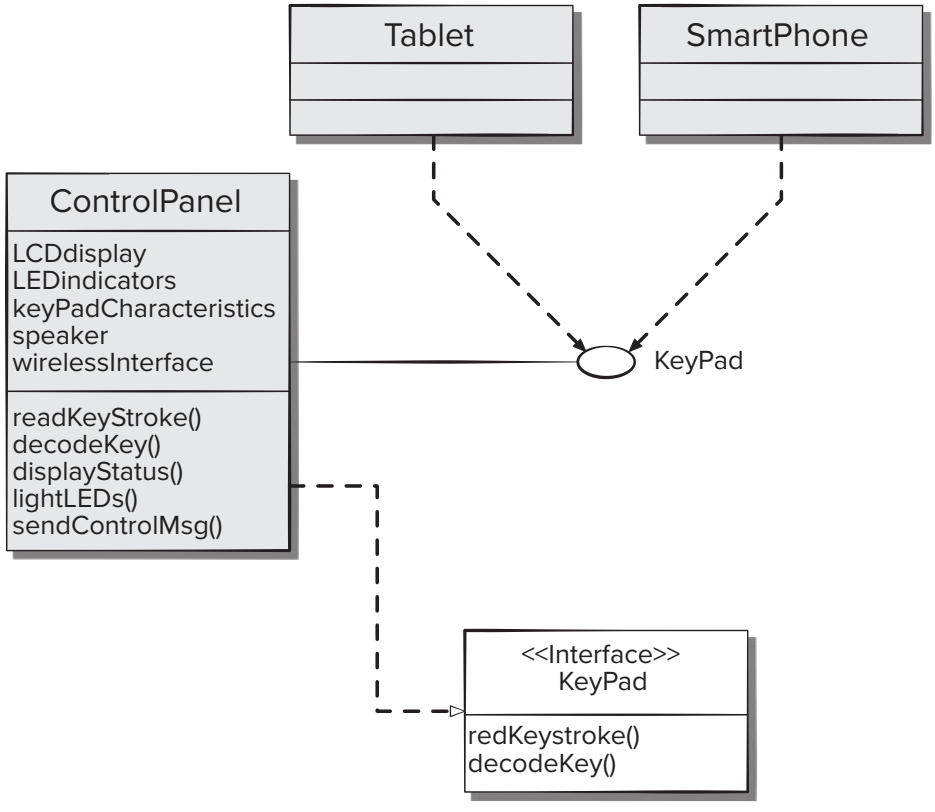
There are three important elements of interface design: (1) the user interface (UI); (2) external interfaces; to other systems, devices, networks, or other producers or consumers of information; and (3) internal interfaces between various design components. These interface design elements allow the software to communicate externally and enable internal communication and collaboration among the components that populate the software architecture.

UI design (increasingly called *UX* or *user experience design*) is a major software engineering action and is considered in detail in Chapter 12. UX design focuses on ensuring the usability of the UI design. A usable design incorporates carefully chosen aesthetic elements (e.g., layout, color, graphics, information layout), ergonomic elements (e.g., interaction mechanisms, information placement, metaphors, UI navigation), and technical elements (e.g., UX patterns, reusable components). In general, the UI is a unique subsystem within the overall application architecture designed to provide the end user with a satisfying user experience.

The design of external interfaces requires definitive information about the entity to which information is sent or received. In every case, this information should be collected during requirements engineering (Chapter 7) and verified once the interface design commences.[10] The design of external interfaces should incorporate error checking and appropriate security features.

10 Interface characteristics can change with time. Therefore, a designer should ensure that the specification for the interface is accurate and complete.

FIGURE 9.5
Interface representation for ControlPanel



The design of internal interfaces is closely aligned with component-level design (Chapter 11). Design realizations of analysis classes represent all operations and the messaging schemes required to enable communication and collaboration between operations in various classes. Each message must be designed to accommodate the requisite information transfer and the specific functional requirements of the operation that has been requested.

In some cases, an interface is modeled in much the same way as a class. An interface is a set of operations that describes some part of the behavior of a class and provides access to these operations.

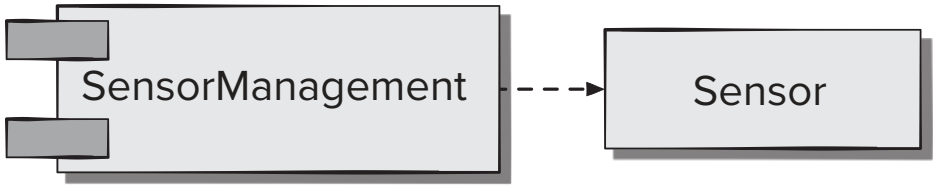
For example, the *SafeHome* security function makes use of a control panel that allows a homeowner to control certain features of the security function. In an advanced version of the system, control panel functions may be implemented via a mobile platform (e.g., smartphone or tablet) and are represented in Figure 9.5.

9.4.5 Component-Level Design Elements

The component-level design for software is the equivalent to a set of detailed drawings (and specifications) for each room in a house. These drawings depict wiring and plumbing within each room, the location of electrical receptacles and wall switches,

FIGURE 9.6

A UML component diagram



faucets, sinks, showers, tubs, drains, cabinets, and closets, and every other detail associated with a room.

The component-level design for software fully describes the internal detail of each software component. To accomplish this, the component-level design defines data structures for all local data objects and algorithmic detail for all processing that occurs within a component and an interface that allows access to all component operations (behaviors).

Within the context of object-oriented software engineering, a component is represented in UML diagrammatic form, as shown in Figure 9.6. In this figure, a component named **SensorManagement** (part of the *SafeHome* security function) is represented. A dashed arrow connects the component to a class named **Sensor** that is assigned to it. The **SensorManagement** component performs all functions associated with *SafeHome* sensors including monitoring and configuring them. Further discussion of component design is presented in Chapter 11.

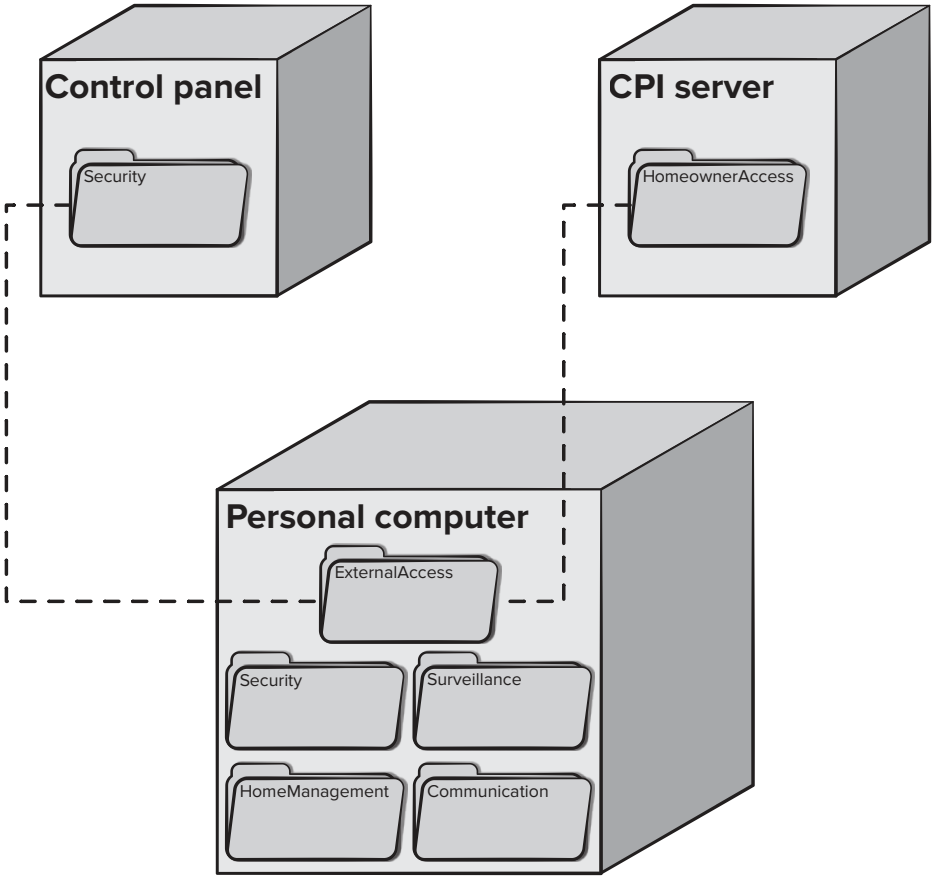
The design details of a component can be modeled at many different levels of abstraction. A UML activity diagram can be used to represent processing logic. Algorithmic structure details for a component can be represented using either pseudocode (a programming languagelike representation described in Chapter 11) or some other diagrammatic form (e.g., flowchart). Data structure details are usually modeled using pseudocode or the programming language to be used for implementation.

9.4.6 Deployment-Level Design Elements

Deployment-level design elements indicate how software functionality and subsystems will be allocated within the physical computing environment that will support the software. For example, the elements of the *SafeHome* product are configured to operate within three primary computing environments—a mobile device—in this case a PC, the *SafeHome* control panel, and a server housed at CPI Corp. (providing Internet-based access to the system).

During design, a UML deployment diagram is developed and then refined, as shown in Figure 9.7. In the figure, three computing environments are shown (in the full design there would be more details included: sensors, cameras, and the functionality delivered by mobile platforms). The subsystems (functionality) housed within each computing element are indicated. For example, the personal computer houses subsystems that implement security, surveillance, home management, and communications features. In addition, an external access subsystem has been designed to manage all attempts to access the *SafeHome* system from an external source. Each subsystem would be elaborated to indicate the components that it implements.

FIGURE 9.7
A UML deployment diagram



The diagram shown in Figure 9.7 is in *descriptor form.* This means that the deployment diagram shows the computing environment but does not explicitly indicate configuration details. For example, the "personal computer" is not further identified. It could be a Mac, a Windows-based PC, a Linux box, or a mobile platform with its associated operating system. These details are provided when the deployment diagram is revisited in *instance form* during the latter stages of design or as construction begins. Each instance of the deployment (a specific, named hardware configuration) is identified.

9.5 SUMMARY

Software design commences as the first iteration of requirements engineering concludes. The intent of software design is to apply a set of principles, concepts, and practices that lead to the development of a high-quality system or product. The goal of design is to create a model of software that will implement all customer requirements

correctly and bring delight to those who use it. Software designers must sift through many design alternatives and converge on a solution that best suits the needs of project stakeholders.

The design process moves from a "big picture" view of software to a narrower view that defines the detail required to implement a system. The process begins by focusing on architecture. Subsystems are defined, communication mechanisms among subsystems are established, components are identified, and a detailed description of each component is developed. In addition, external, internal, and user interfaces are designed at the same time.

Design concepts have evolved over the first 60 years of software engineering work. They describe attributes of computer software that should be present regardless of the software engineering process that is chosen, the design methods that are applied, or the programming languages that are used. In essence, design concepts emphasize the need for abstraction as a mechanism for creating reusable software components; the importance of architecture as a way to better understand the overall structure of a system; the benefits of pattern-based engineering as a technique for designing software with proven capabilities; the value of separation of concerns and effective modularity as a way to make software more understandable, more testable, and more maintainable; the consequences of information hiding as a mechanism for reducing the propagation of side effects when errors do occur; the impact of functional independence as a criterion for building effective modules; the use of refinement as a design mechanism; the application of refactoring for optimizing the design that is derived; the importance of object-oriented classes and the characteristics that are related to them; the need to use abstraction to reduce coupling between components; and the importance of design for testing.

The design model encompasses four different elements. As each of these elements is developed, a more complete view of the design evolves. The architectural element uses information derived from the application domain, the requirements model, and available catalogs for patterns and styles to derive a complete structural representation of the software, its subsystems, and components. Interface design elements model external and internal interfaces and the user interface. Component-level elements define each of the modules (components) that populate the architecture. Finally, deployment-level design elements allocate the architecture, its components, and the interfaces to the physical configuration that will house the software.

PROBLEMS AND POINTS TO PONDER

9.1. Do you design software when you "write" a program? What makes software design different from coding?

9.2. If a software design is not a program (and it isn't), then what is it?

9.3. How do we assess the quality of a software design?

9.4. Describe software architecture in your own words.

9.5. Describe separation of concerns in your own words. Is there a case when a "divide and conquer" strategy may not be appropriate? How might such a case affect the argument for modularity?

9.6. Discuss the relationship between the concept of information hiding as an attribute of effective modularity and the concept of module independence.

9.7. How are the concepts of coupling and software portability related? Provide examples to support your discussion.

9.8. Apply a "stepwise refinement approach" to develop three different levels of procedural abstractions for one or more of the following programs: (1) Develop a check writer that, given a numeric dollar amount, will print the amount in words normally required on a check. (2) Iteratively solve for the roots of a transcendental equation. (3) Develop a simple task-scheduling algorithm for an operating system.

9.9. Does *refactoring* mean that you modify the entire design iteratively? If not, what does it mean?

9.10. Briefly describe each of the four elements of the design model.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

10

ARCHITECTURAL DESIGN—A RECOMMENDED APPROACH

Design has been described as a multistep process in which representations of data and program structure, interface characteristics, and procedural detail are synthesized from information requirements. As we noted in Chapter 9, design is information driven. Software design methods are derived from consideration of each of the three domains of the analysis model. The decisions made while considering the data, functional, and behavioral domains serve as guides for the creation of the software architectural design.

KEY CONCEPTS

agility and architecture 185
archetypes 196
architectural considerations 193
architectural decisions 195
architectural description language 184
architectural descriptions 184
architectural design 196
architectural patterns 187
architectural styles 186
architecture 182
architecture conformance checking 204
architecture trade-off analysis method (ATAM) 201
layered architectures 189
refining the architecture 198
taxonomy of architectural styles 187

QUICK LOOK

What is it? Architectural design represents the structure of data and program components that are required to build a computer-based system. It considers the architectural style that the system will take, the structure and properties of the components that constitute the system, and the interrelationships that occur among all architectural components of a system.

Who does it? Although a software engineer can design both data and architecture, the job is often allocated to specialists when large, complex systems are to be built. A database or data warehouse designer creates the data architecture for a system. The "system architect" selects an appropriate architectural style from the requirements derived during software requirements analysis.

Why is it important? You wouldn't attempt to build a house without a blueprint, would you? You also wouldn't begin drawing blueprints by sketching the plumbing layout for the house. You'd need to look at the big picture—the house itself—before you worry about details. That's what architectural design does—it provides you with the big picture and ensures that you've got it right.

What are the steps? Architectural design begins with data design and then proceeds to the derivation of one or more representations of the architectural structure of the system. Alternative architectural styles or patterns are analyzed to derive the structure that is best suited to customer requirements and quality attributes. Once an alternative has been selected, the architecture is elaborated, using an architectural design method.

What is the work product? An architecture model encompassing data architecture and program structure is created during architectural design. In addition, component properties and relationships (interactions) are described.

How do I ensure that I've done it right? At each stage, software design work products are reviewed for clarity, correctness, completeness, and consistency with requirements and with one another.

Philippe Kruchten, Grady Booch, Kurt Bittner, and Rich Reitman [Mic09] suggest that software architecture identifies a system's "structural elements and their interfaces," along with the "behavior" of individual components and subsystems. They write that the job of architectural design is to create "coherent, well-planned representations" of the system and software.

Methods to create such representations of the data and architectural layers of the design model are presented in this chapter. The objective is to provide a systematic approach for the derivation of the architectural design—the preliminary blueprint from which software is constructed.

10.1 Software Architecture

In their landmark book on the subject, Shaw and Garlan [Sha15] argue that since the earliest days of computer programming, "software systems have had architectures, and programmers have been responsible for the interactions among the modules and the global properties of the assemblage." Today, effective software architecture and its explicit representation and design have become dominant themes in software engineering.

10.1.1 What Is Architecture?

When you consider the architecture of a building, many different attributes come to mind. At the most simplistic level, you think about the overall shape of the physical structure. But in reality, architecture is much more. It is the manner in which the various components of the building are integrated to form a cohesive whole. It is the way in which the building fits into its environment and meshes with other buildings in its vicinity. It is the degree to which the building meets its stated purpose and satisfies the needs of its owner. It is the aesthetic feel of the structure—the visual impact of the building—and the way textures, colors, and materials are combined to create the external facade and the internal "living environment." It is small details—the design of lighting fixtures, the type of flooring, the placement of wall hangings; the list is almost endless. And finally, it is art.

Architecture is also something else. It is "thousands of decisions, both big and small" [Tyr05]. Some of these decisions are made early in design and can have a profound impact on all other design actions. Others are delayed until later, thereby eliminating overly restrictive constraints that would lead to a poor implementation of the architectural style.

Just like the plans for a house are merely a representation of the building, the software architecture representation is not an operational product. Rather, it is a representation that enables you to (1) analyze the effectiveness of the design in meeting its stated requirements, (2) consider architectural alternatives at a stage when making design changes is still relatively easy, and (3) reduce the risks associated with the construction of the software.

This definition emphasizes the role of "software components" in any architectural representation. In the context of architectural design, a software component can be something as simple as a program module or an object-oriented class, but it can also be extended to include databases and "middleware" that enable the configuration of

a network of clients and servers. The properties of components are those characteristics that are necessary to an understanding of how the components interact with other components. At the architectural level, internal properties (e.g., details of an algorithm) are not specified. The relationships between components can be as simple as a procedure call from one module to another or as complex as a database access protocol.

We believe that a software design can be thought of as an instance of some software architecture. However, the elements and structures that are defined as parts of particular software architectures are the root of every design. It is our recommendation that design should begin with a consideration of the software architecture.

10.1.2 Why Is Architecture Important?

In a book dedicated to software architecture, Bass and his colleagues [Bas12] identify three key reasons that software architecture is important:

- Software architecture provides a representation that facilitates communication among all stakeholders.
- The architecture highlights early design decisions that will have a profound impact on all software engineering work that follows.
- The architecture constitutes a relatively small model of how the system components are structured and work together.

The architectural design model and the architectural patterns contained within it are transferable. That is, architecture genres, styles, and patterns (Sections 10.3 through 10.6) can be applied to the design of other systems and represent a set of abstractions that enable software engineers to describe architecture in predictable ways.

Making good decisions while defining the software architecture is critical to the success of a software product. The software architecture sets the structure of the system and determines its quality [Das15].

10.1.3 Architectural Descriptions

Each of us has a mental image of what the word *architecture* means. The implication is that different stakeholders will see a given software architecture from different viewpoints that are driven by different sets of concerns. This implies that an architectural description is actually a set of work products that reflect different views of the system.

Smolander, Rossi, and Purao [Smo08] have identified multiple metaphors, representing different views of the same architecture that stakeholders use to understand the term *software architecture*. The *blueprint metaphor* seems to be most familiar to the stakeholders who write programs to implement a system. Developers regard architecture descriptions as a means of transferring explicit information from architects to designers to software engineers charged with producing the system components.

The *language metaphor* views architecture as a facilitator of communication across stakeholder groups. This view is preferred by stakeholders with a high customer focus (e.g., managers or marketing experts). The architectural description needs to be concise and easy to understand because it forms the basis for negotiation, particularly in determining system boundaries.

The *decision metaphor* represents architecture as the product of decisions involving trade-offs among properties such as cost, usability, maintainability, and performance that have resource consequences for the system being designed. Stakeholders such as project managers view architectural decisions as the basis for allocating project resources and tasks. These decisions may affect the sequence of tasks and shape the structure of the software team.

The *literature metaphor* is used to document architectural solutions constructed in the past. This view supports the construction of artifacts and the transfer of knowledge between designers and software maintenance staff. It also supports stakeholders whose concern is reuse of components and designs.

An *architectural description* (AD) represents a system using multiple views, where each *view* is "a representation of a whole system from the perspective of a related set of [stakeholder] concerns." The IEEE Computer Society standard IEEE-Std-42010:2011(E), *Systems and software engineering—Architectural description* [IEE11], describes the use of architecture viewpoints, architecture frameworks, and architecture description languages as a means of codifying the conventions and common practices for architectural description.

10.1.4 Architectural Decisions

Each view developed as part of an architectural description addresses a specific stakeholder concern. To develop each view (and the architectural description as a whole), the system architect considers a variety of alternatives and ultimately decides on the specific architectural features that best meet the concern. Architectural decisions themselves can be considered to be one view of the architecture. The reasons that decisions were made provide insight into the structure of a system and its conformance to stakeholder concerns.

As a system architect, you can use the template suggested in the sidebar to document each major decision. By doing this, you provide a rationale for your work and establish a historical record that can be useful when design modifications must be made. For agile developers, a lightweight *architectural decision record* (ADR) might simply contain a title, a context (assumptions and constraints), the decision (resolution), status (proposed accepted rejected), and consequences (implications) [Nyg11].

Grady Booch [Boo11a] writes that when setting out to build an innovative product, software engineers often feel compelled to plunge right in, build stuff, fix what doesn't work, improve what does work, and then repeat the process. After doing this a few times, they begin to recognize that the architecture should be defined first and decisions associated with architectural choices must be stated explicitly. It may not be possible to predict the right choices before building a new product. However, if innovators find that architectural decisions are worth repeating after testing their prototypes in the field, then a *dominant design*[1] for this type of product may begin to emerge. Without documenting what worked and what did not, it is hard for software engineers to decide when to innovate and when to use previously created architecture.

1 *Dominant design* describes an innovative software architecture or process that becomes an industry standard after a period of successful adaptation and use in the marketplace.

INFO



Architecture Decision Description Template

Each major architectural decision can be documented for later review by stakeholders who want to understand the architecture description that has been proposed. The template presented in this sidebar is an adapted and abbreviated version of a template proposed by Tyree and Ackerman [Tyr05].

Design issue: Describe the architectural design issues that are to be addressed.

Resolution: State the approach you've chosen to address the design issue.

Category: Specify the design category that the issue and resolution address (e.g., data design, content structure, component structure, integration, presentation).

Assumptions: Indicate any assumptions that helped shape the decision.

Constraints: Specify any environmental constraints that helped shape the decision (e.g., technology standards, available patterns, project-related issues).

Alternatives: Briefly describe the architectural design alternatives that were considered and why they were rejected.

Argument: State why you chose the resolution over other alternatives.

Implications: Indicate the design consequences of making the decision. How will the resolution affect other architectural design issues? Will the resolution constrain the design in any way?

Related decisions: What other documented decisions are related to this decision?

Related concerns: What other requirements are related to this decision?

Work products: Indicate where this decision will be reflected in the architecture description.

Notes: Reference any team notes or other documentation that was used to make the decision.

10.2 AGILITY AND ARCHITECTURE

The view of some agile developers is that architectural design is equated with "big design upfront." In their view, this leads to unnecessary documentation and the implementation of unnecessary features. However, most agile developers would agree [Fal10] that it is important to focus on software architecture when a system is complex (i.e., when a product has a large number of requirements, lots of stakeholders, or a large number of global users). For this reason, it is important to integrate new architectural design practices into agile process models.

To make early architectural decisions and avoid the rework required to correct the quality problems encountered when the wrong architecture is chosen, agile developers

need to anticipate architectural elements[2] and implied structure that emerges from the collection of user stories gathered (Chapter 7). By creating an architectural prototype (e.g., a *walking skeleton*) and developing explicit architectural work products to communicate the right information to the necessary stakeholders, an agile team can satisfy the need for architectural design.

Using a technique called *storyboarding*, the architect contributes architectural user stories to the project and works with the product owner to prioritize the architectural stories with the business user stories as "sprints" (work units) are planned. The architect works with the team during the sprint to ensure that the evolving software continues to show high architectural quality as defined by the nonfunctional product requirements. If quality is high, the team is left alone to continue development on its own. If not, the architect joins the team for the duration of the sprint. After the sprint is completed, the architect reviews the working prototype for quality before the team presents it to the stakeholders in a formal sprint review. Well-run agile projects make use of iterative work product delivery (including architectural documentation) with each sprint. Reviewing the work products and code as it emerges from each sprint is a useful form of architectural review.

Responsibility-driven architecture (RDA) is a process that focuses on when, how, and who should make the architectural decisions on a project team. This approach also emphasizes the role of architect as being a servant-leader rather than an autocratic decision maker and is consistent with the agile philosophy. The architect acts as facilitator and focuses on how the development team works to accommodate stakeholder's nontechnical concerns (e.g., business, security, usability).

Agile teams usually have the freedom to make system changes as new requirements emerge. Architects want to make sure that the important parts of the architecture were carefully considered and that developers have consulted the appropriate stakeholders. Both concerns may be satisfied by making use of a practice called *progressive sign-off* in which the evolving product is documented and approved as each successive prototype is completed [Bla10].

Using a process that is compatible with the agile philosophy provides verifiable sign-off for regulators and auditors, without preventing the empowered agile teams from making the decisions needed. At the end of the project, the team has a complete set of work products and the architecture has been reviewed for quality as it evolved.

10.3 Architectural Styles

When a builder uses the phrase "center hall colonial" to describe a house, most people familiar with houses in the United States will be able to conjure a general image of what the house will look like and what the floor plan is likely to be. The builder has used an *architectural style* as a descriptive mechanism to differentiate the house from other styles (e.g., A-frame, raised ranch, Cape Cod). But more important,

2 An excellent discussion of architectural agility can be found in [Bro10a].

the architectural style is also a template for construction. Further details of the house must be defined, its final dimensions must be specified, customized features may be added, building materials are to be determined, but the style—a "center hall colonial"—guides the builder in his work.

The software that is built for computer-based systems also exhibits one of many architectural styles. Each style describes a system category that encompasses (1) a set of components (e.g., a database, computational modules) that perform a function required by a system, (2) a set of connectors that enable "communication, coordination and cooperation" among components, (3) constraints that define how components can be integrated to form the system, and (4) semantic models that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts [Bas12].

An *architectural style* is a transformation that is imposed on the design of an entire system. The intent is to establish a structure for all components of the system. In the case where an existing architecture is to be refactored (Chapter 27), the imposition of an architectural style will result in fundamental changes to the structure of the software including a reassignment of the functionality of components [Bos00].

An *architectural pattern*, like an architectural style, imposes a transformation on the design of an architecture. However, a pattern differs from a style in a number of fundamental ways: (1) the scope of a pattern is less broad, focusing on one aspect of the architecture rather than the architecture in its entirety, (2) a pattern imposes a rule on the architecture, describing how the software will handle some aspect of its functionality at the infrastructure level (e.g., concurrency) [Bos00], and (3) architectural patterns (Section 10.3.2) tend to address specific behavioral issues within the context of the architecture (e.g., how real-time applications handle synchronization or interrupts). Patterns can be used in conjunction with an architectural style to shape the overall structure of a system.

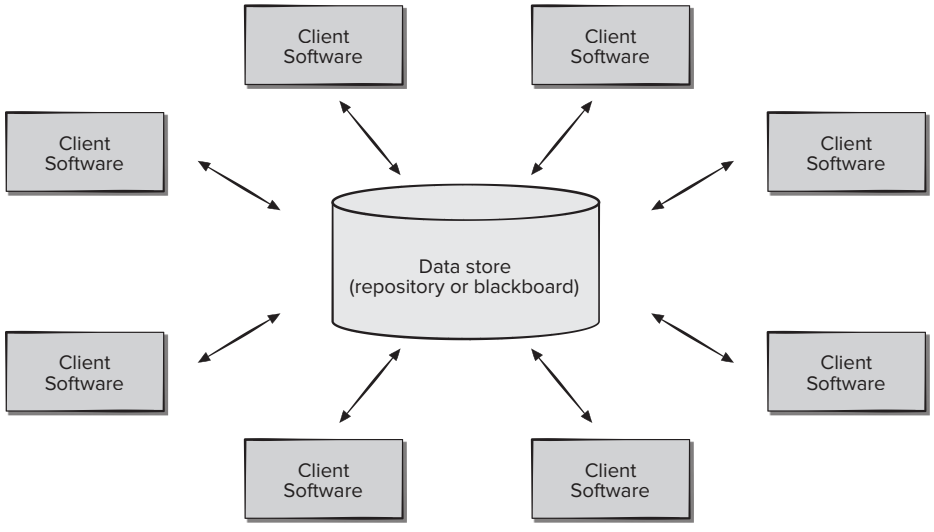
10.3.1 A Brief Taxonomy of Architectural Styles

Although millions of computer-based systems have been created over the past 60 years, the vast majority can be categorized into one of a relatively small number of architectural styles.

Data-Centered Architectures. A data store (e.g., a file or database) resides at the center of this architecture and is accessed frequently by other components that update, add, delete, or otherwise modify data within the store. Figure 10.1 illustrates a typical data-centered style. Client software accesses a central repository. In some cases, the data repository is passive. That is, client software accesses the data independent of any changes to the data or the actions of other client software. A variation on this approach transforms the repository into a "blackboard" that sends notifications to client software when data of interest to the client changes.

Data-centered architectures promote *integrability* [Bas12]. That is, existing components can be changed and new client components added to the architecture without concern about other clients (because the client components operate independently). In addition, data can be passed among clients using the blackboard mechanism (i.e., the blackboard component serves to coordinate the transfer of information between clients). Client components independently execute processes.

FIGURE 10.1
Data-centered architecture



Data-Flow Architectures. This architecture is applied when input data are to be transformed through a series of computational or manipulative components into output data. A pipe-and-filter pattern (Figure 10.2) has a set of components, called *filters*, connected by *pipes* that transmit data from one component to the next. Each filter works independently of those components upstream and downstream, is designed to expect data input of a certain form, and produces data output (to the next filter) of a specified form. However, the filter does not require knowledge of the workings of its neighboring filters.

FIGURE 10.2 **Data-flow architecture**

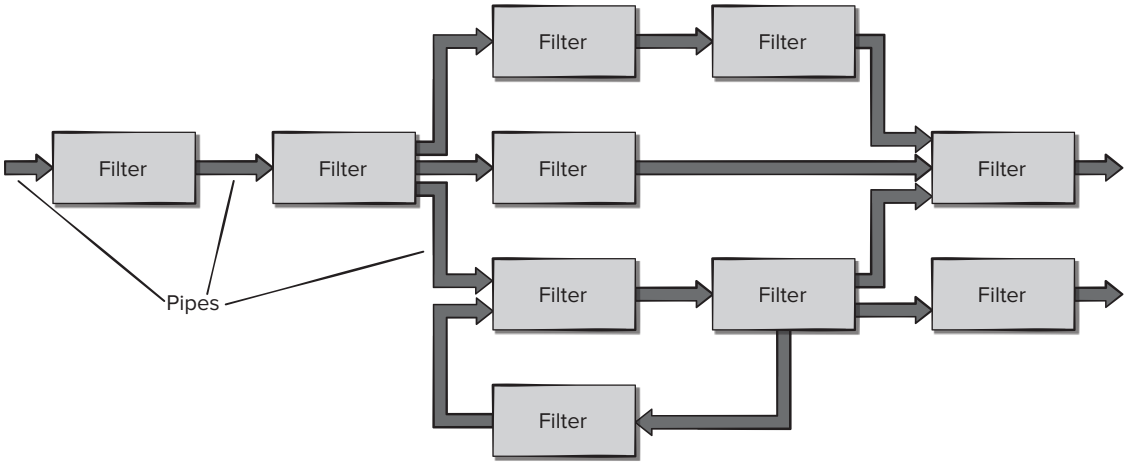
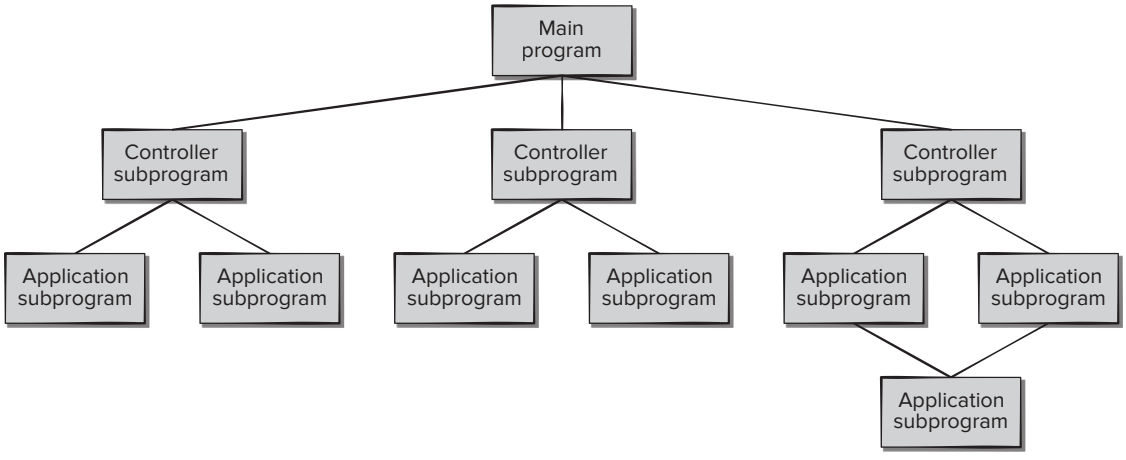


FIGURE 10.3 Main program/subprogram architecture



Call-and-Return Architectures. This architectural style enables you to achieve a program structure that is relatively easy to modify and scale. Two substyles [Bas12] that exist within this category:

- *Main program/subprogram architectures.* This classic program structure decomposes function into a control hierarchy where a "main" program invokes several program components, which in turn may invoke still other components. Figure 10.3 illustrates an architecture of this type.
- *Remote procedure call architectures.* The components of a main program/subprogram architecture are distributed across multiple computers on a network.

Object-Oriented Architectures. The components of a system encapsulate data and the operations that must be applied to manipulate the data. Communication and coordination between components are accomplished via message passing. Figure 10.4 contains a UML communication diagram that shows the message passing for the login portion of a system implemented using an object-oriented architecture. Communications diagrams are described in more details in Appendix 1 of this book.

Layered Architectures. The basic structure of a layered architecture is illustrated in Figure 10.5. A number of different layers are defined, each accomplishing operations that progressively become closer to the machine instruction set. At the outer layer, components service user interface operations. At the inner layer, components perform operating system interfacing. Intermediate layers provide utility services and application software functions.

Model-View-Controller (MVC) architecture [Kra88] is one of a number of suggested mobile infrastructure models often used in Web development. The *model* contains all application-specific content and processing logic. The *view* contains all

FIGURE 10.4

UML communication diagram

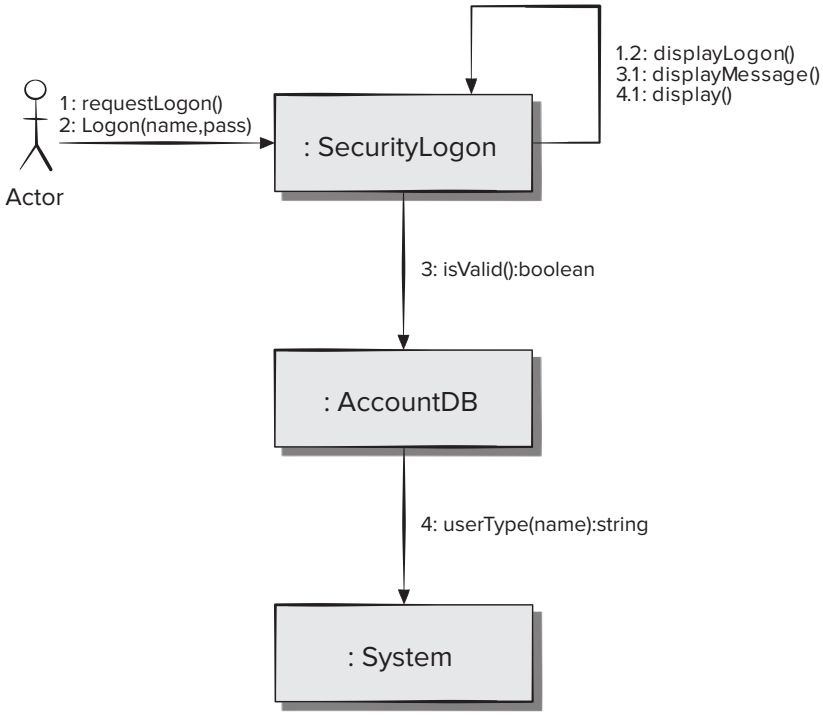


FIGURE 10.5

Layered architecture

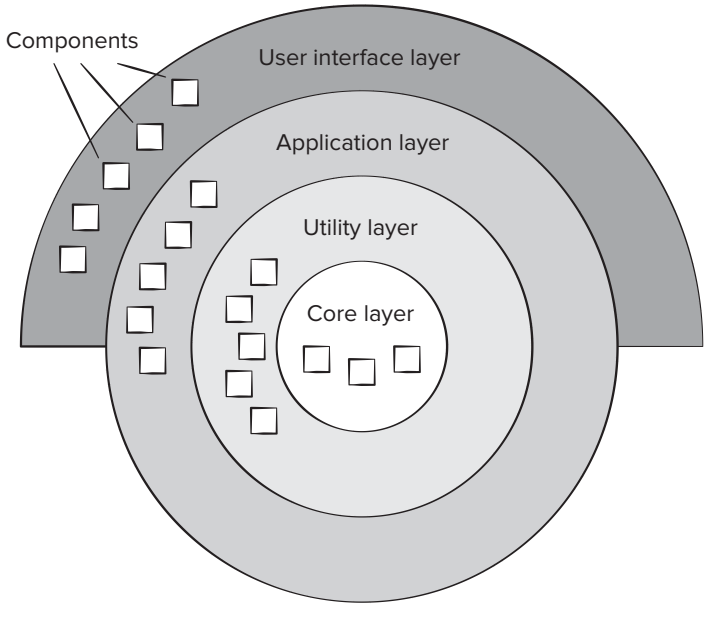
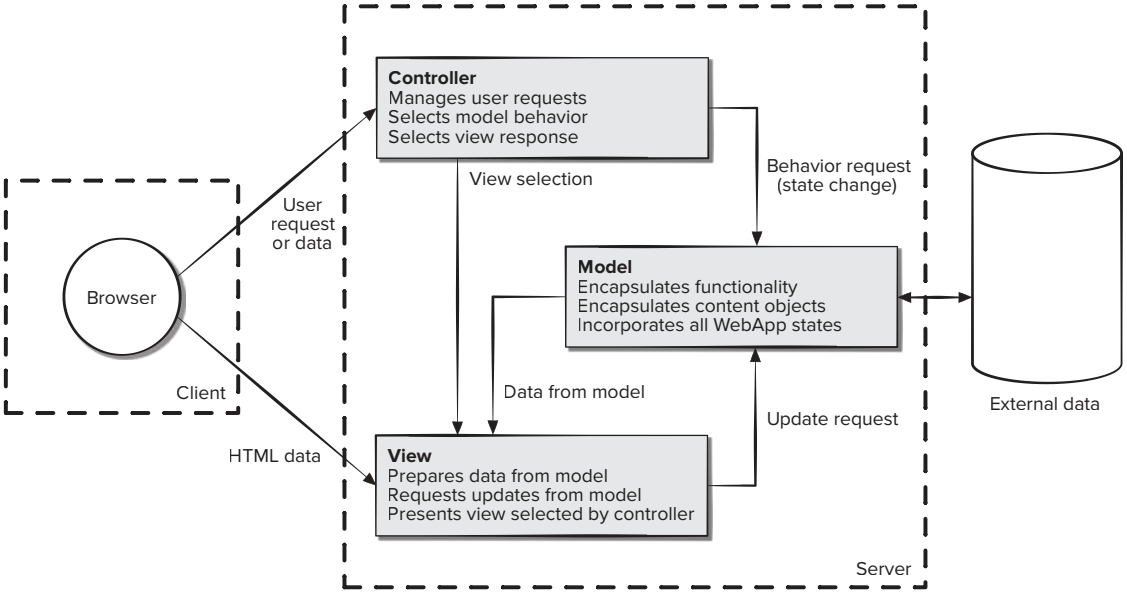


FIGURE 10.6 The MVC architecture



Source: Adapted from Jacyntho, Mark Douglas, Schwabe, Daniel and Rossi, Gustavo, "An Architecture for Structuring Complex Web Applications," 2002, available at http://www-di.inf.puc-rio.br/schwabe/papers/OOHDMJava2%20Report.pdf

interface-specific functions and enables the presentation of content and processing logic required by the end user. The *controller* manages access to the model and the view and coordinates the flow of data between them. A schematic representation of the MVC architecture is shown in Figure 10.6.

Referring to the figure, user requests are handled by the controller. The controller also selects the view object that is applicable based on the user request. Once the type of request is determined, a behavior request is transmitted to the model, which implements the functionality or retrieves the content required to accommodate the request. The model object can access data stored in a corporate database, as part of a local data store, or as a collection of independent files. The data developed by the model must be formatted and organized by the appropriate view object and then transmitted from the application server back to the client-based browser for display on the customer's machine.

These architectural styles are only a small subset of those available.[3] Once requirements engineering uncovers the characteristics and constraints of the system to be built, the architectural style and/or combination of patterns that best fits those characteristics and constraints can be chosen. In many cases, more than one pattern might be appropriate and alternative architectural styles can be designed and evaluated. For example, a layered style (appropriate for most systems) can be combined with a data-centered architecture in many database applications.

3 See [Roz11], [Tay09], [Bus07], [Gor06], or [Bas12] for a detailed discussion of architectural styles and patterns.

SAFEHOME



Choosing an Architectural Style

The scene: Jamie's cubicle, as design modeling begins.

The players: Jamie and Ed—members of the *SafeHome* software engineering team.

The conversation:

Ed (frowning): We've been modeling the security function using UML . . . you know, classes, relationships, that sort of stuff. So I guess the object-oriented architecture is the right way to go.

Jamie: But . . .?

Ed: But . . . I have trouble visualizing what an object-oriented architecture is. I get the call-and-return architecture, sort of a conventional process hierarchy, but OO . . . I don't know, it seems sort of amorphous.

Jamie (smiling): Amorphous, huh?

Ed: Yeah . . . what I mean is I can't visualize a real structure, just design classes floating in space.

Jamie: Well, that's not true. There are class hierarchies . . . think of the hierarchy (aggregation) we did for the **FloorPlan** object [Figure 9.3]. An OO architecture is a combination of that structure and the interconnections—you know, collaborations—between the classes. We can show it by fully describing the attributes and operations, the messaging that goes on, and the structure of the classes.

Ed: I'm going to spend an hour mapping out a call-and-return architecture; then I'll go back and consider an OO architecture.

Jamie: Doug'll have no problem with that. He said that we should consider architectural alternatives. By the way, there's absolutely no reason why both of these architectures couldn't be used in combination with one another.

Ed: Good. I'm on it.

Choosing the right architecture style can be tricky. Real-world problems often follow more than one problem frame, and a combination architectural model may result. For example, the model-view-controller (MVC) architecture used in WebApp design[4] might be viewed as combining two problem frames (command behavior and information display). In MVC, the end user's command is sent from the browser window to a command processor (controller) that manages access to the content (model) and instructs the information rendering model (view) to translate it for display by the browser software.

10.3.2 Architectural Patterns

As the requirements model is developed, you'll notice that the software must address several broad problems that span the entire application. For example, the requirements model for virtually every e-commerce application is faced with the following problem: *How do we offer a broad array of goods to many different customers and allow those customers to find and purchase our goods easily?*

The requirements model also defines a context in which this question must be answered. For example, an e-commerce business that sells golf equipment to consumers will operate in a different context than an e-commerce business that sells high-priced industrial equipment to medium and large corporations. In addition, a set of limitations and constraints may affect the way you address the problem to be solved.

4 The MVC architecture is considered in more detail in Chapter 13.

Architectural patterns address an application-specific problem within a specific context and under a set of limitations and constraints. The pattern proposes an architectural solution that can serve as the basis for architectural design.

Previously in this chapter, we noted that most applications fit within a specific domain or genre and that one or more architectural styles may be appropriate for that genre. For example, the overall architectural style for an application might be call and return or object oriented. But within that style, you will encounter a set of common problems that might best be addressed with specific architectural patterns. Some of these problems and a more complete discussion of architectural patterns are presented in Chapter 14.

10.3.3 Organization and Refinement

Because the design process often leaves you with a number of architectural alternatives, it is important to establish a set of design criteria that can be used to assess an architectural design that is derived. The following questions [Bas12] provide insight into an architectural style:

> **Control.** How is control managed within the architecture? Does a distinct control hierarchy exist, and if so, what is the role of components within this control hierarchy? How do components transfer control within the system? How is control shared among components? What is the control topology (i.e., the geometric form that the control takes)? Is control synchronized, or do components operate asynchronously?
>
> **Data.** How are data communicated between components? Is the flow of data continuous, or are data objects passed to the system sporadically? What is the mode of data transfer (i.e., are data passed from one component to another or are data available globally to be shared among system components)? Do data components (e.g., a blackboard or repository) exist, and if so, what is their role? How do functional components interact with data components? Are data components passive or active (i.e., does the data component actively interact with other components in the system)? How do data and control interact within the system?

The answers to these questions provide the designer with an early assessment of design quality and lay the foundation for more detailed analysis of the architecture.

Evolutionary process models (Chapter 2) have become very popular. This implies the software architectures may need to evolve as each product increment is planned and implemented. In Chapter 9, we described this process as refactoring—improving the internal structure of the system without changing its external behavior.

10.4 ARCHITECTURAL CONSIDERATIONS

Buschmann and Henny [Bus10a, Bus10b] suggest several architectural considerations that can provide software engineers with guidance as architecture decisions are made.

- **Economy.** The best software is uncluttered and relies on abstraction to reduce unnecessary detail. It avoids complexity due to unnecessary functions and features.

- **Visibility.** As the design model is created, architectural decisions and the reasons for them should be obvious to software engineers who examine the model later. Important design and domain concepts must be communicated effectively.
- **Spacing.** Separation of concerns (Chapter 9) in a design is sometimes referred to as *spacing*. Sufficient spacing leads to modular designs, but too much spacing leads to fragmentation and loss of visibility.
- **Symmetry.** Architectural symmetry implies that a system is consistent and balanced in its attributes. Symmetric designs are easier to understand, comprehend, and communicate. As an example of architectural symmetry, consider a **customer account** object whose life cycle is modeled directly by a software architecture that requires both *open()* and *close()* methods. Architectural symmetry can be both structural and behavioral.
- **Emergence.** Emergent, self-organized behavior and control are often the key to creating scalable, efficient, and economic software architectures. For example, many real-time software applications are event driven. The sequence and duration of these events that define the system's behavior is an emergent quality. Because it is very difficult to plan for every possible sequence of events, a system architect should create a flexible system that accommodates this emergent behavior.

These considerations do not exist in isolation. They interact with each other and are moderated by each other. For example, spacing can be both reinforced and reduced by economy. Visibility can be balanced by spacing.

The architectural description for a software product is not explicitly visible in the source code used to implement it. As a consequence, code modifications made over time (e.g., software maintenance activities) can cause slow erosion of the software architecture. The challenge for a designer is to find suitable abstractions for the architectural information. These abstractions have the potential to add structuring that improves readability and maintainability of the source code [Bro10b].

SAFEHOME



Evaluating Architectural Decisions

The scene: Jamie's cubicle, as design modeling continues.

The players: Jamie and Ed, members of the *SafeHome* software engineering team.

The conversation:

Ed: I finished my call-return architectural model of the security function.

Jamie: Great! Do you think it meets our needs?

Ed: It doesn't introduce any unneeded features, so it seems to be economic.

Jamie: How about visibility?

Ed: Well, I understand the model, and there's no problem implementing the security requirements needed for this product.

Jamie: I get that you understand the architecture, but you may not be the programmer for this part of the project. I'm a little worried

about spacing. This design may not be as modular as an object-oriented design.

Ed: Maybe, but that may limit our ability to reuse some of our code when we have to create the mobile version of *SafeHome*.

Jamie: What about symmetry?

Ed: Well, that's harder for me to assess. It seems to me the only place for symmetry in the security function is adding and deleting PIN information.

Jamie: That will get more complicated when we add remote security features to the mobile app.

Ed: That's true, I guess.

(They both pause for a moment, pondering the architectural issues.)

Jamie: *SafeHome* is a real-time system, so state transition and sequencing of events will be tough to predict.

Ed: Yeah, but the emergent behavior of this system can be handled with a finite state model.

Jamie: How?

Ed: The model can be implemented based on the call-return architecture. Interrupts can be handled easily in many programming languages.

Jamie: Do you think we need to do the same kind of analysis for the object-oriented architecture we were initially considering?

Ed: I suppose it might be a good idea, because architecture is hard to change once implementation starts.

Jamie: It's also important for us to map the nonfunctional requirements besides security on top of these architectures to be sure they have been considered thoroughly.

Ed: Also, true.

10.5 ARCHITECTURAL DECISIONS

Decisions about system architecture identify key design issues and the rationale behind chosen architectural solutions. System architecture decisions encompass software system organization, selection of structural elements and their interfaces as defined by their intended collaborations, and the composition of these elements into increasingly larger subsystems [Kru09]. In addition, choices of architectural patterns, application technologies, middleware assets, and programming language can also be made. The outcome of the architectural decisions influences the system's nonfunctional characteristics and many of its quality attributes [Zim11] and can be documented with *developer notes*. These notes document key design decisions along with their justification, provide a reference for new project team members, and serve as a repository for lessons learned.

In general, software architectural practice focuses on architectural views that represent and document the needs of various stakeholders. It is possible, however, to define a *decision view* that cuts across several views of information contained in traditional architectural representations. The decision view captures both the architecture design decisions and their rationale.

Service-oriented architecture decision (SOAD)[5] modeling [Zim11] is a knowledge management framework that provides support for capturing architectural decision dependencies in a manner that allows them to guide future development activities.

5 SOAD is analogous to the use of architecture patterns discussed in Chapter 14.

A SOAD *guidance model* contains knowledge about architectural decisions required when applying an architectural style in a particular application genre. It is based on architectural information obtained from completed projects that employed the architectural style in that genre. The guidance model documents places where design problems exist and architectural decisions must be made, along with quality attributes that should be considered in selecting from among potential alternatives. Potential alternative solutions (with their pros and cons) from previous software applications are included to assist the architect in making the best decision possible.

A SOAD *decision model* documents both the architectural decisions required and records the decisions actually made on previous projects with their justifications. The guidance model feeds the architectural decision model in a *tailoring* step that allows the architect to delete irrelevant issues, enhance important issues, or add new issues. A decision model can make use of more than one guidance model and provides feedback to the guidance model after the project is completed. This feedback may be accomplished by *harvesting* lessons learned from project postmortem reviews.

10.6 ARCHITECTURAL DESIGN

As architectural design begins, context must be established. To accomplish this, the external entities (e.g., other systems, devices, people) that interact with the software and the nature of their interaction are described. This information can generally be acquired from the requirements model. Once context is modeled and all external software interfaces have been described, you can identify a set of architectural archetypes.

An *archetype* is an abstraction (similar to a class) that represents one element of system behavior. The set of archetypes provides a collection of abstractions that must be modeled architecturally if the system is to be constructed, but the archetypes themselves do not provide enough implementation detail. Therefore, the designer specifies the structure of the system by defining and refining software components that implement each archetype. This process continues iteratively until a complete architectural structure has been derived.

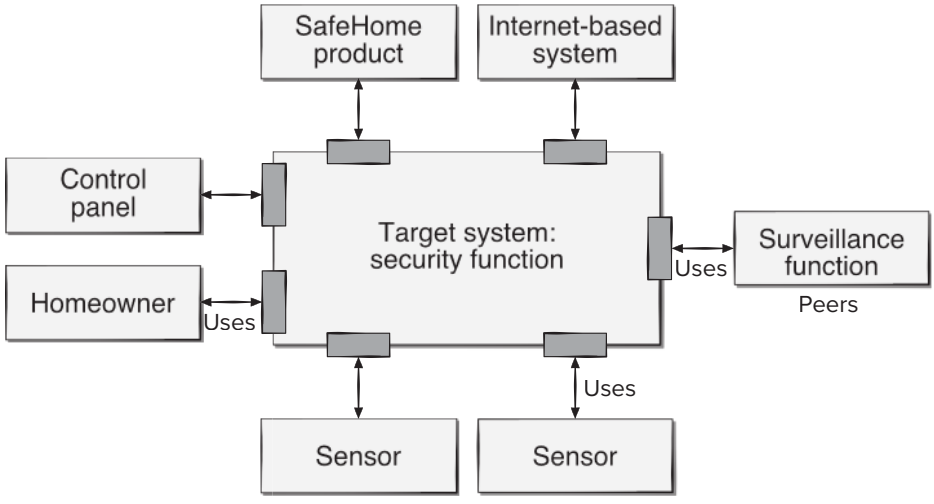
Several questions [Boo11b] must be asked and answered as a software engineer creates meaningful architectural diagrams. Does the diagram show how the system responds to inputs or events? What visualizations might there be to help emphasize areas of risk? How can hidden system design patterns be made more obvious to other developers? Can multiple viewpoints show the best way to refactor specific parts of the system? Can design trade-offs be represented in a meaningful way? If a diagrammatic representation of software architecture answers these questions, it will have value to software engineers that use it.

10.6.1 Representing the System in Context

UML does not contain specific diagrams that represent the system in context. Software engineers wishing to stick with UML and represent the system in context would do so with a combination of use case, class, component, activity, sequence, and collaboration diagrams. Some software architects may make use of an *architectural context diagram* (ACD) to model the manner in which software interacts with entities external to its boundaries. An architectural context diagram for the *SafeHome* security functions is shown in Figure 10.7.

FIGURE 10.7

Architectural context diagram for the *SafeHome* security function



To illustrate the use of the ACD, consider the home security function of the *SafeHome* product shown in Figure 10.7. The overall *SafeHome* product controller and the Internet-based system are both superordinate to the security function and are shown above the function. The surveillance function is a *peer system* and uses (is used by) the home security function in later versions of the product. The homeowner and control panels are actors that produce and consume information that is, respectively, used and produced by the security software. Finally, sensors are used by the security software and are shown as subordinate to it (by drawing them below the target system).

As part of the architectural design, the details of each interface shown in Figure 10.7 would have to be specified. All data that flow into and out of the target system must be identified at this stage.

10.6.2 Defining Archetypes

An *archetype* is a class or pattern that represents a core abstraction that is critical to the design of an architecture for the target system. In general, a relatively small set of archetypes is required to design even relatively complex systems. The target system architecture is composed of these archetypes, which represent stable elements of the architecture but may be instantiated many different ways based on the behavior of the system.

In many cases, archetypes can be derived by examining the analysis classes defined as part of the requirements model. Continuing the discussion of the *SafeHome* home security function, you might define the following archetypes:

- **Node.** Represents a cohesive collection of input and output elements of the home security function. For example, a node might be composed of (1) various sensors and (2) a variety of alarm (output) indicators.
- **Detector.** An abstraction that encompasses all sensing equipment that feeds information into the target system.

- **Indicator.** An abstraction that represents all mechanisms (e.g., alarm siren, flashing lights, bell) for indicating that an alarm condition is occurring.
- **Controller.** An abstraction that depicts the mechanism that allows the arming or disarming of a node. If controllers reside on a network, they have the ability to communicate with one another.

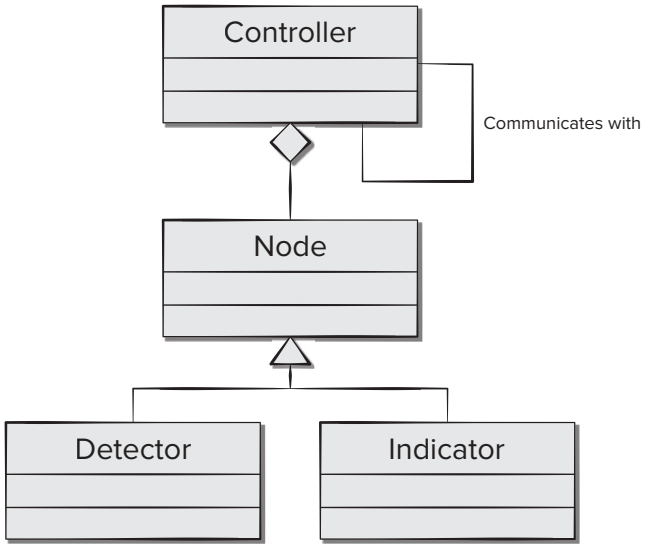
Each of these archetypes is depicted using UML notation, as shown in Figure 10.8. Recall that the archetypes form the basis for the architecture but are abstractions that must be further refined as architectural design proceeds. For example, **Detector** might be refined into a class hierarchy of sensors.

10.6.3 Refining the Architecture into Components

As the software architecture is refined into components, the structure of the system begins to emerge. But how are these components chosen? To answer this question, you begin with the classes that were described as part of the requirements model.[6] These analysis classes represent entities within the application (business) domain that must be addressed within the software architecture. Hence, the application domain is one source for the derivation and refinement of components. Another source is the infrastructure domain. The architecture must accommodate many infrastructure components that enable application components but have no business connection to the application domain. For example, memory management components, communication components, database components, and task management components are often integrated into the software architecture.

FIGURE 10.8

UML relationships for *SafeHome* security function archetype
Source: Adapted from Bosch, Jan, Design & Use of Software Architectures. Pearson Education, 2000.



6 If a conventional (non-object-oriented) approach is chosen, components may be derived from the subprogram calling hierarchy (see Figure 10.3).

The interfaces depicted in the architecture context diagram (Section 10.6.1) imply one or more specialized components that process the data that flows across the interface. In some cases (e.g., a graphical user interface), a complete subsystem architecture with many components must be designed.

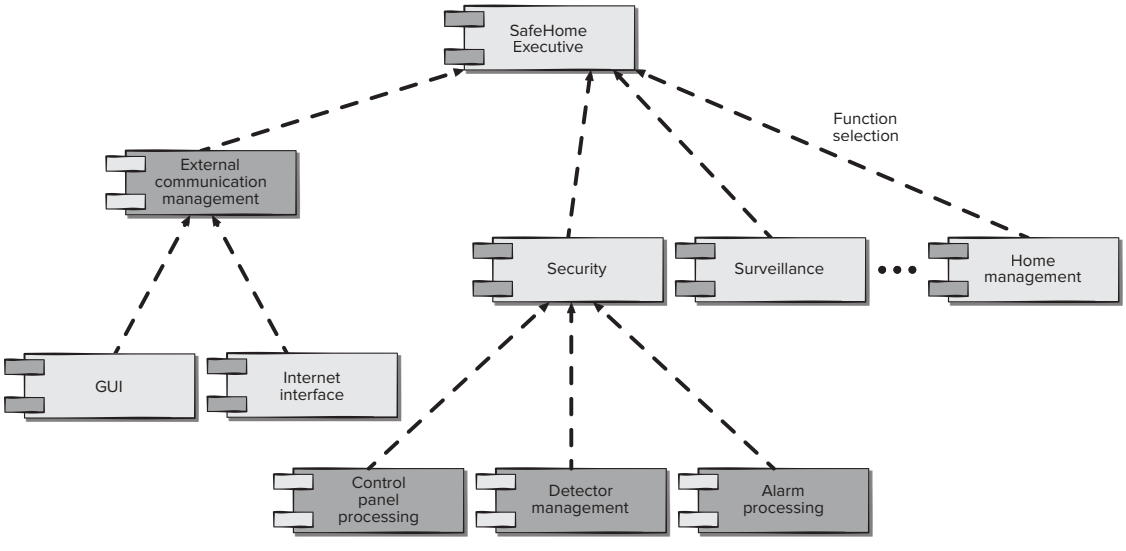
Continuing the *SafeHome* home security function example, you might define the set of top-level components that addresses the following functionality:

- ***External communication management.*** Coordinates communication of the security function with external entities such as other Internet-based systems and external alarm notification.
- ***Control panel processing.*** Manages all control panel functionality.
- ***Detector management.*** Coordinates access to all detectors attached to the system.
- ***Alarm processing.*** Verifies and acts on all alarm conditions.

Each of these top-level components would have to be elaborated iteratively and then positioned within the overall *SafeHome* architecture. Design classes (with appropriate attributes and operations) would be defined for each. It is important to note, however, that the design details of all attributes and operations would not be specified until component-level design (Chapter 11).

The overall architectural structure (represented as a UML component diagram) is illustrated in Figure 10.9. Transactions are acquired by *external communication management* as they move in from components that process the *SafeHome* GUI and the Internet interface. This information is managed by a *SafeHome* executive component that selects the appropriate product function (in this case security). The *control panel processing* component interacts with the homeowner to arm and disarm the security

FIGURE 10.9 **Overall architectural structure for *SafeHome* with top-level components**



function. The *detector management* component polls sensors to detect an alarm condition, and the *alarm processing* component produces output when an alarm is detected.

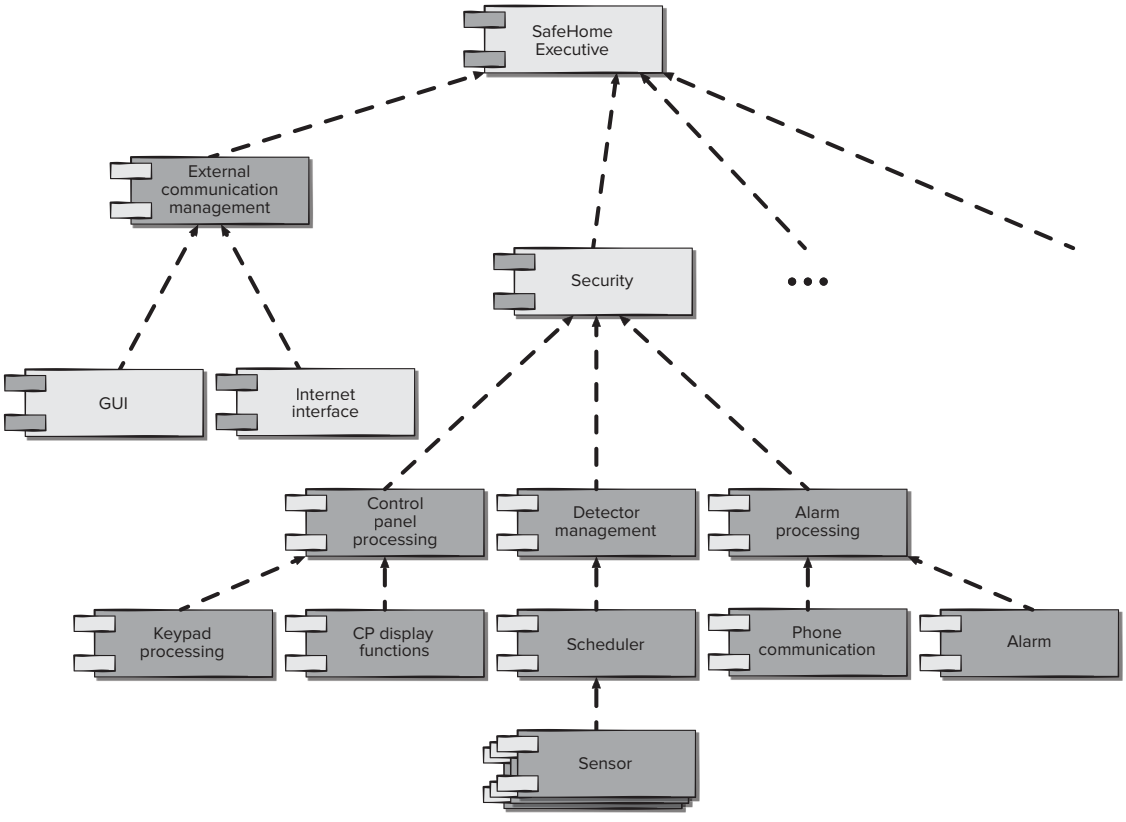
10.6.4 Describing Instantiations of the System

The architectural design that has been modeled to this point is still relatively high level. The context of the system has been represented, archetypes that indicate the important abstractions within the problem domain have been defined, the overall structure of the system is apparent, and the major software components have been identified. However, further refinement (recall that all design is iterative) is still necessary.

To accomplish this, an actual instantiation of the architecture is developed. By this we mean that the architecture is applied to a specific problem with the intent of demonstrating that the structure and components are appropriate.

Figure 10.10 illustrates an instantiation of the *SafeHome* architecture for the security system. Components shown in Figure 10.9 are elaborated to show additional detail. For example, the *detector management* component interacts with a *scheduler* infrastructure component that implements polling of each *sensor* object used by the security system. Similar elaboration is performed for each of the components represented in Figure 10.10.

FIGURE 10.10 **An instantiation of the security function with component elaboration**



10.7 ASSESSING ALTERNATIVE ARCHITECTURAL DESIGNS

In their book on the evaluation of software architectures, Clements and his colleagues [Cle03] state, "To put it bluntly, an architecture is a bet, a wager on the success of a system."

The big question for a software architect and the software engineers who will work to build a system is simple: Will the architectural bet pay off?

To help answer this question, architectural design should result in a number of architectural alternatives that are each assessed to determine which is the most appropriate for the problem to be solved.

The Software Engineering Institute (SEI) has developed an *architecture trade-off analysis method* (ATAM) [Kaz98] that establishes an iterative evaluation process for software architectures. The design analysis activities that follow are performed iteratively:

1. **Collect scenarios.** A set of use cases (Chapters 7 and 8) is developed to represent the system from the user's point of view.
2. **Elicit requirements, constraints, and environment description.** This information is required as part of requirements engineering and is used to be certain that all stakeholder concerns have been addressed.
3. **Describe the architectural styles and patterns that have been chosen to address the scenarios and requirements.** The architectural style(s) should be described using one of the following architectural views:
   - *Module view* for analysis of work assignments with components and the degree to which information hiding has been achieved.
   - *Process view* for analysis of system performance.
   - *Data flow view* for analysis of the degree to which the architecture meets functional requirements.
4. **Evaluate quality attributes by considering each attribute in isolation.** The number of quality attributes chosen for analysis is a function of the time available for review and the degree to which quality attributes are relevant to the system at hand. Quality attributes for architectural design assessment include reliability, performance, security, maintainability, flexibility, testability, portability, reusability, and interoperability.
5. **Identify the sensitivity of quality attributes to various architectural attributes for a specific architectural style.** This can be accomplished by making small changes in the architecture and determining how sensitive a quality attribute, say performance, is to the change. Any attributes that are significantly affected by variation in the architecture are termed *sensitivity points.*
6. **Critique candidate architectures (developed in step 3) using the sensitivity analysis conducted in step 5.** The SEI describes this approach in the following manner [Kaz98]:

   > Once the architectural sensitivity points have been determined, finding trade-off points is simply the identification of architectural elements to which multiple attributes are sensitive. For example, the performance of a client-server architecture might

> be highly sensitive to the number of servers (performance increases, within some range, by increasing the number of servers). . . . The number of servers, then, is a trade-off point with respect to this architecture.

These six steps represent the first ATAM iteration. Based on the results of steps 5 and 6, some architecture alternatives may be eliminated, one or more of the remaining architectures may be modified and represented in more detail, and then the ATAM steps are reapplied.

SAFEHOME



Architecture Assessment

The scene: Doug Miller's office as architectural design modeling proceeds.

The players: Vinod, Jamie, and Ed, members of the *SafeHome* software engineering team. Also Doug Miller, manager of the software engineering group.

The conversation:

Doug: I know you guys are deriving a couple of different architectures for the *SafeHome* product, and that's a good thing. I guess my question is, how are we going to choose the one that's best?

Ed: I'm working on a call-and-return style, and then either Jamie or I will derive an OO architecture.

Doug: Okay, and how do we choose?

Jamie: I took a CS course in design in my senior year, and I remember that there are a number of ways to do it.

Vinod: There are, but they're a bit academic. Look, I think we can do our assessment and choose the right one using use cases and scenarios.

Doug: Isn't that the same thing?

Vinod: Not when you're talking about architectural assessment. We already have a complete set of use cases. So we apply each to both architectures and see how the system reacts, how components and connectors work in the use case context.

Ed: That's a good idea. Make sure we didn't leave anything out.

Vinod: True, but it also tells us whether the architectural design is convoluted, whether the system has to twist itself into a pretzel to get the job done.

Jamie: Aren't scenarios just another name for use cases?

Vinod: No, in this case a scenario implies something different.

Doug: You're talking about a quality scenario or a change scenario, right?

Vinod: Yes. What we do is go back to the stakeholders and ask them how *SafeHome* is likely to change over the next, say, 3 years. You know, new versions, features, that sort of thing. We build a set of change scenarios. We also develop a set of quality scenarios that defines the attributes we'd like to see in the software architecture.

Jamie: And we apply them to the alternatives.

Vinod: Exactly. The style that handles the use cases and scenarios best is the one we choose.

10.7.1 Architectural Reviews

Architectural reviews are a type of specialized technical review (Chapter 16) that provide a means of assessing the ability of a software architecture to meet the system's quality requirements (e.g., scalability or performance) and to identify any potential

risks. Architectural reviews have the potential to reduce project costs by detecting design problems early.

Unlike requirements reviews that involve representatives of all stakeholders, architecture reviews often involve only software engineering team members supplemented by independent experts. However, software-based systems are built by people with a variety of different needs and points of view. Architects often focus on the long-term impact of the system's nonfunctional requirements as the architecture is created. Senior managers assess the architecture within the context of business goals and objectives. Project managers are often driven by short-term considerations of delivery dates and budget. Software engineers are often focused on their own technology interests and feature delivery. Each of these (and other) constituencies must agree that the software architecture chosen has distinct advantages over any other alternatives. Therefore, a wise software architect should build consensus among members of the software team (and other stakeholders) to achieve the architectural vision for the final software product [Wri11].

The most common architectural review techniques used in industry are: experience-based reasoning, prototype evaluation, scenario review (Chapter 8), and use of checklists. Many architectural reviews occur early in the project life cycle; they should also occur after new components or packages are acquired in component-based design (Chapter 11). One of the most commonly cited problems facing software engineers when conducting architectural reviews is missing or inadequate architectural work products, thereby making review difficult to complete [Bab09].

10.7.2 Pattern-Based Architecture Review

Formal technical reviews (Chapter 16) can be applied to software architecture and provide a means for managing system quality attributes, uncovering errors, and avoiding unnecessary rework. However, in situations in which short build cycles, tight deadlines, volatile requirements, and/or small teams are the norm, a lightweight architectural review process known as *pattern-based architecture review* (PBAR) might be the best option.

PBAR is an evaluation method based on architectural patterns[7] that leverages the patterns' relationships to quality attributes. A PBAR is a face-to-face audit meeting involving all developers and other interested stakeholders. An external reviewer with expertise in architecture, architecture patterns, quality attributes, and the application domain is also in attendance. The system architect is the primary presenter.

A PBAR should be scheduled after the first working prototype or *walking skeleton*[8] is completed. The PBAR encompasses the following iterative steps [Har11]:

1. Identify and discuss the quality attributes most important to the system by walking through the relevant use cases (Chapter 8).
2. Discuss a diagram of the system's architecture in relation to its requirements.

---

7 An *architectural pattern* is a generalized solution to an architectural design problem with a specific set of conditions or constraints. Patterns are discussed in detail in Chapter 14.

8 A *walking skeleton* contains a baseline architecture that supports the functional requirements with the highest priorities in the business case and the most challenging quality attributes.

3. Help the reviewer identify the architecture patterns used and match the system's structure to the patterns' structure.
4. Using existing documentation and past use cases, examine the architecture and quality attributes to determine each pattern's effect on the system's quality attributes.
5. Identify and discuss all quality issues raised by architecture patterns used in the design.
6. Develop a short summary of the issues uncovered during the meeting, and make appropriate revisions to the walking skeleton.

PBARs are well suited to small, agile teams and require a relatively small amount of extra project time and effort. With its short preparation and review time, PBAR can accommodate changing requirements and short build cycles and, at the same time, help improve the team's understanding of the system architecture.

10.7.3 Architecture Conformance Checking

As the software process moves through design and into construction, software engineers must work to ensure that an implemented and evolving system conforms to its planned architecture. Many things (e.g., conflicting requirements, technical difficulties, deadline pressures) cause deviations from a defined architecture. If architecture is not checked for conformance periodically, uncontrolled deviations can cause *architecture erosion* and affect the quality of the system [Pas10].

Static architecture-conformance analysis (SACA) assesses whether an implemented software system is consistent with its architectural model. The formalism (e.g., UML) used to model the system architecture presents the static organization of system components and how the components interact. Often the architectural model is used by a project manager to plan and allocate work tasks, as well as to assess implementation progress.

10.8 SUMMARY

Software architecture provides a holistic view of the system to be built. It depicts the structure and organization of software components, their properties, and the connections between them. Software components include program modules and the various data representations that are manipulated by the program. Therefore, data design is an integral part of the derivation of the software architecture. Architecture highlights early design decisions and provides a mechanism for considering the benefits of alternative system structures.

Architectural design can coexist with agile methods by applying a hybrid architectural design framework that makes use of existing techniques derived from popular agile methods. Once an architecture is developed, it can be assessed to ensure conformance with business goals, software requirements, and quality attributes.

Several different architectural styles and patterns are available to the software engineer and may be applied within a given architectural genre. Each style describes a system category that encompasses a set of components that perform a function required by a system; a set of connectors that enable communication, coordination, and cooperation among components; constraints that define how components can be integrated

to form the system; and semantic models that enable a designer to understand the overall properties of a system.

In a general sense, architectural design is accomplished using four distinct steps. First, the system must be represented in context. That is, the designer should define the external entities that the software interacts with and the nature of the interaction. Once context has been specified, the designer should identify a set of top-level abstractions, called archetypes, that represent pivotal elements of the system's behavior or function. After abstractions have been defined, the design begins to move closer to the implementation domain. Components are identified and represented within the context of an architecture that supports them. Finally, specific instantiations of the architecture are developed to "prove" the design in a real-world context.

PROBLEMS AND POINTS TO PONDER

10.1. Using the architecture of a house or building as a metaphor, draw comparisons with software architecture. How are the disciplines of classical architecture and the software architecture similar? How do they differ?

10.2. Present two or three examples of applications for each of the architectural styles noted in Section 10.3.1.

10.3. Some of the architectural styles noted in Section 10.3.1 are hierarchical in nature, and others are not. Make a list of each type. How would the architectural styles that are not hierarchical be implemented?

10.4. The terms *architectural style, architectural pattern,* and *framework* (not discussed in this book) are often encountered in discussions of software architecture. Do some research, and describe how each of these terms differs from its counterparts.

10.5. Select an application with which you are familiar. Answer each of the questions posed for control and data in Section 10.3.3.

10.6. Research the ATAM (using [Kaz98]), and present a detailed discussion of the six steps presented in Section 10.7.1.

10.7. If you haven't done so, complete Problem 8.3. Use the design approach described in this chapter to develop a software architecture for the pothole tracking and repair system (PHTRS).

10.8. Use the architectural decision template from Section 10.1.4 to document one of the architectural decisions for PHTRS architecture developed in Problem 10.7.

10.9. Select a mobile application you are familiar with, and assess it using the architecture considerations (economy, visibility, spacing, symmetry, emergence) from Section 10.4.

10.10. List the strengths and weakness of the PHTRS architecture you created for Problem 10.7.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

11 COMPONENT-LEVEL DESIGN

Component-level design occurs after the first iteration of architectural design has been completed. At this stage, the overall data and program structure of the software has been established. The intent is to translate the design model into operational software. But the level of abstraction of the existing design model is relatively high, and the abstraction level of the operational program is low. The translation can be challenging, opening the door to the introduction of subtle errors that are difficult to find and correct in later stages of the software process. Component-level design bridges the gap between architectural design and coding.

KEY CONCEPTS

cohesion 216
component. 207
component-based development 228
content design. 226
coupling 218
dependency inversion principle 214
design guidelines 215
interface segregation principle 214
Liskov substitution principle. 214
object-oriented view 207
open-closed principle 212
process-related view. 211
traditional components. 227
traditional view 209
WebApp component 226

QUICK LOOK

What is it? A complete set of software components is defined during architectural design. But the internal data structures and processing details of each component are not represented at a level of abstraction that is close to code. Component-level design defines the data structures, algorithms, interface characteristics, and communication mechanisms allocated to each software component.

Who does it? A software engineer performs component-level design.

Why is it important? You need to determine whether the software will work before you build it. The component-level design represents the software in a way that allows you to review the details of the design for correctness and consistency with other design representations.

What are the steps? Design representations of data, architecture, and interfaces form the foundation for component-level design. The class definition or processing narrative for each component is translated into a detailed design that makes use of diagrammatic or text-based forms that specify internal data structures, local interface detail, and processing logic.

What is the work product? The design for each component, represented in graphical, tabular, or text-based notation, is the primary work product produced during component-level design.

How do I ensure that I've done it right? A design review is conducted. The design is examined to determine whether data structures, interfaces, processing sequences, and logical conditions are correct.

Component-level design will reduce the number of errors introduced during coding. As you translate the design model into source code, you should follow a set of design principles that not only perform the translation but also do not "introduce bugs to start with."

11.1 What Is a Component?

A *component* is a modular building block for computer software. More formally, the *OMG Unified Modeling Language Specification* [OMG03a] defines a component as "a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces."

As we discussed in Chapter 10, components populate the software architecture and play a role in achieving the objectives and requirements of the system to be built. Because components reside within the software architecture, they must communicate and collaborate with other components and with entities (e.g., other systems, devices, people) that exist outside the boundaries of the software.

The true meaning of the term *component* will differ depending on the point of view of the software engineer who uses it. In the sections that follow, we examine three important views of what a component is and how it is used as design modeling proceeds.

11.1.1 An Object-Oriented View

In the context of object-oriented software engineering, a component contains a set of collaborating classes.[1] Each class within a component has been fully elaborated to include all attributes and operations that are relevant to its implementation. As part of the design elaboration, all interfaces that enable the classes to communicate and collaborate with other design classes must also be defined. To accomplish this, you begin with the analysis model and elaborate analysis classes (for components that relate to the problem domain) and infrastructure classes (for components that provide support services for the problem domain).

Recall that analysis modeling and design modeling are both iterative actions. Elaborating the original analysis class may require additional analysis steps, which are then followed with design modeling steps to represent the elaborated design class (the details of the component). To illustrate this process of design elaboration, consider software to be built for a sophisticated print shop. The overall intent of the software is to collect the customer's requirements at the front counter, cost a print job, and then pass the job on to an automated production facility. During requirements engineering, an analysis class called **PrintJob** was derived.

The attributes and operations defined during analysis are noted at the top of Figure 11.1. During architectural design, **PrintJob** is defined as a component within the software architecture and is represented using the shorthand UML notation[2] shown in the middle right of the figure. Note that **PrintJob** has two interfaces, *computeJob,*

1 In some cases, a component may contain a single class.

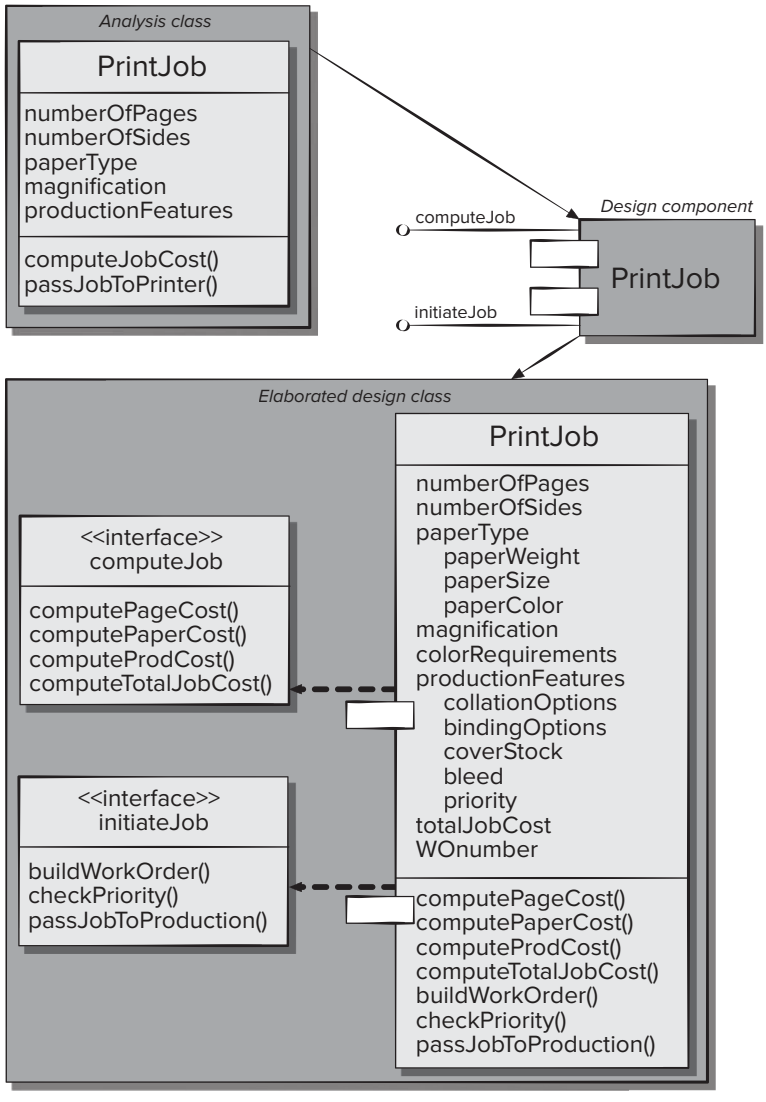
2 Readers who are unfamiliar with UML notation should refer to Appendix 1.

which provides job costing capability, and *initiateJob,* which passes the job along to the production facility. These are represented using the "lollipop" symbols shown to the left of the component box.

Component-level design begins at this point. The details of the component **PrintJob** must be elaborated to provide sufficient information to guide implementation. The original analysis class is elaborated to flesh out all attributes and operations required to implement the class as the component **PrintJob.** Referring to the lower right portion of Figure 11.1, the elaborated design class **PrintJob** contains more

FIGURE 11.1

Elaboration of a design component



detailed attribute information as well as an expanded description of operations required to implement the component. The interfaces *computeJob* and *initiateJob* imply communication and collaboration with other components (not shown here). For example, the operation *computePageCost()* (part of the *computeJob* interface) might collaborate with a **PricingTable** component that contains job pricing information. The *checkPriority()* operation (part of the *initiateJob* interface) might collaborate with a **JobQueue** component to determine the types and priorities of jobs currently awaiting production.

This elaboration activity is applied to every component defined as part of the architectural design. Once it is completed, further elaboration is applied to each attribute, operation, and interface. The data structures appropriate for each attribute must be specified. In addition, the algorithmic detail required to implement the processing logic associated with each operation is designed. This procedural design activity is discussed later in this chapter. Finally, the mechanisms required to implement the interface are designed. For object-oriented software, this may encompass the description of all messaging that is required to effect communication between objects within the system.

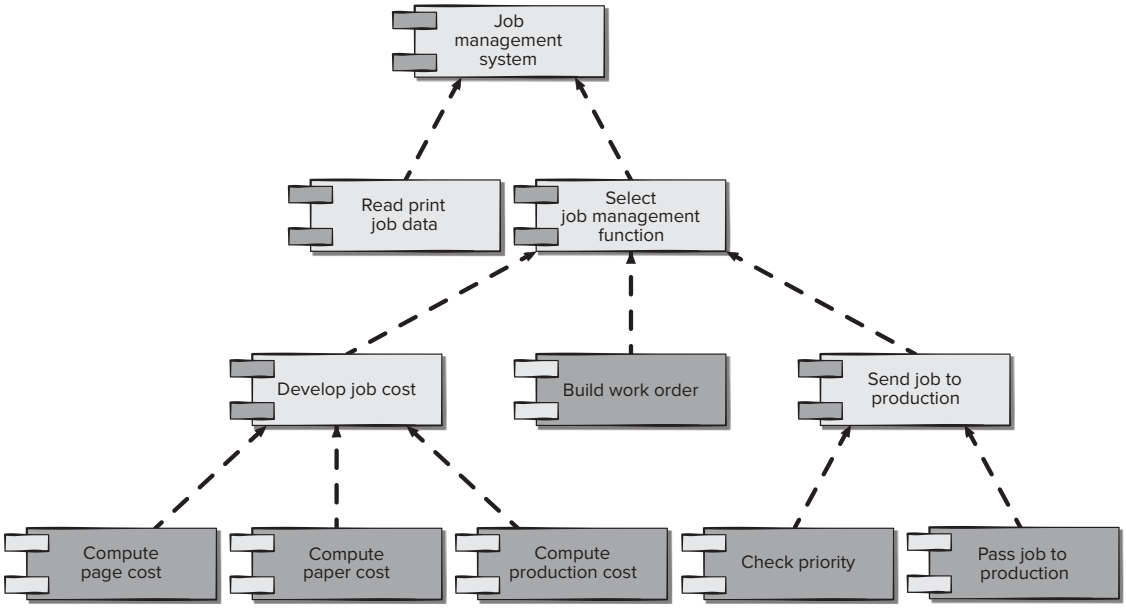
11.1.2 The Traditional View

In the context of traditional software engineering, a component is a functional element of a program that incorporates processing logic, the internal data structures that are required to implement the processing logic, and an interface that enables the component to be invoked and data to be passed to it. A traditional component, also called a *module,* resides within the software architecture and serves one of three important roles: (1) a *control component* that coordinates the invocation of all other problem domain components, (2) a *problem domain component* that implements a complete or partial function that is required by the customer, or (3) an *infrastructure component* that is responsible for functions that support the processing required in the problem domain.

Like object-oriented components, traditional software components are derived from the analysis model. In this case, however, the component elaboration element of the analysis model serves as the basis for the derivation. Each component representing the component hierarchy is mapped (Section 10.6) into a module hierarchy. Control components (modules) reside near the top of the hierarchy (program architecture), and problem domain components tend to reside toward the bottom of the hierarchy. To achieve effective modularity, design concepts like functional independence (Chapter 9) are applied as components are elaborated.

To illustrate this process of design elaboration for traditional components, again consider software to be built for the print shop noted earlier. A hierarchical architecture is derived and shown in Figure 11.2. Each box represents a software component. Note that the shaded boxes are equivalent in function to the operations defined for the **PrintJob** class discussed in Section 11.1.1. In this case, however, each operation is represented as a separate module that is invoked as shown in the figure. Other modules are used to control processing and are therefore control components.

FIGURE 11.2 Structure chart for a traditional system

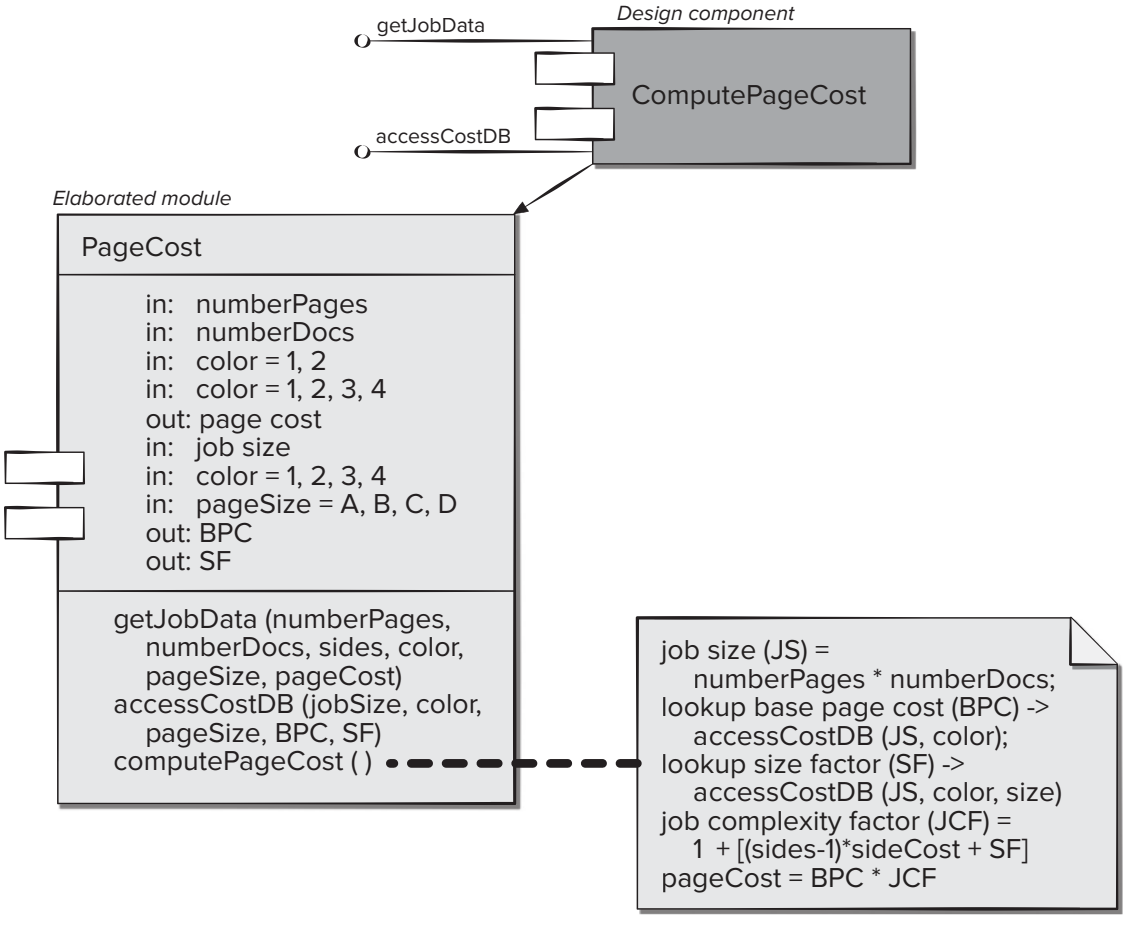


During component-level design, each module in Figure 11.2 is elaborated. The module interface is defined explicitly. That is, each data or control object that flows across the interface is represented. The data structures that are used internal to the module are defined. The algorithm that allows the module to accomplish its intended function is designed using the stepwise refinement approach discussed in Chapter 9. The behavior of the module is sometimes represented using a state diagram.

To illustrate this process, consider the module *ComputePageCost.* The intent of this module is to compute the printing cost per page based on specifications provided by the customer. Data required to perform this function are: number of pages in the document, total number of documents to be produced, one- or two-side printing, color requirements, and size requirements. These data are passed to *ComputePageCost* via the module's interface. *ComputePageCost* uses these data to determine a page cost that is based on the size and complexity of the job—a function of all data passed to the module via the interface. Page cost is inversely proportional to the size of the job and directly proportional to the complexity of the job.

As the design for each software component is elaborated, the focus shifts to the design of specific data structures and procedural design to manipulate the data structures. Figure 11.3 represents the component-level design using a modified UML notation. The *ComputePageCost* module accesses data by invoking the module *getJobData,* which allows all relevant data to be passed to the component, and a database interface, *accessCostsDB,* which enables the module to access a database that contains all printing costs. As design continues, the *ComputePageCost* module is

FIGURE 11.3 Component-level design for *ComputePageCost*



elaborated to provide algorithm detail and interface detail (Figure 11.3). Algorithm detail can be represented using the pseudocode text shown in the figure or with a UML activity diagram. The interfaces are represented as a collection of input and output data objects or items. Design elaboration continues until sufficient detail is provided to guide construction of the component. However, don't forget the architecture that must house the components or the global data structures that may serve many components.

11.1.3 A Process-Related View

The object-oriented and traditional views of component-level design presented in Sections 11.1.1 and 11.1.2 assume that the component is being designed from scratch. That is, you always create a new component based on specifications derived from the requirements model. There is, of course, another approach.

Over the past four decades, the software engineering community has emphasized the need to build systems that make use of existing software components or design patterns. To do this, a catalog of proven design or code-level components needs to be made available to you as design work proceeds. As the software architecture is developed, you choose components or design patterns from the catalog and use them to populate the architecture. Because these components have been created with reusability in mind, a complete description of their interface, the function(s) they perform, and the communication and collaboration they require are all available to you. We will save a discussion on the pros and cons of component-based software engineering (CBSE) for Section 11.4.4.

11.2 DESIGNING CLASS-BASED COMPONENTS

As we have already noted, component-level design draws on information developed as part of the requirements model (Chapter 8) and represented as part of the architectural model (Chapter 10). When an object-oriented software engineering approach is chosen, component-level design focuses on the elaboration of problem domain specific classes and the definition and refinement of infrastructure classes contained in the requirements model. The detailed description of the attributes, operations, and interfaces used by these classes is the design detail required as a precursor to the construction activity.

11.2.1 Basic Design Principles

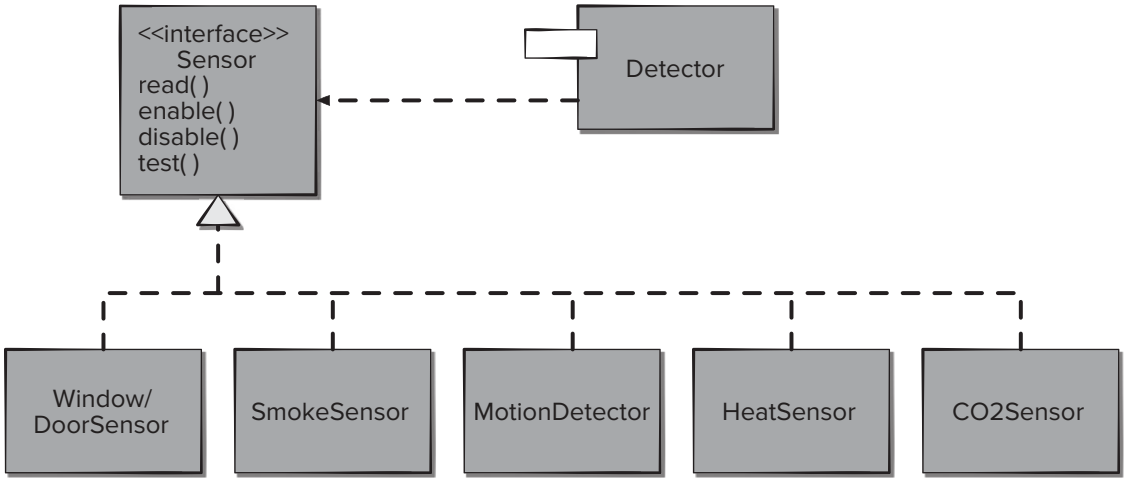
Four basic design principles are applicable to component-level design and have been widely adopted when object-oriented software engineering is applied. The underlying motivation for the application of these principles is to create designs that are more amenable to change and to reduce the propagation of side effects when changes do occur. You can use these principles as a guide as each software component is developed.

The Open-Closed Principle (OCP). *"A module [component] should be open for extension but closed for modification"* [Mar00]. This statement seems to be a contradiction, but it represents one of the most important characteristics of a good component-level design. Stated simply, you should specify the component in a way that allows it to be extended (within the functional domain that it addresses) without the need to make internal (code or logic-level) modifications to the component itself. To accomplish this, you create abstractions that serve as a buffer between the functionality that is likely to be extended and the design class itself.

For example, assume that the *SafeHome* security function makes use of a **Detector** class that must check the status of each type of security sensor. It is likely that as time passes, the number and types of security sensors will grow. If internal processing logic is implemented as a sequence of if-then-else constructs, each addressing a different sensor type, the addition of a new sensor type will require additional internal processing logic (still another if-then-else). This is a violation of OCP.

One way to accomplish OCP for the **Detector** class is illustrated in Figure 11.4. The *sensor* interface presents a consistent view of sensors to the detector component. If a new type of sensor is added, no change is required for the **Detector** class (component). The OCP is preserved.

FIGURE 11.4 Following the OCP



SAFEHOME



The OCP in Action

The scene: Vinod's cubicle.

The players: Vinod and Shakira, members of the *SafeHome* software engineering team.

The conversation:

Vinod: I just got a call from Doug [the team manager]. He says marketing wants to add a new sensor.

Shakira (smirking): Not again, jeez!

Vinod: Yeah . . . and you're not going to believe what these guys have come up with.

Shakira: Amaze me.

Vinod (laughing): They call it a doggie angst sensor.

Shakira: Say what?

Vinod: It's for people who leave their pets home in apartments or condos or houses that are close to one another. The dog starts to bark. The neighbor gets angry and complains. With this sensor, if the dog barks for more than, say, a minute, the sensor sets a special alarm mode that calls the owner on his or her cell phone.

Shakira: You're kidding me, right?

Vinod: Nope. Doug wants to know how much time it's going to take to add it to the security function.

Shakira (thinking a moment): Not much . . . look. [She shows Vinod Figure 11.4.] We've isolated the actual sensor classes behind the **sensor** interface. As long as we have specs for the doggie sensor, adding it should be a piece of cake. Only thing I'll have to do is create an appropriate component . . . uh, class, for it. No change to the **Detector** component at all.

Vinod: So I'll tell Doug it's no big deal.

Shakira: Knowing Doug, he'll keep us focused and not deliver the doggie thing until the next release.

Vinod: That's not a bad thing, but can you implement now if he wants you to?

Shakira: Yeah, the way we designed the interface lets me do it with no hassle.

Vinod (thinking a moment): Have you ever heard of the open-closed principle?

Shakira (shrugging): Never heard of it.

Vinod (smiling): Not a problem.

The Liskov Substitution Principle (LSP). *"Subclasses should be substitutable for their base classes"* [Mar00]. This design principle, originally proposed by Barbara Liskov [Lis88], suggests that a component that uses a base class should continue to function properly if a class derived from the base class is passed to the component instead. LSP demands that any class derived from a base class must honor any implied contract between the base class and the components that use it. In the context of this discussion, a "contract" is a *precondition* that must be true before the component uses a base class and a *postcondition* that should be true after the component uses a base class. When you create derived classes, be sure they conform to the pre- and post-conditions.

The Dependency Inversion Principle (DIP). *"Depend on abstractions. Do not depend on concretions"* [Mar00]. As we have seen in the discussion of the OCP, abstractions are the place where a design can be extended without great complication. The more a component depends on other concrete components (rather than on abstractions such as an interface), the more difficult it will be to extend. Just remember that code is the ultimate concretion. If you dispense with design and hack out code, you're violating DIP.

The Interface Segregation Principle (ISP). *"Many client-specific interfaces are better than one general purpose interface"* [Mar00]. There are many instances in which multiple client components use the operations provided by a server class. ISP suggests that you should create a specialized interface to serve each major category of clients. Only those operations that are relevant to an individual client category should be specified in the interface for that client. If multiple clients require the same operations, it should be specified in each of the specialized interfaces.

As an example, consider the **FloorPlan** class that is used for the *SafeHome* security and surveillance functions (Chapter 10). For the security functions, **FloorPlan** is used only during configuration activities and uses the operations *placeDevice(), showDevice(), groupDevice(),* and *removeDevice()* to place, show, group, and remove sensors from the floor plan. The *SafeHome* surveillance function uses the four operations noted for security, but also requires special operations to manage cameras: *showFOV()* and *showDeviceID().* Hence, the ISP suggests that client components from the two *SafeHome* functions have specialized interfaces defined for them. The interface for security would encompass only the operations *placeDevice(), showDevice(), groupDevice(),* and *removeDevice().* The interface for surveillance would incorporate the operations *placeDevice(), showDevice(), groupDevice(),* and *removeDevice(),* along with *showFOV()* and *showDeviceID().*

Although component-level design principles provide useful guidance, components themselves do not exist in a vacuum. In many cases, individual components or classes are organized into subsystems or packages. It is reasonable to ask how this packaging activity should occur. Exactly how should components be organized as the design proceeds? Martin [Mar00] suggests additional packaging principles that are applicable to component-level design. These principles follow.

The Reuse/Release Equivalency Principle (REP). *"The granule of reuse is the granule of release"* [Mar00]. When classes or components are designed for reuse,

an implicit contract is established between the developer of the reusable entity and the people who will use it. The developer commits to establish a release control system that supports and maintains older versions of the entity while the users slowly upgrade to the most current version. Rather than addressing each class individually, it is often advisable to group reusable classes into packages that can be managed and controlled as newer versions evolve. Designing components for reuse requires more than good technical design. It also requires effective configuration control mechanisms (Chapter 22).

The Common Closure Principle (CCP). *"Classes that change together belong together"* [Mar00]. Classes should be packaged cohesively. That is, when classes are packaged as part of a design, they should address the same functional or behavioral area. When some characteristic of that area must change, it is likely that only those classes within the package will require modification. This leads to more effective change control and release management.

The Common Reuse Principle (CRP). *"Classes that aren't reused together should not be grouped together"* [Mar00]. When one or more classes with a package changes, the release number of the package changes. All other classes or packages that rely on the package that has been changed must now update to the most recent release of the package and be tested to ensure that the new release operated without incident. If classes are not grouped cohesively, it is possible that a class with no relationship to other classes within a package is changed. This will precipitate unnecessary integration and testing. For this reason, only classes that are reused together should be included within a package.

11.2.2 Component-Level Design Guidelines

In addition to the principles discussed in Section 11.2.1, a set of pragmatic design guidelines can be applied as component-level design proceeds. These guidelines apply to components, their interfaces, and the dependencies and inheritance characteristics that have an impact on the resultant design. Ambler [Amb02b] suggests the following guidelines:

Components. Naming conventions should be established for components that are specified as part of the architectural model and then refined and elaborated as part of the component-level model. Architectural component names should be drawn from the problem domain and should have meaning to all stakeholders who view the architectural model. For example, the class name **FloorPlan** is meaningful to everyone reading it regardless of technical background. On the other hand, infrastructure components or elaborated component-level classes should be named to reflect implementation-specific meaning. If a linked list is to be managed as part of the **FloorPlan** implementation, the operation *manageList()* is appropriate, even if a nontechnical person might misinterpret it.[3]

You can choose to use stereotypes to help identify the nature of components at the detailed design level. For example, `<<infrastructure>>` might be used to identify an

3 It is unlikely that someone from marketing or the customer organization (a nontechnical type) would examine detailed design information.

infrastructure component, <<database>> could be used to identify a database that services one or more design classes or the entire system, and <<table>> can be used to identify a table within a database.

Interfaces. Interfaces provide important information about communication and collaboration (as well as helping us to achieve the OPC). However, unfettered representation of interfaces tends to complicate component diagrams. Ambler [Amb02c] recommends that (1) lollipop representation of an interface should be used in lieu of the more formal UML box and dashed arrow approach, when diagrams grow complex, (2) for consistency, interfaces should flow from the left-hand side of the component box, (3) only those interfaces that are relevant to the component under consideration should be shown, even if other interfaces are available. These recommendations are intended to simplify the visual nature of UML component diagrams.

Dependencies and Inheritance. For improved readability, it is a good idea to model dependencies from left to right and inheritance from bottom (derived classes) to top (base classes). In addition, components' interdependencies should be represented via interfaces, rather than by representation of a component-to-component dependency. Following the philosophy of the OCP, this will help to make the system more maintainable.

11.2.3 Cohesion

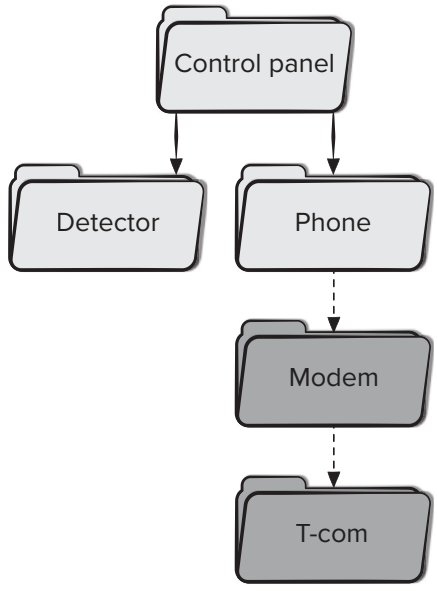
In Chapter 9, we described cohesion as the "single-mindedness" of a component. Within the context of component-level design for object-oriented systems, *cohesion* implies that a component or class encapsulates only attributes and operations that are closely related to each other and to the class or component itself. Lethbridge and Laganiére [Let04] define several different types of cohesion (listed in order of the level of the cohesion):[4]

> **Functional.** Exhibited primarily by operations, this level of cohesion occurs when a module performs one and only one computation and then returns a result.
>
> **Layer.** Exhibited by packages, components, and classes, this type of cohesion occurs when a higher layer accesses the services of a lower layer, but lower layers do not access higher layers. Consider, for example, the *SafeHome* security function requirement to make an outgoing phone call if an alarm is sensed. It might be possible to define a set of layered packages, as shown in Figure 11.5. The shaded packages contain infrastructure components. Access is from the control panel package downward.
>
> **Communicational.** All operations that access the same data are defined within one class. In general, such classes focus solely on the data in question, accessing and storing it.

4 In general, the higher the level of cohesion, the easier the component is to implement, test, and maintain.

FIGURE 11.5

Layer cohesion



Classes and components that exhibit functional, layer, and communicational cohesion are relatively easy to implement, test, and maintain. You should strive to achieve these levels of cohesion whenever possible. It is important to note, however, that pragmatic design and implementation issues sometimes force you to opt for lower levels of cohesion.

SAFEHOME



Cohesion in Action

The scene: Jamie's cubicle.

The players: Jamie and Ed, members of the *SafeHome* software engineering team who are working on the surveillance function.

The conversation:

Ed: I have a first-cut design of the **camera** component.

Jamie: Wanna do a quick review?

Ed: I guess . . . but really, I'd like your input on something.

(Jamie gestures for him to continue.)

Ed: We originally defined five operations for **camera**. Look . . .

determineType() tells me the type of camera.

translateLocation() allows me to move the camera around the floor plan.

displayID() gets the camera ID and displays it near the camera icon.

displayView() shows me the field of view of the camera graphically.

displayZoom() shows me the magnification of the camera graphically.